ТЕХНОЛОГИИ
РАЗРАБОТКИ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

ТРПО

(Программная инженерия)

Часть 2. РАЗРАБОТКА ТРЕБОВАНИЙ

© Ржавин В.В.

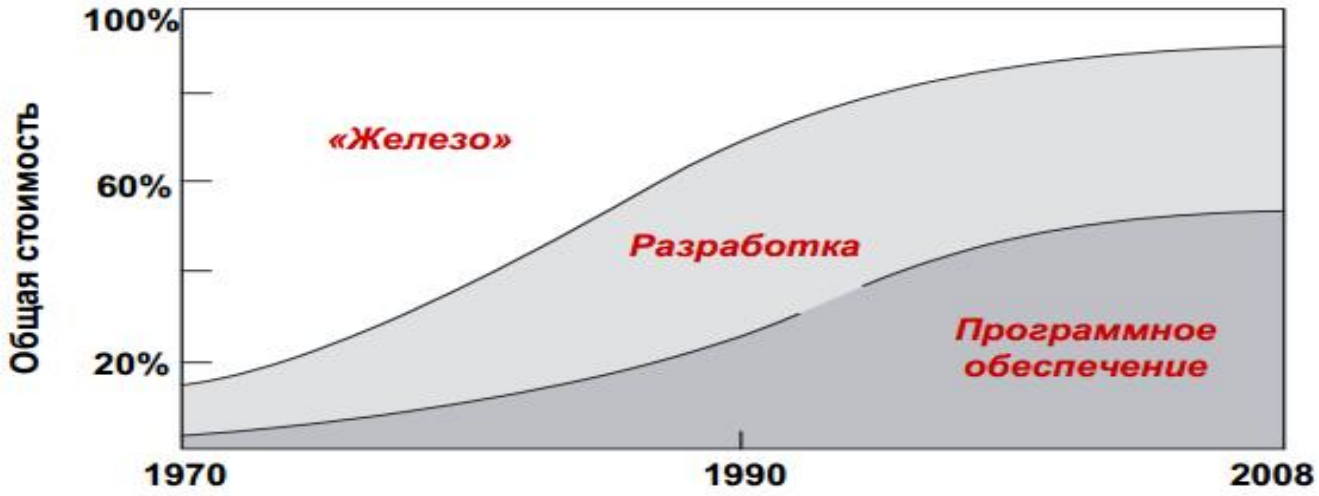
ЛИТЕРАТУРА

- К. Вигерс
  Разработка требований к программному обеспечению
- А. Коберн
  Современные методы описания функциональных требований к системам
- У. Леффингуэлл
  Принципы работы с требованиями к программному обеспечению. Унифицированный подход

ВВЕДЕНИЕ

Значительно возрастает доля программной составляющей

и это ведет к существенным проблемам

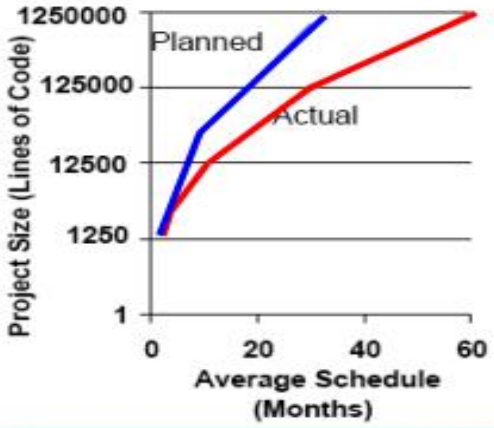


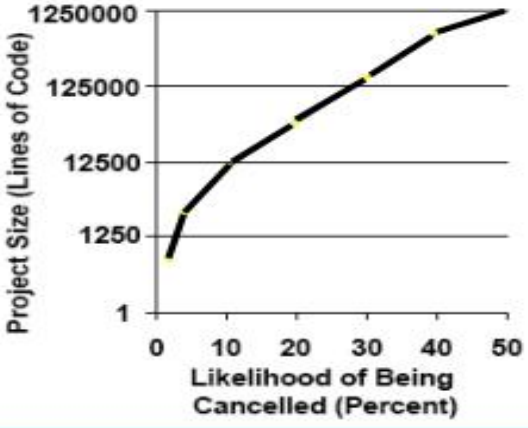
Распределение составляющих стоимости

Рост софтверных составляющих

Тип самолета	Год	Функции, контролируемые ПО
F-4	1960	8%
A-7	1964	10%
F-111	1970	20%
F-15	1975	35%
F-16	1982	45%
B-2	1990	65%
F-22	2000	80%

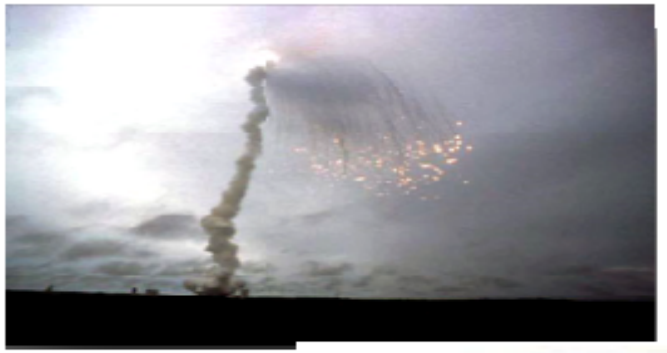
➔ *Влияние на бизнес*





Аварии все еще продолжают беспокоить производителей

- **Аэрокосмическое агентство**
  На 40с полета бортовой компьютер прототипа стратегической ракеты стоимостью $1 млрд. ошибочно выдал команду на самоуничтожение



- **Производитель автомобилей**
  Компания вынуждена была отозвать 75 тысяч автомобилей, которые из-за ошибки в ПО вдруг начинали тормозить на большой скорости

- **Производитель мед. оборудования:**
  Из-за некачественного ПО пришлось изымать из эксплуатации 42 тысячи дефибрилляторов

Зачем нужен Системный Инжиниринг

Системный Инжиниринг отвечает за всю **картину в целом**, обеспечивая выполнение требований в течение всего жизненного цикла изделия.

- Повышение вероятности успеха создания Системы
  - Понимание природы Системы и ее поведения в окружающей среде
  - Определение характеристик Системы с точки зрения пользователя
- Уменьшение рисков
  - Определение и оценка рисков, неопределенностей и изменяемых параметров в процессе принятия решений
  - Соответствие нормативным документам и требованиям регулирующих организаций
- Уменьшение общей стоимости жизненного цикла изделия
  - Улучшение процесса принятия решений в планировании, разработке, управлении, эксплуатации

Такой подход начинается с понимания потребностей заказчика, определения функциональности изделия и обязательных запланированных проверок (аттестаций, приемочных испытаний, контроля) на самых ранних стадиях жизненного цикла создания изделия.

Системный инжиниринг
(System Engineering)

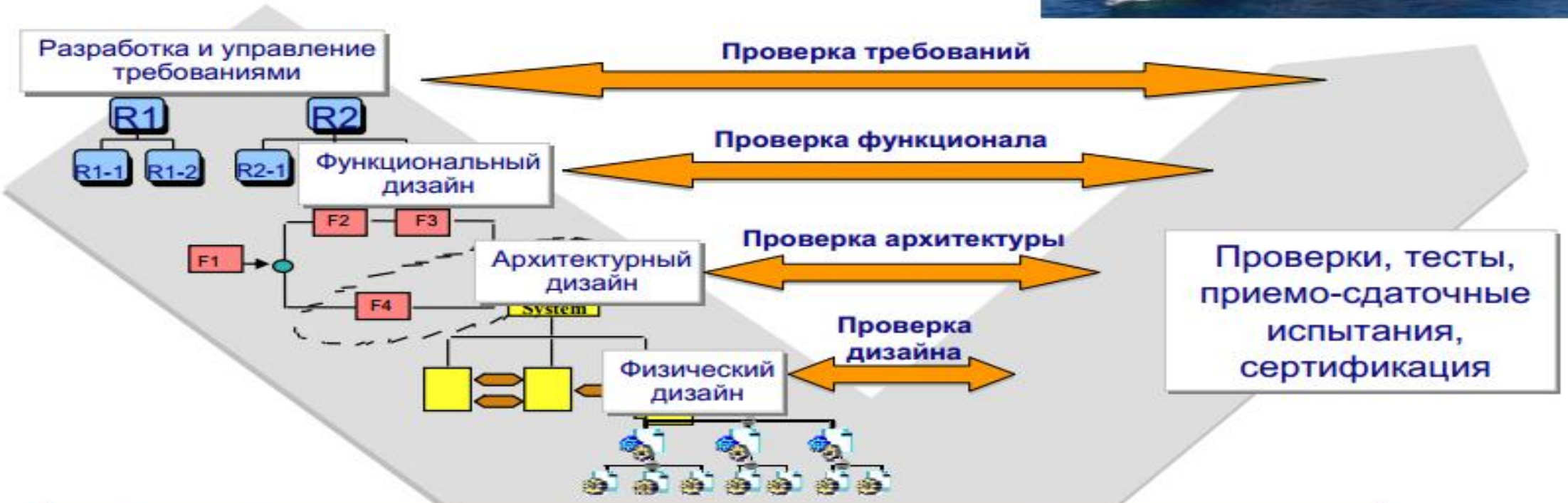
- **Системный инжиниринг** - целостный, ориентированный на изделие подход, отвечающий за создание и выполнение процессов, охватывающих различные инженерные дисциплины и обеспечивающих удовлетворение нужд заказчиков и непосредственных пользователей изделия

- Системный инжиниринг – есть **междисциплинарный** подход, используемый для контроля за разработками **сложных, инновационных** изделий и ***систем*** *

  - **Система** – это набор компонентов (*которые и сами могут быть системами*), соединенных и связанных определенным образом так, чтобы обеспечить функционирование, которое сами по себе компоненты обеспечить не могут.

  - **Корабль, самолет, ракета** - есть система.. и их основные компоненты (*корпус, крылья, система управления, силовые установки, вооружение, программное обеспечение …*), также являются системами (*или оборудованием*)

Для чего это все нужно ?



- Разработка продукта в полном соответствии с требованиями
- Учет изменений на всех уровнях разработки
- Тесты, проверки, сертификация проверят требования
- Обеспечивается сквозной мониторинг производства продукта
- **Конечный продукт соответствует требованиям на все 100%**

Решения для системного инжиниринга

Системный инжиниринг требует взаимодействия множества инструментов и дисциплин для обеспечения и поддержки

- Управления совместными бизнес-процессами:
  - Управление портфелями
  - Управление программами
  - Управление требованиями
  - Управление изменениями
  - Управление конфигурациями
  - Управление поставками
  - Управление потоками задач
  - Контроль за соблюдением стандартов
- Моделирование и тестирование систем и изделий (*в зависимости от их типа*) :
  - UML & SysML
  - Rhapsody
  - Simulink
  - Modelica
  - MCAD & ECAD

Кто создает и использует требования

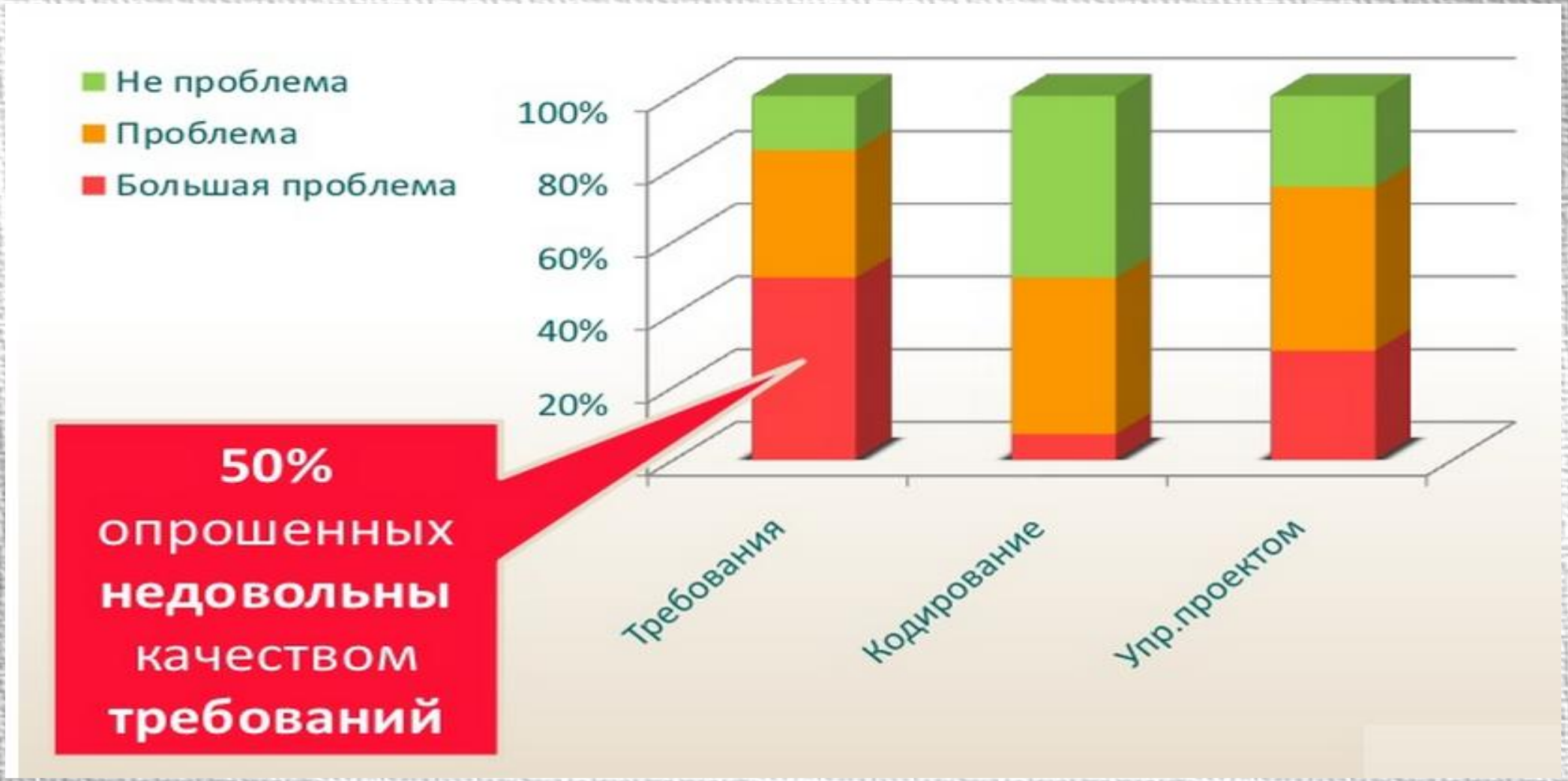
Как и кем используются требования?

- **Специалист по АТ** - постановка задачи, определение рамок проекта;
- **Представитель заказчика** - постановка задачи, определение рамок проекта, контроль работы исполнителя, приемка результатов работы;
- **Архитектор системы** - разработка архитектуры, проектирование подсистем;
- **Программист** - разработка программного кода;
- **Тестировщик** - составление тест-плана, тестовых сценариев;
- **Менеджер проекта** - планирование и контроль исполнения работ.

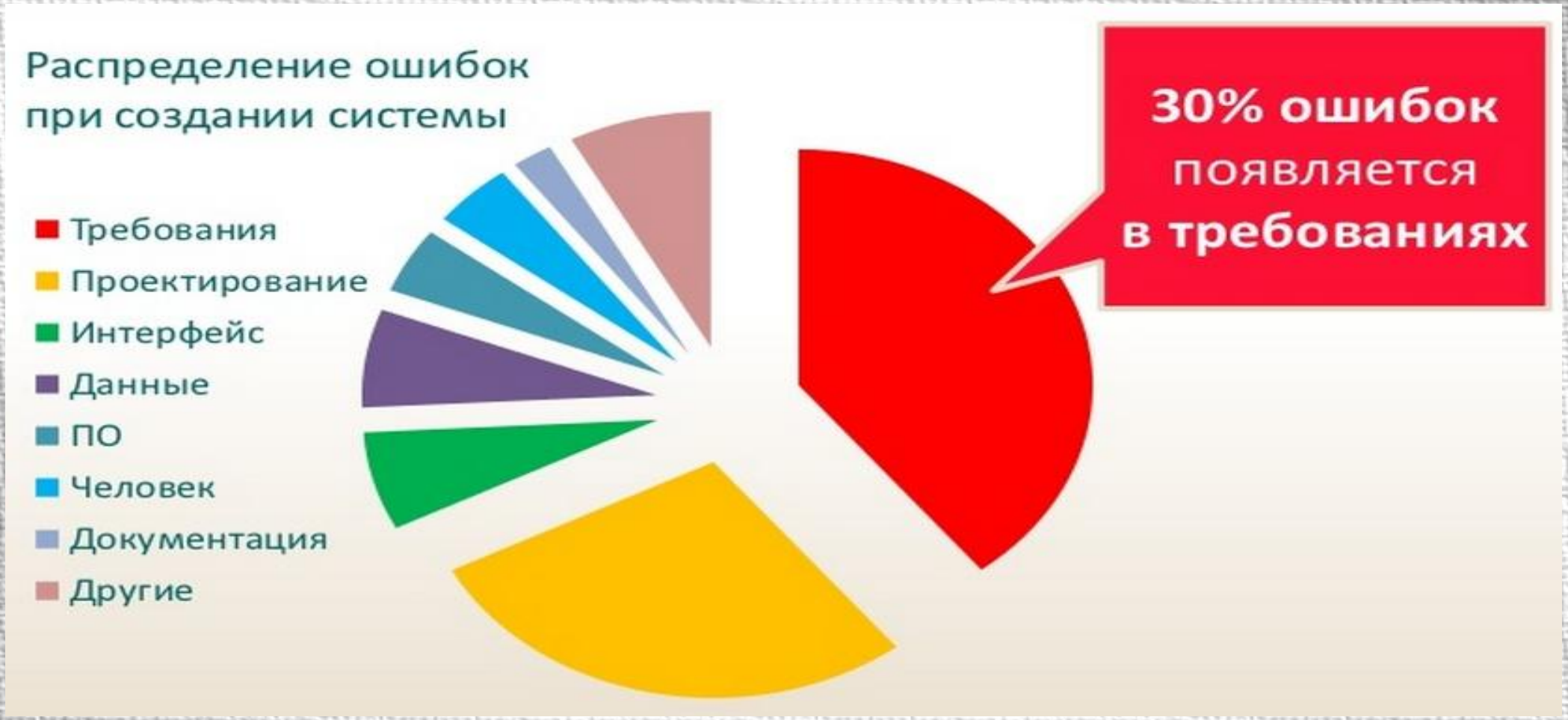
Для всех упомянутых выше лиц обычно используют обобщающий термин «**заинтересованные лица**" (*stakeholders*).

Чем завершаются IT-проекты

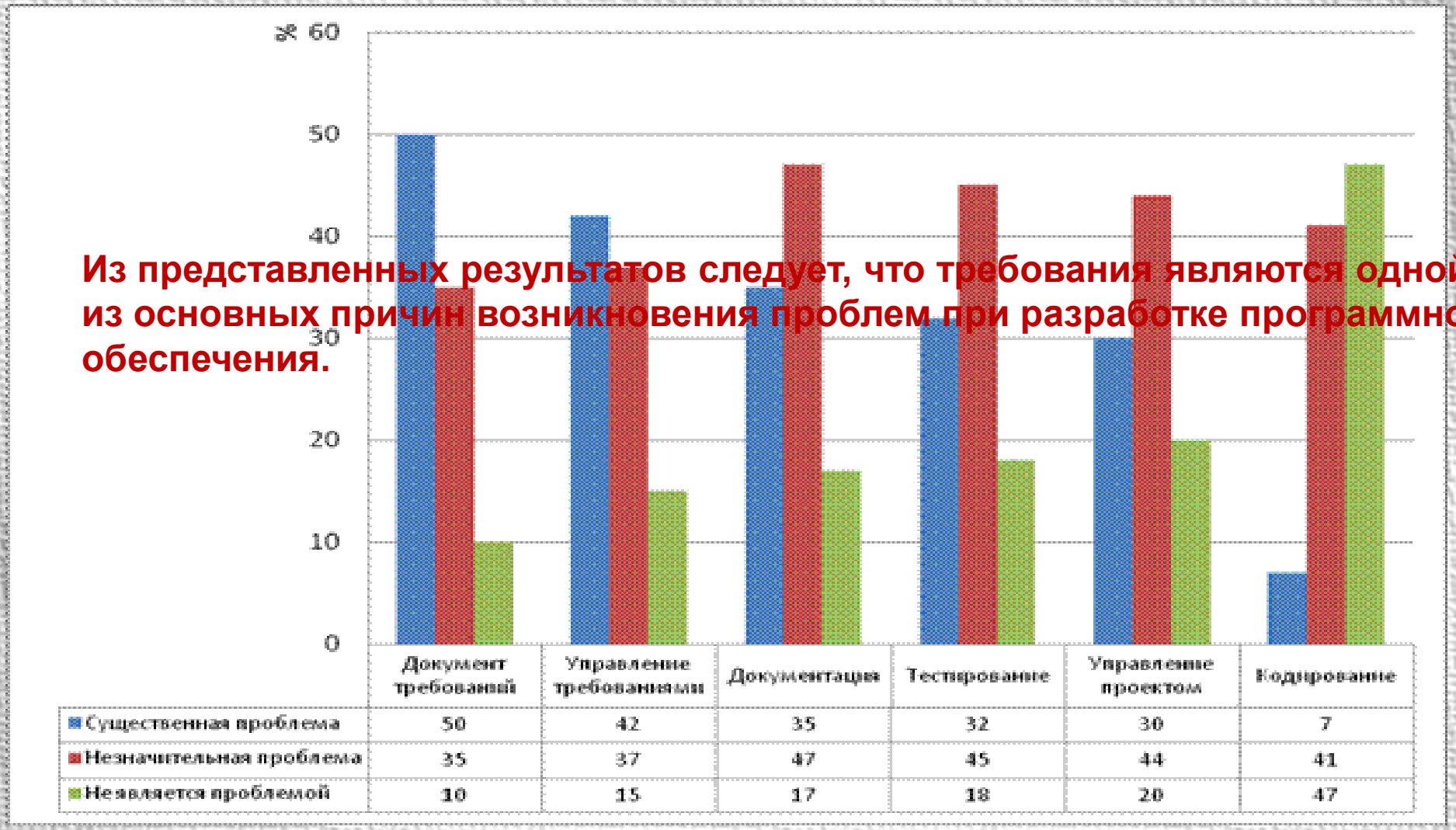
В чем причина провала IT-проектов



Распределение ошибок



Проблемы разработки программного обеспечения



	Документ требований	Управление требованиями	Документация	Тестирование	Управление проектом	Кодирование
Существенная проблема	50	42	35	32	30	7
Незначительная проблема	35	37	47	45	44	41
Не является проблемой	10	15	17	18	20	47

Из представленных результатов следует, что требования являются одной из основных причин возникновения проблем при разработке программного обеспечения.

1. Исследования последних лет показали, что успешные компании тратят на разработку требований 28% своих ресурсов и 38,5% человеко-часов.
2. Европейское исследование показало, что компании, которые тратят вдвое больше усилий на разработку требований, чем остальные (а среднее значение по индустрии - 14%), разрабатывают продукт быстрее.

Сколько стоит устранение ошибки?

Ключевые факторы, создающие проблемы

- Недостаток исходной информации от клиентов - 13% всех проектов;
- Неполные требования и спецификации -12% всех проектов;
- Изменение требований спецификаций -12% всех проектов.

К чему все это приводит ...
... и как найти выход из этого нерадостного положения?

Проблемы бизнеса

Продукт не удовлетворяет заказчика	46%
Поздний выход на рынок / упущенный спрос	33%
Слабая коммерциализация / раскрутка	26%
Качество продукта	24%
Ценовая политика	23%
Нечеткое позиционирование продукта	19%

The CIO's Guide to the PERFECT Launch: Translating Innovation to Business Benefit, AMR Research, 2005

Организационные возможности

Улучшить связь и взаимодействие между дисциплинами / доменами	**71%**
Повысить доступность требований	**49%**
Уметь прогнозировать поведение системы до тестирования	**46%**
Внедрить новый или переделать имеющийся процесс разработки, чтобы охватить множественные дисциплины / домены	**43%**
(... что-то, не имеющее отношения к данной теме ...)	39%

Инжиниринг требований – составная часть системного инжиниринга

Более **80%** разработок заканчиваются плачевно только из-за неудовлетворительного формирования требований, их анализа и управления им

IDC, November 2007

Почти **80%** ошибок вносится на стадии формирования требований

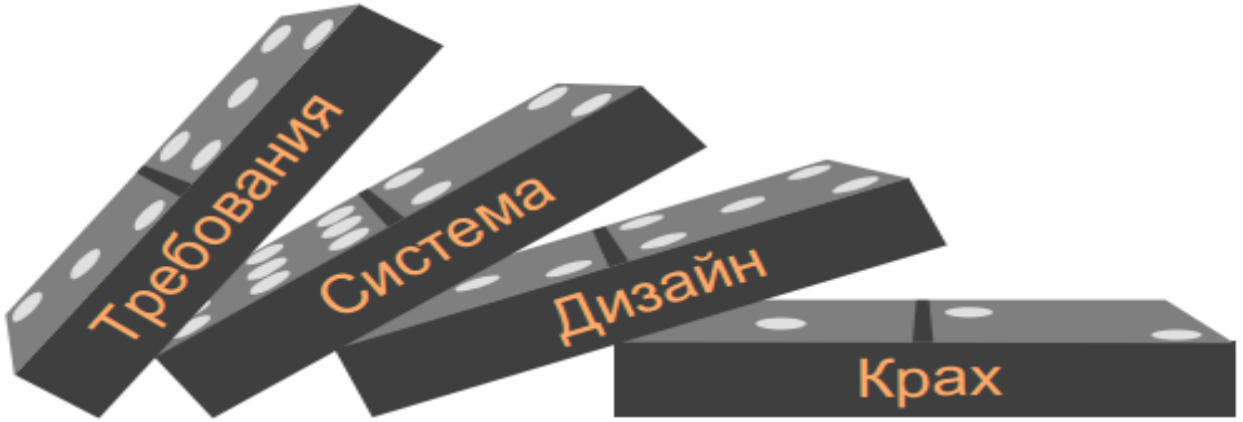
NASA, 2006

Проблемы в работе с требованиями ведут к излишним доработкам и переделкам, плохому качеству, задержкам и провалу проектов

Source:
- Defense System Management College, DAU, 2004

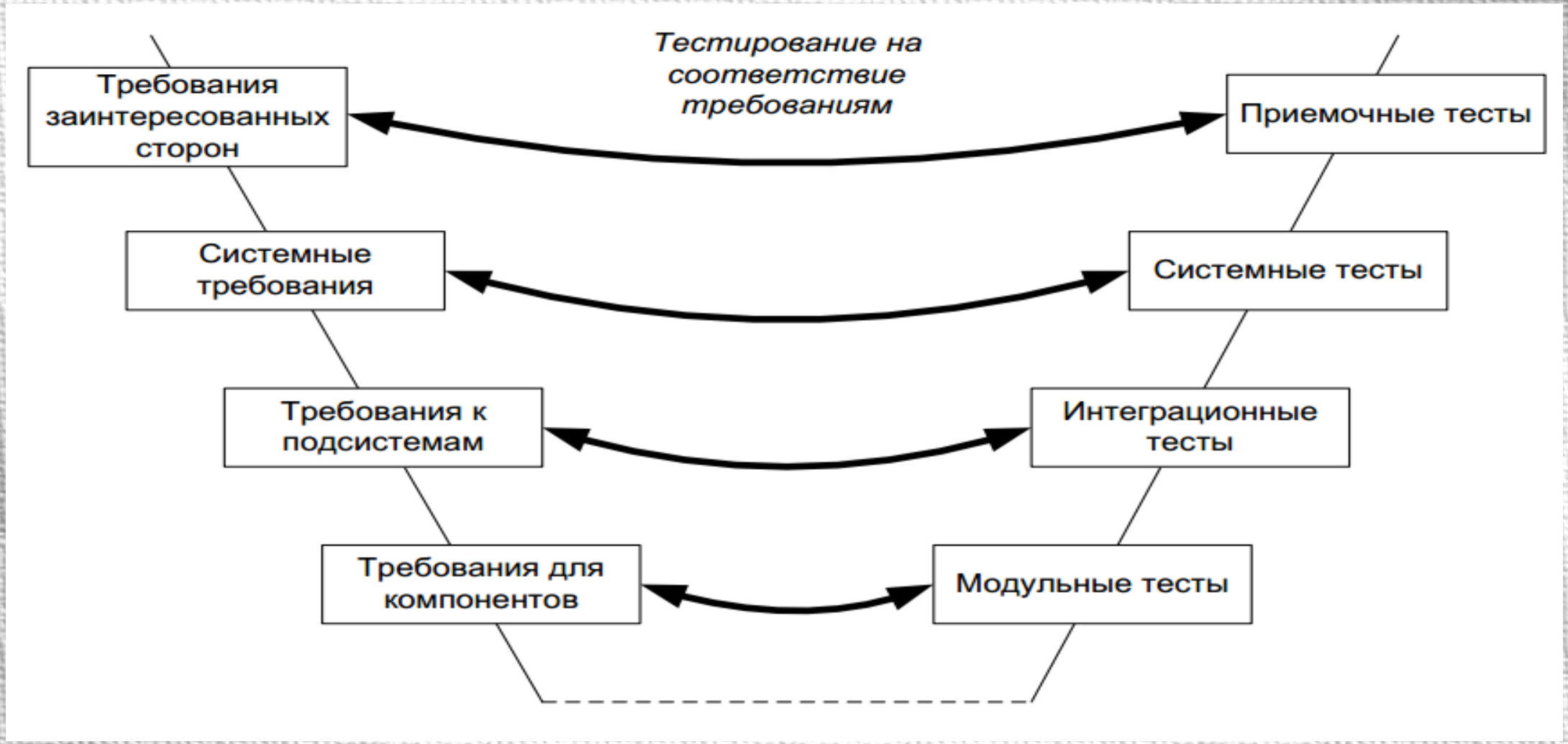
Очень важен правильный старт и эффективная работа

- Около 60%-70% общего числа всех проектов заканчиваются плачевным результатом только из-за неудовлетворительного формирования требований, их анализа, управления и контроля
  – *Meta Group, 2003*



- Задержки - средний проект «опаздывает» на 220%
- Неэффективная работа - 40-50% времени специалиста тратится на задачи, не связанные с выполнением его непосредственных обязанностей: *поиск нужных документов, отслеживание изменений...*

Требования и процесс выполнения проекта



Роль требований на каждом уровне реализации проекта

Роль и место требований в методологии разработки ПО

Независимо от выбранной методологии разработки, можно выделить следующие этапы жизненного цикла ПО:

1. ***Анализ рынка, технико-экономическое обоснование проекта***
2. ***Сбор и анализ требований к ПО***
3. ***Проектирование архитектуры системы***
4. ***Кодирование***
5. ***Тестирование***
6. ***Интеграция, развертывание***
7. ***Эксплуатация и обслуживание (поддержка)***

Наиболее активно интенсивно управление требованиями осуществляется на этапах 2 и 3, менее активно на этапах 4-5.

Области применения требований

- Планирование
- Управление рисками
- Управление объемом проекта
- Проектирование
- Тестирование
- Управление изменениями

Заключение

Требования –
это и компас и карта, которые помогут вам прийти к намеченной цели.

ТРЕБОВАНИЯ – КЛАССИФИКАЦИЯ СВОЙСТВА ДОКУМЕНТАЦИЯ

Определения

- **Требование** (requirement):
  - возможность, которую должна обеспечивать система;
  - некое свойство ПО, необходимое пользователю для решения проблемы при достижении послав ленной цели;
  - некое свойство ПО, которым должна обладать система или её компонент, чтобы удовлетворить условиям договора/контракта, стандарта, спецификации или другого формального документа.
- **Управление требованиями** (requirements management):
  - набор организованных, стандартизованных, систематических процессов и методов работы с требованиями в значительном/сложном проекте;
  - системный подход к выявлению, организации, документированию требований к системе, в ходе которого вырабатывается и обеспечивается соглашение между заказчиком и исполнителем по поводу меняющихся требований к системе.
- **Функция системы** (feature):
  - высокоуровневое описание желаемого поведения системы/продукта;
  - обслуживание, предоставляемое системой для выполнения одной/нескольких потребностей заинтересованного лица (ЗЛ).

- **Под техникой** мы понимаем деятельность, состоящую в выполнении определенной последовательности действий, нацеленных на решение одной вполне конкретной задачи. При этом часто ни одно из действий, предпринимаемых в рамках работы в соответствии с данной техникой, не имеет смысла выполнять отдельно.
- **Методом** называется набор идей и техник, объединенных этими идеями и образующих единую структуру обработки информации, нацеленную на решение ряда связанных задач из некоторой области. *Техники, используемые в рамках метода, можно применять отдельно или в рамках других методов для решения тех же задач.*
- **Процессом** мы будем называть такой метод, который,
  - во-первых, включает инструментальную поддержку ряда используемых в нем техник,
  - во-вторых, предназначен для решения не просто группы, а всех значимых задач в некоторой четко определенной области.

Что в себя включает дисциплина по управлению требований?

К действиям по управлению требованиями относятся:

- определение основной версии требований (моментальный срез требований для конкретной версии продукта);
- просмотр предлагаемых изменений требований и оценка вероятности воздействия каждого изменения до его принятия;
- включение одобренных изменений требований в проект установленным способом;
- согласование плана проекта с требованиями;
- обсуждение новых обязательств, основанных на оценке влияния изменения требований;
- отслеживание отдельных требований до их дизайна, исходного кода и вариантов тестирования;
- отслеживание статуса требований и действий по изменению на протяжении всего проекта.

Строжайшее и единственное правило построения систем программного обеспечения (ПО) - решить точно, ЧТО же строить. Никакая другая часть концептуальной работы не является такой трудной, как выяснение деталей технических требований, в том числе и взаимодействие с людьми, с механизмами и с иными системами ПО. Никакая другая часть работы так не портит результат, если она выполнена плохо. Ошибки никакого другого этапа работы не исправляются так трудно.

Ф.Брукс, американский учёный в области теории вычислительных систем, автор книги «Мифический человеко-месяц»

ЧТО ТАКОЕ ТРЕБОВАНИЕ

ТРЕБОВАНИЕ (requirement)

- возможность, которую должна обеспечивать система;
- некое свойство ПО, необходимое пользователю для решения проблемы при достижении поставленной цели;
- некое свойство ПО, которым должна обладать система или её компонент, чтобы удовлетворить условиям договора/контракта, стандарта, спецификации или другого формального документа.

Что такое требования и зачем нужны требования?

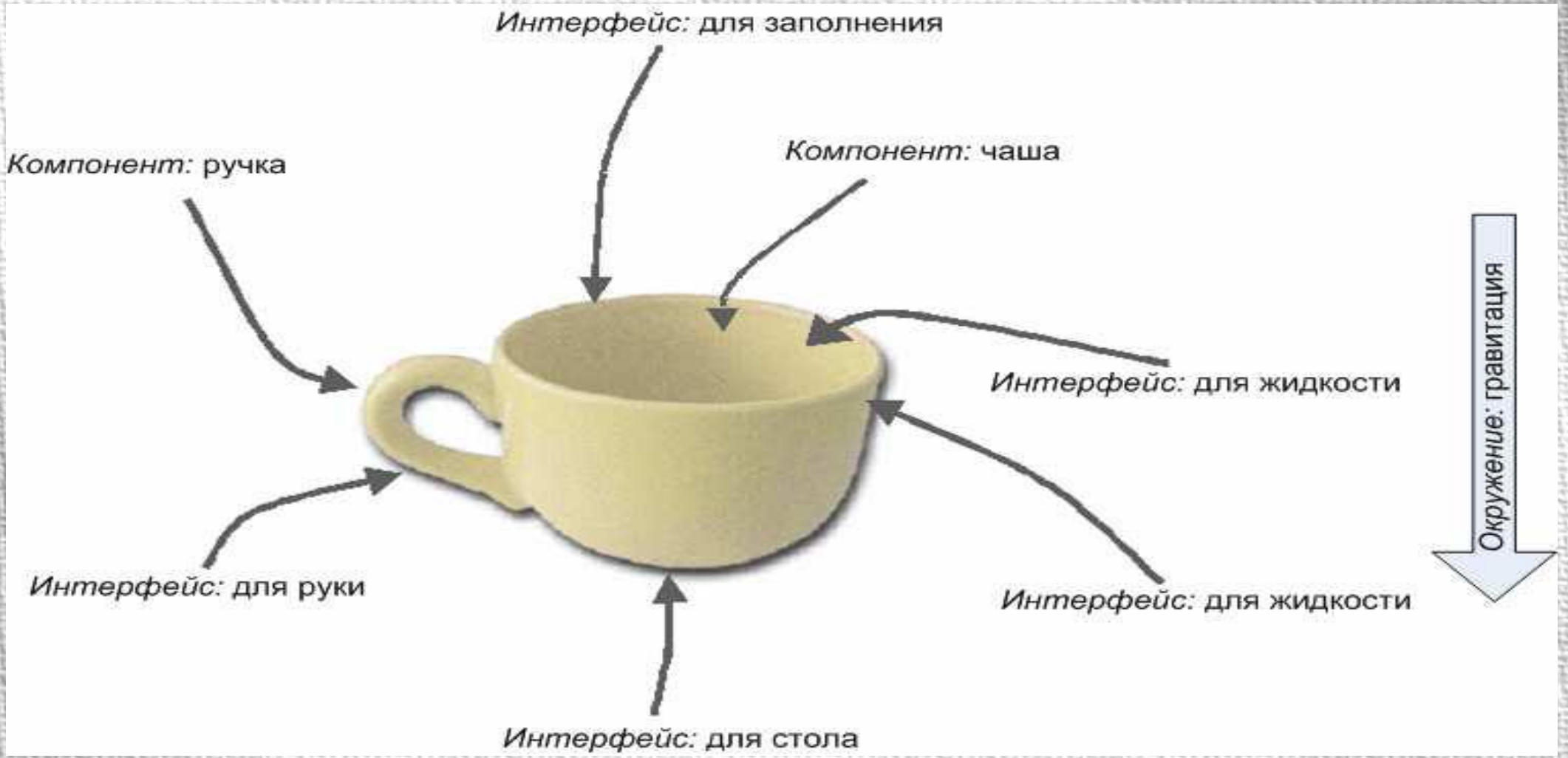
Требования определяют **цель**	• «получить «правильный» продукт для выхода с ним на рынок в подходящее для этого время, для захвата рыночной ниши» • «получить прибыль от реализации проекта»
Требования определяют **потребности (проблемы):**	• Требования заинтересованных сторон, бизнес требования
Требования определяют **решение**:	• Процедуры, регламенты, системные требования
Требования определяют **ограничения** связанные с решением или проектом по его реализации:	• Сроки, бюджет, персонал, применяемые технологии, соответствие требованиям законодательства и т.д.

Вертолет и гравитация

Для того чтобы правильно сформулировать требования к системе, необходимо понять требования к той системе, в чей состав она входит. Зачастую, от этого зависит правильное функционирование всей системы. Например, способность вертолета летать обеспечивается наличием у Земли гравитации и атмосферы.

Возьмем очень простой пример – чашку

ЧАШКА КАК ПРИМЕР ПРОСТОЙ СИСТЕМЫ



Очевидно, что чашка состоит из компонентов: ручки и чаши. Для каких целей нужны эти компоненты? - чаша нужна для того, чтобы содержать в себе жидкость, а ручка нужна для того, чтобы человек мог держать чашку, не обжигаясь. Отсюда вывод - цель, или **требования для чашки - это позволить человеку донести горячую жидкость до рта, не разлив ее и не получив ожога.**

Чашка полна интерфейсов. Она может быть поставлена на плоскую поверхность для устойчивости; ее можно держать в человеческой руке; она может быть наполнена жидкостью и опустошена; должен быть интерфейс с жидкостью для продолжительного ее содержания; и она должна подходить для перемещения жидкости в человеческий рот.

Тем не менее, можно сделать и другие наблюдения:.

- *Сама по себе чашка не может удовлетворить цель, сформулированную нами выше, так как чашка зависит от движений человеческой руки, которая ее держит.*
- *Чаша, как компонент чашки, критически зависит от гравитации, а также правильного использования. Если чашку перевернуть вверх дном, это приведет к выливанию жидкости, и, возможно, ожогу.*

Итак, способность простой чашки удовлетворить сформулированную нами цель зависит от:

a) свойств, которые появились в результате взаимодействия ее компонентов;

b) соответствующих интерфейсов с внешними компонентами;

c) ее корректного включения в общую систему - чашка удерживается и переносится человеческой рукой;

d) присутствия соответствующих внешних условий - в условиях невесомости для удовлетворения поставленной нами цели явно потребуется другое решение.

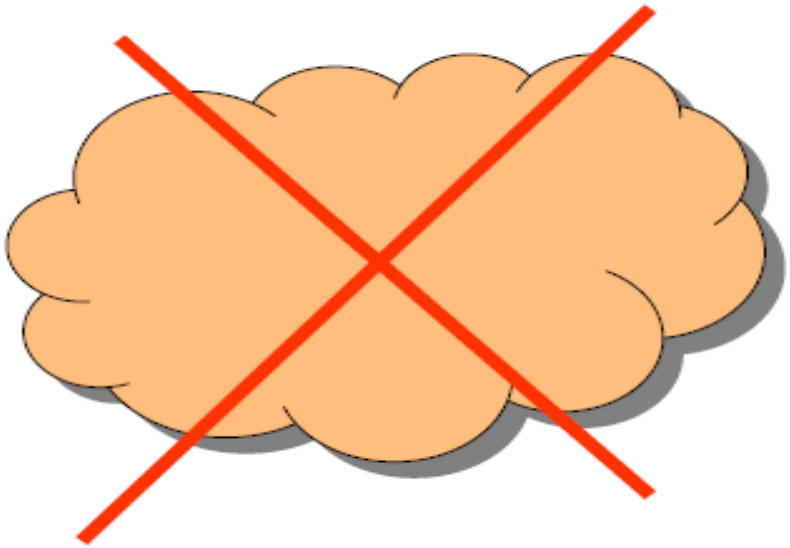
Вывод

Резюмируя вышесказанное, необходимо отметить, что разработка требований должна обязательно принимать во внимание природу системы. Нельзя рассматривать системные свойства изолированно, - необходим комплексный подход: учет условий, которые привносит внешнее окружение, накладываемые кем-то или чем-то ограничения, а также интерфейсы с окружающими системами.

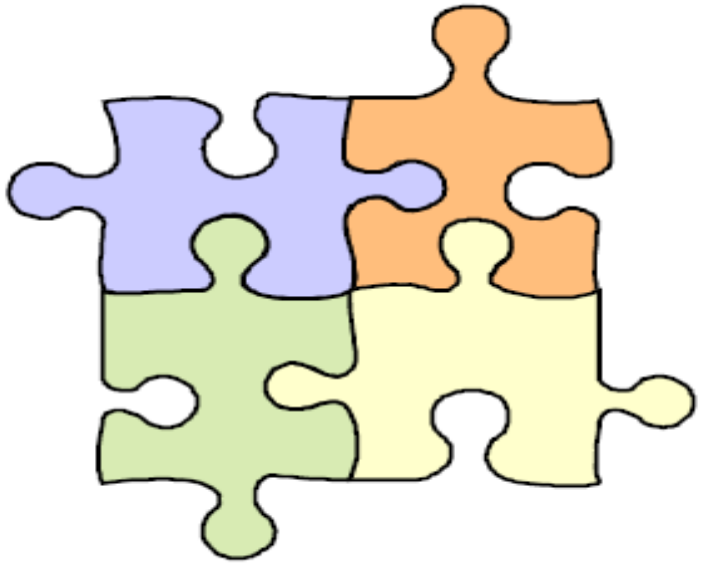
Требования – это тот первичный информационный ресурс, на основании которого проектируется и создается ПС.

Первичные данные о требованиях поступают из различных источников (*stakeholders*), характеризуются противоречивостью, неполнотой, нечёткостью и изменчивостью.

Дискретность требований



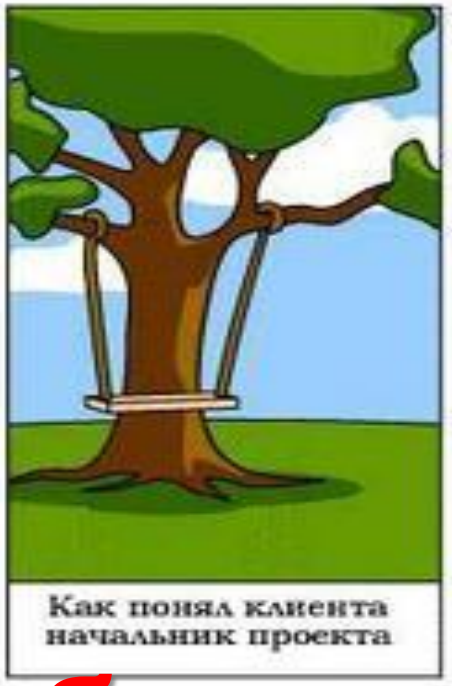
Бесформенный набор предложений



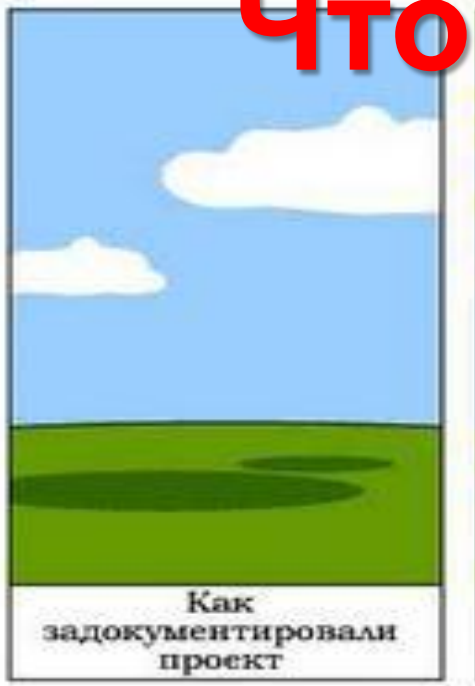
Отдельные (дискретные) утверждения, составляющие целостную картину

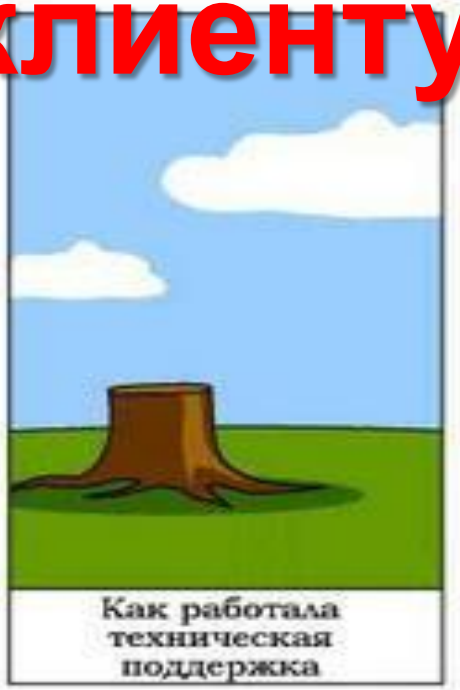
Слон и слепые

www.oper.ru



Что было нужно клиенту?





SWEBOK
(Software Engineering Body of Knowledge)

SWEBOK (**Software Engineering Body of Knowledge**) — документ, подготавливаемый комитетом *Software Engineering Coordinating Committee*, в который вовлечено сообщество IEEE Computer Society. Назначение SWEBOK — в объединении знаний по инженерии программного обеспечения.

Документ является одним из трёх документов, призванных обеспечить следующее:

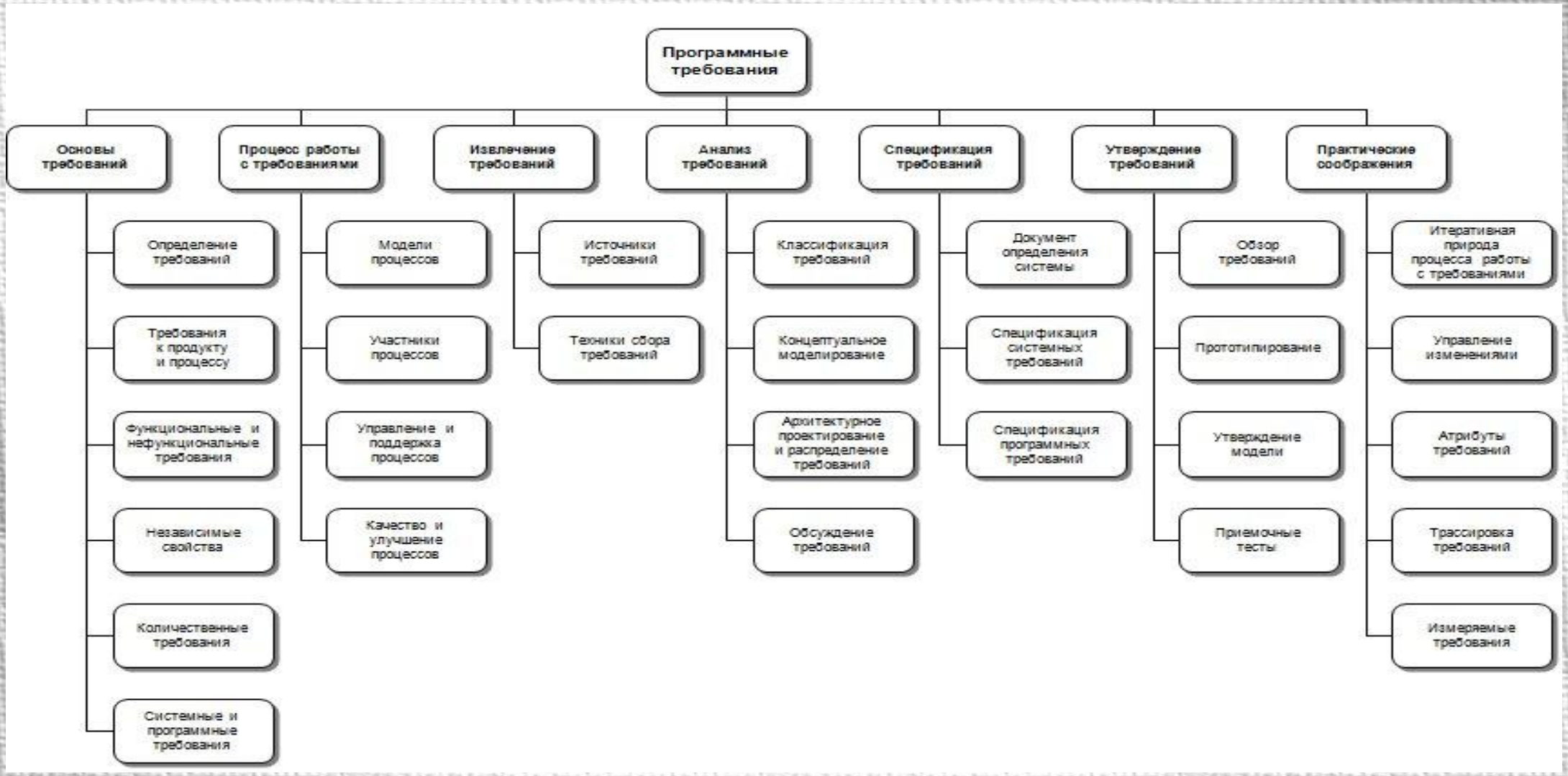
- определить необходимый набор знаний и рекомендуемые практики;
- определить этические и профессиональные стандарты;
- определить учебную программу для студентов, аспирантов и продолжающих обучение.

Связь с другими областями знаний

Область знаний управления требованиями включает 7 секций, каждая из которых представлена в виде ключевых тем. Кроме того, данная область знаний тесно связана со следующими областями:

- **Проектирование (**Software Design)
- **Тестирование** (Software Testing)
- С**опровождение** ПО (Software Maintenance)
- **Управление конфигурацией программного обеспечения** (Software Configuration Management)
- **Управление IT проектом** (Software Engineering Management)
- **Процесс программной инженерии** (Software Engineering Process)
- **Качество ПО** (Software Quality)

Область знаний “Программные требования” [SWEBOK, 2004]

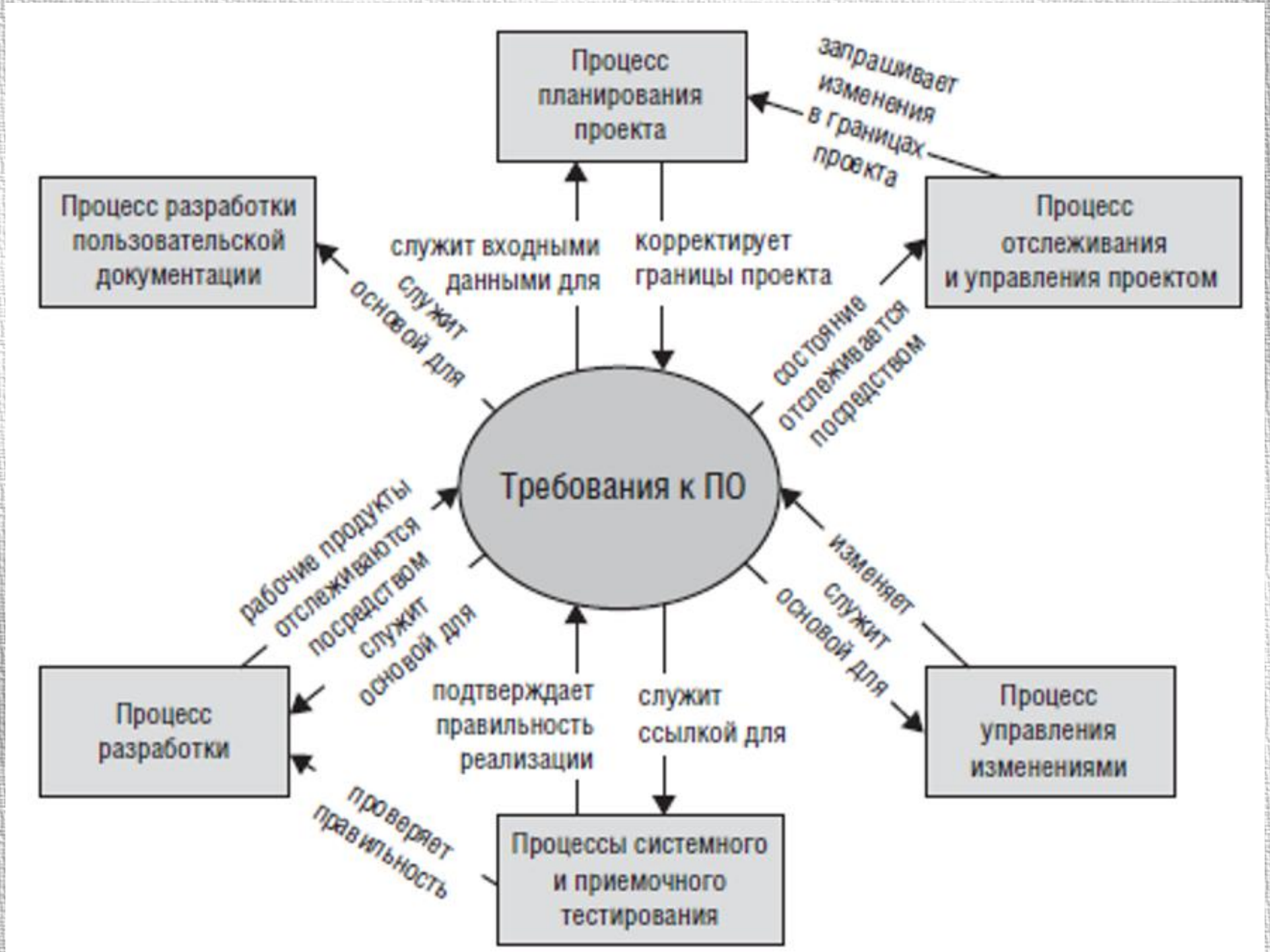


ИСТОЧНИКИ ТРЕБОВАНИЙ

- Федеральное и муниципальное отраслевое законодательство (конституция, законы, распоряжения)
- Нормативное обеспечение организации (регламенты, положения, уставы, приказы)
- Текущая организация деятельности объекта автоматизации
- Модели деятельности (диаграммы бизнес-процессов)
- Журналы использования существующих программно-аппаратных систем
- Конкурирующие программные продукты
- Заинтересованные лица

Как и кем используются требования?

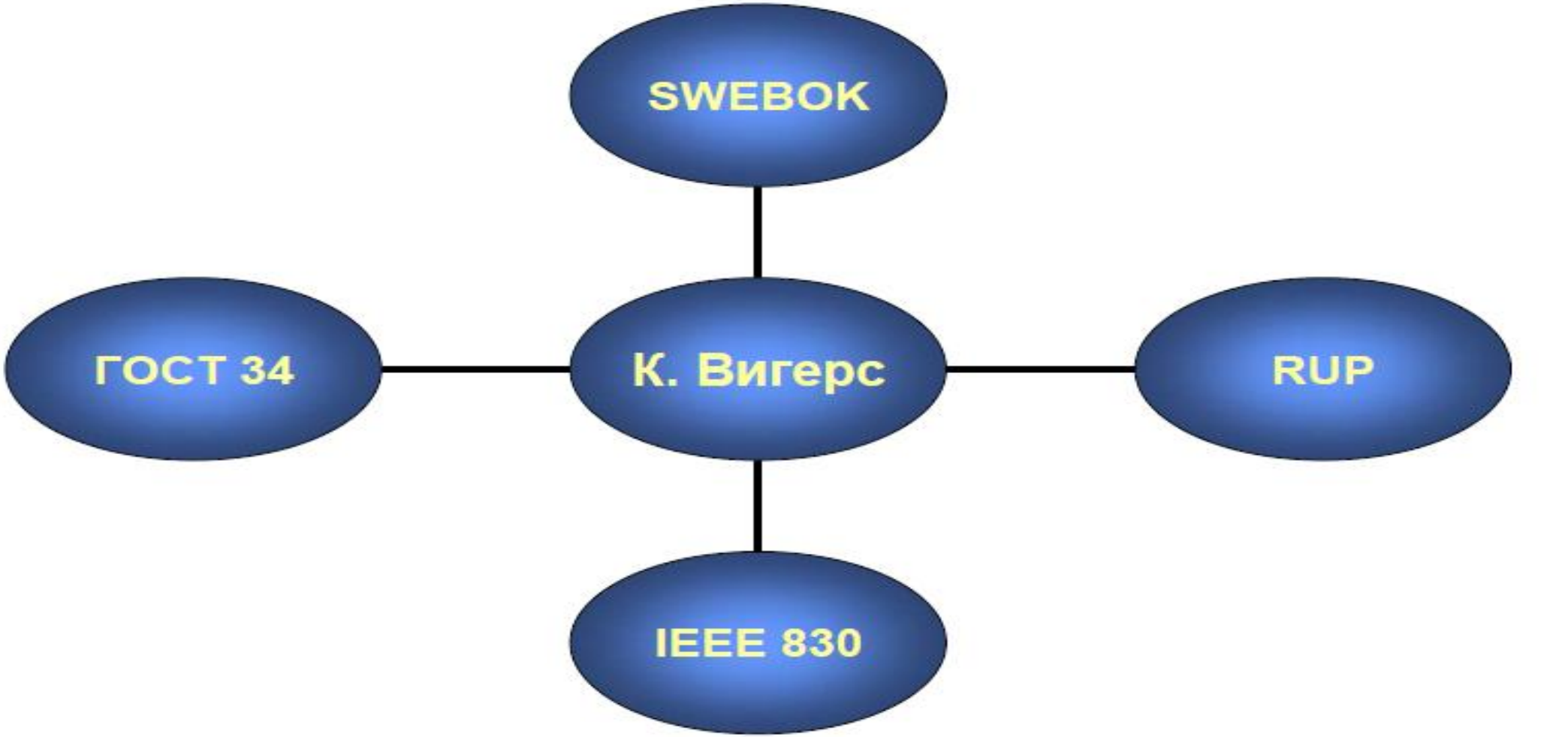
- ***Специалист по АТ*** – постановка задачи, определение рамок проекта
- ***Представитель заказчика*** – постановка задачи, определение рамок проекта, контроль работы исполнителя, приёмка результатов работы
- ***Архитектор системы*** – разработка архитектуры, проектирование подсистем
- ***Программист*** – разработка программного кода
- ***Тестировщик*** – составление тест-плана, тестовых сценариев
- ***Менеджер проекта*** – планирование и контроль исполнения работ


Процесс планирования проекта
запрашивает изменения в границах проекта
Процесс разработки пользовательской документации
Процесс отслеживания и управления проектом
служит входными данными для
корректирует границы проекта
служит основой для
состояние отслеживается посредством
Требования к ПО
рабочие продукты отслеживаются посредством
служит основой для
изменяет
служит основой для
Процесс разработки
подтверждает правильность реализации
служит ссылкой для
Процесс управления изменениями
проверяет правильность
Процессы системного и приемочного тестирования

КЛАССИФИКАЦИЯ ТРЕБОВАНИЙ

- Классификация **Вигерса**
- Классификация **Леффингвелла**
- Классификация **FURPS+**
- Классификация **ITIL v.3**

ИСТОЧНИКИ КЛАССИФИКАЦИИ



ОБОБЩЕННАЯ КЛАССИФИКАЦИЯ

КЛАССИФИКАЦИЯ 1
(ПО ПРЕДМЕТУ)

Требования к проекту

Вопросы формулирования требований к проекту, т.е. к тому, как Разработчик будет выполнять работы по созданию целевой системы, казалось бы, не лежат в компетенции Заказчика. Без регламентации процесса Заказчиком легко можно было бы обойтись, если бы все проекты всегда выполнялись точно и в срок. Однако, к сожалению, мировая статистика результатов программных проектов говорит об обратном. Заказчик, вступая в договорные отношения с Разработчиком, несёт различные риски, главными из которых является риск получить продукт с опозданием, либо ненадлежащего качества.

Основные мероприятия по контролю и снижению риска – ***регламентация процесса создания программного обеспечения и его аудит***.

К требованиям проекта относятся:

- физические ресурсы, необходимые команде разработки, такие как рабочие станции, специальные аппаратные устройства, тестовые лаборатории, средства и оборудование тестирования, командные комнаты и оборудование для видеоконференций;
- потребности в обучении персонала;
- пользовательская документация, включая обучающие материалы, пособия, справочные руководства и информация о выпусках ПО;
- документация для поддержки, такая как ресурсы службы технической поддержки, а также информация о техническом обеспечении и обслуживании аппаратных устройств;
- инфраструктурные изменения, которые необходимо внести в рабочую среду;

- требования и процедуры для выпуска продукта, установки в рабочей среде, конфигурирования и тестирования;
- требования и процедуры для перехода со старой на новую систему, например требования по переносу и преобразованию данных, по настройке безопасности, переносу производства и обучению для восполнения недостатка квалификации — это требования иногда называют *требованиями по переходу* (transition requirements);
- требования по сертификации продукта и его соответствия требованиям регулирующих органов;
- скорректированные политики, процессы, организационные структуры и аналогичные документы;

- сорсинг, приобретение и лицензирование ПО сторонних производителей и компонентов оборудования;
- требования по бета-тестированию, производству, упаковке, маркетингу и дистрибуции;
- соглашения об уровне обслуживания с клиентами;
- требования по правовой защите (патенты, товарные знаки или авторское право) интеллектуальной собственности, связанной с разрабатываемым ПО.

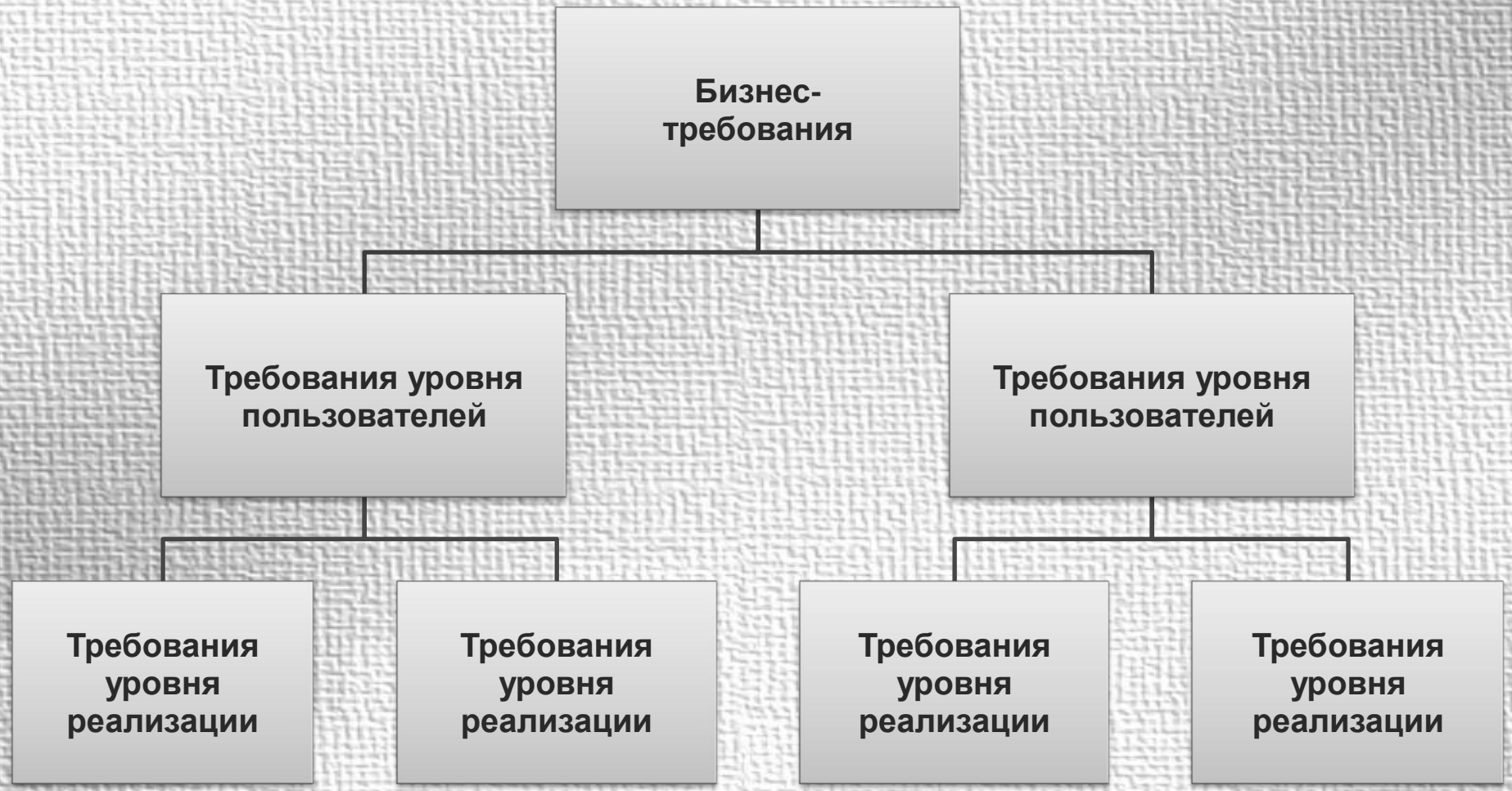
Это требования к проекту, но не к продукту. Спецификация требований содержит требования к продукту и не содержит деталей дизайна или реализации (кроме известных ограничений), данных о планировании проекта, сведений о тестировании и тому подобной информации. **Удалите указанные элементы из требований**, чтобы из этого документа было абсолютно ясно, что надлежит построить команде разработчиков.

Требования к продукту

В своей основе требования – это то, что формулирует заказчик. Цель, которую он преследует – получить хороший конечный продукт: функциональный и удобный в использовании.

Поэтому требования к продукту являются основополагающим классом требований.

КЛАССИФИКАЦИЯ 2 (ПО УРОВНЮ)



БИЗНЕС ТРЕБОВАНИЯ
(BUSINESS REQUIREMENTS)

Назначение:

Бизнес-требования описывают, **почему** организации нужна такая система, то есть цели, которые организация намерена достичь с ее помощью. Основное их содержание — бизнес-цели организации или клиента, заказывающих систему.

Для проектов, разрабатываемых в соответствии с ГОСТ, «бизнес требования» описывают цели и назначение разрабатываемой системы, и могут размещаться в соответствующем разделе технического задания.

Примеры бизнес требований:

- *Ускорение обработки заказов на 30 %.*
- *Снижение издержек ведения бухгалтерии на 200 тыс. рублей в год.*

Где описываются:

Концепция системы (границы и содержание проекта), Business Requirements Document (BRD).

Свойства БТ

- В случае с информационными системами или разработкой нестандартного приложения бизнес-требования должен определять тот, кто платит, а требования пользователей — те, кто будет стучать по клавишам и непосредственно работать с продуктом.
- Всем участникам проекта рекомендуете проверить согласованность бизнес-требований и требований пользователей.
- Бизнес-требования отражают организационную стратегию или бюджетные ограничения, скрытые от пользователей.
- Источником бизнес требований, как правило, является отдел маркетинга и высшее руководство.
- Бизнес требования описывают критерии успешности проекта с точки зрения спонсоров проекта.
- **Бизнес требования являются требованиями высшего уровня, остальные требования подчинены им.**

Модель выявления требований

Давайте поговорим о том, откуда берутся требования.

И бизнес-требования (БТ), и нефункциональные требования напрямую вытекают из потребностей бизнеса.

Из бизнес-требований, помимо функциональных (ФТ) и нефункциональных требований, напрямую могут вытекать ещё и требования причастных сторон. В свою очередь, из требований причастных сторон могут вытекать ФТ и НФТ.

Культурные образцы — это те требования, которые можно получить на основе стороннего опыта. То есть, изучая какой-то аналогичный готовый продукт на рынке, мы можем сформировать требования к своему продукту (даже если эти требования никто из стейкхолдеров не озвучивал напрямую).

Все виды требований возникают в конкретном контексте, то есть следуют из Операционного контекста, к которому относятся активы, спрос, технологии и так далее.

Согласно классификации требований по BABOK®Guide именно *бизнес-требования* являются основой всех остальных видов требований: пользовательских (требования стейхолдеров) и требований к решению (функциональных и нефункциональных).

Можно трассировать бизнес-потребность к бизнес-требованиям и бизнес-правилам

Бизнес-потребность	Бизнес-требование	Бизнес-правила
Проблема: BN-1. Потеря 20% клиентских заявок от 50 входящих обращений в неделю **Следствие**: 75 тысяч рублей недополученного дохода при 50%-ной конверсии заявки в продажу стоимостью 15 тысяч рублей	BReq-1. Сократить долю потерянных (полученных, но необработанных в плановый срок) заявок с 20% до 0% к 01.10.2023	BR-1. Плановый срок обработки клиентской заявки не более 48 часов BR-2. Заявки от юридических лиц на сумму от 100 тысяч рублей получают наивысший приоритет BR-3. Заявки с наивысшим приоритетом должны быть обработаны в течение 24 часов с момента их поступления

Примеры бизнес-требований по видам критичных активностей

Вид БТ: Значимые характеристики продуктов или услуг

Примеры БТ:

- Скорость обслуживания — очередь на кассе должна быть не длиннее 4 покупателей.
- Своевременность доставки — клиент должен получить пиццу горячей и не умереть ожидая ее.
- Простота получения услуги — оформление предварительно одобренного кредита должно завершаться за одно посещение клиента.

Вид БТ: Распознавание и обработка событий

Примеры БТ:

- Банк должен своевременно получать оповещения о сбоях в работе банкоматов.
- К моменту начала этапа работ, все нужные материалы и оборудование должны находиться на объекте строительства и быть готовыми к использованию.
- Сотрудники должны быть своевременно проинформированы о назначенных им задачах.

Вид БТ: Принятие решений

Примеры БТ:

- По каждому обнаруженному дефекту, на основе имеющейся информации, должно быть оперативно принято решение о способе его устранения в соответствии с протоколом реакции на аварийные ситуации.
- Решение о выдаче кредита должно приниматься на основании…
- Отпускная цена на товар для конкретного клиента определяется на основании… в зависимости от …

Вид БТ: Сбор, обработка, хранение и предоставление информации

Примеры БТ:

- С момента обнаружения дефекта и до его полного устранения должна регистрироваться и храниться следующая информация.
- Инвестиционный банк в конце каждого рабочего дня должен направлять Регулятору информацию о…
- Для принятия решений об участии в тендере необходимы результаты предварительной оценки себестоимости проекта.

Вид БТ: Обеспечение возможности выполнения действий

Примеры БТ:

- В случае неудовлетворенности качеством обслуживания, клиент должен иметь возможность направить жалобу по телефону, по электронной почте или через сайт компании.
- Клиент должен иметь возможность отказаться от заказа до момента его отправки.
- Покупатели должны иметь возможность оформить и оплатить покупку самостоятельно посредством мобильного приложения.

Вид БТ: Предотвращение возможности выполнения действий

Примеры БТ:

- Должна исключаться возможность доступа к защищенным ресурсам лиц, не являющихся сотрудниками компании.

Работа с «магическими» числами

Чтобы проверить требование на наличие данной проблемы, можно определить чувствительность (столбец «чуть-чуть» в таблице) требования и его границы (столбец «гораздо» в таблице).

Для успеха проекта важно своевременно обнаруживать некорректные требования и решать, что с ними делать.

Для тестирования бизнес-требований на «магические числа» можно использовать модель, представленную на рисунке.

Примеры некорректных бизнес-требований

- Обеспечить высокое качество обслуживания — Как проверить, что качество обслуживания высокое?
- Сократить среднее время обработки заказа до 14 минут — Почему до 14 минут, чем это обосновано? А почему не до 1 минуты? Какую пользу принесет сокращение времени обработки заказа?
- Обеспечить 100% достоверность учетных данных — Как мы узнаем какая достоверность? Как можно определить, что данные достоверны? В результате чего возникает недостоверность? С этим реально можно что-то сделать? Выполнимо ли данное требование?
- Гарантировать правильность принимаемых решений — Что выступает гарантом? Это выполнимо?
- Внедрить ERP — Кому и зачем это нужно? В чем проблема?

ПОЛЬЗОВАТЕЛЬСКИЕ ТРЕБОВАНИЯ
(USER REQUIREMENTS)

Назначение:

ПТ определяют набор пользовательских задач, которые должна решать ИС, а также способы (сценарии) их решения в системе. Пользовательские требования описывают, **что** пользователь должен делать с системой. Область пользовательских требований также включает описания атрибутов или характеристик продукта, которые важны для удовлетворения пользователей.

Пример ПТ:

- *система должна обеспечить возможность ввода информации о заказе, с последующим ее сохранением и возможностью передачи информации о заказе к любому сотруднику.*

Где описываются:

Диаграммы вариантов использования, сценарии взаимодействия, пользовательские истории , функциональные модели в различных нотациях.

Способ представления ПТ

Пользовательские требования могут выражаться в виде

- **фраз утверждений,**
- **в виде вариантов использования (use case),**
- **пользовательских историй (user story),**
- **сценариев взаимодействия (scenario).**

Свойства ПТ

- ***Пользовательские требования*** отражают пожелания непосредственных пользователей системы.
- Хорошей практикой является оформление пользовательских требований в виде вариантов использования (*use cases*).
- Работе с пользовательскими требованиями следует уделять пристальное внимание, так как мнение о системе, сложившееся у непосредственных пользователей является ключевым фактором успешности внедрения и эксплуатации системы.
- Требования пользователей часто бывают плохо структурированными, дублирующимися, противоречивыми. Поэтому для создания системы важен третий уровень, в котором осуществляется формализация требований.

- Важно помнить, что в большинстве проектов есть несколько классов пользователей, а также других участников, потребности которых тоже надо выявить.
- Зачастую на практике «пользовательские требования» идут в разрез с «бизнес требованиями». Для примера, заказчик хочет сделать дешевую систему, пользователь, напротив, желает работать с удобным пользовательским интерфейсом, реализация которого дорого обойдется заказчику. Подобные противоречия должен идентифицировать и решать системный аналитик.

ТРЕБОВАНИЯ УРОВНЯ РЕАЛИЗАЦИИ
(REALIZATION REQUIREMENTS)

Требования уровня реализации (требования к решению) — определяют функциональность и нефункциональные характеристики, необходимые для реализации программного продукта. Эти требования определяют технические аспекты системы, такие как программно-аппаратные требования, производительность, надежность и безопасность.

Каждый уровень требований имеет свой набор характеристик и принципов разработки, и все они должны быть взаимосвязаны и согласованы друг с другом. Обычно требования на каждом уровне подвергаются проверке, верификации и валидации для обеспечения соответствия спецификации и критериям качества..

ПРИМЕРЫ

1. Бизнес-требование
   - *“…..система должна сократить в 2 раза срок выполнения обрабатываемых на предприятии заказов …..”*
2. Пользовательское требование
   - *“…система должна обеспечить возможность ввода информации о заказе, с последующим ее сохранением и возможностью передачи информации о заказе к любому сотруднику….”*
3. Функциональные требования
   - “ …необходимо разработать диалоговые средства (Web-интерфейс) для работы пользователя с данными о заказах на предприятии,
   - … для каждого заказа данные могут быть: введены, отредактированы, удалены и / или сохранены в БД системы, с возможностью доступа для них одного или нескольких пользователей ….”

КЛАССИФИКАЦИЯ 3
(ПО ТИПУ)

ФУНКЦИОНАЛЬНЫЕ ТРЕБОВАНИЯ

ФУНКЦИОНАЛЬНЫЕ ТРЕБОВАНИЯ
(FUNCTIONAL REQUIREMENTS)

Функциональные требования — описывают функции и возможности программной системы, то есть то, что она должна делать. Это может быть выполнение определенных операций, обработка данных, отображение информации, и т.д. Функциональные требования являются основными и наиболее важными требованиями для разработки программного обеспечения.

НЕФУНКЦИОНАЛЬНЫЕ ТРЕБОВАНИЯ

- *Атрибуты качества*
- *Бизнес-правила*
- *Ограничения*
- *Внешние интерфейсы*

Далеко не заплывать!

Нефункциональные требования описывают **ограничения**, накладываемые на работу системы, например, количество одновременно работающих пользователей, и стандарты, которым должна соответствовать система.

Требования, отличные от функциональных, могут описывать не что система делает, а **как хорошо** она это делает.

Они могут описывать важные характеристики или свойства системы. К ним относятся доступность, легкость и простота использования, производительность и другие характеристики системы. Некоторые считают нефункциональные требования тем же, что атрибуты качества, но это слишком узкое понимание. Например, ограничения проекта или реализации также являются нефункциональными требованиями, как требования внешних интерфейсов.

Другие нефункциональные требования описывают среду, в которой работает система, например платформу, переносимость, совместимость и ограничения. Многие продукты также должны подчиняться определенным правилам, требованиям регулирующих органов или требовать сертификации. Такими могут быть требования по локализации для продуктов, в которых должны учитываться региональные стандарты, языки, законы, валюты, терминология, орфография и другие характеристики пользователей. Хотя такие требования определяются с использованием нефункциональной терминологии, бизнес-аналитик на их основе может определить много функций, чтобы система обладала всеми необходимыми свойствами и вела себя соответствующим образом в разных ситуациях.

Классификация характеристик качества

Атрибуты качества ПО
(по стандарту ISO 9126:2001)

- Функциональность («что делает ПО»)
  - Пригодность к определенной работе(suitability)
  - Точность, правильность (accuracy)
  - Способность к взаимодействию (interoperability)
  - Соответствие стандартам и правилам (compliance)
  - Защищенность (security)
- Надежность («насколько надежно ПО работает»)
  - Зрелость, завершенность (обратна к частоте отказов) (maturity)
  - Устойчивость к отказам (fault tolerance)
  - Способность к восстановлению работоспособности при отказах (recoverability)
  - Соответствие стандартам надежности (reliability compliance)

- Практичность, удобство использования («насколько удобно пользоваться ПО»)
  - Понятность (understandability)
  - Удобство обучения (learnability)
  - Работоспособность (operability)
  - Привлекательность (attractiveness, добавлен в 2001)
  - Соответствие стандартам практичности (usability compliance)
- Эффективность («насколько эффективно работает ПО»)
  - Временные характеристики (time behaviour)
  - Использование ресурсов (resource utilisation)
  - Соответствие стандартам эффективности (efficiency compliance)

- Сопровождаемость («насколько удобно вносить изменения и поправки в ПО»)
  - Анализируемость (analyzability)
  - Изменяемость, удобство внесения изменений (changeability)
  - Риск возникновения неожиданных эффектов при внесении изменений (stability)
  - Контролируемость, удобство проверки (testability)
  - Соответствие стандартам сопровождаемости (maintainability compliance)
- Переносимость, мобильность («насколько удобно переносить ПО в другое окружение»)
  - Адаптируемость (adaptability)
  - Устанавливаемость, удобство установки (installability)
  - Способность к сосуществованию с другим ПО (coexistence)
  - Удобство замены другого ПО данным (replaceability)
  - Соответствие стандартам переносимости (portability compliance)

Категории нефункциональных требований

- **Основные:**
  - Удобство использования (usability)
  - Надежность и устойчивость к сбоям (reliability)
  - Производительность (performance)
  - Удобство эксплуатации и технического обслуживания (maintainability)
- **Дополнительные:**
  - Ограничение на дизайн
  - Требования реализации
  - Требования интерфейса
  - Требования аппаратного обеспечения
  - Требования документации
  - Требования лицензий и юридических норм

Пример

Атрибутов качества много разных, но есть два базовых атрибута — это **надежность и производительность**.

Что такое надежность? Это в общем-то вероятность работы вашей системы в случае различных нештатных ситуаций. Выключили свет, нет связи, пожар, землетрясение и т.д. Проектирование высоконадежных систем, это в первую очередь все возможное резервирование: основная база данных, реплика, избыточность в рамках одного ЦОД, автоматическая система переключения основной базы данных на резервную, георасределение и т.д.

Производительность - это, то, какую нагрузку может выдержать ваш сервис. Какое количество запросов в секунду может обработать с надлежащей скоростью. И вот тут возникают вопросы. Дело в том, что при некоторых сценариях, производительность не удовлетворяет потребителей. А потребители кто?

Оказывается, у вновь разрабатываемой системы два абсолютно разных потребителя. Первый потребитель — это человек, пользователь. Это различные “фронты” приложения. Нагрузка 1-2 запроса в секунду максимум. А второй потребитель — это внутренние информационные системы. И вот, как раз во втором случае, производительность системы оказывается недостаточной. В данном сценарии нагрузка может достигать 100 и более запросов в секунду. Скорость ответа варьируется от 0,8 до 2 секунд. И что тут такого? Ну 2 секунды, не 20-ть. Что тут может не удовлетворять потребителя?

Требование «Надежность»

Подкатегория	Пример
Работоспособность	Система должна быть доступна 99% времени
Прочность	Для каждого неверного ввода пользователем система должна отображать соответствующее сообщение об ошибке.
Точность	Денежные расчеты должны выполняться и храниться с точностью до двух десятых
Восстанавливаемость	После восстановления системы из состояния неработоспособности обработка данных должна производиться в том же режиме, что и до сбоя.
Корректность	После выпуска релиза система может иметь не более чем 20 незначительных ошибок.

Требование «Производительность»

Подкатегория	Пример
Скорость обработки данных	Система должна обрабатывать 1000 процедур бронирования билетов в минуту.
Время ответа	Среднее время отображения списка полетов должно быть не более 10 секунд.
Время восстановления	Среднее время восстановления должно быть менее часа.
Время загрузки /выхода	Система должна быть работоспособной в течение одной минуты после загрузки.
Емкость	Система должна обслуживать 5000 пользователей одновременно.
Использование ресурсов	Система должна хранить в БД не более 1 млн. транзакций. При превышении лимита старые транзакции архивируются.

Требование «Удобство использования»

Подкатегория	Пример
Доступность	Функциональность бронирования билета на самолет должна быть доступна с домашней страницы
Эстетичность	Поля ввода на одной странице должны быть выровнены по вертикали
Соответствие интерфейсу пользователя	Пользовательский интерфейс должен соответствовать стандарту IBM
Эргономичность	При открытии диалогового окна курсор должен быть на первом поле ввода
Легкость в использовании	Среднее время процедуры бронирования должно быть не более 2 минут

Требование «Удобство эксплуатации и технического обслуживания»

- Тестируемость
- Приспособляемость
- Совместимость
- Способность к обновлению
- Расширяемость
- Переносимость
- Возможность многократного применения
- Взаимодействие с другими ИС
- Способность к аудиту
- Способность к локализации

КАТЕГОРИИ *runtime u designtime*

Принято разделять нефункциональные требования на две большие категории, соответствующие структурным и поведенческим аспектам приложения.

1. Первая категория (***runtime***) содержит атрибуты, имеющие значение при исполнении приложения, то есть в режиме его работы.
2. Вторая категория (***designtime***) определяет атрибуты, относящиеся к аспектам проектирования приложения.

Далее мы приведем таблицу основных критериев качества, которые вам необходимо учитывать при описании или проектировании архитектуры.

Используйте этот перечень для подробного описания архитектуры разрабатываемого приложения.

АТРИБУТЫ КАЧЕСТВА В RUNTIME

Название атрибута	Описание
Availability	**Доступность** - требования ко времени непрерывной работы приложения, например, 24x7, минимальное время простоя и т.п.
Reliability	**Надежность** - поведение приложения при наступлении нештатных ситуаций, например, автоматический перезапуск, восстановление работы, дублирование важных данных, резервирование логики
Durability	**Требования к долговременному хранению результатов работы приложения**, например, использование базы данных, требования ко времени продолжительности хранения данных

Название атрибута	Описание
Usability	**Требования к удобству использования** приложения с точки зрения использования, поддержки
Scalability	**Масштабируемость** - требования к горизонтальному или вертикальному масштабированию приложения
Security	Требования к безопасности работы или использования приложения, связанные с разграничением доступа, работой с приватными данным, снижения подверженности рискам от внешних атак
Configurability	Требования к конфигурируемости работы приложения, взаимодействия и расположения компонентов

Название атрибута	Описание
Performance	Требования к производительности решения, количество одновременно работающих пользователей, обслуживаемых транзакций, времени реакции, продолжительности вычислений, скорости и пропускной способности каналов связи
Restrictions	Описание ограничений, накладываемых на объем доступной памяти, процессорного времени, дискового пространства, пропускную способность сети, при которых приложение должно эффективно выполнять возложенные на него задачи

АТРИБУТЫ КАЧЕСТВА В DESIGN TIME

Название атрибута	Описание
Reusability	Требования к повторному использованию реализации или компонентов приложения, а также реализация приложения с возможностью повторного его использования для различных задач
Extensibility	Требования к расширяемости приложения в связи с появлением новых функциональных требований
Portability	Требования к переносимости приложения на различные платформы
Interoperability	Требования к взаимодействию между компонентами решения, между внешними компонентами, использование стандартных протоколов и технологий взаимодействия

Название атрибута	Описание
Supportability	Требования к различным аспектам поддержки приложения, таким как дешевизна и скорость разработки, прозрачность поведения приложения, простота анализа ошибок и проблем в работе
Modularity	Требования к разделению приложения на модули
Testability	Требования к возможности автоматического и ручного тестирования приложения, наличие необходимого инструментария
Localizability	Требования к возможности и простоте локализации приложения, перечень языков, на которые предполагается локализация приложения
Compatibility	Особые требования к совместимости между версиями приложений, между различными приложениями и внешними подсистемами

Спектр нефункциональных требований

Некоторые люди считают нефункциональные требования тем же, что атрибуты качества, но это слишком узкое понимание. Например, ограничения проекта или реализации также являются нефункциональными требованиями, как требования внешних интерфейсов.

Другие нефункциональные требования описывают среду, в которой работает система, например платформу, переносимость, совместимость и ограничения. Многие продукты также должны подчиняться определенным правилам, требованиям регулирующих органов или требовать сертификации. Такими могут быть требования по локализации для продуктов, в которых должны учитываться региональные стандарты, языки, законы, валюты, терминология, орфография и другие характеристики пользователей.

Хотя такие требования определяются с использованием нефункциональной терминологии, бизнес-аналитик на их основе может определить много функций, чтобы система обладала всеми необходимыми свойствами и вела себя соответствующим образом в разных ситуациях.

Особенности нефункциональных требований

- Заказчики часто забывают про эти требования и не предоставляют их, пока не будут заданы соответствующие вопросы.
- Заказчики обычно не в курсе стоимости улучшения определенных возможностей.
- У нетехнических пользователей часто возникают проблемы с пониманием смысла некоторых технических требований.
- Некоторые требования являются сложными в измерении, например: «Система должна быть простой для обучения».

КЛАССИФИКАЦИЯ 4

В сложных проектах принято разделять спецификацию системных требований (system requirements) и спецификацию программных требований (software requirements). При таком подходе программные требования порождаются системными требованиями и детализируют требования к компонентам и подсистемам программного обеспечения.

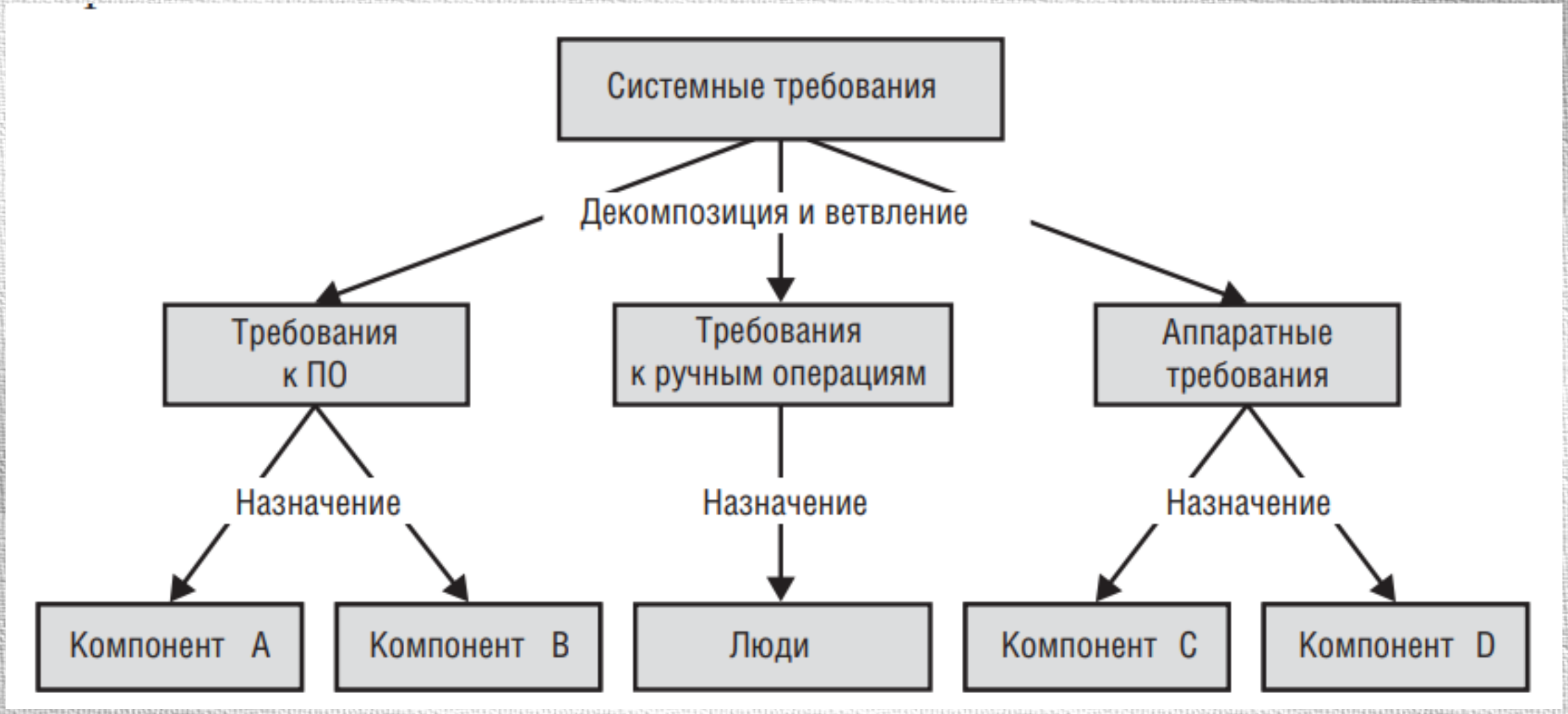
СИСТЕМНЫЕ ТРЕБОВАНИЯ
(SYSTEM REQUIREMENTS)

Термин системные требования описывает требования к продукту, который содержит многие компоненты или подсистемы (ISO/IEC/IEEE 2011). В этом смысле «система» это не просто любая информационная система. Говоря о системе, мы подразумеваем программное обеспечение или подсистемы ПО и оборудования. Люди и процессы тоже часть системы, поэтому определенные системные функции могут распространяться и на людей.

- Хороший пример «системы» — рабочее место кассира в супермаркете. В нем есть сканер штрих-кода, интегрированный с весами, а также ручной сканер штрих-кода. У кассира есть клавиатура, монитор и выдвижной ящик-касса. Есть также устройство чтения кредитных карточек и клавиатура для ввода ПИН-кода и чтения карт постоянного покупателя. На рабочем месте может быть до трех принтеров: для печати чека, квитанции кредитной карты и купонов на скидку. Все эти устройства взаимодействуют под управлением программного обеспечения.

На основе системных требований бизнес-аналитик формулирует конкретную функциональность, которую должны поддерживать тот или иной компонент или подсистема, а также интерфейсы взаимодействия между ними.

Системные требования



Как трактовать СТ?

Существуют различные трактовки такого понятия как «системные требования» (*system requirements*).

1. Классифицируются как составная часть группы функциональных требований (не путайте с как таковыми “функциональными требованиями”). Описывают высокоуровневые требования к программному обеспечению, содержащему несколько или много взаимосвязанных подсистем и приложений. При этом, система может быть как целиком программной, так и состоять из программной и аппаратной частей. В общем случае, частью системы может быть персонал, выполняющий определенные функции *системы*, например, авторизация выполнения определенных операций с использованием программно-аппаратных подсистем.

В данное подмножество целесообразно относить наиболее важные, существенные требования, которые относятся в целом к системе и не содержат избыточной детализации.

2. В практике компьютерной инженерии бытует другой, более узкий контекст использования данного понятия: под системными требованиями в узком смысле понимается требования, выдвигаемые прикладной программной системой (в частности – информационной) к среде своего функционирования (системной, аппаратной). Пример таких требований – тактовая частота процессора, объём памяти, требования к выбору операционной системы.

Стандарт IEEE 1233 является одним из признанных руководств по разработке системных требований.

ВОЗМОЖНЫЕ СИСТЕМНЫЕ ТРЕБОВАНИЯ

- требования к структуре системы;
- требования к режимам функционирования системы;
- требования к персоналу;
- требования к надежности;
- требования к безопасности;
- требования к эргономике и технической эстетике;
- требования к транспортабельности (для подвижных АС);
- требования к защите информации от несанкционированного доступа;
- ...

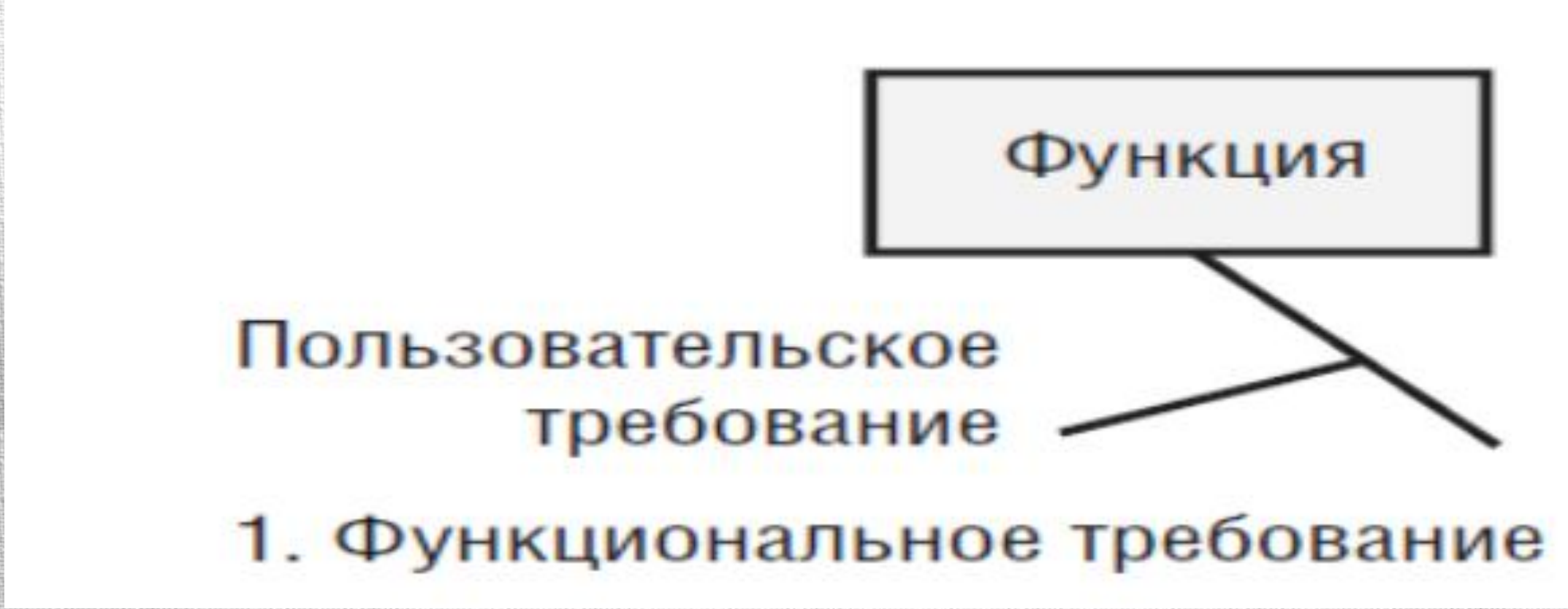
ХАРАКТЕРИСТИКА (FEATURE)

Характеристика (feature) — это набор логически связанных функциональных требований, которые представляют ценность для пользователя и удовлетворяют бизнес-цели. Желательные характеристики продукта, которые перечисляет клиент, не эквивалентны тем, что входят в список необходимых для решения задач пользователей.

В качестве примеров характеристик продуктов можно привести избранные страницы или закладки веб-браузера, средства проверки орфографии, запись макрокоманды, автоматическое обновление определений вирусов в антивирусной программе.

Характеристики могут охватывать множество пользовательских требований, и для каждого варианта необходимо, чтобы множество функциональных требований было реализовано для выполнения пользователем его задач.

Взаимоотношения между функциями, пользовательским и функциональными требованиями



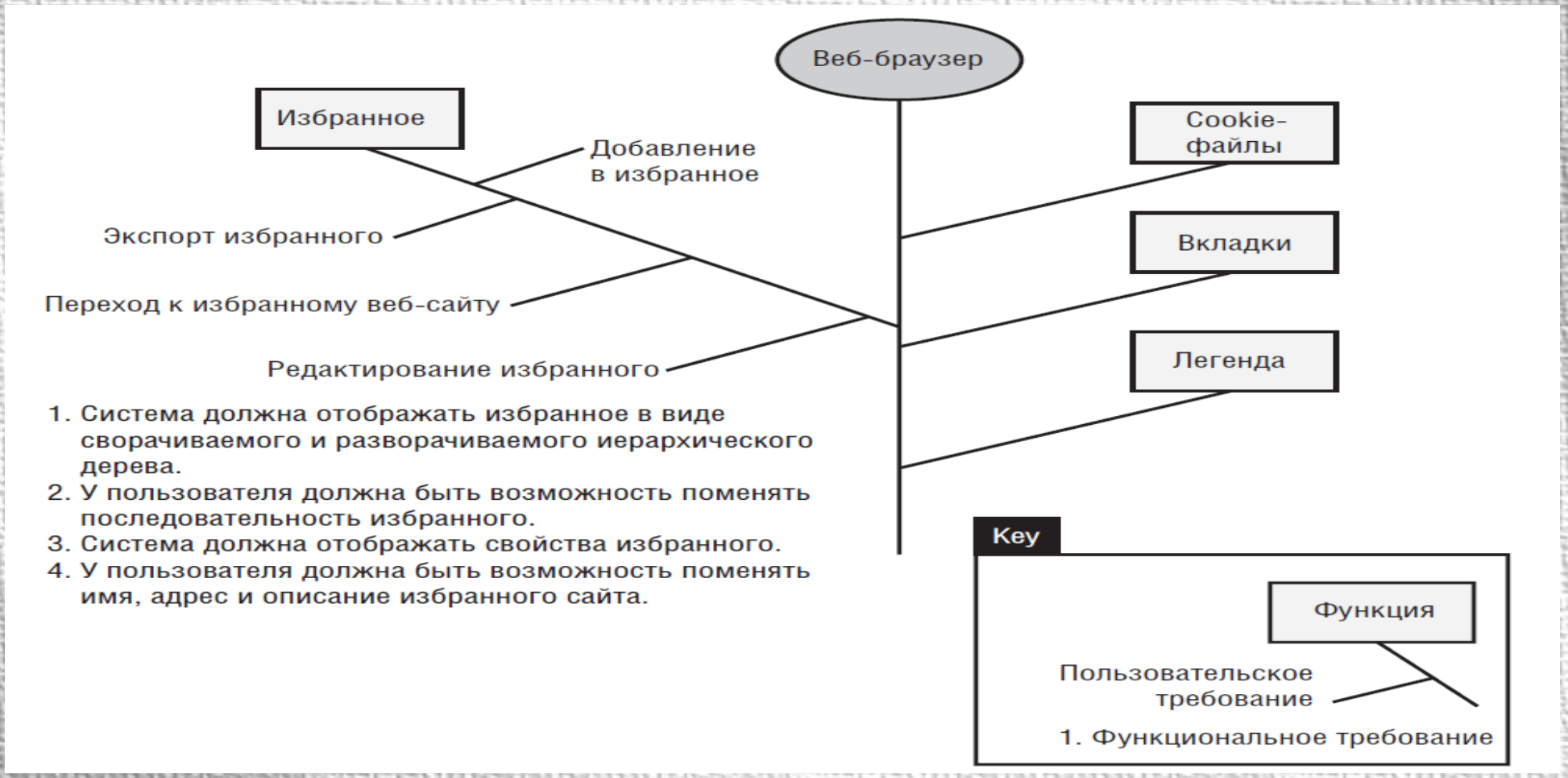
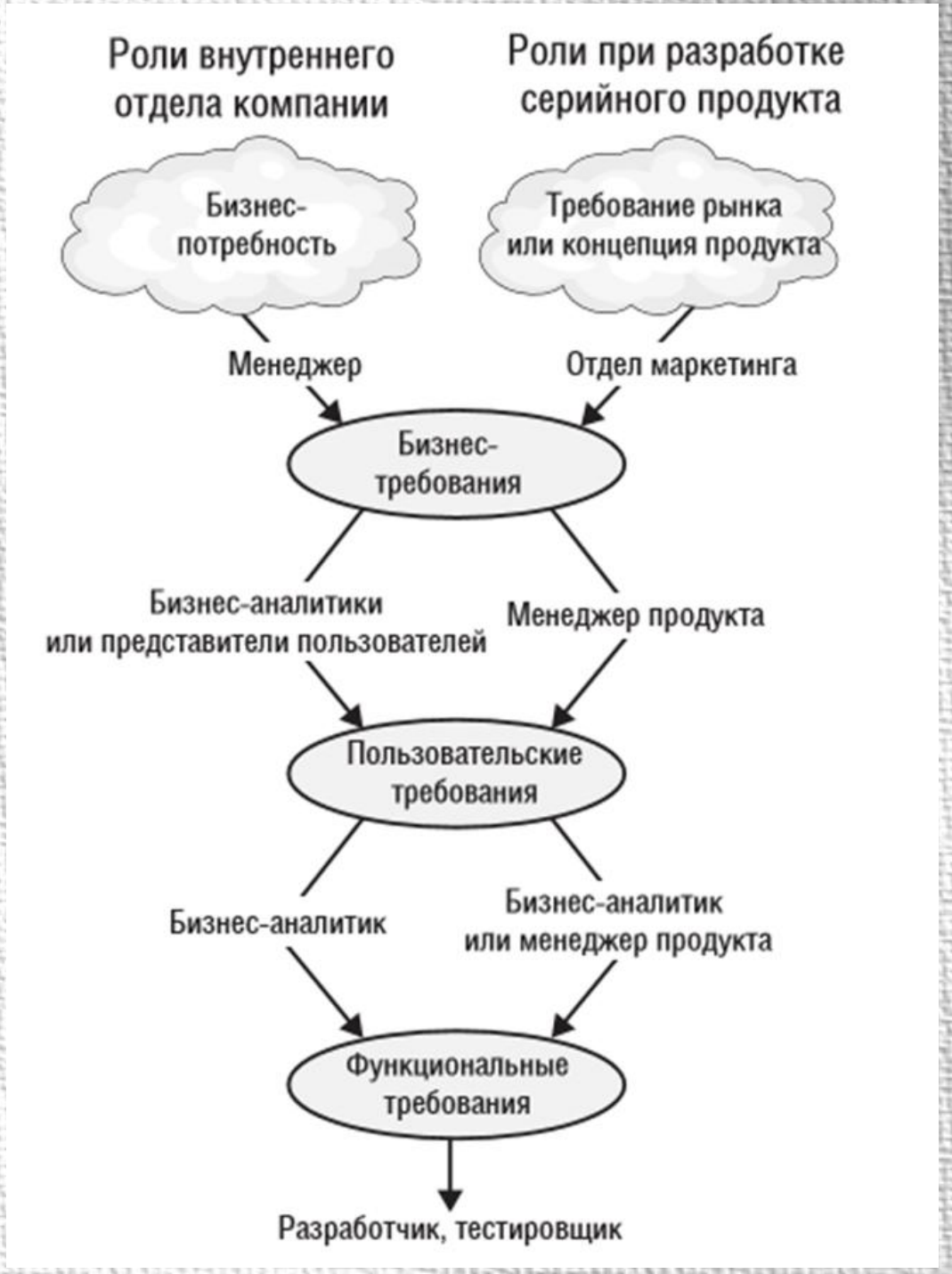


Рисунок иллюстрирует дерево функций (feature tree) — модель анализа, которая показывает, как функцию можно разложить на иерархию более мелких функций, которые связаны с конкретными пользовательскими требованиями и ведут к определению наборов функциональных требований

Пример участия различных заинтересованных лиц в разработке требований



КЛАССИФИКАЦИЯ К.ВИГЕРСА

Классификация К.Вигерса

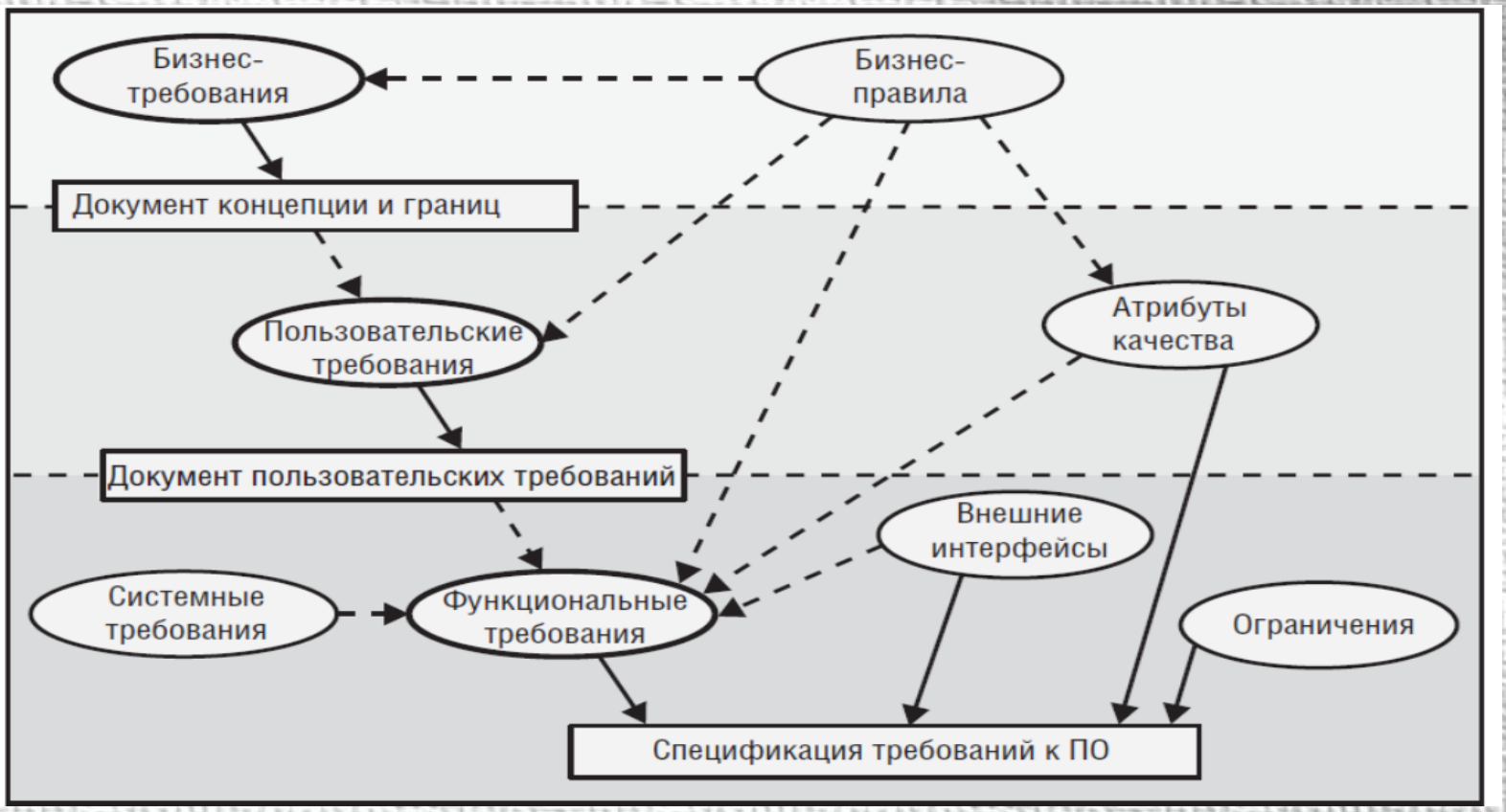
Группа функциональных требований

Группа нефункциональных требований

Бизнес требования

Пользовательские требования

Функциональные требования

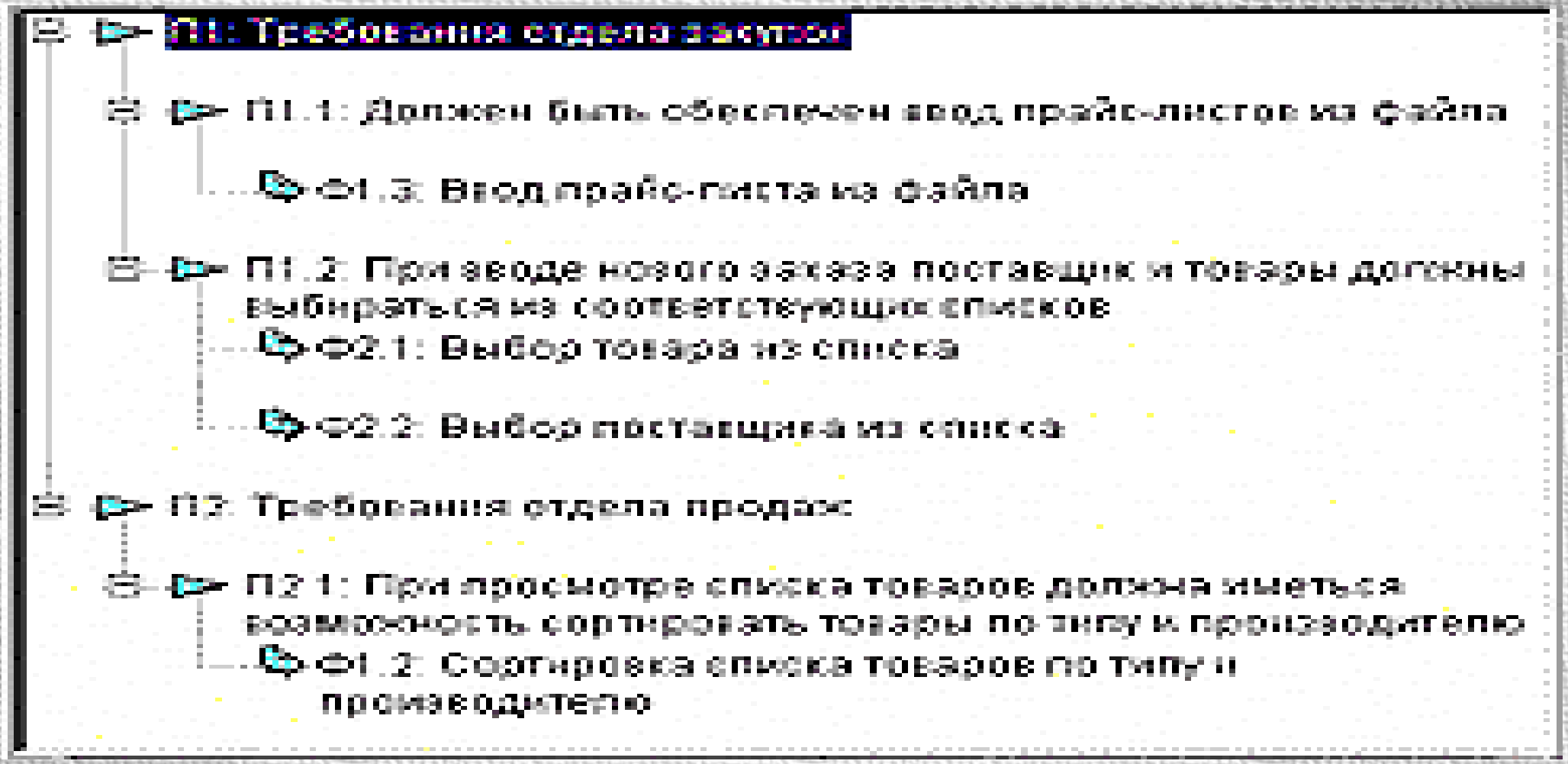


Сплошные линии означают «содержатся в», а пунктирные — «являются отправной точкой» или «влияют на»

Комментарий

- Овалы обозначают типы информации требований, а прямоугольники — документы, в которых хранится эта информация. Сплошные линии указывают, что в указанном документе хранится информация определенного типа.
- Бизнес-правила и системные требования хранятся отдельно от требований к ПО, обычно соответственно в каталоге бизнес-правил или в спецификации системных требований.) Пунктирная линия указывает, что информация одного типа является источником или влияет на информацию другого типа или на требование.
- На этой схеме не показаны требования к данным. Функции манипулируют данными, поэтому требования к данным могут присутствовать на всех трех уровнях.

Древовидное представление зависимости пользовательских и функциональных требований



Нефункциональные требования
(по К.Вигерсу)

Если функциональные требования задают “**что**” система должна делать, то нефункциональные – с соблюдением “**каких условий**” .

Нефункциональные требования регламентируют внутренние и внешние условия или атрибуты функционирования системы.

К ***нефункциональным требованиям***, относят:

1. **Бизнес правила**. Набор принятых приемов организации бизнес процессов, которые должны быть учтены при разработке ПО
2. **Атрибуты (характеристики) качества**. Чаще всего это характеристики надежности и отказоустойчивости системы.
3. **Внешние интерфейсы.** Набор программных интерфейсов, которые необходимо реализовать для интеграции в существующую в организации среду программных средств.
4. **Ограничения.** Законодательные и другие ограничения накладываемые на создаваемое ПО.

❶ Бизнес-правила

БП – это положение, определяющее или ограничивающее какие-либо стороны бизнеса. Его назначение – защитить структуру бизнеса, контролировать или влиять на его операции. Рассмотрим различные *типы правил*, с которыми можно столкнуться:

1. **Факты - э**то всего лишь верные утверждения о бизнесе. Факты также называют *инвариантами* – неизменными истинами о сущностях и их атрибутах.
   - *Со стоимости доставки налог с продаж не берется.*
   - *Стоимость билетов не возвращается, если покупатель изменяет маршрут.*
2. **Ограничения о**пределяют, какие операции может выполнять система и её пользователи. Часто применяемые слова: *может, не может, должен, только*.
   - *При отгрузке заказа менеджер должен запросить у бухгалтера товарно-транспортную накладную и счет-фактуру.*

3. **Активаторы операций –** правило, при определенных условиях приводящих к какому-либо действию. Выражение вида *«Если <некоторое условие верно или наступило определенное событие>, то <что-то произойдет>»,* — это ключ, который описывает активатор операции.
   - *Если заказанный товар отсутствует на складе, то заказ передается в производственный отдел.*
4. **Вычисления.**
   - *Цена единицы товара снижается на 10% при заказе от 6 до 10 единиц, на 20% — при заказе от 11 до 20 единиц и на 30% — при заказе свыше 20 единиц;*
5. **Выводы.** Вывод создает новый факт на основе других фактов или вычислений. Выводы зачастую записывают в формате «если — то», применяемом также при записи бизнес-правил, активирующих операции; тем не менее, раздел «то» вывода заключает в себе факт или предположение, а не действие.
   - *Если платеж не поступил в течение 30 календарных дней с момента отправки счета, счет считается просроченным;*
   - *Если поставщик не может поставить заказанный товар в течение пяти дней с момента получения заказа, заказ считается невыполненным;*

❷ Атрибуты качества

Основные атрибуты качества:

- **Надежность** (reliability) - это свойство ПС функционировать с уровнем ошибок (сбоев), не превыщающем заданный (напр., д.б. обработаны не менее чем 99% всех запросов к Web-серверу);
- **Производительность** (*performance*) – это свойство ПС выполнять свои основные функции с определенным уровнем быстродействия, напр.: обработка запросов к БД в диапазоне от 200 до 400 мс;
- **Переносимость** (portability) - это характеристика ПС, которая обратно пропорционально величине затрат, необходимых на перенос системы в другую операционную среду (напр., из среды MS Windows в среду Unix), значение этой характеристики находиться в интервале [0,1];

- **Удобство использования** (*usability*) - это характеристика ПС, которая обратно пропорционально величине затрат, необходимых для первоначального обучения ее пользователей, и для дальнейшей ее эксплуатации, значение этой характеристики находиться в интервале [0,1].
- **Защищенность** (*security*) - это характеристика ПС, которая обеспечивает заданный уровень безопасности обработки данных в системе (напр., вероятность несанкционированного проникновения в систему д.б. не более 1%);
- **Сопровождаемость** (*maintainability*) - это характеристика ПС, которая обратно пропорционально величине затрат, необходимых для внесение в систему требуемых зменений в ходе ее эксплуатации, значение этой характеристики находиться в интервале [0,1];

❸ Внешние интерфейсы

Среди внешних интерфейсов в большинстве современных АИС выделяют

1. интерфейс пользователя (User Interface, UI).
2. интерфейсы с внешними устройствами (аппаратные интерфейсы)
3. программные интерфейсы «Система должна взаимодействовать с существующей системой «Наша библиотека» для доступа к базе данных учебно-методической литературы»
4. интерфейсы передачи информации (коммуникационные интерфейсы)

Внешние интерфейсы часто подменяются “пользовательским интерфейсом”. На самом деле вопросы организации пользовательского интерфейса безусловно важны в данной категории требований, однако, конкретизация аспектов взаимодействия с другими системами, операционной средой (например, запись в журнал событий операционной системы), возможностями мониторинга при эксплуатации – все это не столько функциональные требования (к которым ошибочно приписывают иногда такие характеристики), сколько вопросы интерфейсов, так как функциональные требования связаны непосредственно с *функциональностью* системы, направленной на решение *бизнес-потребностей*.

3.1. Интерфейсы пользователя

Основные характеристики UI:

- ссылки на стандарты графического интерфейса пользователей или стилевые рекомендации для семейства продукта, которые необходимо соблюдать;
- стандарты шрифтов, значков, названий кнопок, изображений, *цветовых схем*, последовательностей полей вкладок, часто используемых элементов управления и т.п.;
- конфигурация экрана или ограничения разрешения;
- стандартные кнопки, функции или ссылки перемещения, одинаковые для всех экранов, например кнопка справки;
- быстрые клавиши;
- стандарты отображения сообщений;
- стандарты конфигурации для упрощения локализации ПО;
- специальные возможности для пользователей с проблемами со зрением.

3.2. Интерфейсы оборудования

Опишите характеристики каждого интерфейса между компонентами ПО и оборудования системы. В описание могут входить типы поддерживаемых устройств, взаимодействия данных и элементов управлений между ПО и оборудованием, а также протоколы взаимодействия, которые будут использоваться.

3.3. Интерфейсы ПО

Опишите соединения продукта и других компонентов ПО (идентифицированные по имени и версии), в том числе базы данных, операционные системы, средства, библиотеки и интегрированные коммерческие компоненты. Укажите назначение элементов сообщений, данных и элементов управления, обмен которыми происходит между компонентами ПО. Опишите службы, необходимые внешним компонентам ПО, и природу взаимодействия между компонентами. Определите данные, к которым будут иметь доступ компоненты ПО.

3.4. Интерфейсы передачи информации

Укажите требования для любых функций взаимодействия, которые будут использоваться продуктом, включая электронную почту, Web-браузер, протоколы сетевого соединения и электронные формы. Определите соответствующие форматы сообщений. Опишите особенности безопасности взаимодействия или шифрования, частоты передачи данных и механизмов синхронизации.

❹ Ограничения

Ограничения - формулировки условий, модифицирующих требования или наборы требований, сужая выбор возможных решений по их реализации. выбор платформы реализации и/или развертывания (протоколы, серверы приложений, баз данных, ...), которые, в свою очередь, могут относиться, например, к внешним интерфейсам.

После этого аналитики обычно работают с представителями пользователей для определения **пользовательских требований**. В компаниях, разрабатывающих коммерческие продукты, часто назначают менеджера продукта, который определяет, какие функции должны включаться в новый продукт. Каждое пользовательское требование должно быть сопоставлено бизнес-требованию. На основе пользовательских требований аналитик или менеджер продукта определяет **функции**, которые дадут возможность пользователям выполнять их задачи. Разработчикам необходимы функциональные и нефункциональные требования, чтобы создавать решения с желаемой функциональностью, не выходя за рамки налагаемых ограничений. Тестировщики определяют, как проверять правильность реализации требований.

ТРИ ГЛАВНЫХ ДОКУМЕНТА ТРЕБОВАНИЙ

1. **документ концепции и границ**
2. **документ пользовательских требований**
3. **спецификация программных требований**

Не всегда обязательно создавать эти три отдельных документа в каждом проекте. Часто имеет смысл объединять часть информации, особенно в небольших проектах. Однако надо понимать, что эти три документа содержат разную информацию, разрабатываются на разных этапах проекта, возможно даже разными людьми с разными целями и разной целевой аудиторией.

Пример описания требований

Бизнес требования

Улучшить пользовательский опыт за счет уведомления пользователя о процессах

Пользовательские требования

Необходимо уведомлять пользователя о следующих событиях:
оформление заказа
сборка заказа
передача заказа в доставку
доставка заказа

Функциональные требования

1. Реализовать сервис уведомления пользователей.
2. Сервис уведомлений должен обрабатывать события, поступающие от сервисов A, B, C и отправлять события в сервис D.

Описание API

Метод POST /inform

Назначение Уведомление пользователя

Ограничения
1. В случае внутренней ошибки возвращать ошибку 500 Internal Server Error
2. В случае ошибки валидации входящих параметров возвращать ошибку 400 Bad Request

Логика
1. Сервис A, B, C присылают уведомления для пользователя на эндпоинт POST /inform.
2. Параметры входящего запроса валидируются и осуществляется маппинг в параметры исходящего запроса.
3. Осуществляется вызов эндпоинта POST /notify сервиса D.

Маппинг данных

Сервис A, B, C	Сервис D	Примечание
client_id (integer)	consumer_id (string)	
order_id (integer)	order_id (string)	
stage(string)	message (string)	
status (string)	status (string)	
	additionalText (string)	Если stage = "order_confirmed" то заполняется: "Благодарим за заказ"

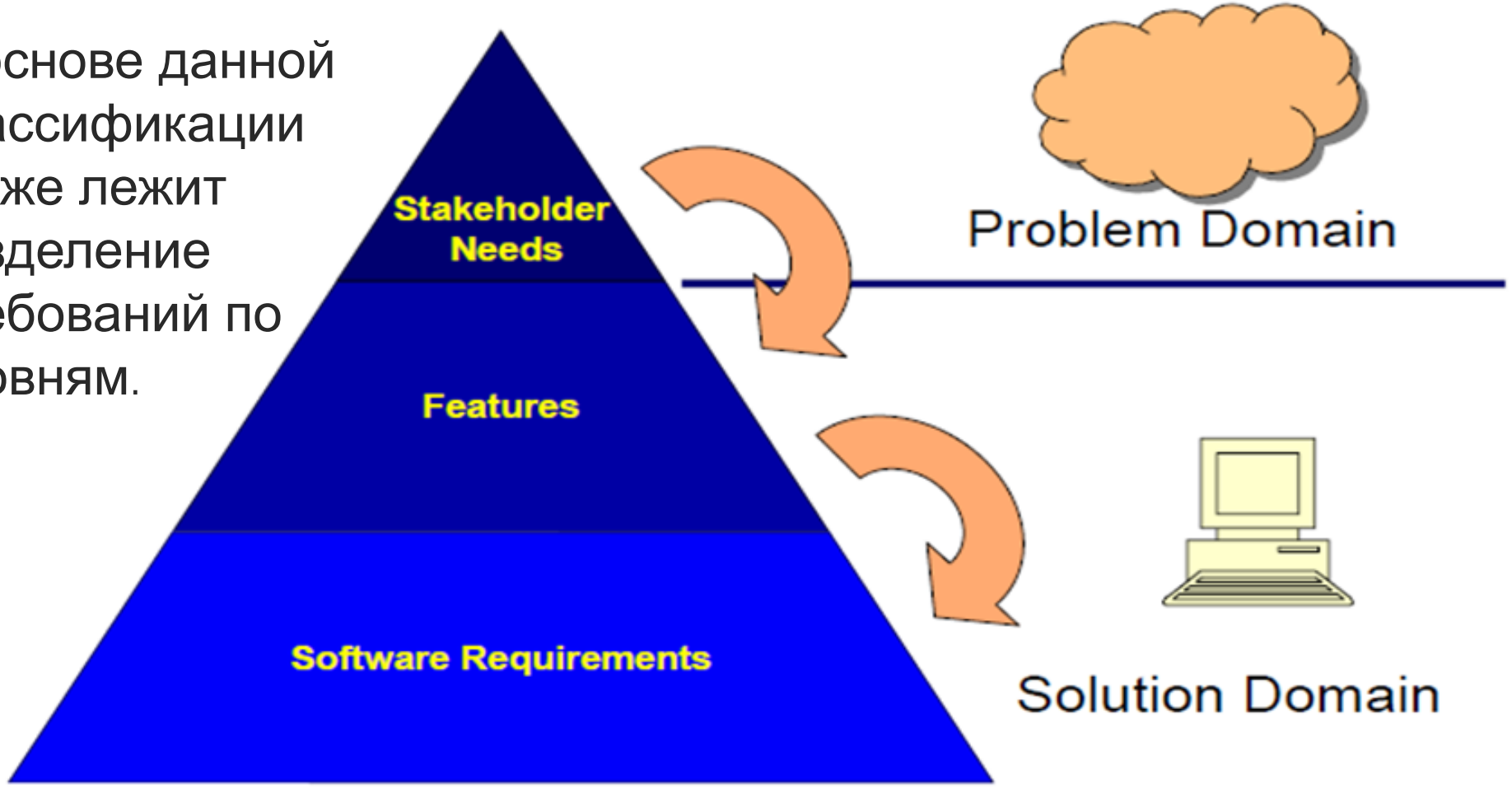
Нефункциональные требования

1. Время обработки сообщения сервисом не превышает 1 сек для 99.99% запросов.
2. Средняя нагрузка на сервис - 100 RPS
3. Максимальная нагрузка на сервис - 200 RPS
4. Категория конфиденциальности обрабатываемых данных: общедоступные данные
5. Обеспечивается подсчет метрик по количеству успешно и неуспешно обработанных запросов на POST /inform.
6. Уровень логирования сервиса уведомлений - "ERROR"

КЛАССИФИКАЦИЯ ЛЕФФИНГВЕЛЛА

СХЕМА ЛЕФФИНГВЕЛЛА

В основе данной классификации также лежит разделение требований по уровням.



Уровни требований

1. **Потребность (need)** - отражение проблемы бизнеса. персоналии или процесса, которая должно быть соотнесена с использованием или приобретением новой системы.
2. **Характеристика, функция, фича (feature)** - множество логически связанных функциональных требований, которые обеспечивают возможности пользователя и удовлетворяют бизнес целям.
3. **Требования к программному обеспечению (software requirements).**
4. **Вариант использования (UseCase)**

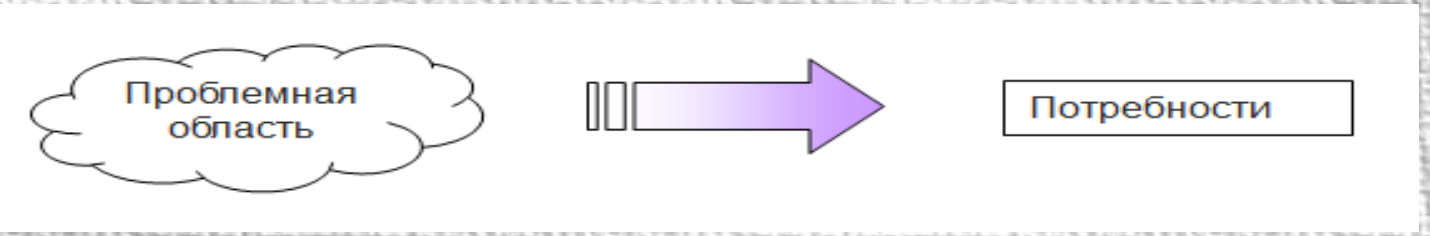
Процесс формулирования требований

Начинается этот процесс с анализа проблемной области. **Проблемная область** — это вотчина реальных пользователей и других заинтересованных лиц, чьи потребности мы должны удовлетворить, чтобы построить нужную систему. У пользователей есть технические или бизнес-задачи, для решения которых им нужно ПО.

Задача разработчиков состоит в том, чтобы понять их потребности в их предметной области на их языке и построить систему, удовлетворяющую эти потребности.

1. ПОТРЕБНОСТИ

- С точки зрения разработчиков проблемная область достаточно туманна, поэтому ее принято изображать в виде облака. Из этого облака проблем в результате анализа выявляются **потребности** в программном продукте.



- Анализ проблемы — это процесс осознания реальных проблем и потребностей пользователя и предложения решений для удовлетворения этих потребностей.
- Цель анализа проблемы состоит в том, чтобы добиться лучшего понимания решаемой проблемы до начала разработки.

Согласно классификации требований по BABOK®Guide именно *бизнес-требования* являются основой всех остальных видов требований: пользовательских (требования стейхолдеров) и требований к решению (функциональных и нефункциональных).

Можно трассировать бизнес-потребность к бизнес-требованиям и бизнес-правилам

Бизнес-потребность	Бизнес-требование	Бизнес-правила
Проблема: BN-1. Потеря 20% клиентских заявок от 50 входящих обращений в неделю **Следствие**: 75 тысяч рублей недополученного дохода при 50%-ной конверсии заявки в продажу стоимостью 15 тысяч рублей	BReq-1. Сократить долю потерянных (полученных, но необработанных в плановый срок) заявок с 20% до 0% к 01.10.2023	BR-1. Плановый срок обработки клиентской заявки не более 48 часов BR-2. Заявки от юридических лиц на сумму от 100 тысяч рублей получают наивысший приоритет BR-3. Заявки с наивысшим приоритетом должны быть обработаны в течение 24 часов с момента их поступления

2. ФУНКЦИИ

Решения о том, каким образом проектируемая система будет решать проблемы заказчика, обычно сводят в список такого вида:

- *У автомобиля будут автоматические стеклоподъемники*
- *Диаграммы динамики обнаружения неисправностей будут снабжены визуальными средствами оценки прогресса*
- *Необходимо предусмотреть возможность ввода заказов через Интернет*
- *Будет поддерживаться автоматический цикл двойной сварки*

Эти описания называются ***функциями*** (features) или ***характеристиками*** создаваемой системы.

Функция — это предоставляемая системой услуга для удовлетворения одной или нескольких потребностей заинтересованных лиц.

В чем разница между требованиями и функциями?

Пример

Представим, что вы проектируете онлайн-магазин книг. Для данного проекта вот пример **функции**:

заказ за один шаг

Вот несколько **требований**, связанных с этой функцией:

- пользователь должен иметь возможность включить функцию заказа за один шаг в своем аккаунте;
- пользователь должен иметь возможность выключить функцию заказа за один шаг в своем аккаунте;
- пользователь должен иметь возможность заказать книги всего за один шаг;
- …
- пользователь должен иметь возможность отменить заказ в течение 30 минут после осуществления заказа.

Key feature

Key feature переводится на русский язык как «ключевая особенность», «главная особенность», «основная характеристика».

Ключевые возможности

«**Ключевая возможность» (key feature)** - **это набор логичес-ки связанных функциональных и нефункциональных требований**, которые обеспечивают возможности пользователя и удовлетворяют «бизнес требованиям». В области коммерческого программного обеспечения «ключевая возможность» представляет собой узнаваемую всеми заинтересованными лицами группу требований, которые важны при принятии решения о покупке - элемент маркированного списка в описании продукта.

В отличие от «бизнес требований», которые описывают цели организации, «ключевые возможности» на высоком уровне абстракции **описывают функциональность и характеристики качества разрабатываемой информационной системы, т.е. являются высокоуровневыми требованиями к системе**.

Иногда, такие возможности детализируются в смысле функциональных требований в некоторых agile-техниках, например, FDD – Feature-Driven Development (как вы видите вплоть до самого названия целого комплекса практик, метода разработки).

Вигерс, описывает ***feature*** как “***множество логически связанных функциональных требований, которые предоставляют определенные возможности для пользователя и удовлетво-ряют бизнес-целям <организации***”.

С точки зрения маркетинга программного обеспечения, как отмечает Вигерс, ***feature*** это «группа требований, узнаваемая заинтересованными лицами, которые вовлечены в процесс принятия решения по приобретению ПО – это список <***отличительных особенностей или возможностей, характеристик***>, присутствующий в описании продукта».

В то же время, Леффингвелл и Видриг определяют feature как “*сервисы, которые оказывает система для удовлетворения одной или более потребностей заинтересованных лиц (stakeholders needs)*”. Ими же отмечается, что в реальных проектах могут быть не определены stakeholders needs (а их часто выделяют, особенно если у проекта/продукта есть много заинтересованных лиц со своими потребностями, и эти потребности могут носить взаимоисключающий характер), но существовать features могут и самостоятельно (как и stakeholder needs), и конечно же возможно существование и stakeholder needs и features со взаимной трассировкой.

Необходимо также отметить, что Features обладают определенным дуализмом в своей интерпретации, зависимым от контекста конкретного продукта

- с одной стороны это может быть «тот самый список характеристик, указанный на коробке продукта» в случае создания «коробочного ПО»,
- с другой стороны это может список высокоуровневых возможностей системы, например при заказной разработке ПО автоматизации бизнес-процессов конкретной организации.

Features могут быть разного уровня детализации – от выражения высокоуровневых возможностей системы (например, «Расчет заработной платы …»), до достаточно конкретных требований (например, «Автоматическое уведомление Клиента по e-mail о резервировании товара на складе»)

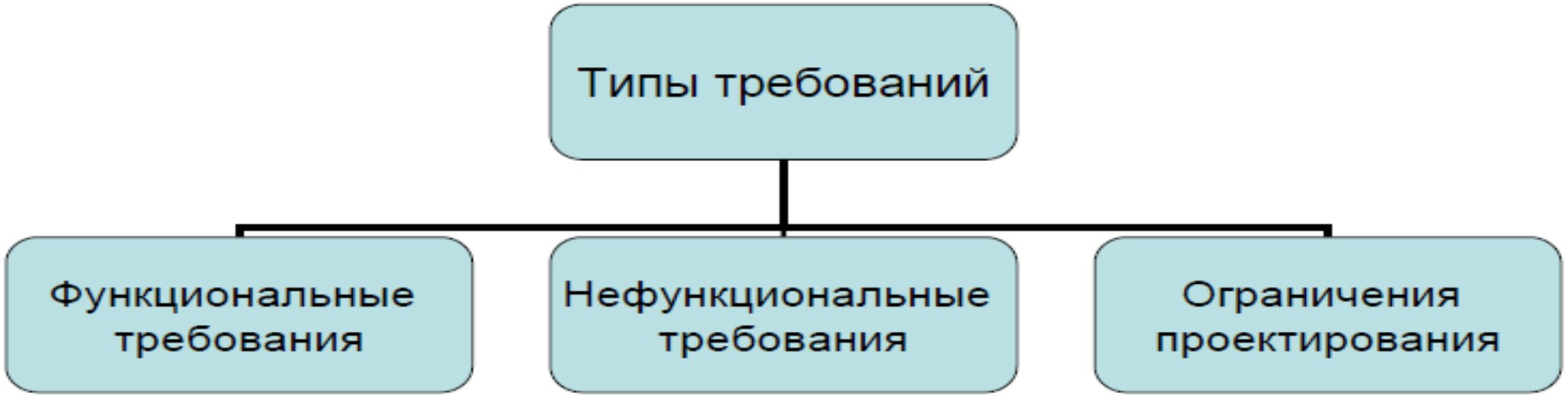
3. ТРЕБОВАНИЯ К ПРОГРАММНОМУ ОБЕСПЕЧЕНИЮ (SOFTWARE REQUIREMENTS)

После того, как определен набор функций и достигнуто соглашение с клиентом, можно переходить к определению более конкретных требований, которым должно удовлетворять решение.
Этими требованиями являются **требования к программному обеспечению** (***software requirements***)

Эти программные требования являются исходными данными для построения самого ПО и для проведения его контроля.

Четкость, степень
конкретизации
Проблемы
Потребности
Функции
Требования к ПО

Детализация Software Requirements (Леффингвелл)



- **Функциональные требования*** выражают поведение системы – входы, выходы и функции, которые система предоставляет пользователю.
- **Нефункциональные требования** (Usability, Reliability, Performance, Supportability)
- **Ограничения проектирования** – ограничения, относящиеся к дизайну или процессу создания системы, которые не влияют на внешнее поведение, но должны быть учтены для соответствия различного вида обязательствам

Все требования обычно делят на три группы:

- **функциональные требования к ПО;**
- **нефункциональные требования к ПО;**
- **ограничения проектирования**.

Функциональные требования описывают, что делает система, это требования к первой составляющей качества — функциональности. Эти требования обычно ориентированы на действия (Когда пользователь нажимает кнопку «Обработать заказ», система сохраняет данные заказа в БД и определяет его статус как «В очереди на обработку»).

Функциональные требования описывают, как система должна вести себя, когда ей предоставляются определенные входные данные или условия. Но одних функциональных требований недостаточно для полного описания требований к системе — необходимо также учитывать требования к другим составляющим качества, задаваемые нефункциональными требованиями.

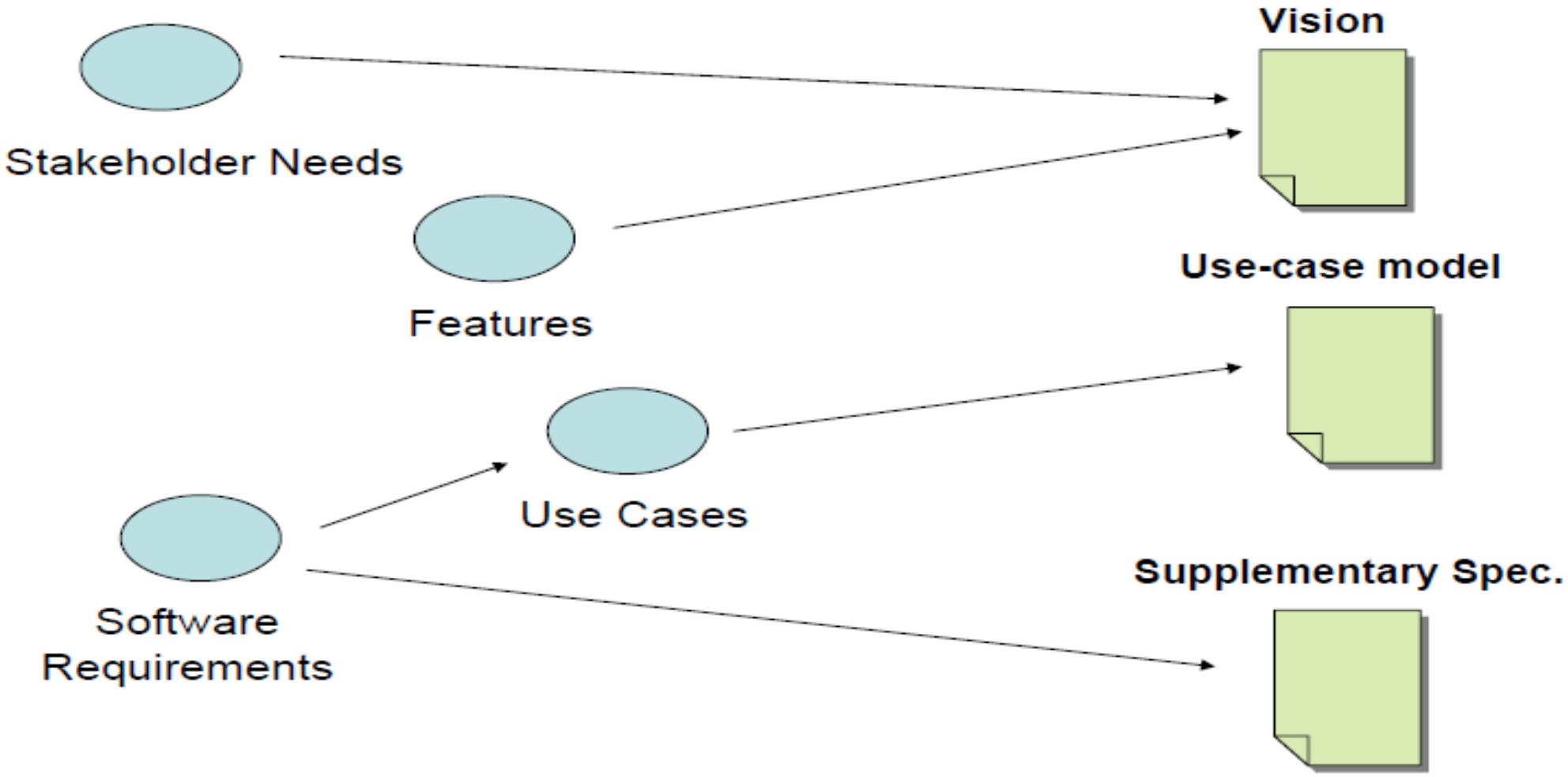
Ограничения проектирования, как правило, касаются вариантов проектирования системы или процессов, используемых при ее построении. Ограничения проектирования налагаются на проект системы или процессы, с помощью которых система создается. Они должны выполняться для удовлетворения технических, деловых или контрактных обязательств.

Ограничения проектирования

Можно указать следующие источники ограничений проектирования:

- **операционные среды**
- **совместимость с существующими системами**
- **прикладные стандарты**
- **корпоративные практические наработки и стандарты.**

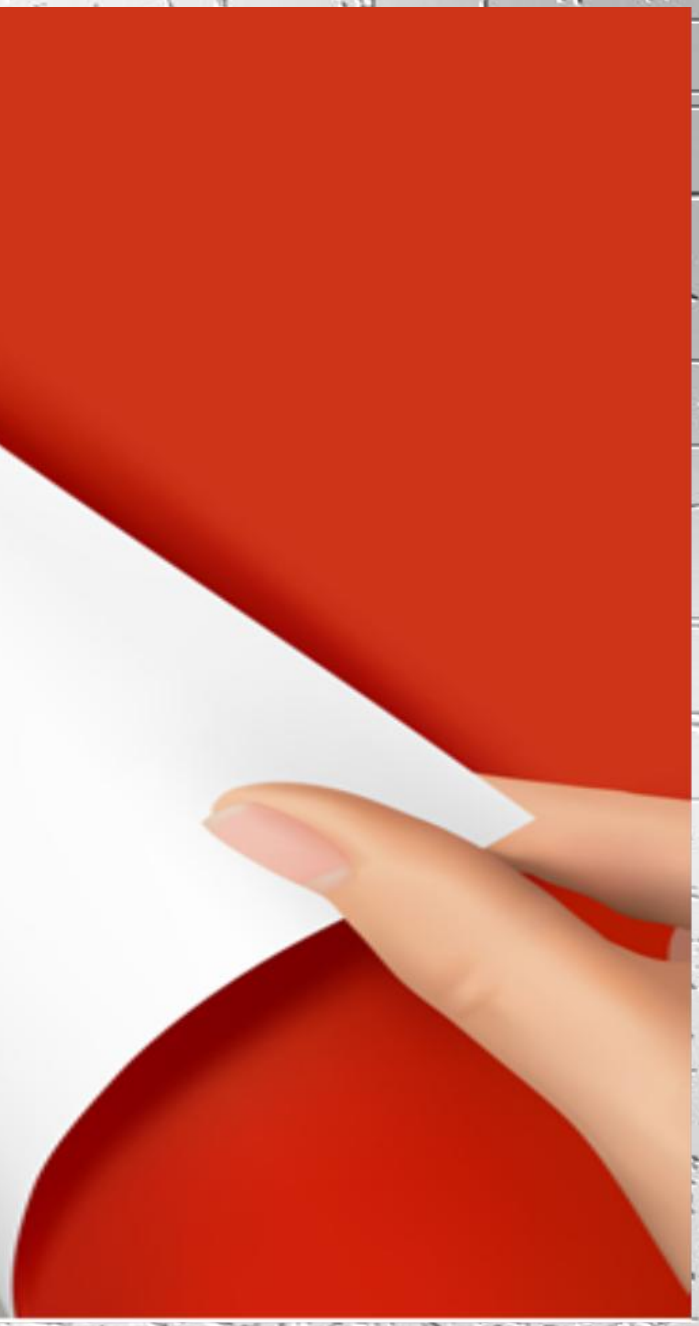
Классификация и артефакты



Иерархия требований и трассировка

Для отслеживания зависимости требований разных типов друг от друга и прослеживания изменений требований используется понятие трассировки требований. Обычно трассировка требований применяется для того, чтобы показать связь различных типов требований. На рисунке представлена иерархия описанных ранее типов требований со связями трассировки между ними:

ВИДЫ ТРЕБОВАНИЙ И ЗАДАЧИ УПРАВЛЕНИЯ ИМИ ПО BABOK®GUIDE



Трассировка требований по BABOK®Guide

- бизнес-потребность в виде проблемы означает потерю денег в компании,
- бизнес-потребность как возможность – потенциал их заработать.

Хотя рекомендуется формулировать бизнес-потребность в конкретных деньгах, на практике это не всегда получается. В этом случае следует все же сделать примерную оценку в численном выражении. Например, если бизнес-потребность сводится к улучшению клиентской лояльности, ее следует выразить в значениях ключевых метрик, таких как NPS (Net Promote Score), CSAT (Customer Satisfaction Score), отток клиентов ChR (Churn Rate) и коэффициент удержания CRR (Customer Retention Rate). Причем 2 последние метрики (ChR и CRR) уже напрямую коррелируют с деньгами в виде LTV (Lifetime Value). Например, если необходимо сократить отток клиентов на 15%, ценность этого бизнес-требования уже можно рассчитать, зная значения LTV или ARPU (Average revenue per user).

Ключевые показатели результативности процесса (метрики) – характеристики, по которым можно судить о том, насколько эффективно выполняется процесс и достигаются ли запланированные результаты.

Время: среднее время обработки заявки;
объем продукции в единицу времени

Стоимость: себестоимость продукта (услуги);
стоимость труда

Качество: процент брака;
уровень обслуживания

Метрики проекта

- Количество фаз / действий / работ
- Продолжительность каждой работы
- Стоимость ресурсов, стоимость работы, общая стоимость
- Степень загрузки ресурсов и исполнителей на отдельных этапах
- Количество завершенных работ
- Количество изменений в проекте
- Задержки выпуска
- Стоимость изменения требований

Метрики программного обеспечения

1) Трудоемкость и емкостная сложность (асимптотическая оценка),
2) Количество строк кода (LOC – lines-of-code),
3) Цикломатическая сложность,
4) Анализ функциональных точек,
5) Количество ошибок на 1000 строк кода,
6) Степень покрытия кода тестированием,
7) Покрытие требований,
8) Количество классов и интерфейсов,
9) Связность.

Ключевые метрики

Основные типы метрик:

Метрики вовлеченности: количество активных пользователей, время, проведенное в приложении, частота использования.

Метрики удержания: коэффициент удержания, коэффициент оттока.

Метрики конверсии: коэффициент конверсии, стоимость привлечения клиента, пожизненная ценность клиента.

Метрики доходности: общий доход, средний доход на пользователя, маржинальность.

- NPS (Net Promote Score) - индекс потребительской лояльности
- CSAT (Customer Satisfaction Score),
- отток клиентов ChR (Churn Rate)
- коэффициент удержания CRR (Customer Retention Rate).

Контекстная диаграмма

Зачем нужна контекстная диаграмма?

Обсуждение и визуализация назначения и границ системы

Контекстная диаграмма прежде всего позволяет быстро, кратко и ёмко описать назначение и границы системы, выявить и устранить коллективные расхождения в их понимании, показать и договориться о её масштабе.

Быстрое выявление функциональных системных требований

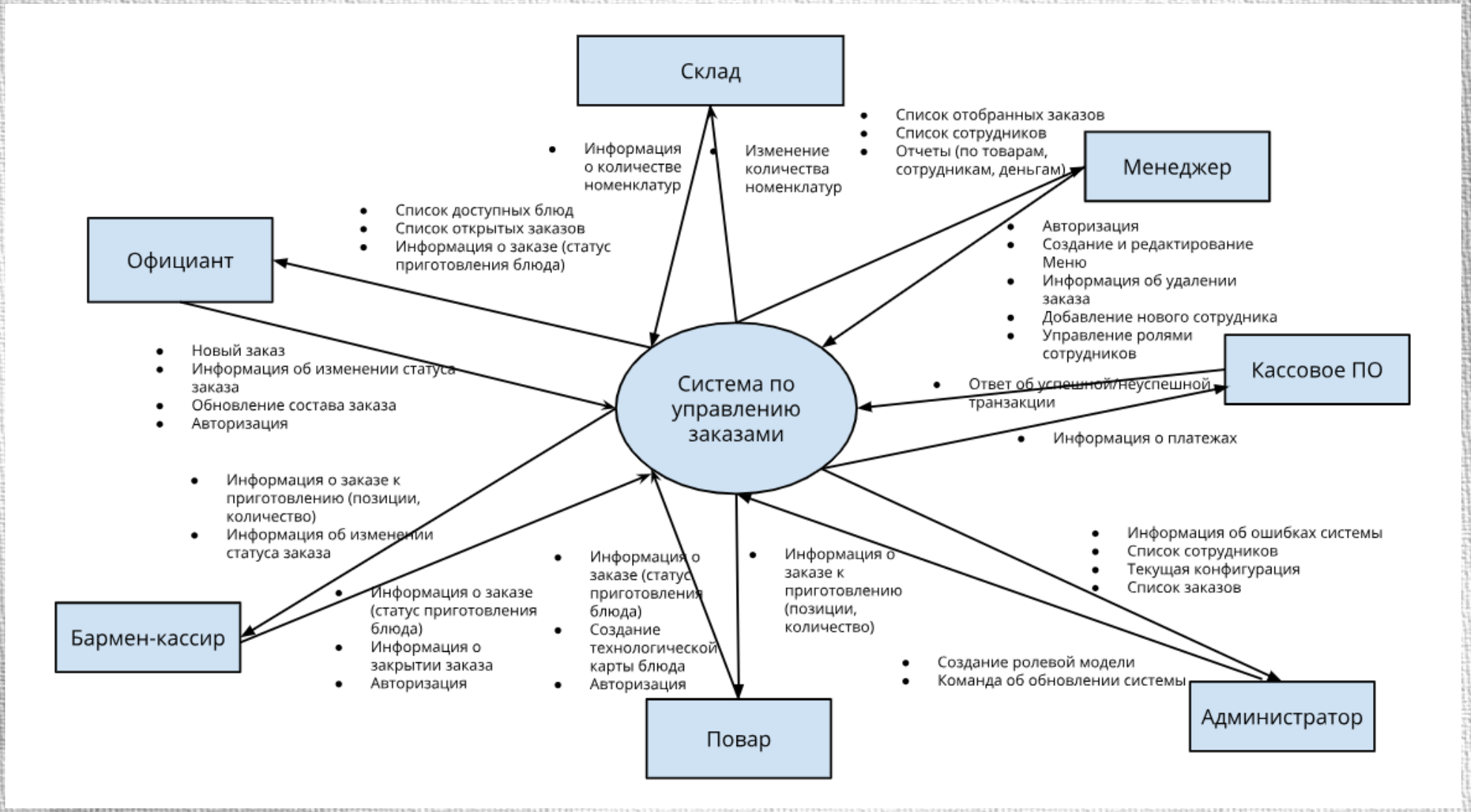
Второе её назначение — служить источником для быстрой генерации первичного набора системных функциональных требований при необходимости проектирования системы не «сверху-вниз», от бизнес-модели, бизнес-требований, модели деятельности организации, требований заинтересованных лиц, модели использования, как предлагает нам системная инженерия, а «из середины».

Как устроена контекстная диаграмма

Контекстная диаграмма относится к категории диаграмм, описывающих систему на уровне «чёрного ящика» — а именно, только внешние свойства (в данном случае — потоки данных), но не содержание системы.

Контекстная диаграмма содержит 3 основных компонента:

1. Проектируемый объект (например, система)
2. Взаимодействующие с проектируемым объектом элементы окружения (группы пользователей, смежные системы)



Пример контекстной диаграммы для программной системы управления Заказами в ресторане

- Потоки данных могут передаваться между окружением и (программной) системой любым образом — с помощью графического пользовательского интерфейса (GUI), командной строки (CLI), программных вызовов (API), почтовых сообщений и т.д.
- Если система имеет физические интерфейсы, то это могут быть разнообразные джойстики, рукоятки управления, специализированные клавиатуры, датчики распознавания движения, изображения, жестов и т.д.
- В стандартной форме не принято указывать виды интерфейсов взаимодействия и тем более протоколы, чтобы не усложнять диаграмму и не пытаться принимать вторичные решения, пока не приняты первичные.

Как применять контекстную диаграмму после её создания

Диаграмма может использоваться в документе концепции системы, документе системных требований или вики-документации для получения обзорного представления о назначении системы у читателя.

Контекстная диаграмма не создаётся один раз и навсегда, она может эволюционировать в ходе проекта. Другое дело, что каждое изменение диаграммы по сути означает изменение рамок системы и должно внимательно отрабатываться проектной командой.

Выявление и контроль полноты (функциональных) системных требований

При создании системных требований возникает риск упустить что-то важное или наоборот, избыточно проработать очевидное.

Чтобы не упустить что-то важное среди системных функций, можно применять:

- Трассировку системных требований на требования заинтересованных лиц
- Модель использования системы (обычно в форме набора сценариев использования, use case'ов)
- Контекстную диаграмму системы

Контекстная диаграмма может эффективно использоваться для выявления первичного набора системных функциональных требований. Каждый поток данных на диаграмме по сути означает, подразумевает какую-то функцию.

Наибольшие гарантии даёт применение всех 3-х методов, однако контекстная диаграмма — это самый простой и дешёвый их них, поэтому часто стоит начинать проработку системных требований именно с неё.

Откуда взялась контекстная диаграмма и почему до сих пор актуальна

Контекстная диаграмма в той или иной форме использовалась человечеством в разное время, однако свою каноническую форму получила у Тома ДеМарко в 70-80-х годах в семействе методологий Structured Analysis and Structured Design как верхний уровень диаграммы потоков данных — DFD (Data Flow Diagram).

При создании нотации и языка UML его авторы не взяли в него контекстную диаграмму, а использовали диаграмму схожего назначения — диаграмму использования (Use Case Diagram, UCD).

Диаграмма использования фокусируется только на части потоков данных, применение которых помогает агентам в достижении важных для них результатов и, таким образом, относится к более высокому уровню — уровню модели использования.
По задумке авторов UML отсутствие всех потоков данных не позволяет использовать UCD для выявления функциональных требований непосредственно, а требует предварительной проработки сценариев использования и извлечения ФТ уже из них.
Таким образом диаграммы не взаимозаменяемы, а скорее дополняют друг друга и поэтому для сложных систем полезно строить обе диаграммы.

Контроль знаний

1. Какие виды классификации требований существуют?
2. Какие требования относятся к группе функциональных/ нефункиональных требований (по Вигерсу)?
3. Какие документы описывают требования на каждом из уровней (по Вигерсу)?
4. Какие требования относят к категории нефункциональных требований (по Вигерсу)?
5. Как классифицируются бизнес правила?
6. Назовите основные характеристики (атрибуты) качества.
7. Какие требования могут предъявляться к системе?

СВОЙСТВА ТРЕБОВАНИЙ

- Полнота
- Ясность
- Корректность
- Верифицируемость
- …

Свойства требований

Хороший набор требований удовлетворяет следующим показателям качества (IEEE 830-1998 «Recommended Practice for Software Requirements Specifications»):

1. **Полнота**
2. **Ясность (краткость, простота, точность, недвусмысленность)**
3. **Верифицируемость (тестируемость, возможность проверки)**
4. **Необходимость и полезность при эксплуатации**
5. **Осуществимость (выполнимость, правдоподобность, реализуемость)**
6. **Элементарность и трассируемость (прослеживаемость)**
7. **Независимость от других требований (атомарность)**
8. **Независимость от реализации (абстрактность)**
9. **Корректность (согласованность, непротиворечивость)**
10. **Постоянство (стабильность)**

Свойства требований

11. Наличие количественной метрики
12. Единичность

1.Полнота требования

- *Полнота требования* означает, что текст требования не требует дополнительной детализации, то есть, в нем предусмотрены все необходимые нюансы, особенности и детали данного требования.
- Различают полноту отдельного требования и полноту системы требований.

При создании программных систем нам приходится иметь дело с характеристиками еще несуществующей системы.

Идея о том, что необходимо сформулировать все требования полностью, т.е. исчерпывающим образом, до начала проектирования, а тем более - реализации системы, изжила себя вместе с так называемым каскадным подходом, который поддерживал последовательную модель реализации системы. Спиральный подход, на котором базируется большинство современных методологий, предусматривает **поэтапное выделение и детализацию требований на всем протяжении цикла разработки системы.**

Тем не менее, требование полноты предъявляется к требованиям, формулируемым к системе. Надо понимать, что данное требование - это скорее тенденция, цель, к которой нужно постараться максимально приблизиться на как можно более ранних стадиях проекта.

Требование полноты можно рассматривать в двух аспектах:

1. **Полнота отдельного требования** - свойство, означающее, что текст требования не требует дополнительной детализации, то есть в нем предусмотрены все необходимые нюансы, особенности и детали данного требования.
2. **Полнота системы требований** - свойство, означающее, что совокупность артефактов, описывающих требования, исчерпывающим образом описывает все то, что требуется от разрабатываемой системы.

2. Ясность

- *Ясность* – недвусмысленность, определенность, однозначность спецификаций.
- Требование обладает свойством *ясности*, если оно сходным образом воспринимается всеми заинтересованными лицами.

Пример 1.

Требование неясное: Система не должна принимать слишком длинные пароли.
Требование ясное: Система не должна принимать пароли >15 символов. Если пользователь вводит >15 символов, сообщение об ошибке должно информировать пользователя об необходимом исправлении пароля.

Пример 2.

Требование неясное: Иногда пользователь будет вводить Код Аэропорта, который система будет распознавать. Но иногда код можно заменить близлежащим городом, и тогда пользователю не нужно знать код аэропорта, т.к. система будет понимать название города.

Требование ясное: Система должна идентифицировать аэропорт на основании кода аэропорта или названия города

На практике *ясность* требований достигается в том числе и в процессе консультаций, в ходе которых происходит "выравнивание *тезаурусов*" совладельцев системы. Хорошим подспорьем в этом служит согласованный сторонами *глоссарий* ключевых понятий *предметной области*.

Требование, которое сформулировано ясно, может быть прослежено, начиная от того документа, где оно сформулировано впервые, вплоть до рабочих спецификаций.

3. Проверяемость

Проверяемость – пригодность к проверке. Тестировщики должны иметь возможность проверить, было ли требование реализовано корректно.

- *Треб.1: Функция поиска должна позволять пользователю искать заказ на основе Фамилии, Имени, Даты,* ***и т.д.***
- *Треб. 2 : Система должна препятствовать одновременному доступу* ***большого*** *числа пользователей.*
- *Треб.3: Код аэропорта должен быть введен.*

Реализованность требования может быть определена через один из четырёх возможных методов: осмотр, демонстрация, тест или анализ.

4. Необходимость и Полезность

- ***Необходимым*** считается требование, невыполнение которого угрожает работоспособности или эффективности ИС.
- **В требовании нет необходимости, если:**
  - Ни одному заинтересованному лицу требование не нужно.
    - ***Пример***. *Пользователь должен иметь возможность просмотра карты аэропорта.*
  - Удаление требования не повлияет на систему, т.к. оно не предоставляет никакой новой информации:
    - ***Пример.*** *Все требования, указанные в документе Концепция, должны быть реализованы и протестированы.*
- ***Полезность при эксплуатации*** **–** следует считать любые свойства, повышающие применимость продукта.

5. Осуществимость (выполнимость, правдоподобность, реализуемость)

Требование должно быть выполнимо в рамках существующих ограничений, таких как время, деньги и доступные ресурсы.

- ***Пример**: Система должна иметь интерфейс на естественном языке, который будет понимать команды на русском языке.*

6. Элементарность и трассируемость (прослеживаемость)

Требование считается **элементарным**, если оно содержит только один *трассируемый элемент,* который дает возможность отследить связь между ним и другими элементами информационной системы.

- ***Пример**: Система должна предоставлять возможность бронировать рейс, покупать билет, бронировать номер в гостинице, бронировать машину, а также предоставлять информацию о развлечениях.*

Трассируемость требования определяется возможностью отследить связь между ним и другими артефактами информационной системы (документами, моделями, текстами программ и пр.)

Различают трассируемость в *прямом* и *обратном* направлениях.

7. Независимость от других требований (атомарность)

Требование является *независимым*, если для его понимания не нужно знать другие требования.

Пример

- *Треб.1: Список доступных рейсов должен включать номер рейса, время отправления и время прибытия для каждого отрезка пути.*
- *Треб.2: Он должен быть отсортирован по ценам.*

8. Независимость от реализации (абстрактность)

Требования должны быть независимыми от реализации

- ***Пример**: Информация должна храниться в текстовом файле.*

9. Корректность (согласованность, непротиворечивость)

Корректность – согласованность, непротиворечивость. Требования не должны противоречить требованиям своего уровня иерархии и требованиям "родительского" уровня. Если требование содержит факты, эти факты должны быть достоверны:

- *Треб.1: Цена заказа должна включать все соответствующие платежи (включая стоимость пересылки – 200 руб.)*

Требование считается *корректным*, если не существует конфликтов между ним и другими требованиями.

Прямые конфликты возникают, когда ожидается различное поведение системы в одной и той же ситуации:

- *Треб.1: Дата должна отображаться в формате ММ/ДД/ГГ.*
- *Треб.2 : Дата должна отображаться в формате ДД/ММ/ГГ.*

Косвенный конфликт возникает, когда требования описывают разную функциональность, но выполнить оба этих требования одновременно невозможно:

- *Треб.1: Система должна иметь интерфейс на естественном языке.*
- *Треб.2: Система должна быть разработана в течение трех месяцев.*

10. Постоянство (стабильность, актуальность)

Приоритет требования представляет количественную оценку степени значимости (важности) требования. Приоритеты требований обычно назначает представитель Заказчика. Разработчик, отталкиваясь от приоритетности требований, управляет процессом реализации информационной системы.

Стабильность требования характеризует прогнозную оценку неизменности требований во времени.

Требование не стало устаревшим с течением времени

11. Наличие количественной метрики

Количественные метрики играют важную роль в верификации и аттестации информационных систем.

В первую очередь это относится к нефункциональным требованиям, которые, как правило, должны иметь под собой количественную основу.

Функциональные требования также могут расширяться количественными мерами, например при помощи аспектов применимости .

12. Единичность

Требование описывает одну и только одну вещь

Характеристика	Объяснение
Единичность	Требование описывает одну и только одну вещь.
Завершённость	Требование полностью определено в одном месте и вся необходимая информация присутствует.
Последовательность	Требование не противоречит другим требованиям и полностью соответствует внешней документации.
Атомарность	Требование «атомарно». То есть оно не может быть разбито на ряд более детальных требований без потери завершённости.
Отслеживаемость	Требование полностью или частично соответствует деловым нуждам как заявлено заинтересованными лицами и документировано.
Актуальность	Требование не стало устаревшим с течением времени.

Выполнимость	Требование может быть реализовано в пределах проекта.
Недвусмысленность . .	Требование кратко определено без обращения к техническому жаргону, акронимам и другим скрытым формулировкам. Оно выражает объективные факты, не субъективные мнения. Возможна одна и только одна интерпретация. Определение не содержит нечётких фраз. Использование отрицательных утверждений и составных утверждений запрещено
Проверяемость	Реализованность требования может быть определена через один из четырёх возможных методов: осмотр, демонстрация, тест или анализ.
Обязательность	Требование представляет определённую заинтересованным лицом характеристику, отсутствие которой приведёт к неполноценности решения, которая не может быть проигнорирована. Необязательное требование — противоречие самому понятию требования

Каких требований не должно быть

Спецификация требований не должна содержать деталей проектирования или реализации.

Требования должны отвечать на вопрос: "что должна делать система", абстрагируясь от вопроса "как она это должна делать".

ПРОЕКТНАЯ ДОКУМЕНТАЦИЯ И СТАНДАРТЫ

- Техническое задание
- Спецификация требований SRS
- ГОСТЫ 19 и 34
- …

Институт инженеров электротехники и электроники

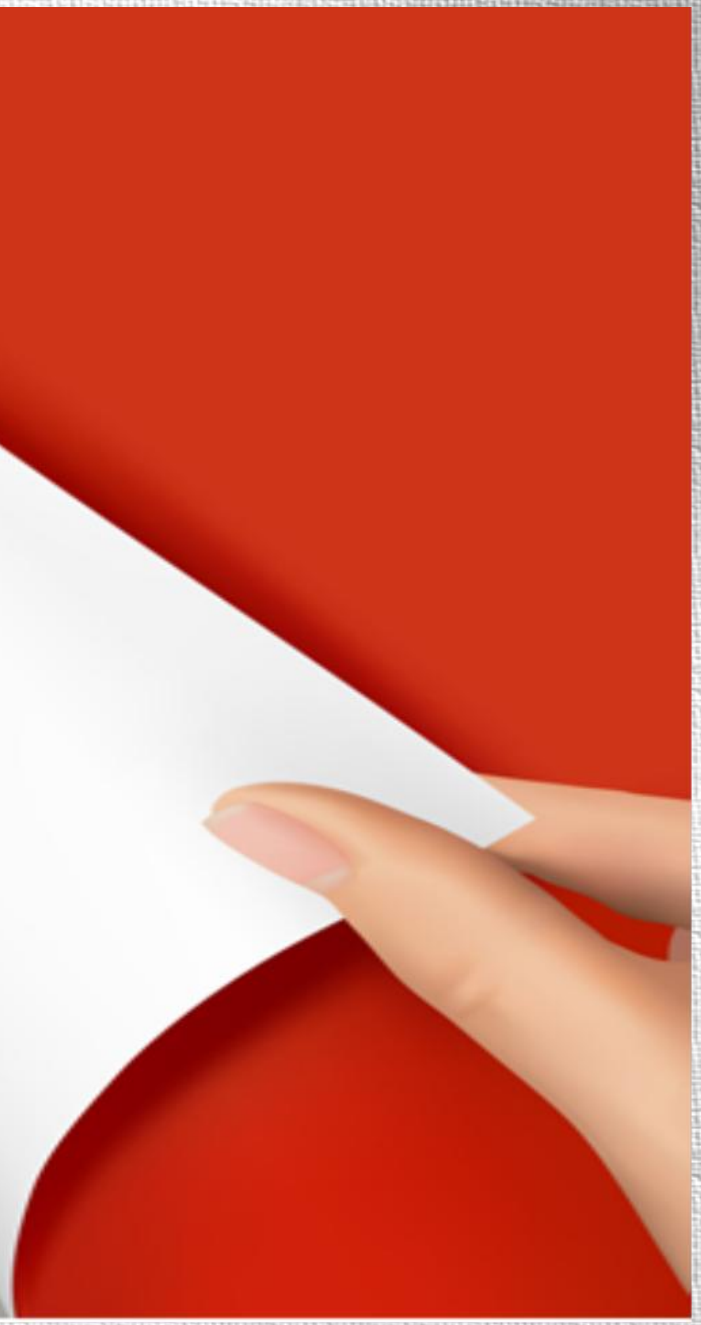
Некоммерческая организация

Институт инженеров электротехники и электроники — IEEE — международная некоммерческая ассоциация специалистов в области техники, мировой **лидер в области разработки стандартов** по радиоэлектронике, электротехнике и аппаратному обеспечению вычислительных систем и сетей.

Штаб-квартира: Пискатауэй, Нью-Джерси, США

Дата основания: 1 января 1963 г.

- ТЕХНИЧЕСКОЕ ЗАДАНИЕ
- СТАНДАРТЫ



Стандарты, регламентирующие работу с требованиями

Разработки IEEE:

- IEEE 1362 "Concept of Operations Document".
- IEEE 1233 "Guide for Developing System Requirements Specifications".
- IEEE Standard 830-1998, "IEEE Recommended Practice for Software Requirements Specifications"
- IEEE Standard Glossary of Software Engineering Terminology/IEEE Std 610.12-1990
- IEEE Guide to the Software Engineering Body of Knowledge (1) - SWEBOK®, 2004.

Стандарты, регламентирующие работу с требованиями

Отечественные ГОСТ:

- ГОСТ 34.602-89. Информационная технология. Техническое задание на создание автоматизированной системы
- ГОСТ 19.201-78. Техническое задание. Требования к содержанию и оформлению.
- РД 50 34.698-90. Автоматизированные системы. Требования к содержанию документов
- ГОСТ 34.601-90. Информационная технология. Автоматизированные системы. Стадии создания.

Дед Мороз & Santa Claus

(проектная документация)

1. **ТЗ —** техническое задание (ГОСТы 34 и 19)
2. **SRS (Software Requirement Specification) — спецификация требований программного обеспечения** — законченное описание поведения программы, которую требуется разработать. Включает ряд пользовательских сценариев (*use cases*), которые описывают все варианты взаимодействия между пользователями и программным обеспечением. В дополнение к пользовательским сценариям, спецификация также содержит нефункциональные требования. В стандарте IEEE 830 содержится рекомендации к структуре и методам описания программных требований — «*Recommended Practice for Software Requirements Specifications*».

SRS ≠ ТЗ

Спецификация требований является частью технического задания в случае создания автоматизированных информационных систем.

За создание спецификации программного обеспечения чаще всего в российской практике отвечает Системный аналитик, иногда — Бизнес-аналитик.

ГОСТ РФ

- **ГОСТ 34.602-89. Информационная технология. Техническое задание на создание автоматизированной системы**
- **ГОСТ 19.201-78. ЕСПД. Техническое задание. Требования к содержанию и оформлению.**
- ГОСТ 34.601-90. Информационная технология. Автоматизированные системы. Стадии создания.
- РД 50 34.698-90 «Автоматизированные системы. Требования к содержанию документов»

Федеральный закон о техническом регулировании (№184-ФЗ)

С 27 декабря 2002 года, согласно Федеральному закону о техническом регулировании (№184-ФЗ) **ГОСТ перестал носить обязательный характер и может применяться добровольно** (за исключением случаев, когда указания на применение безопасности здоровья граждан или окружающей среды).

Король умер!

Да здравствует король?

В этом же году было введено понятие технического регламента, который в будущем должен заменить государственные стандарты.

Технический регламент — документ, который устанавливает обязательные требования к продукции, выпускаемой на Российский рынок.

Аналитический отчет → Техническое задание

- Техническое задание является исходным материалом для создания информационной системы или другого продукта. Поэтому техническое задание (сокращенно ТЗ) в первую очередь должно содержать основные технические требования к продукту и отвечать на вопрос, **что данная система должна делать, как работать и при каких условиях**.
- Как правило, этапу составления технического задания предшествует проведение обследования предметной области, которое завершается созданием **аналитического отчета**. Именно аналитический отчет (или аналитическая записка) ложится в основу документа Техническое задание.

Если в аналитическом отчете требования заказчика могут быть изложены в общем виде и проиллюстрированы UML-диаграммами, в техническом задании следует подробно описать все функциональные и пользовательские требования к системе. Чем подробнее будет составлено техническое задание, тем меньше спорных ситуаций возникнет между заказчиком и разработчиком во время приемочных испытаний.

Таким образом, техническое задание является документом, который позволяет как разработчику, так и заказчику представить конечный продукт и впоследствии выполнить проверку на соответствие предъявленным требованиям.

Разделы ТЗ

Любое техническое задание должно содержать разделы, отражающие сведения:

- что надо сделать;
- для чего, с какой целью надо сделать это;
- где, в какой области применения, на каком объекте это должно **решать** задачи и **выполнять** свои функции;
- какие требования будут предъявлены к этому;
- какие работы потребуется выполнить, чтобы сделать это;
- каков порядок проведения испытаний и приемки-сдачи работ заказчику;
- как должно быть задокументировано проведение работ;
- на основании каких нормативно-технических документов должны проводиться работы?

Список и описание разделов, которые должно содержать техническое задание согласно ГОСТ

ГОСТ 19.201-78 Техническое задание. Требования к содержанию и оформлению	ГОСТ 34.602.89 Техническое задание на создание автоматизированной системы
1. Введение	1. Общие сведения
2. Основания для разработки	
3. Назначение разработки	2. Назначение и цели создания системы
	3. Характеристика объекта автоматизации
4. Требования к программе или программному изделию	4. Требования к системе
4.1. Требования к функциональным характеристикам	4.2. Требования к функциям (задачам), выполняемым системой
	4.1. Требования к системе в целом
	4.1.1. Требования к структуре и функционированию системы
	4.1.3. Показатели назначения
4.2. Требования к надежности	4.1.4. Требования к надежности
	4.1.5. Требования к безопасности
	4.1.6. Требования к эргономике и технической эстетике

4.3. Условия эксплуатации	4.1.2. Требования к численности и квалификации персонала системы и режиму его работы
	4. 1.9. Требования к защите информации от несанкционированного доступа
	4. 1.10. Требования по сохранности информации при авариях
	4. 1.11. Требования к защите от влияния внешних воздействий
	4. 1.12. Требования к патентной чистоте
	4. 1.13. Требования по стандартизации и унификации
4.4. Требования к составу и параметрам технических средств	4. 1.8. Требования к эксплуатации, техническому обслуживанию, ремонту и хранению компонентов системы
4.5. Требования к информационной и программной совместимости	
4.6. Требования к маркировке и упаковке	

4.7. Требования к транспортированию и хранению	4. 1.7. Требования к транспортабельности для подвижных систем
4.8. Специальные требования	4. 1.14. Дополнительные требования
	4.3. Требования к видам обеспечения
5. Требования к программной документации	8. Требования к документированию
6. Технико-экономические показатели	
7. Стадии и этапы разработки	5. Состав и содержание работ по созданию системы
8. Порядок контроля и приемки	6. Порядок контроля и приемки системы
	7. Требования к составу и содержанию работ по подготовке объекта автоматизации к вводу системы в действие
	9.Источники разработки

Основные разделы ТЗ

Итак, документ **Техническое задание** должен, по сути, отражать все требования к проектируемому продукту, выделенные на этапе аналитического исследования объекта автоматизации.

Опираясь на таблицу, приведенную выше, мы можем выделить основные разделы технического задания:

1. *Общие сведения о системе (программе);*
2. *Назначение, цели и задачи системы (программы);*
3. *Требования к системе (функциональные требования, пользовательские требования, требования к системе в целом и т.д.);*
4. *Требования к видам обеспечения;*
5. *Требования к документированию;*
6. *Стадии и этапы разработки;*
7. *Порядок контроля и приемки системы (программы).*

Недостатки ГОСТ 19.102-77

ГОСТ 19.102-77, описывая стадии разработки программ, вообще не предусматривает каких-либо мероприятий по их внедрению, кроме передачи для сопровождения или изготовления. Отсутствует понятие «пользователя» программы, не поддержано методически документирование программ «непрерывного действия». Не рассмотрены в ЕСПД вопросы интеграции программ и межпрограммного взаимодействия, слабо нормированы вопросы информационного и организационного обеспечения. Предложенная степень детализации структуры программных документов недостаточна, а львиная доля внимания уделяется второстепенным по современным меркам вопросам (например, маркировке и упаковке машинных носителей).

Стадии ЖЦ

Анализ требований

по ISO/IEC 15288:2002

- Формирование концепции
- Разработка
- Реализация
- Эксплуатация
- Поддержка
- Снятие с эксплуатации

по ГОСТ 34.601-90

- Формирование требований к АС
- Разработка концепции АС.
- Техническое задание.
- Эскизный проект.
- Технический проект.
- Рабочая документация.
- Ввод в действие.
- Сопровождение АС

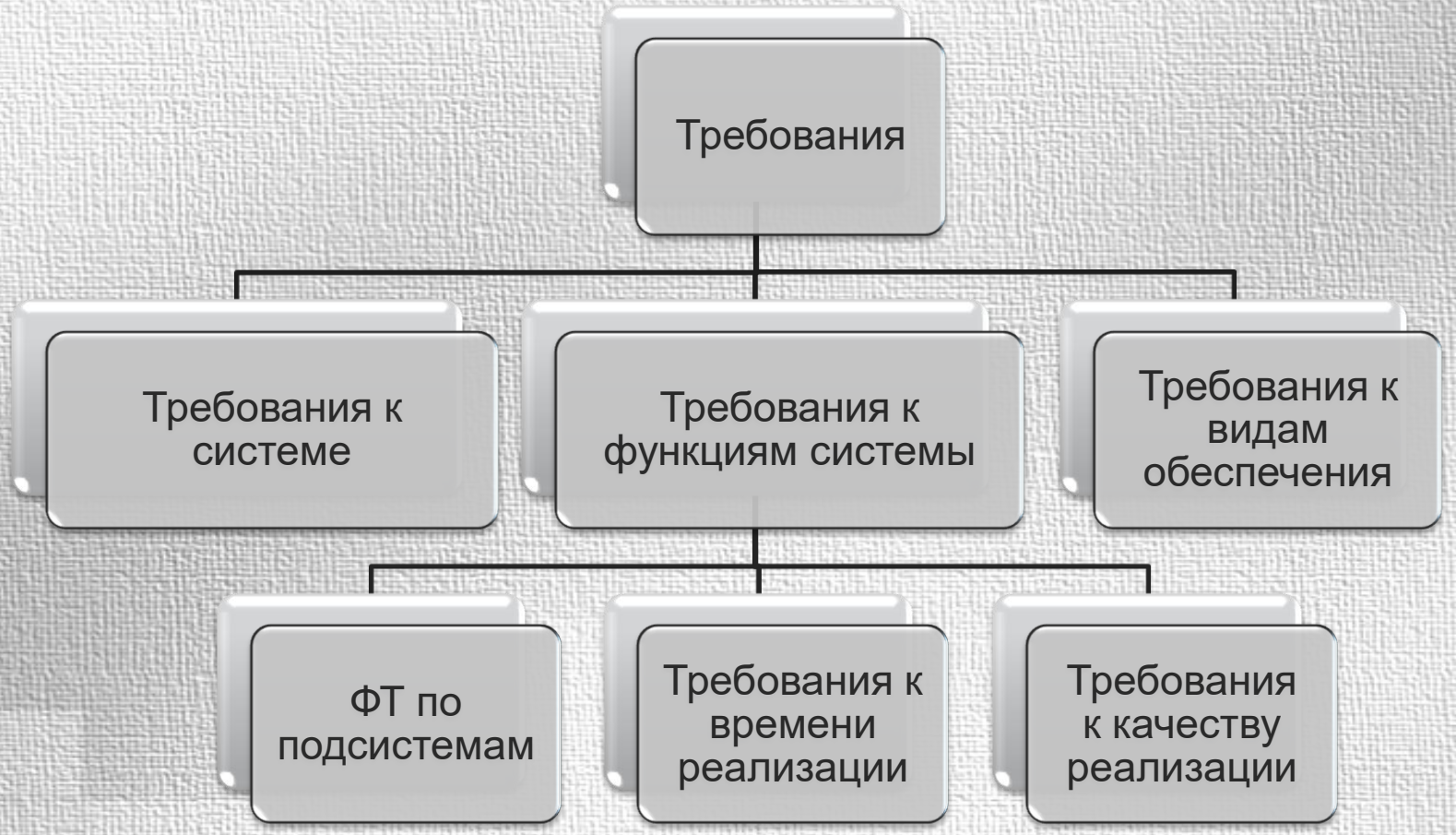
Проектирование

Реализация

Внедрение

Эксплуатация

Классификация требований по ГОСТ 34.602-89



Представление требований в ГОСТ

- ГОСТ 34.602 определяет состав и правила оформления документа «Техническое задание на создание (развитие или модернизацию) системы» (см. Автоматизированная система, в ГОСТ 34.003).
- Документ Техническое задание устанавливается как основной документ, определяющий «требования и порядок создания автоматизированной системы».
- Предполагая, что на основании этого документа будет производиться разработка системы и ее приемка.

Соотнесение типов требований (К.Вигерс) и ГОСТ (34.602-89)

Описание соответствий

Бизнес-требования	**2.4.2. В подразделе «Цели создания системы»** приводят наименования и требуемые значения технических, технологических, производственно-экономических или других показателей объекта автоматизации, которые должны быть достигнуты в результате создания АС, и указывают критерии оценки достижения целей создания системы. **ГОСТ 34.601-90** АВТОМАТИЗИРОВАННЫЕ СИСТЕМЫ. СТАДИИ СОЗДАНИЯ. 2. На этапе 1.2. "Формирование требований пользователя к АС" проводят: а) подготовку исходных данных для формирования требований АС (характеристика объекта автоматизации, описание требований к системе, ограничения допустимых затрат на разработку, ввод в действие и эксплуатацию, эффект, ожидаемый от системы, условия создания и функционирования системы); б) формулировку и оформление требований пользователя к АС 3. На этапе 1.3. "Оформление отчёта о выполненной работе и заявки на разработку АС (технико-технического задания)" проводят оформление отчета о выполненных работах на данной стадии … (по ГОСТ 7.32-2001)
Функциональные требования	**2.6.2. В подразделе «Требование к функциям (задачам), выполняемым системой, приводят:** а) по каждой подсистеме перечень функций, задач или их комплексов (в том числе обеспечивающих взаимодействие частей системы), подлежащих автоматизации; …
Внешние интерфейсы	**2.6.1.1. В требованиях к структуре и функционированию системы приводят**: в) требования к характеристикам взаимосвязей создаваемой системы со смежными системами, требования к ее совместимости, в том числе указания о способах обмена информацией (автоматически, пересылкой документов, по телефону и т. п.);

Описание соответствий

Атрибуты качества	**2.6.2. В подразделе «Требование к функциям (задачам)», выполняемым системой, приводят:** в) требования к качеству реализации каждой функции :,:, характеристики необходимой точности и времени выполнения : **2.6.1.3. В требованиях к показателям назначения АС приводят значения параметров, характеризующие степень соответствия системы ее назначению**. а) степень приспособляемости системы к изменению процессов № методов управления, к отклонениям параметров объекта управления; б) допустимые пределы модернизации и развития системы;
Ограничения	**2.6.3. В подразделе «Требования к видам обеспечения» в зависимости от вида системы приводят требования к:** 1) математическому, 2) информационному, 3) лингвистическому, 4) программному, 5) техническому, 6) метрологическому, 7) организационному, 8) методическому и другие видам обеспечения системы. Другие подразделы допускают включение различных ограничений.

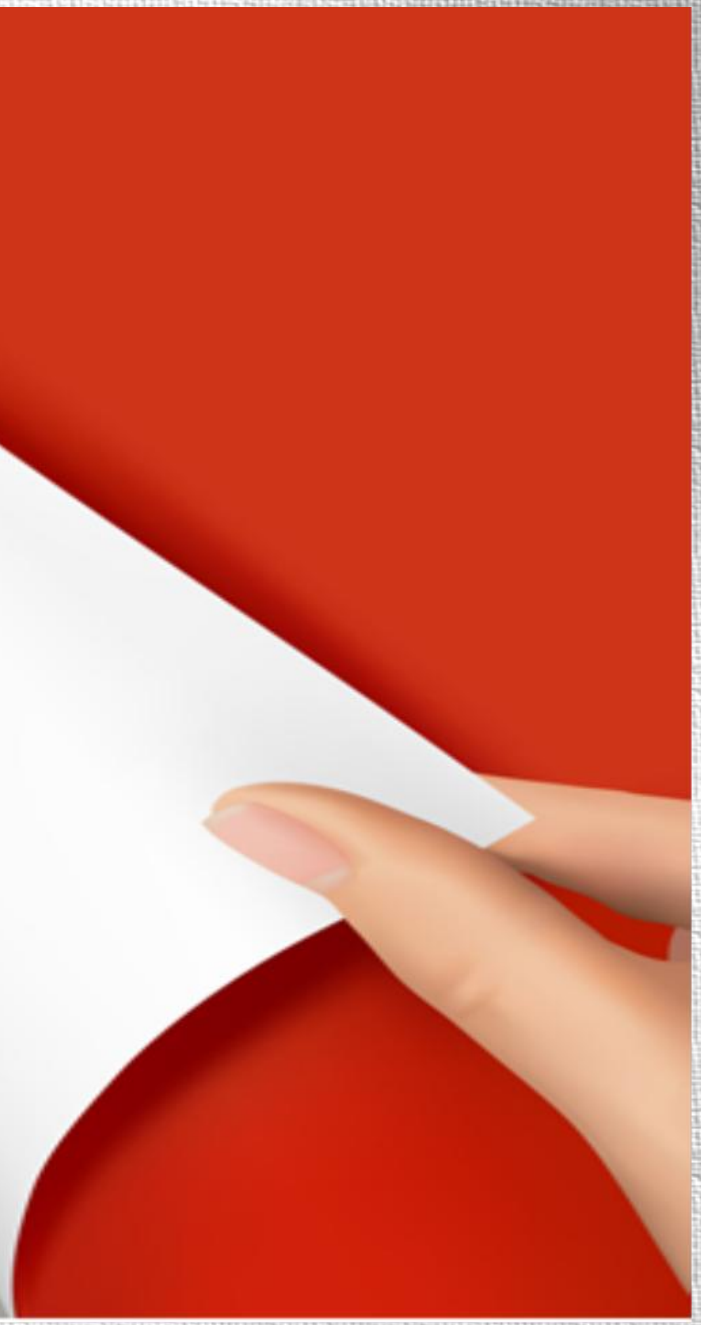
Дополнительные замечания

- Пользовательские требования, в том виде, в котором они определены в классификации приведенной К. Вигерсом, не имеют **явного** отражения в документе Техническое задание ни в других документах ГОСТ.

- Этап работ «1.2. Формирование требований пользователя к АС» на стадии «1. Формирование требований к АС» (ГОСТ 34.601-90. АВТОМАТИЗИРОВАННЫЕ СИСТЕМЫ СТАДИИ СОЗДАНИЯ.) не является эквивалентом пользовательских требований «по Вигерсу».

- При необходимости, с определенными оговорками, в качестве контейнера для вариантов использования может быть использован документ «Описание постановки задачи» (см. РД 50-34.698-90).

1. BUSINESS REQUIREMENTS DOCUMENT (BRD)
2. SOFTWARE REQUIREMENT SPECIFICATIONS (SRS) DOCUMENT
3. FUNCTIONAL REQUIREMENT SPECIFICATIONS (FRS) DOCUMENT

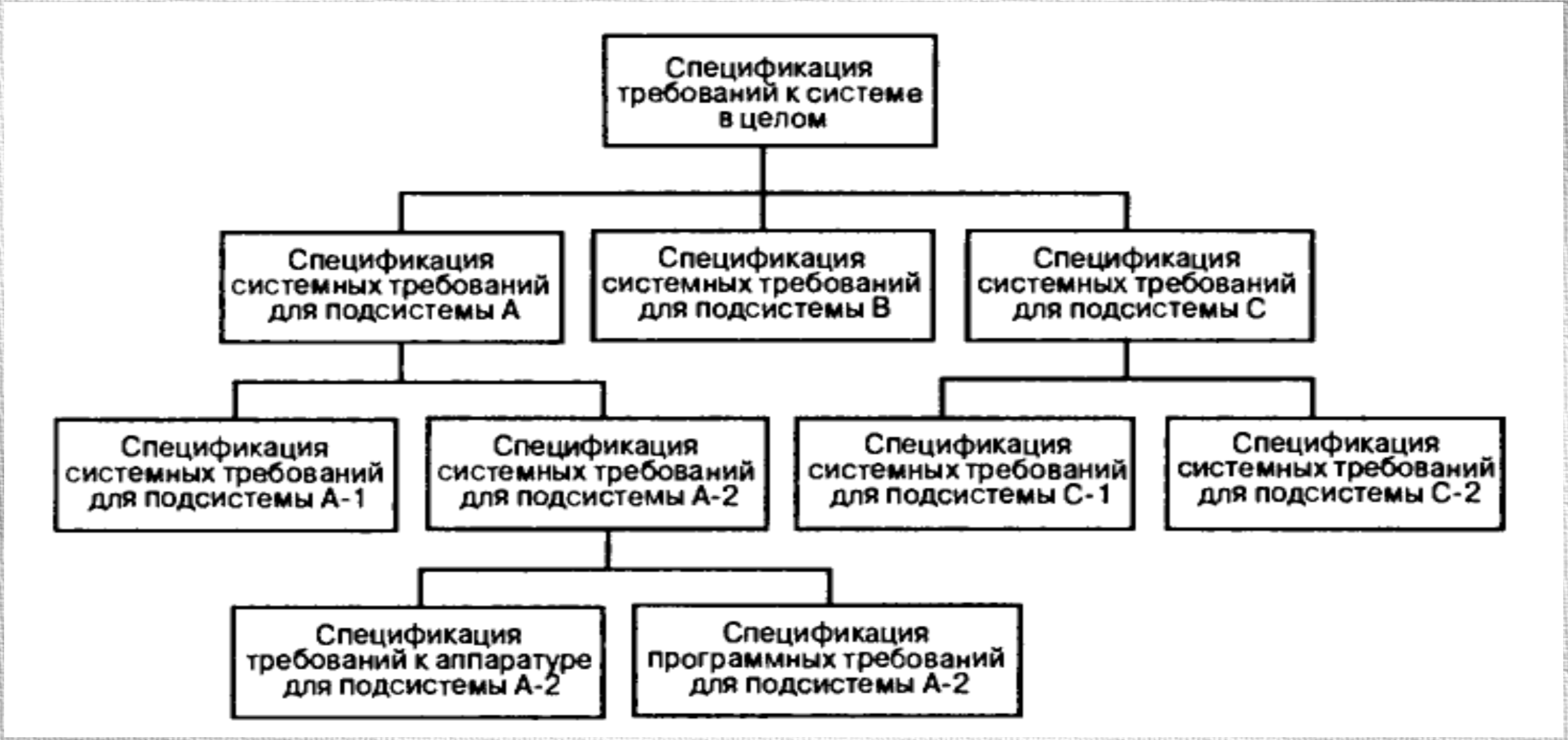


Для учета всех требований приходится вести много документов. Рассмотрим несколько случаев.

- Некий родительский документ определяет требования ко всей системе в целом: к аппаратному и программному обеспечению, персоналу и процедурам; а другой документ только к программному обеспечению. Первый документ называется *спецификацией требований к системе (System Requirement Specification).* А второй – *спецификацией требований к программному обеспечению (SRS).*
- Один документ содержит общие определении функций системы, а другой более конкретные. Первый документ часто называется *документом-концепцией (vision document)*, а второй *спецификацией требований к программному обеспечению (SRS)*.

- Один документ содержит полный набор требований к семейству продуктов. А в другом представлены требования конкретному приложению и конкретной версии. Первый из них именуется *документом требований к семейству продуктов* или *концепцией семейства продуктов.* А второй - *спецификацией требований к программному обеспечению (SRS)* определенной реализации некоего конкретного приложения из семейства продуктов.
- Один документ описывает общие бизнес-требования и бизнес окружение, в котором будет существовать система, а другой определяет её внешнее поведение. Первый называется *документом бизнес требований (BRD)* или *документов требований маркетинга,* а второй - *спецификацией требований к программному обеспечению (SRS).*

Иерархия спецификаций



BRD vs SRS vs FRS

Когда аналитик начинает изучать документирование требований проекта, ему приходится сталкиваться с разными видами требований. Такими как Business Requirements Document (BRD), Software Requirement Specifications (SRS) и Functional Requirement Specifications (FRS). Если быть кратким то, BRD содержит «высокоуровневые» бизнес-требования, SRS содержит «детальные» функциональные и нефункциональные требования, а FRD/FRS содержит «детализированные» функциональные требования вместе с диаграммами потока данных и UML.

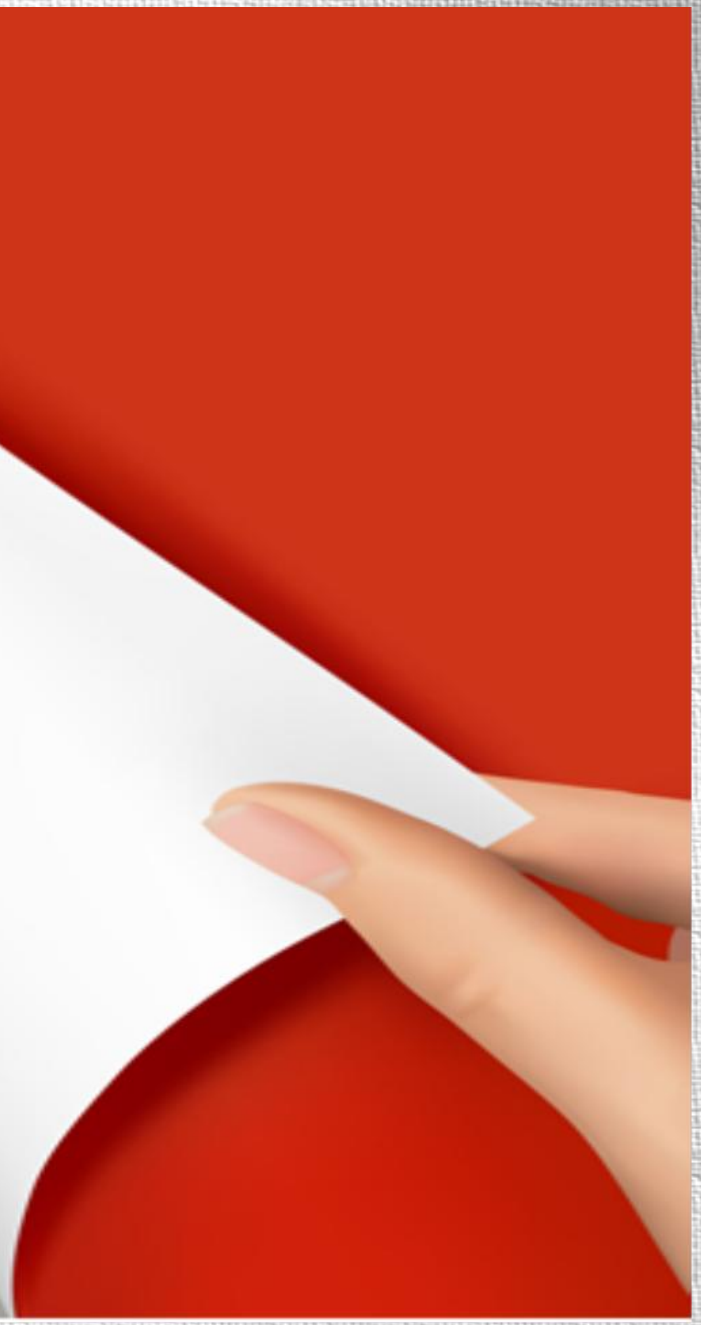
1. **Business Requirements Document (BRD)**
2. **Software Requirement Specifications (SRS) Document**
3. **Functional Requirement Specifications (FRS) Document**

	Business Requirements Document (BRD)	Software Requirement Specifications (SRS)	Functional Requirement Specifications (FRS)
Иные названия		Product Requirements Specification (PRS), System Requirements Specification	Functional Specifications Document (FRD), Product Specifications Document (PSD), Functional Specs (FS)
Кто создает	Бизнес-аналитик	Системный/Бизнес-аналитик	Системный/Бизнес-аналитик/Руководитель проекта
Что содержит	Высокоуровневые бизнес требования	Детальные функциональные требования, не функциональные требования, примеры использования	Детальные функциональные требования, схемы, диаграммы
Кто использует	Бизнес-пользователи, менеджеры проекта	Менеджеры проекта, эксперты предментых областей, Технические руководители проекта	Разработчики, тестировщики
Когда создается	Этап инициации проекта	Этап планирования проекта	Этап плаинрования проекта
Отвечает на вопрос	Зачем реализовывать требования	Какие требования должны быть выполнены для удовлетворения потребностей бизнеса	Каким образом система должна функционировать

Бизнес-анализ управляется определенными стандартами и рамками. Они должны соблюдаться для проведения практического анализа требований в любом проекте. Однако, нет общепринятых принципов для BRD, SRS и FRS относительно их структуры, содержания и детализации. Это приводит к тому, что организации применяют эти документы на основе своих процессов и стандартов, ресурсов, доступных для них, и типа проекта.

БИЗНЕС - ТРЕБОВАНИЯ

(BRD — Business Requirements Document)



BRD – один из самых широко используемых документов требований и BABOK – всемирно признанный стандарт бизнес-анализа, определяет его следующим образом:

Business Requirements Document (BRD) – это документ, который описывает бизнес-требования к продукту или проекту, их приоритеты, основные характеристики и ограничения, а также определяет причины их необходимости.

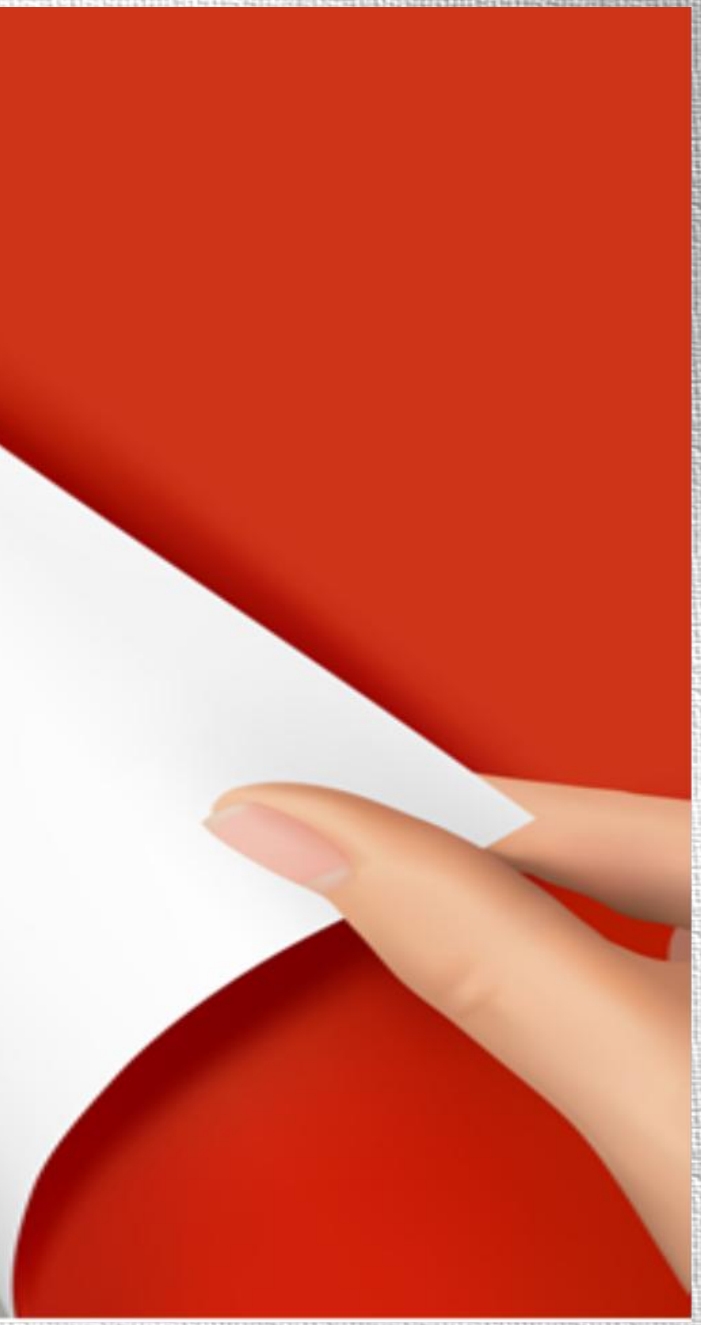
BRD обычно является одним из первых документов, создаваемых в жизненном цикле проекта. Он описывает общие цели компании, которые она пытается достичь или потребности, которые она пытается удовлетворить, создавая услугу или продукт. Кроме того, BRD также содержит потребности заинтересованных сторон, которые будут взаимодействовать с конечным продуктом. Таким образом, он отвечает на вопрос «почему» (а не «что» или «как») требования будут выполнены и какие результаты ожидаются от системы. Следует отметить, что все будущие требования, улучшения и запросы на изменения в рамках проекта должны быть обоснованы целями и потребностями компании (или бизнеса), перечисленными в BRD.

BRD всегда готовится бизнес-аналитиком и создается после проведения анализа и общения с заинтересованными сторонами клиента. После того, как BRD будет подготовлен, он обычно проверяется и утверждается. Это необходимо, чтобы, документ в точности отражал ожидания бизнеса и ключевых заинтересованных сторон.

Основной аудиторией BRD являются заказчики проекта, руководители и аналитики.

СПЕЦИФИКАЦИЯ ПРОГРАММНЫХ ТРЕБОВАНИЙ

(SRS — Software Requirement Specification)



Документы IEEE

- IEEE 1362 “Concept of Operations Document”.
- IEEE 1233 «Guide for Developing System Requirements Specifications».
- **IEEE Standard 830-1998, «IEEE Recommended Practice for Software Requirements Specifications»**
- IEEE Standard Glossary of Software Engineering Terminology/IEEE Std 610.12-1990
- IEEE Guide to the Software Engineering Body of Knowledge (1) - SWEBOK®, 2004.

Спецификация требований — это документ, в котором излагаются конкретные потребности проекта или системы. Спецификация требований важна, потому что она служит основой для всей будущей работы над проектом. Спецификация требований к программному обеспечению (SRS) отличается от спецификации бизнес-требований (BRS), хотя они и связаны между собой. SRS фокусируется на том, что система должна делать, в то время как BRS фокусируется на том, зачем нужна система и как она будет использоваться. Структура документа с требованиями к программному обеспечению может варьироваться, но всегда должна включать разделы, посвященные назначению, области применения, функциям, функциям, ограничениям, предположениям и зависимостям.

Рекомендуемая стандартом IEEE 830 структура SRS

- **Введение**
  - — Цели
  - — Соглашения о терминах
  - — Предполагаемая аудитория и последовательность восприятия
  - — Масштаб проекта
  - — Ссылки на источники

- **Общее описание**
  - — Видение продукта
  - — Функциональность продукта
  - — Классы и характеристики пользователей
  - — Среда функционирования продукта (операционная среда)
  - — Рамки, ограничения, правила и стандарты
  - — Документация для пользователей
  - — Допущения и зависимости

- **Функциональность системы**
  - ***Функциональный блок X (таких блоков может быть несколько)***

— Описание и приоритет

— Причинно-следственные связи, алгоритмы (движение процессов, workflows)

— Функциональные требования

- **Требования к внешним интерфейсам**
  - — Интерфейсы пользователя
  - — Программные интерфейсы
  - — Интерфейсы оборудования
  - — Интерфейсы связи и коммуникации

- **Нефункциональные требования**
  — Требования к производительности
  — Требования к сохранности (данных)
  — Критерии качества программного обеспечения
  — Требования к безопасности системы

Прочие требования

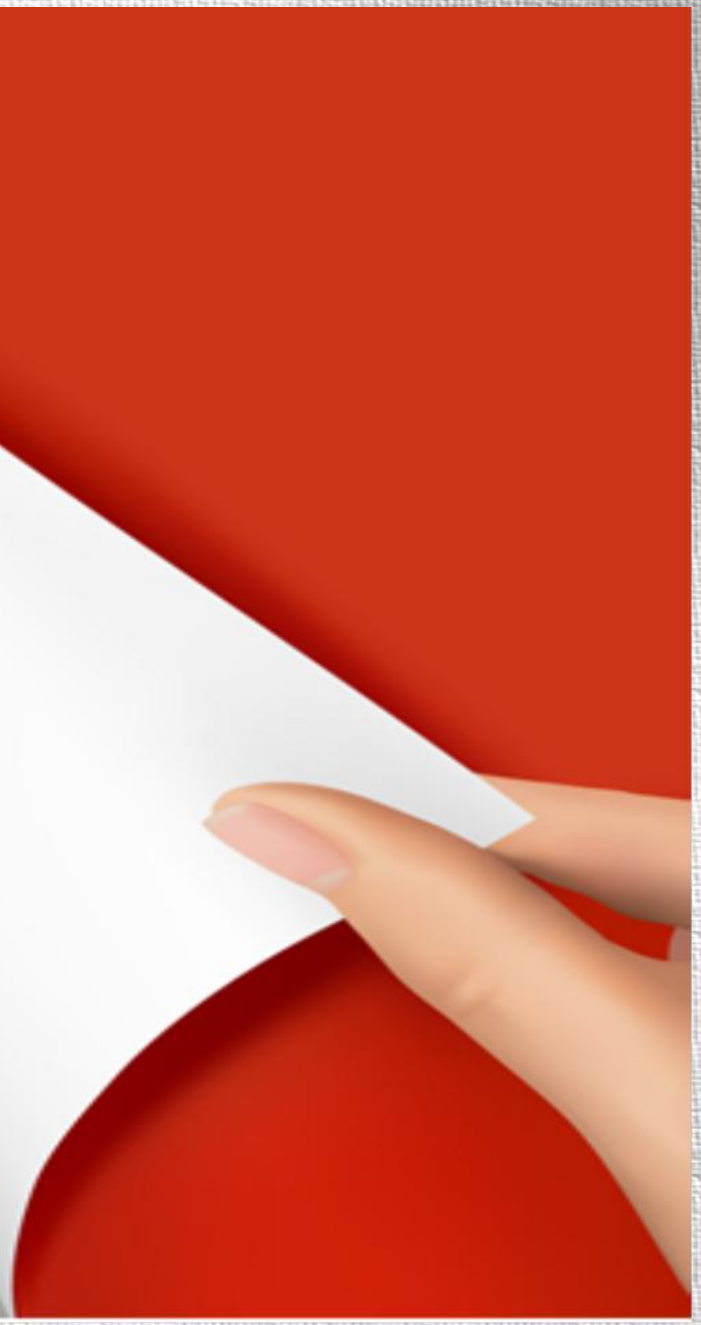
Приложение А: Глоссарий

Приложение Б: Модели процессов и предметной области и другие диаграмм

Приложение В: Список ключевых задач

СПЕЦИФИКАЦИИ ФУНКЦИОНАЛЬНЫХ ТРЕБОВАНИЙ

(FRS - Functional Requirement Specifications Document)



Документ FRS является самым подробным и детализированным из трех. Он, наконец, описывает «как» именно должна функционировать система, чтобы удовлетворить всем требованиям, перечисленным в BRD и SRS.

Документ «Спецификации функциональных требований» (FRS) — это детализированный и низкоуровневый документ, который подробно описывает все функциональные требования проекта.

Пример: Модуль входа в систему будет содержать следующие поля:

Ввод имени пользователя. Это поле представляет собой текстовое поле, в которое пользователь может ввести имя пользователя, являющееся идентификатором электронной почты его зарегистрированной компании.

Ввод пароля. Это поле представляет собой текстовое поле, в которое пользователь может ввести пароль. Введенный пароль не должен отображаться пользователю и должен быть зашифрован в виде «*».

Кнопка «Отправить». При нажатии этой кнопки проверяется, действительны ли введенные учетные данные. В случае, если имя пользователя или пароль неверны, система должна отобразить предупреждающее сообщение «Неверное имя пользователя или пароль»…

Документ FRS очень подробный и однозначно излагает все функциональные требования. В документе указываются все поля и взаимодействия пользователей на каждой странице разрабатываемого программного обеспечения. Кроме того, для облегчения понимания в FRS добавлены диаграммы потоков процессов, диаграммы UML и т.д..

Следует отметить, что документ FRS создается с точки зрения пользователя и описывает, как будет вести себя программное обеспечение при взаимодействии с ним внешнего пользователя. Команда разработчиков обращается к этому артефакту проекта, чтобы понять, что именно им нужно создать, а команда тестирования/ контроля качества — чтобы узнать, каковы различные сценарии и тестовые примеры, на которых должно быть протестировано программное обеспечение.

FRS готовится бизнес-аналитиком/системным аналитиком проекта, а после завершения проверяется руководителем проекта. После этого FRS передается клиентам для окончательного рассмотрения и утверждения. После утверждения FRS становится стандартным документом, который определяет, как должно функционировать программное обеспечение.

Основной аудиторией FRS являются технические руководители, группы разработчиков и группы тестирования.

РАЗРАБОТКА И УПРАВЛЕНИЕ ТРЕБОВАНИЯМИ

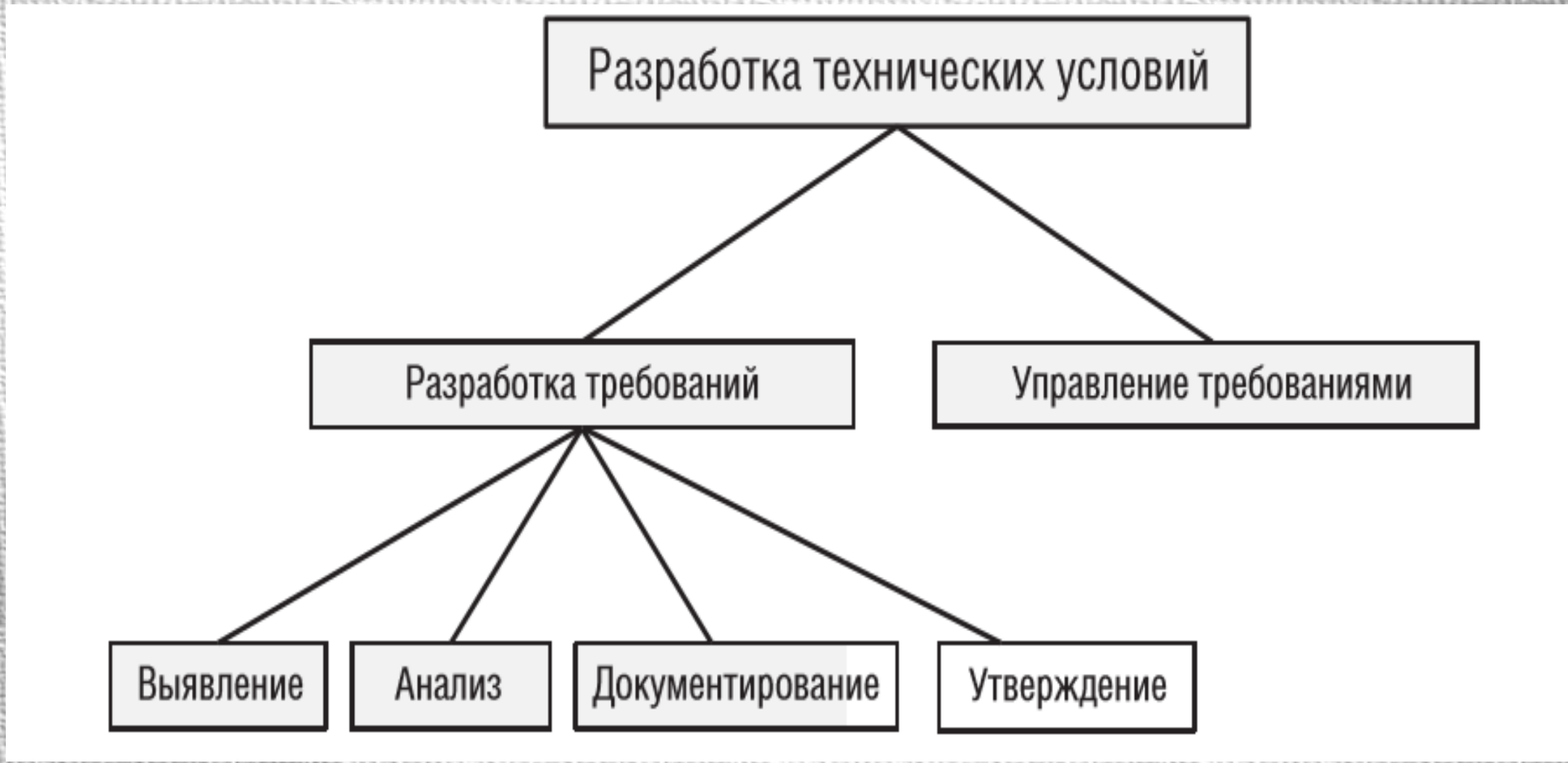
Основные этапы работы с требованиями

1. Планирование
2. Сбор требований (выявление)
3. Анализ требований
4. Документирование
5. Проверка требований
6. Управление требованиями

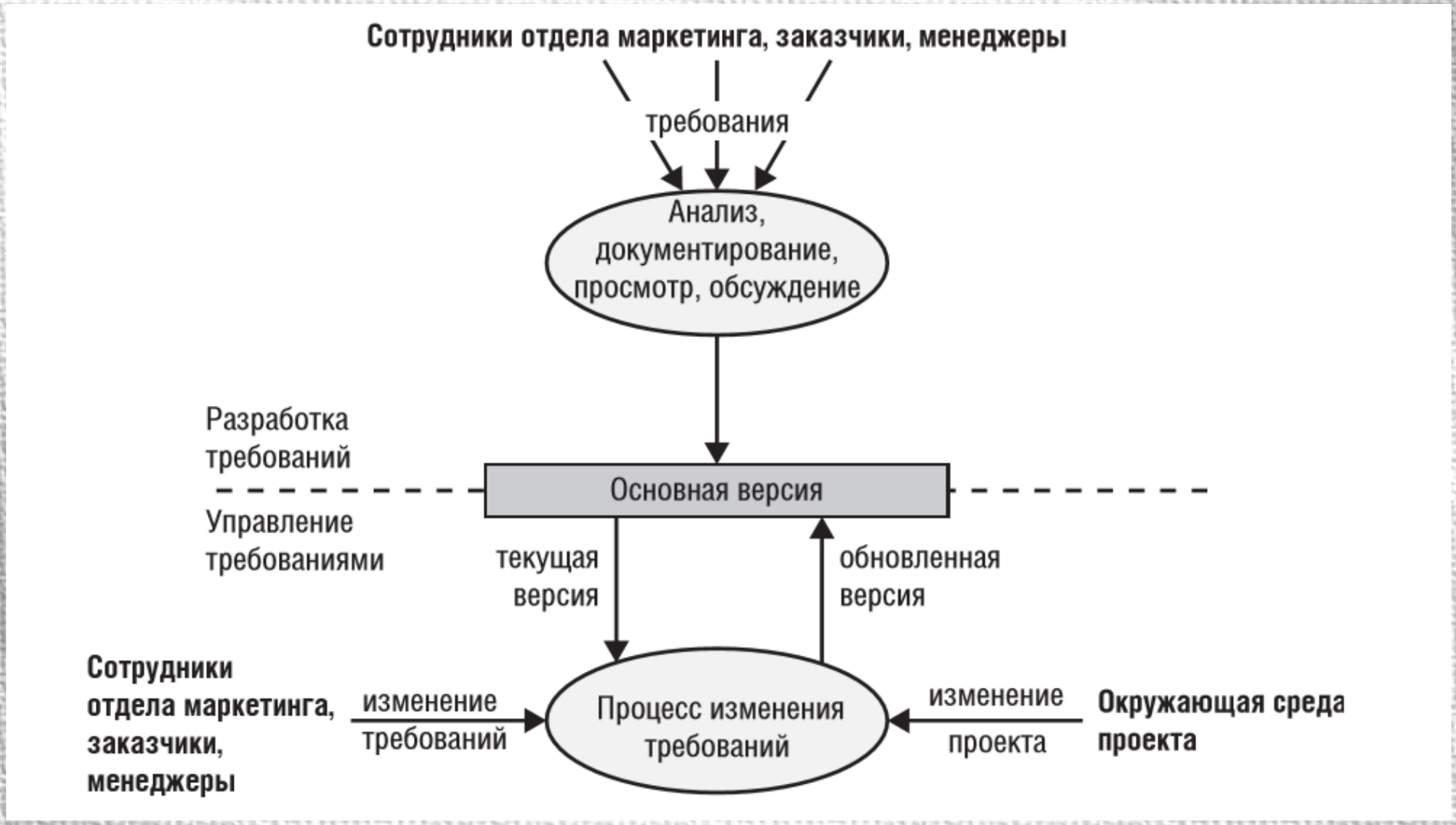
РАЗРАБОТКА ТЕХНИЧЕСКИХ УСЛОВИЙ

Некоторые авторы называют всю эту предметную область **разработкой технических условий**, вторые употребляют термин управление требованиями (requirements management), а третьи считают эту деятельность подмножеством более общей предметной области бизнес-анализа. Лучше всего разделить область разработки технических условий на

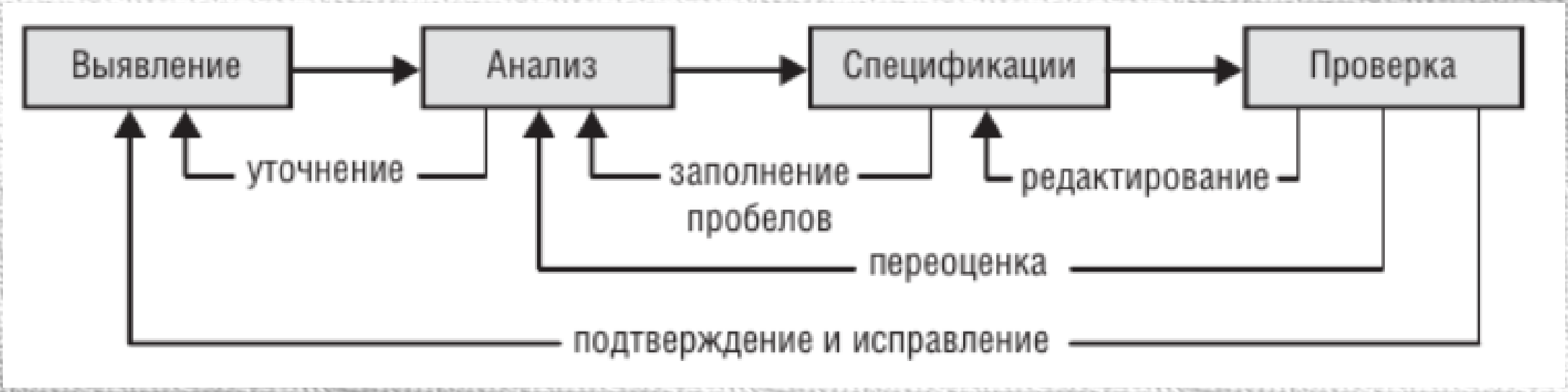
- **разработку требований** (requirements development) и
- **управление требованиями** (requirements management).


Разработка технических условий
Разработка требований
Управление требованиями
Выявление
Анализ
Документирование
Утверждение

Разделение областей разработки требований и управления ими (Вигерс)



Итеративный процесс формулирования требований



На практике эти действия выполняются попеременно, поэтапно и повторяются. «**Поступательная очистка деталей**» — вот, что должно быть основным девизом при разработке требований и переходе от начальных концепций к более точному пониманию и выражению.

Будучи аналитиком, вы будете задавать клиентам вопросы, слушать их ответы и наблюдать, что они делают (**этап выявления требований**). Вы обработаете эту информацию и разберетесь в ней, классифицируете ее на разные категории и свяжете потребности клиента с возможными программными требованиями (**этап анализа**). В результате анализа может оказаться, что нужно уточнить некоторые требования, поэтому вы возвращаетесь назад и дополняете набор требований. После этого вы структурируете информацию от пользователей и выведенные требования в виде письменных требований-утверждений и диаграмм (**спецификация**). При письменной фиксации может потребоваться вернуться назад и выполнить дополнительный анализ, чтобы закрыть пробелы в знаниях.

Затем вы просите ряд заинтересованных лиц подтвердить, что представленное вами точно и полно отражает потребности, и исправить ошибки (**проверка**). Это выполняется по отношению к набору требований, которые важнее и своевременнее всего для начала разработки ПО.

В процессе проверки может потребоваться переписать некоторые неясные требования, повторить некоторые действия анализа или даже вернуться и выполнить дополнительное выявление требований. После этого вы переходите к следующему этапу проекта и весь цикл повторяется. **Такой итеративный процесс продолжается на всем протяжении разработки требований и возможно — как в проектах гибкой разработки — на протяжении всего времени проекта**.

Примерный процесс создания требований

Из-за разнообразия проектов по разработке ПО и организационных культур единого, шаблонного подхода к созданию требований не существует. На рисунке показан каркас создания требований, который с разумными исправлениями подойдет для большинства проектов.

Итеративный процесс формулирования требований

Первые семь действий обычно однократно выполняются на ранних стадиях работы над проектом (тем не менее, команде разработчиков придется периодически проверять эти действия). Остальные необходимы для каждого очередного выпуска или этапа работы над проектом. Многие из этих действий могут выполняться итеративно и попеременно. Например, шаги 8, 9 и 10 можно выполнять небольшими порциями, выполняя пересмотр (шаг 12) после каждой итерации.

Разработка без управления требованиями – непредсказуемый результат

- Если требования плохие, то все остальные ваши усилия, процессы и инструменты лишь помогут вам как можно быстрей создать плохой продукт.
- Если вы еще только начинаете задумываться об улучшении ваших процессов, то помните, что начать стоит именно с процесса управления требованиями, потому что здесь действует простой принцип :

  что посеешь, то и пожнешь

- Если требования плохие, то все остальные ваши усилия, процессы и инструменты лишь помогут вам как можно быстрей создать **плохой продукт**.

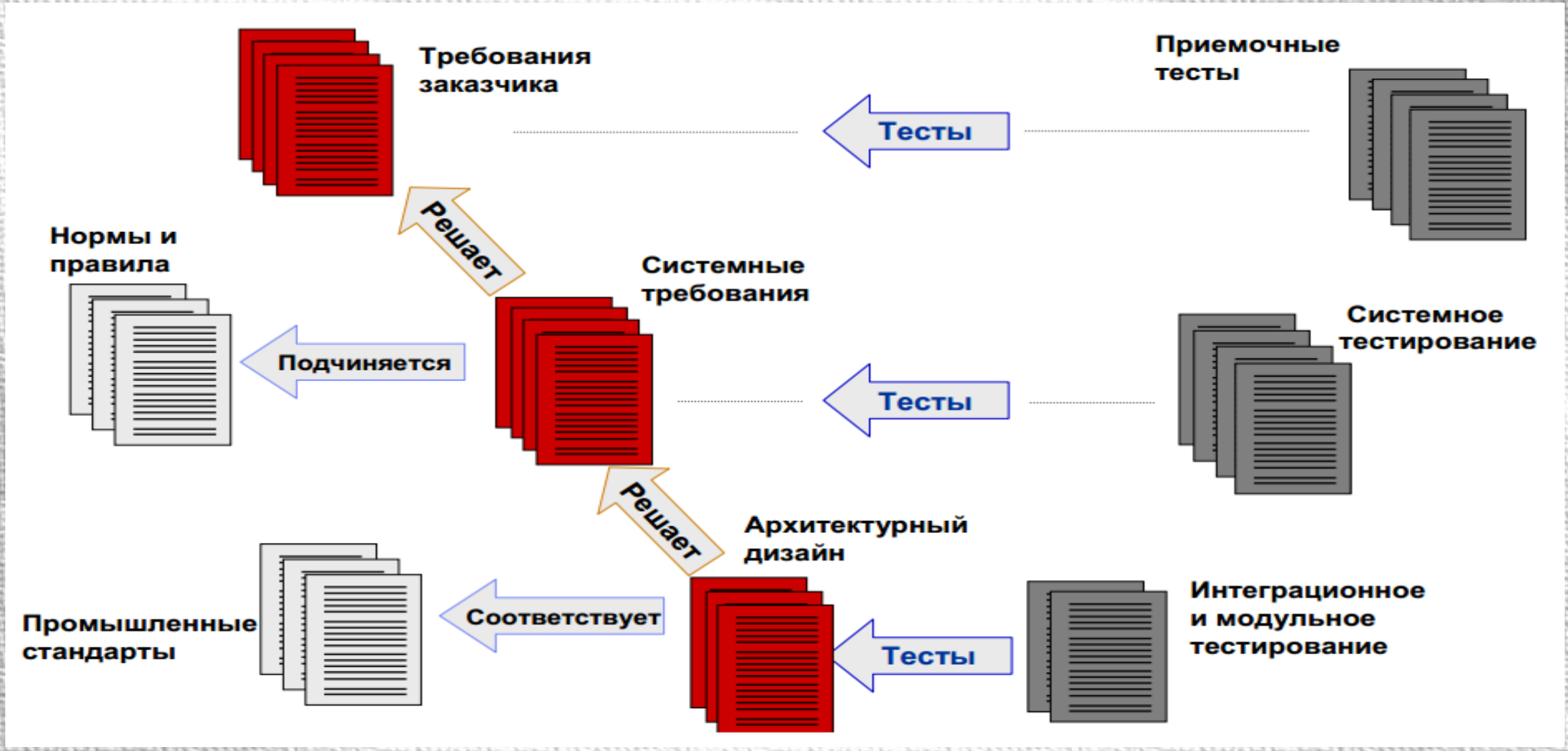
Вся наша жизнь – это работа с требованиями.... В БЫТУ

- Если жена звонит вам и просит по пути с работы домой купить – хлеб, сахар, масло, молоко и вино - то это ничто иное, как **сбор и формирование требований .**
- Если в разговоре выясняется, что она имела ввиду:
  - Хлеб белый – 1 батон
  - Масло растительное - 1 бутылка
  - Сахар-песок– 2 пачки
  - Молоко топленое – 1 литр

  то это ничто иное, как **декомпозиция и детализация требований**.
- Разговор заканчивается тем, что вы приходите к выводу, что вино покупать не стоит, потому что в гости придет друг, которому пить противопоказано ... - это ничто иное, как **работа с ограничениями или учет ограничивающих факторов.**

- Если вы решаете, что хлеб, масло и сахар вы купите в одном магазине, а вот молоко в другом - это выглядит как **структуризация требований**.
- Если чуть позже жена вновь звонит вам и говорит, что она передумала и просит вместо молока купить сметану, то это **изменение требований.**
- Если уже дома вы сверяете купленное с тем списком, что диктовала жена, - это **проверка реализации требований или тестирование**.
- И если обнаруживаете, что вместо сметаны вы все-таки купили молоко - это значит, что вы **не учли изменение**, ранее внесенное в одно из требований.

Вся наша жизнь – это работа с требованиями.... НА РАБОТЕ



Разработка технических условий
Планирование
Разработка требований
Управление требованиями

- Типы документов, в которых будут описываться требования.
- Типы требований, которые будут описаны в каждом документе.
- Атрибуты каждого типа требования.
- Значения, доступные для каждого типа требования

ПЛАНИРОВАНИЕ ТРЕБОВАНИЙ

Планирование

Процесс работы с требованиями начинается с планирования. На этапе планирования системный аналитик создает план управления требованиями и шаблоны необходимой документации. Планирование – первый шаг при работе с требованиями, он начинается на этапе предпроектного обследования.

План управления требованиями является одним из наиболее важных документов в процессе управления требованиями. В данном документе определяются типы требований и атрибуты каждого типа, отношения между требованиями, документы, использующиеся в данном процессе. Также системный аналитик определяет и заносит в план решение об использовании специального инструментального средства для управления требованиями. Расширенный вариант плана управления требованиями может содержать описание ролей, участвующих в процессе; задач, выполняемых каждой ролью, и другую служебную информацию.

На этапе планирования также создаются шаблоны необходимых документов:

- **глоссария,**
- **технического задания,**
- **документа-концепции.**

При работе с государственным заказчиком для описания требований чаще всего используется техническое задание, разработанное в соответствие с **ГОСТ 34.602-89** или **ГОСТ 19.201-78**. Если нет жестких требований со стороны заказчика на соответствие государственным стандартам, можно использовать спецификацию требований на основе стандарта **IEEE 830-1998.**

Современные инструментальные средства позволяют создавать автоматические отчеты с необходимой информацией. Если принято решение о том, что документация будет создаваться автоматически с использованием отчетов, на этапе планирования необходимо создать шаблоны таких отчетов. Шаблоны отчетов, также как и шаблоны документов, должны быть разработаны с учетов требований государственных стандартов.

Для чего необходимо планирование требований

Решение следующих проблем:

- Неучтенные требования
- Недостаточный уровень детализации требований для команды
- Смешанные уровни абстракции и типы требований
- Внезапное изменение требований
- Отсутствие опыта и навыков аналитической команды
- Сложность планирования и контроля аналитических задач
- Срыв сроков и разрастание бюджета проекта

Связь с другими областями разработки ПО

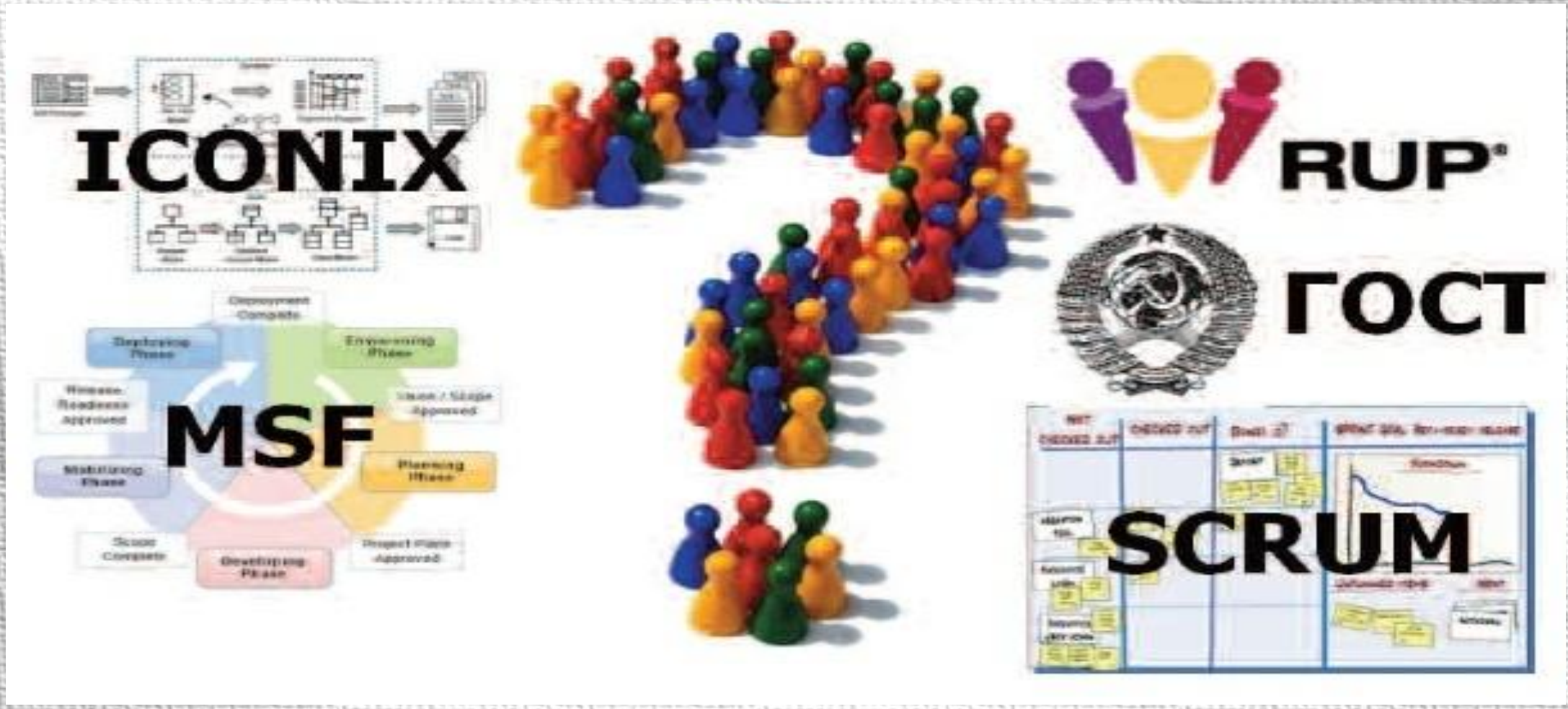
Виды деятельности при планировании требований

Планирование требований - это этап, на котором определяются:

*Мы планируем выход на пенсию

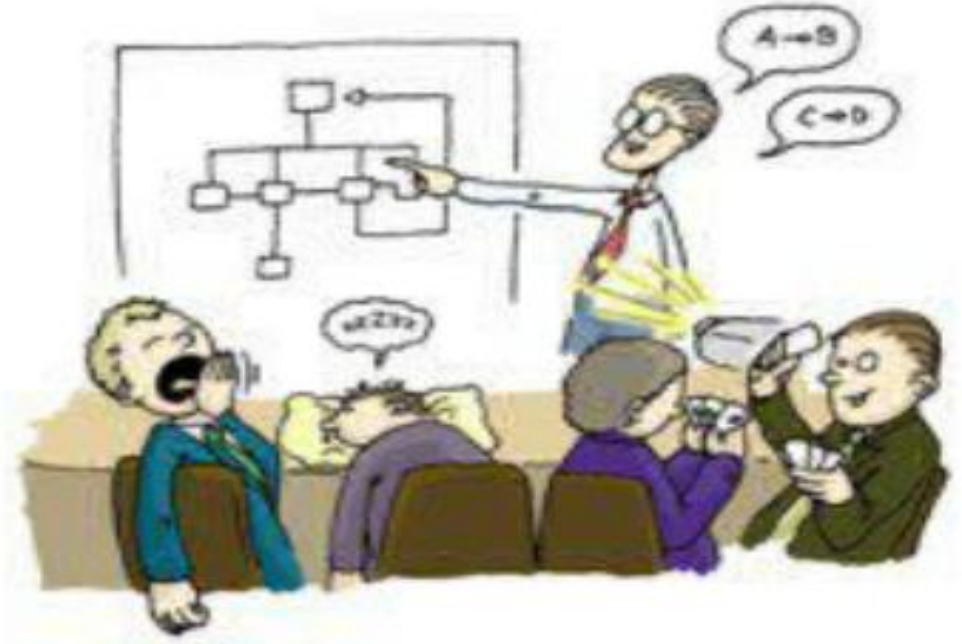
- Методология
- Аналитические процессы
- Регламенты работ
- Описание командных ролей
- Шаблоны аналитических документов
- План управления требованиями
- План работ группы аналитиков

1.Выбор методологии



2. Построение аналитического процесса

Процесс разработки требований



- Используйте методологию
- Адаптируйте процесс под ваш проект/компанию
- Учитывайте предпочтения заказчика
- Согласуйте процесс с командой
- Опишите процесс в виде текста и/или диаграмм

Процесс проверки качества требований

Определите и запишите:

- **Кто участвует в проверке**
- **В какой последовательности проверяют**
- **Количество итераций согласования одного документа**
- **Критерии качества требований**
- **Сроки согласования**
- **Протоколы замечаний**

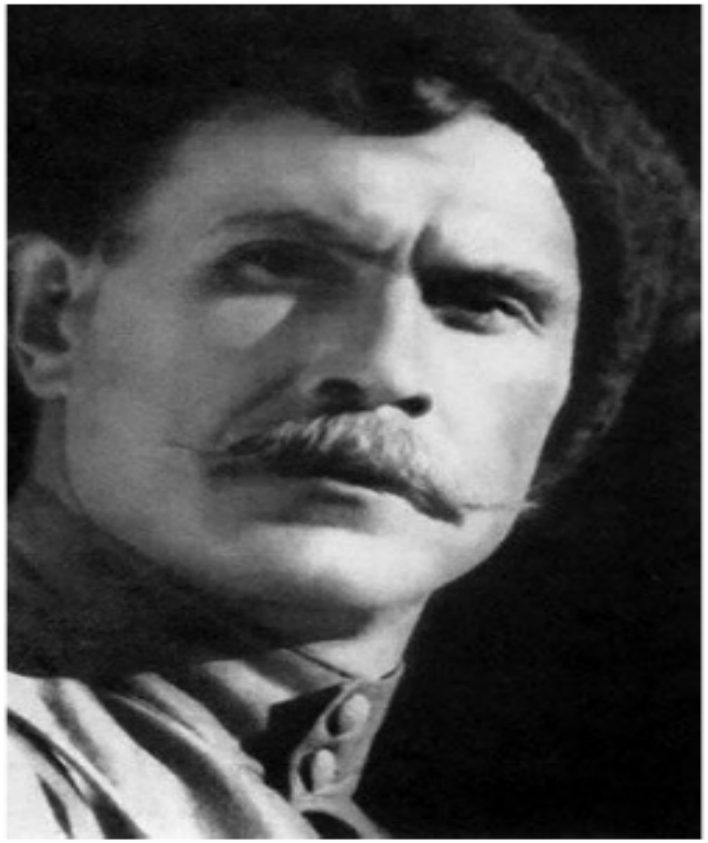
Процесс управления изменениями требований

Основные шаги:

- **Определите источник изменения**
- **Проанализируйте влияния изменений на проект**
- **Примите совместное решение**
- **Зафиксируйте изменения в требованиях**
- **Реализуйте изменения**
- **Протестируйте изменения**

3.Роли и обязанности

Распределение обязанностей, персонал и его навыки



Сотрудник	Навыки	Роль
Чапаев Василий Иванович	Мега мозг, он же Гуру. Знает все и имеет большой опыт руководства командой аналитиков.	Лидер аналитиков
Петька	Студент, мало знает, много учится, чтобы стать гуру. Хорошие аналитические способности	Системный аналитик
Анка	Опыт программирования 5 лет. Хорошо владеет знаниями русского языка	Системный аналитик
Фурманов	Имеет большой опыт в создании документации. Замечает все неточности и ошибки в документах. Имеет большой опыт в предметной области	Рецензент Эксперт предметной области

Лидер аналитиков

- Распределяет аналитические задачи
- Контролирует выполнение задач
- Расставляет приоритеты для требований

Бизнес-аналитик

- Описывает бизнес процессы заказчика
- Подготавливает модель предметной области

Аналитик

В целом, аналитики бывают с приставками «бизнес-» и «системный-». И в том, и в другом случае продукт такой роли - это документация (*техническое задание*), но с разным уровнем детализации. Если не вдаваться в подробности, то бизнес-аналитик разрабатывает ТЗ на уровне связей бизнес-требование – описание функции продукта, а системный аналитик на уровне функция продукта – принцип технической реализации. То есть, системный аналитик «копает глубже», является более специализированной ролью, и необходимость в этой роли возникает только в достаточно крупных проектах. А задачи бизнес-анализа встречаются во всех проектах без исключений.

Упрощенно – бизнес-аналитик занимается тем, что выясняет у заказчика, что же тому нужно и переводит полученную информацию на язык, понятный разработчикам.

Бизнес-аналитик

Бизнес-аналитик отвечает за построение модели предметной области (онтологии) предприятия, бизнес-процессы которого необходимо автоматизировать с помощью разрабатываемой системы. Бизнес-аналитик занимается выявлением «бизнес требований» и др. информации, а именно:

- идентификацией заинтересованных лиц
- изучением внешней и внутренней нормативной документации (законов, методик, распоряжений, стандартов, регламентов и т.д.)
- подготовкой и проведением интервью с экспертами предметной области и заказчиками.

- подготовкой и проведением анкетирования и опросов экспертов предметной области и пользователей будущей системы.

Бизнес-аналитик формализует и систематизирует собранную информацию. Он создает словарь терминов (глоссарий) предметной области, описывает организационно-штатную структуру предприятия, **создает концептуальные функциональные модели деятельности, модели сущностей и связей (ERD), модели потоков данных, модели бизнес-процессов (как есть).** Также бизнес-аналитик фиксирует ограничения бизнеса и бизнес-правила.

Бизнес-аналитик — это одна из ролей участников проекта, а не обязательно название должности. Их также называют

- аналитиками по требованиям,
- системными аналитиками,
- инженерами по требованиям,
- менеджерами по требованиям,
- прикладными аналитиками,
- аналитиками бизнес-систем,
- ИТ- аналитиками
- и просто аналитиками.

Системный аналитик

Аналитик требований, или системный аналитик, отвечает за сбор и управление функциональными требованиями, характеристиками качества и ограничениями к создаваемой системе. Системный аналитик разрабатывает план управления требований, в котором определяет тип требований, их атрибуты, модели и документы, которые будут использоваться при анализе и документировании требований.

Системный аналитик отвечает за формирование технических требований к разрабатываемой системе. После сбора требований системный аналитик проверяет их на непротиворечивость и дублирование, определяет приоритеты, устанавливает отношения прослеживаемости между требованиями. Системный аналитик также разрабатывает варианты использования (UC) системы.

Роли системного аналитика

В зависимости от специфики проекта и его размеров системный аналитик может выполнять одну или несколько проектных ролей, начиная с бизнес-аналитика и заканчивая системным архитектором.

Взаимосвязи системного аналитика с представителями заказчика и проектной командой

Спецификатор требований

Спецификатор требований совместно с системным аналитиком разрабатывает спецификацию требований, детализирует варианты использования и другие требования.

Системный архитектор

Системный архитектор отвечает за разработку технической части концепции создания системы. Системный архитектор разрабатывает варианты концептуальной архитектуры системы, определяет место системы в линейке продуктов организации, порядок взаимодействия с собственными и сторонними продуктами, определяет элементы архитектуры системы, разрабатывает логические модели данных. Системный архитектор также может участвовать в разработке или разрабатывать самостоятельно технико-экономическое обоснование выбранного варианта концепции.

Технический писатель

Технический писатель разрабатывает документацию на информационную систему согласно российским и международным стандартам.

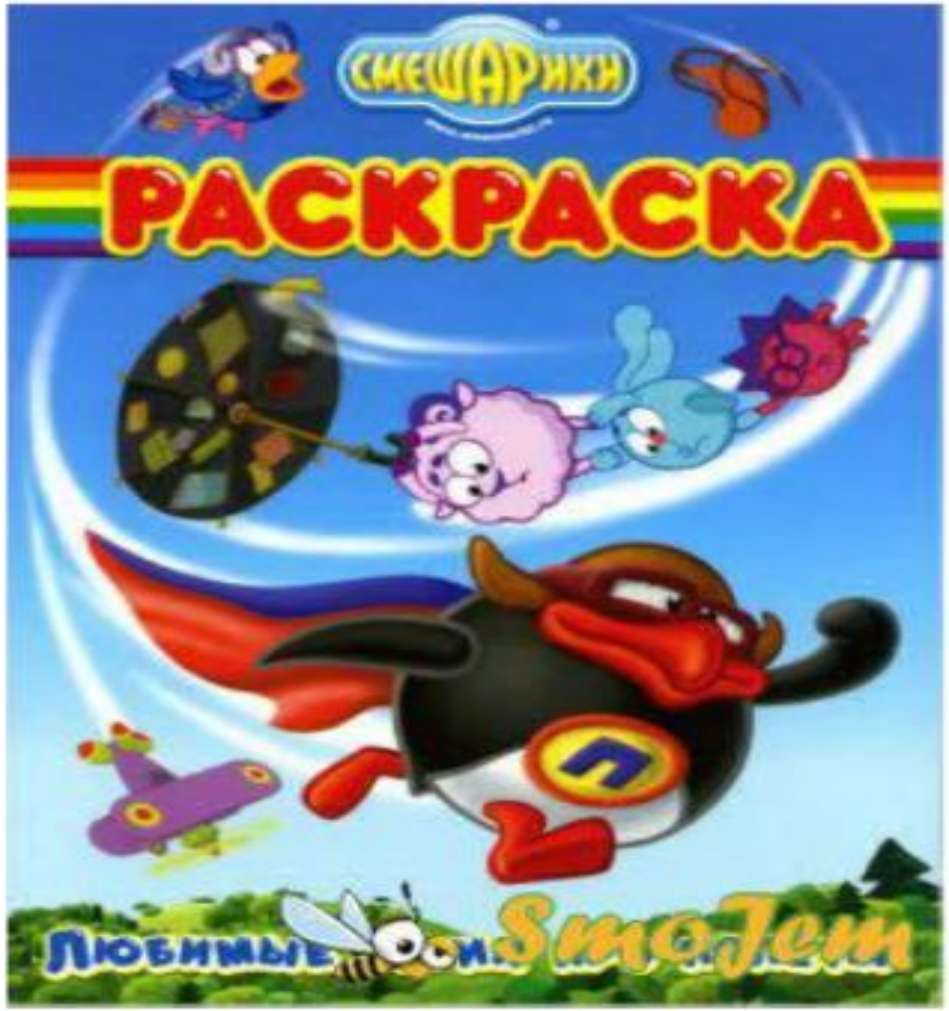
Коммуникатор

Коммуникатор отвечает за организацию, проведение, моделирование и протоколирование совещаний, фокус-групп и мозговых штурмов.

БА vs СА

Бизнес Аналитик	Системный Аналитик
• Знание (изучение) Пр. Обл. • Анализ структуры Орг. • Участие в Стратегии • Выявление З. Л. • Описание БП • Выявление Целей • Выявление Проблем • Выявление Потребностей • Оптимизация БП • Формирование Задач ПО	• Изучение Пр. Обл. • Формулирование Задач ПО • Формулирование ПТ • Изучение ПО конкурентов • Выявление Пользователей • Формулирование ФТ и БПр • Формулирование НеФТ • Участие в разработке Арх. • Участие в Тестировании • Участие во Внедрении

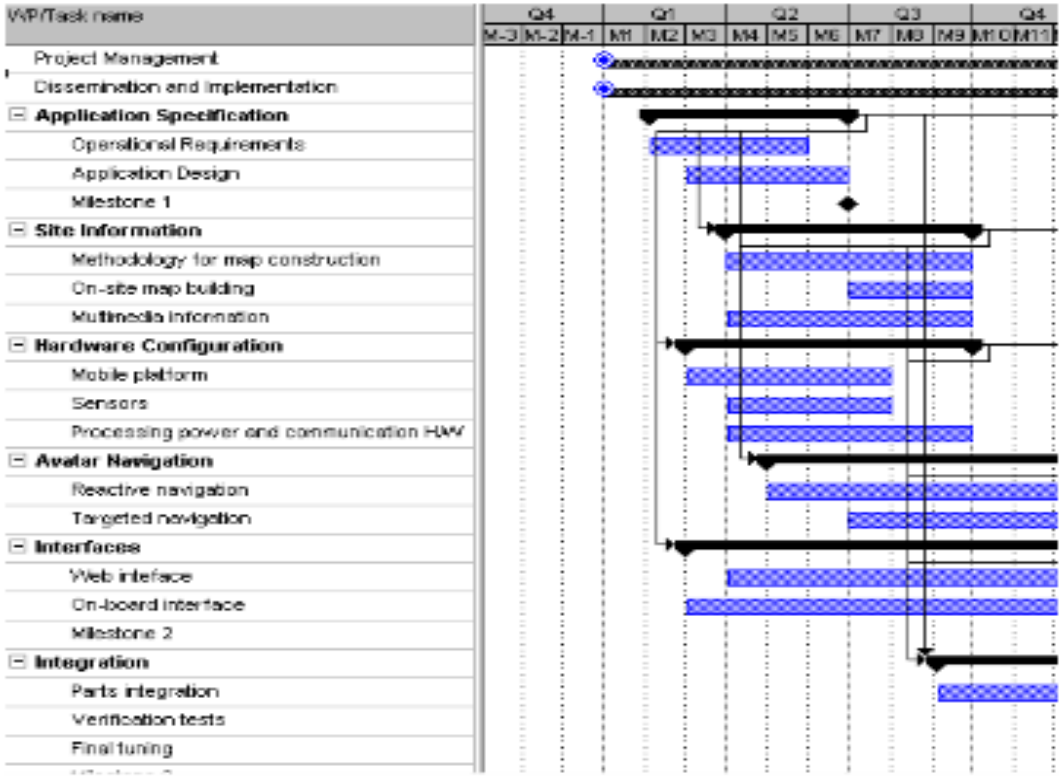
4. Шаблоны документов



Рекомендации:

- **Используйте шаблоны из методологий**
- **Адаптируйте шаблоны под ваш процесс**
- **Убедись в том, что все разделы документа «покрыты» процессом**
- **Используйте принцип «сверху-вниз» при размещении разделов документа**

5. План управления проектом



- **Составление сметы по аналитическим работам**
- **Оценка трудозатрат**
- **Распределение задач**
- **Идентификация контрольных точек**
- **Набор команды аналитиков**

6. План управления требованиями

План разработки и управления требованиями

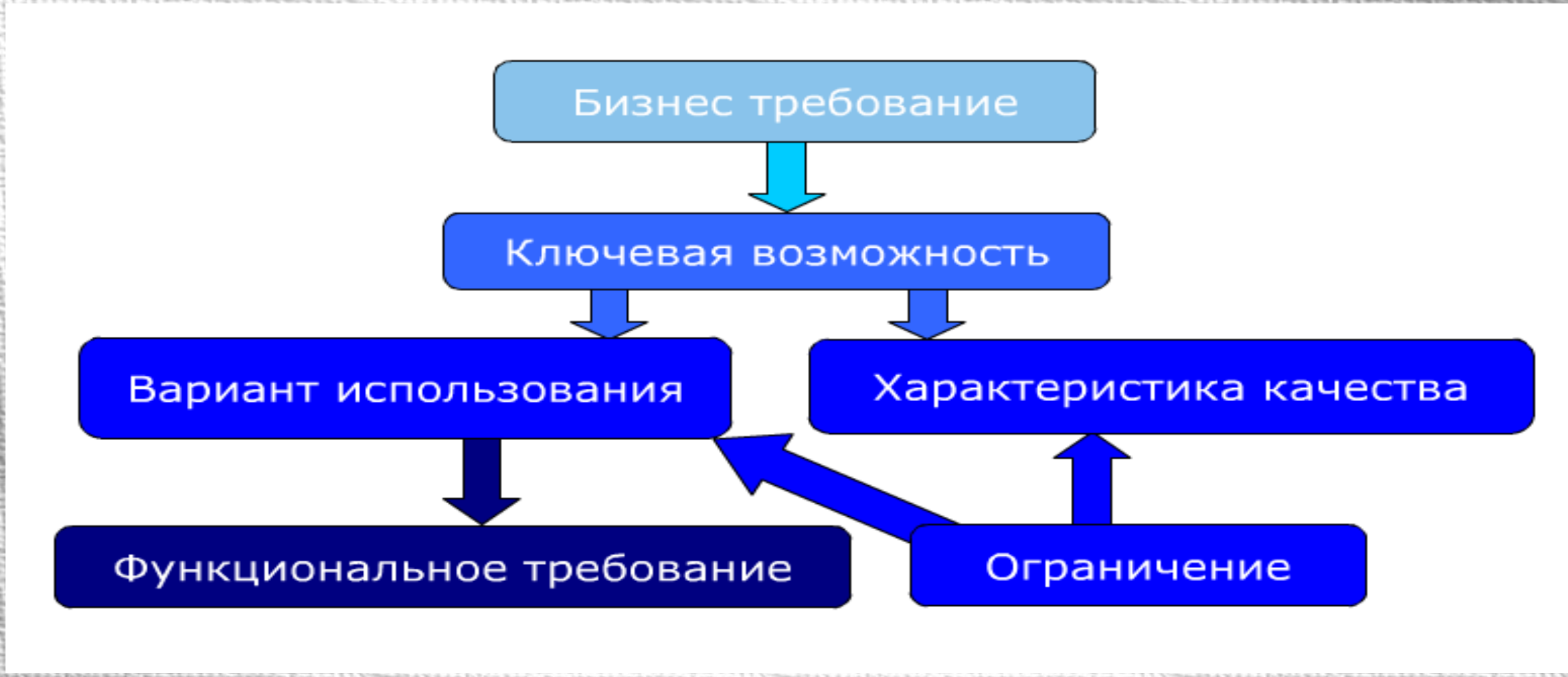
Основные разделы:

- **Список заинтересованных лиц**
- **Типы требований**
- **Трассировка требований**
- **Атрибуты требований**
- **Жизненный цикл требований**
- **Инструменты и ПО**
- **Отчеты и метрики**
- **Обучения и тренинги**

Список заинтересованных лиц

ФИО	Должность	Контактная информация
Василий Теркин	Генеральный директор	666-13-13
Федот Стрелец	Руководитель отдела внедрения	strelets@customer.com
Максим Перепелица	Маркетолог	perepelitsa@customer.com

Типы требований и трассировка



Пирамида Требований

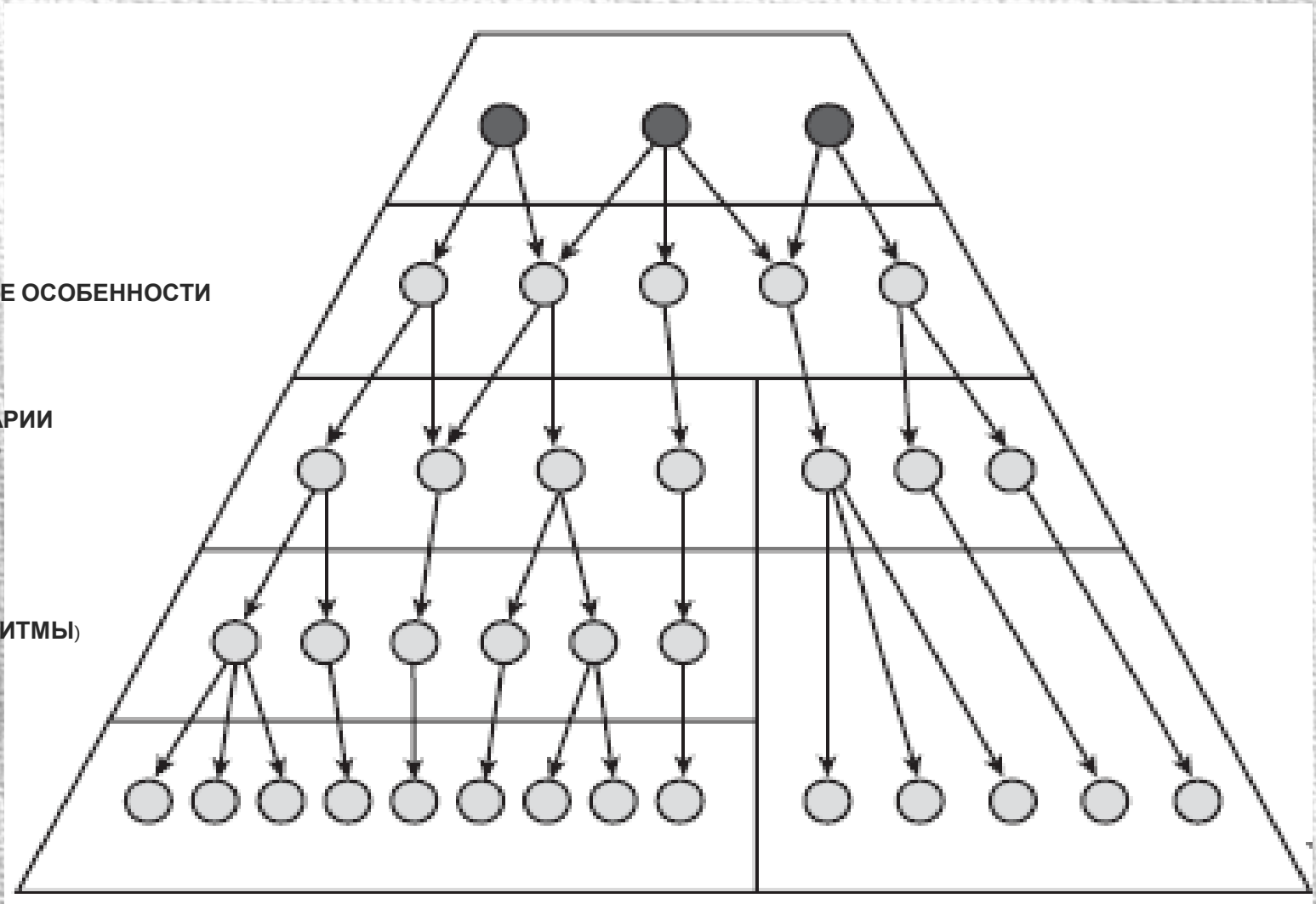
ПОТРЕБНОСТИ

ФУНКЦИОНАЛЬНЫЕ ОСОБЕННОСТИ

USE CASES (СЦЕНАРИИ ИСПОЛЬЗОВАНИЯ)

СЦЕНАРИИ (АЛГОРИТМЫ)

ТЕСТОВЫЕ СЦЕНАРИИ



ДОПОЛНИТЕЛЬНЫЕ ТРЕБОВАНИЯ

ТЕСТОВЫЕ СЦЕНАРИИ

Главное отличие между потребностями и функциональными особенностями – в источнике требований. Потребности поступают от заинтересованных лиц, а функциональные особенности формулируются бизнес-аналитиками.

Атрибуты требований

- **Уникальный идентификатор**
- **Приоритет**
- **Статус**
- **Версия ПО (build)**
- **Трудоемкость**
- **Другие**

Инструменты и программное обеспечение

- Система управления требованиями
- Средство моделирования требований
- Средства коммуникации

Отчеты и метрики

- Матрицы трассировок
- Выборки требований по критериям
- Статистические данные по требованиям
- Другие отчеты

Обучение и тренинги

- План обучения команды
- Индивидуальный план обучения аналитика
- Обмен опытом внутри команды
- Семинары и конференции

Выводы

Когда процесс запущен…

- Используйте результаты планирования на протяжении всего жизненного цикла проекта
- Соблюдайте описанные процессы и регламенты
- Ищите слабые места в процессах и улучшайте их
- Присваивайте артефактам уникальные идентификаторы
- Используйте трассировку для идентификации неучтенных требований
- Управляйте границами системы

Выводы

Планируйте работу с требованиями, даже если вы единственный аналитик на проекте

Выводы

Мы перечислили круг вопросов, которые должны быть освещены в плане управления требованиями. В зависимости от проекта некоторые из них могут меняться. Основная информация, которая должна быть зафиксирована в плане

- Типы документов, в которых будут описываться требования.
- Типы требований, которые будут описаны в каждом документе.
- Атрибуты каждого типа требования.
- Значения, доступные для каждого типа требования.

Один раз создав план управления требованиями для конкретного проекта вы сможете использовать этот документ как шаблон для будущих проектов, поскольку многие решения будут актуальны

ИСТОЧНИК

Планирование требований

Виталий
Григораш

Skype: vitaliy.grigorash

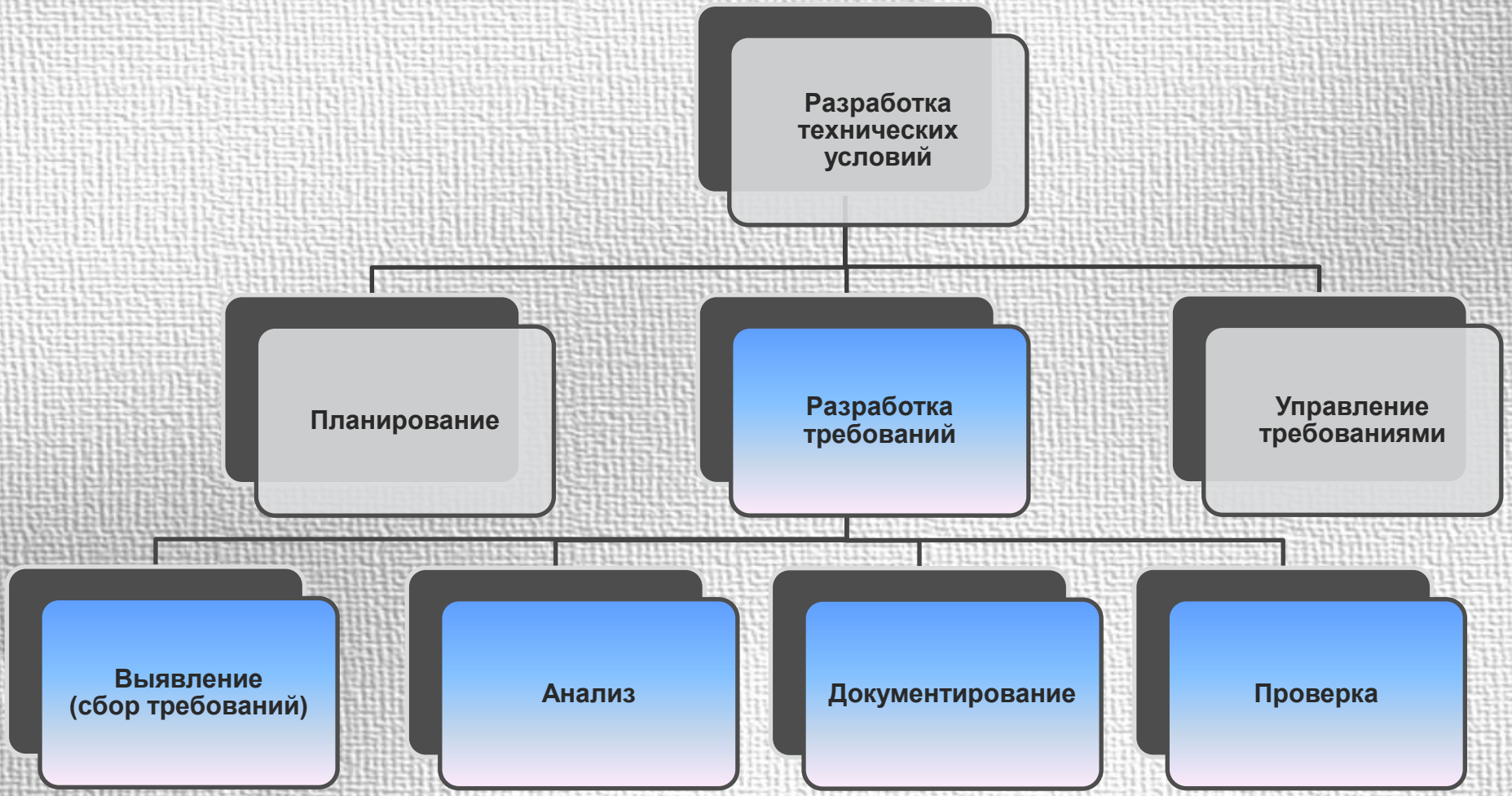
Email: vitaly.grigorash@gmail.com

http://vgrigorash.moikrug.ru

http://www.uml2.ru

- Первичный сбор требований (выявление).
- Формирование и анализ требований.
- Документирование (описание) требований.
- Проверка (утверждение) требований.

РАЗРАБОТКА ТРЕБОВАНИЙ


Разработка технических условий
Планирование
Разработка требований
Управление требованиями
Выявление (сбор требований)
Анализ
Документирование
Проверка

Дело совсем не в том, что они не видят решения проблемы. Дело в том, что они не видят саму проблему.

Гилберт Кит Честертон (Gilbert Keith Chesterton),. писатель, 1874-1936.

Предназначение любой системы или любого продукта это позволить человеку или другому устройству выполнять требуемую функцию. Вот эта самая необходимость «обеспечить функционирование системы» и есть ключевая идея разработки требований в области проблем.

Столкнувшись с необходимостью сбора требований, возможно, вам захочется спросить потенциального пользователя:

Что вы хотите, чтобы эта система делала ?

Часть потенциальных пользователей этот вопрос поставит в тупик, а у другой части пользователей найдется мало что ответить.

Если ваши потенциальные пользователи уже используют какие-либо системы, то у них обычно существуют идеи на тему как их улучшить. В случае если никаких систем у потенциальных пользователей сейчас нет, то и почвы для подобных идей у них тоже нет.

Для того чтобы в самом начале избежать подобных прыжков в область решений, необходимо задать другой вопрос:.

Зачем вам нужна эта система?

или

Какие задачи должна решать необходимая вам система?

Как только речь заходит о задачах, которые должна будет решать рассматриваемая система, люди сразу же начинают думать о том, что они будут делать с помощью системы, вместо того чтобы думать о том, как они это будут делать.

Однако все то, что люди хотят получить с помощью системы, может быть сформулировано даже без описания возможных решений. И это весьма важный момент, потому как, избегая уклона в сторону решения, мы оставляем пространство для работы архитекторов и системных инженеров.

Вот почему некоторые специалисты даже настаивают на том, что слово «система» должно быть полностью исключено из предыдущего вопроса, а сам вопрос должен звучать следующим образом:

Что вы хотите, что бы вы могли делать?

Ответ на этот вопрос должен начинаться фразой:.

Я хотел бы иметь возможность ...

Именно такой ответ является требованием типа «**возможность**» (описывающим возможность). И именно этот тип требований является ключевым в области проблем.

Таким образом, установив, что при разработке требований в области проблем первичной задачей является выявление возможностей, мы формулируем для самих себя следующий вопрос:

А кого, собственно, нужно опрашивать?

Поиск ответа на этот вопрос приводит нас к необходимости идентифицировать некую группу людей, которую мы будем называть «**заинтересованные стороны**» (stakeholders). Под термином «заинтересованные стороны» могут пониматься либо непосредственно люди, либо организации, прямо или косвенно заинтересованные в существовании системы.

Цикл работы с требованиями

Процесс разработки требований включает четыре основных этапа.

1. Сбор требований, нацеленный на *выявление* всех возможных источников требований, ограничений на работу системы и *извлечение* требований из этих источников.
2. Формирование и анализ требований, целью которого является обнаружение и устранение противоречий и неоднозначностей в требованиях, их уточнение и систематизация.

3. Документирование (описание) требований. В результате этой *деятельности* требования должны быть оформлены в виде структурированного набора документов и моделей, который может *систематически* анализироваться, оцениваться с разных *позиций* и в итоге должен быть утвержден как официальная формулировка требований к системе.
4. Проверка (утверждение) требований решает задачу оценки понятности сформулированных требований и их характеристик, необходимых, чтобы разрабатывать ПО на их основе, в первую очередь, непротиворечивости и полноты, а также соответствия корпоративным стандартам на техническую документацию.

1

Сбор требований

Первым и важнейшим этапом SDLC, или жизненного цикла разработки программного обеспечения, является сбор требований.

Сбор требований — это один из самых важных этапов процесса создания любой информационной системы, будь то десктопное, веб или мобильное приложение или же просто доработка уже существующего решения. Прежде, чем начать собирать требования, необходимо выявить всех заинтересованных лиц (стейкхолдеров), которые будут пользоваться системой. Чем точнее будет этот список, тем полнее будут требования.

Сбор требований - это процесс понимания того, что необходимо разработать, и причины, стоящие за разработкой продукта или услуг. По сути, ваша роль как бизнес-аналитика заключается в понимании болевых точек клиента и проблем, с которыми они сталкиваются в текущей среде. Таким образом, понимание того, почему они хотят создать продукт или услугу на месте. Определение объема и целей требований к проекту на ранней стадии инициирования проекта может помочь вам решать факторы риска и проблемы гораздо более интуитивно, чем без надлежащего понимания.

Кто отвечает за сбор требований?

Бизнес-аналитики, системные аналитики, владельцы продуктов и менеджеры проектов в основном отвечают за встречи с заинтересованными сторонами и клиентами и краткое изложение требований, целей и объема проекта.

Выявление и извлечение

Эта деятельность нацелена на **выявление** полного списка требований, их **извлечение** из всех доступных источников. Она состоит из более мелких деятельностей.

Определение источников требований. В его ходе выявляются все заинтересованные лица, документы, эксперты, существующие системы, которые могут служить источниками информации для составления требований к рассматриваемой системе, а также источниками ограничений на ее работу.

Извлечение требований — получение формулировок требований и ограничений из бесед с экспертами или заинтересованными лицами, наблюдением за работой пользователей, анализа документов, отраслевых и корпоративных стандартов, реинжиниринга существующих систем и анализа интерфейсов систем, с которыми придется взаимодействовать, и т.д.

Совет:

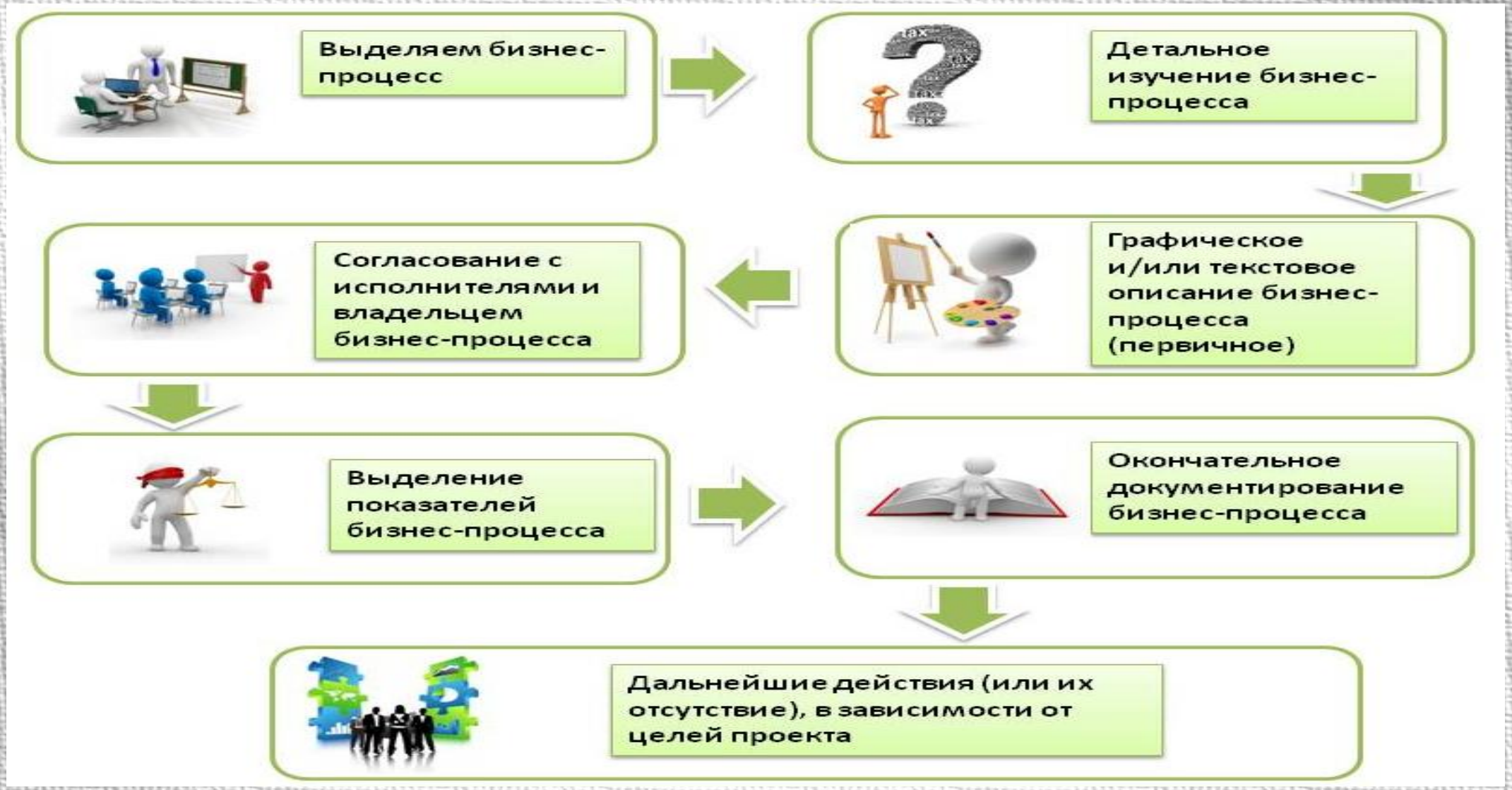
- Запросите существующие регламенты в полном объеме и оцените, которые из них несут полезную информацию для проекта. Изучите их подробно.
- Положения об учетной политике также несут много полезной информации. Они могут содержать особенности учета, не поддерживаемые планируемым к внедрению программным продуктом.

1. Сбор бизнес-требований

Первичный сбор требований (gathering) является первым шагом в процессе создания требований. Сбор требований следует начинать с бизнес-требований, так как все остальные виды требований подчинены им. Основной задачей этапа сбора бизнес-требований является выработка ***образа продукта*** (*product vision*). ***Образ продукта*** **описывает, что сейчас представляет собой продукт и каким он станет впоследствии**.

Ниже на примере бизнес процессов рассмотрена последователь-ность работ при их описании.

Последовательность работ при описании бизнес-процессов



Что и как делать

Шаги	Что и как делать
Выделяем бизнес-процесс	Из общего перечня бизнес-процессов, полученного на предыдущих этапах, выделяем один (по приоритету) для детальной проработки. С остальными затем поступаем аналогично.
Детальное изучение бизнес-процесса	Выделенный бизнес-процесс подвергаем детальному изучению: анализируем полученные первичные документы, отчеты и их структуру, используемые в процессе программы, различные файлы (например, Excel), разговариваем с конечными исполнителями. Собираем различные идеи о том, как можно улучшить процесс. Очень полезно, если удастся понаблюдать за процессом именно в тех условиях, в которых он выполняется (не многие любят, когда за ними наблюдают, но что делать)

Графическое и/или текстовое описание бизнес-процесса (первичное)

Полученную подробную информацию начинаем описывать. Прежде чем описывать процесс, надо определиться, потребует ли он графического описания. Если процесс простой и понятный, функций в нем мало, и, графическое представление не улучшит его понимание или восприятие, то не надо тратить на это время. В этом случае достаточно описать его в текстовом виде в табличной форме. Если же процесс сложный, с различными логическими условиями, то лучше привести его графическую схему. Диаграммы всегда воспринимаются легче. Если Вы решили описать процесс в графическом виде, это вовсе не означает, что не надо приводить его текстовое описание. Т.е. текстовое описание процесса должно быть в любом случае, причем выполненное по одинаковой схеме. Удобно это делать в виде таблицы, в которой указать: исполнителей каждого шага, какую информацию они получают на входе, описание каждого шага, какую информацию формируют на выходе. Ниже мы посмотрим на примере, как это может выглядеть.

Согласование с исполнителями и владельцем бизнес-процесса

Лучший способ понять, насколько удачно вы выбрали стиль изложения информации, это показать результат пользователям (исполнителям) процесса. На самое главное в такой демонстрации это понимание того, насколько правильно Вы поняли, как процесс выполняется. Если обучение проектной команды прошло успешно, то можно ожидать от исполнителей вполне адекватной обратной связи. А если им станет интересно, то продвигаться все начнет гораздо быстрее. Выявленные уточнения и несоответствия необходимо отразить в описании (актуализировать), при необходимости операцию повторить.

Выделение показателей бизнес-процесса

После того, как выработано правильное понимание, как выполняется бизнес-процесс, надо подумать над показателями, которыми можно измерить качество или скорость выполнения процесса. **Это не просто, но необходимо. Показатель должен быть измеряемым, т.е. выражен в числовом выражении и должен существовать простой способ эту величину получить.** Если измеряемый показатель выделить невозможно, есть риск того, что бизнес-процесс выделен неудачно. Кроме того, не будет возможности понять (измерить ведь нельзя), приведут ли изменения процесса к его улучшению или нет.

Окончательное документирование бизнес-процесса

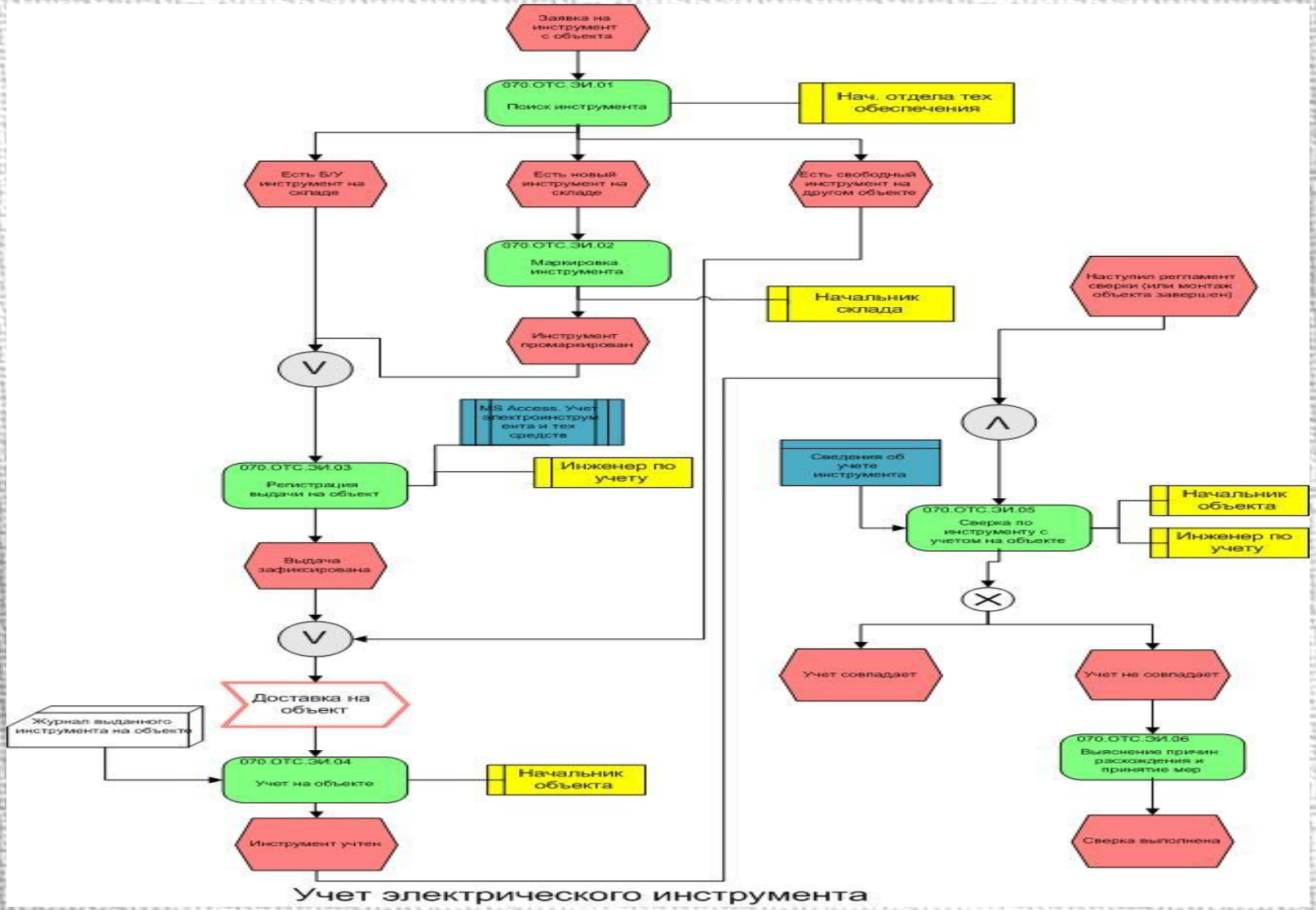
После того, как мы убедились в правильном понимании, как процесс выполняется (или должен выполняться), можно включать его в документацию.

Дальнейшие действия (или их отсутствие), в зависимости от целей проекта

Дальше возможны варианты:

- рассматриваемые процессы будут анализироваться и оптимизироваться, разрабатываться должностные инструкции, приниматься решения о необходимости автоматизации отдельных процессов и т.д.
- это может быть и отдельный проект: описание бизнес-процессов.

Описанный бизнес-процесс в нотации EPC



Учет электрического инструмента

Пример описания бизнес-процесса

Бизнес-процесс:	*Учет электрического инструмента*		070.ОТС.ЭИ
Функции процесса:			
Наименование	*Описание*		*Номер на схеме*
Поиск инструмента	При поступлении заявки с объекта на обеспечение электрическим инструментом, выясняется, есть ли свободный инструмент на складе. Если на складе необходимого инструмента нет, проверяется, нет ли незагруженного инструмента на других объектах (посредством обзвона начальников объектов). Если инструмента недостаточно, делается заявка на закупку в отдел комплектации		070.ОТС.ЭИ01
Маркировка инструмента	Если инструмент новый (ни разу не был в работе и не маркировался), начальник склада выполняет маркировку электрического инструмента (прожиг номера на корпусе). Номер совпадает с номером учета по базе данных учета электрического инструмента		070.ОТС.ЭИ02
Регистрация выдачи на объект	При любых операциях перемещения электрического инструмента между объектами, в базе данных делается отметка. Таким образом, электрический инструмент учитывается по объектам.		070.ОТС.ЭИ03
Учет на объекте	На объекте за сохранность инструмента отвечает начальник объекта, и организует учет самостоятельно. При выдаче в работу монтажникам они расписываются в получении в специальном журнале. Неиспользуемый инструмент начальник объекта должен сдавать на склад		070.ОТС.ЭИ04
Сверка по инструменту с учетом на объекте	Для контроля сохранности инструмента на объекте, инженер по учету в отделе технического обеспечения периодически организует сверку с учетом на объекте. Для этого запрашивается информация об инструменте на объекте (высылается в MS Excel), и выполняется сверка с учетом в базе данных.		070.ОТС.ЭИ05
Выяснение причин расхождение и принятие мер	В случае расхождений по учету между отделом тех. обеспечения и учетом на объекте, выясняются причины (кража, ошибка, и.пр.)		070.ОТС.ЭИ06
Показатели процесса:			
Наименование	*Способ оценки / измерения*		
Стоимость утерянного инструмента	Рыночная стоимость		
Коэффициент загрузки инструмента	Отношение времени хранения на складе или объекте к времени использования		

2. Сбор пользовательских требований

Следующим этапом является сбор пользовательских требований, другими словами вам необходимо выяснить чего ожидают от вас непосредственные пользователи. Естественно, что пользовательские требования должны быть жестко подчинены бизнес требованиям.

Практически любая информационная система имеет несколько категорий пользователей. Очень важно проанализировать потребности каждой из таких групп, достигнув компромисса между противоречащими требованиями различных групп пользователей. Необходимо четко определить приоритетность каждой группы и при разрешении противоречий между требованиями в первую очередь заботится об удовлетворении наиболее приоритетных групп.

Наиболее удачным подходом является оформление пользовательских требований в виде вариантов использования.

Источники требований

1. Основным источником требований к информационной системе, безусловно, являются соображения, высказанные представителями Заказчика.
2. **Артефакты**, описывающие предметную область. Это могут быть
   - документы с описанием бизнес-процессов предприятия, выполненные консалтинговым агентством,
   - либо просто документы (должностные инструкции, распоряжения, своды бизнес-правил), принятые на предприятии.
3. Так называемые "**лучшие практики**", широко используемые в настоящее время в бизнес-консалтинге и при внедрении корпоративных информационных систем. Лучшие практики представляют собой описания моделей деятельности успешных компаний отрасли, используемые длительное время в сотнях и тысячах компаний по всему миру.

4. Федеральное и муниципальное отраслевое законодательство (конституция, законы, распоряжения)
5. Нормативное обеспечение организации (регламенты, положения, уставы, приказы)
6. Текущая организация деятельности объекта автоматизации
7. Модели деятельности (диаграммы бизнес-процессов)
8. Представления и ожидания потребителей и пользователей системы
9. Журналы использования существующих программно-аппаратных систем
10. Конкурирующие программные продукты

Анализ проблемы

ПРОБЛЕМА – ИССЛЕДОВАНИЕ - РЕШЕНИЕ ПРОБЛЕМЫ

Познание мира людьми идет по схеме: "**Проблема – исследование - решение проблемы**"; постановка новой проблемы - исследование - решение и т.д.

ПРОБЛЕМА - это задача /вопрос/, требующая решения. Более полное определение проблемы выглядит примерно так: проблема - это задача, суть которой в основном заключается в противоречии между существующими представлениями о процессе, явлении, веществе, предмете, событии и т.д. и реальными фактами, обнаруженными в действительности опытным путем, или благодаря более глубокому анализу рассматриваемого объекта.

Область проблем и область решений

Анализ проблемы на начальной стадии выявления требований

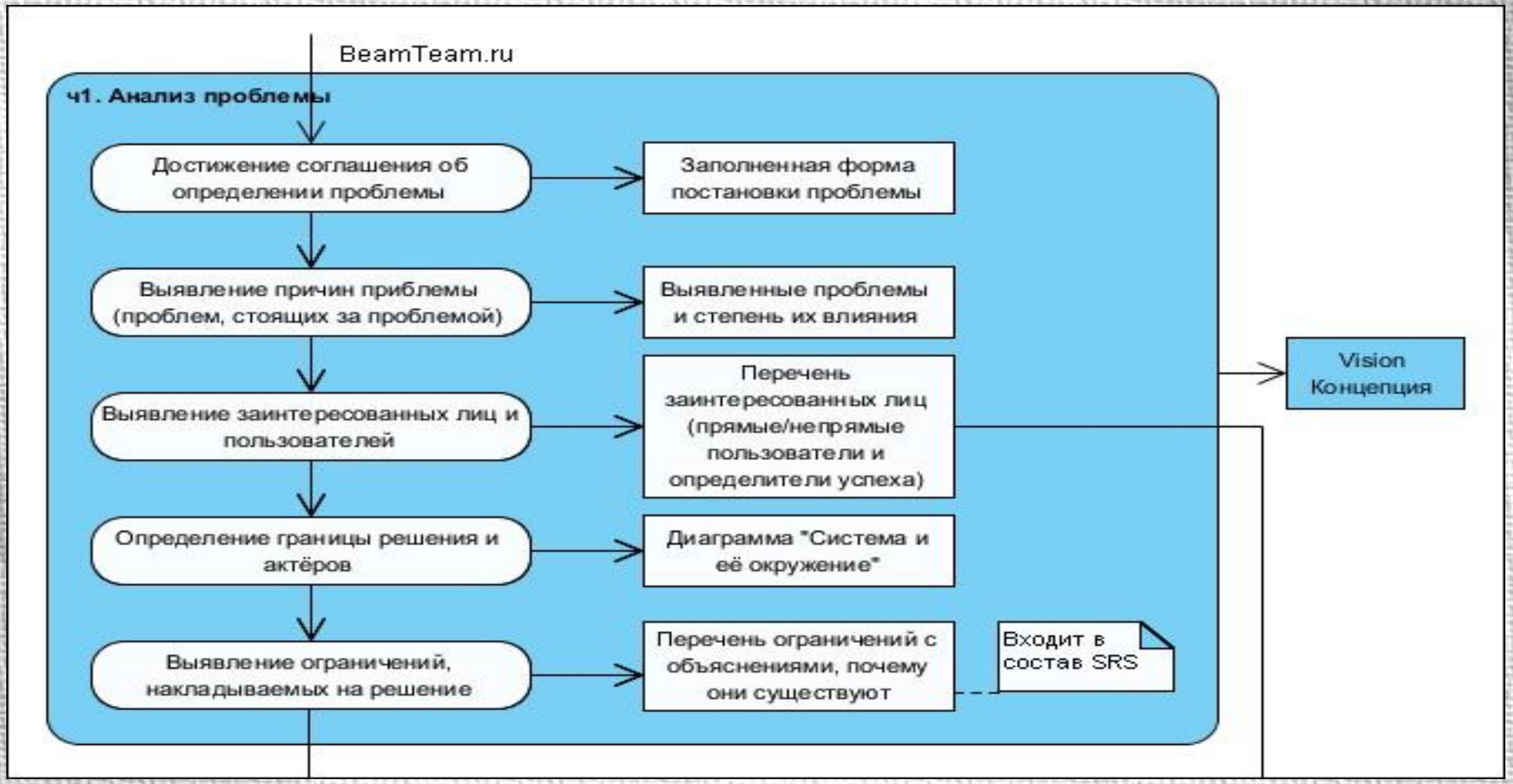
Часто наблюдается ситуация, когда разработанные ПС не могут удовлетворить потребности пользователей и ЗЛ в той мере, как ожидалось. **Увеличение инвестиций в анализ проблемы может дать ощутимый экономический эффект**.

Анализ проблемы проводится на самом ранней стадии процесса выявления требований.

Проблема – это разница между желаемым и воспринимаемым.

Анализ проблемы – это процесс осознания реальных проблем и потребностей пользователей для предложения решений, позволяющих удовлетворить эти потребности.

Шаги анализа проблемы



Этапы анализа проблемы

Процесс анализа проблемы состоит из следующих 5 этапов:

1. **Д**остижение соглашения об определении проблемы;
2. **П**оиск причины проблемы: проблем, стоящих за проблемой;
3. **В**ыявление заинтересованных лиц и пользователей;
4. **О**пределение границы решения и актёров;
5. **В**ыявление ограничений, накладываемых на решение.

1 Достичь соглашения об определении проблемы

Запишите проблему и выясните, все ли согласны с постановкой.

Элемент	Описание
Проблема	[Описание проблемы]
воздействует на	[Лица, на которых она влияет]
результатом чего является	[Каково влияние этой проблемы на заинтересованных лиц и деятельность организации]
Выигрыш от	[Предлагаемое решение]
может состоять в следующем:	[1. Преимущество предлагаемого решения №1; 2. … 3. Преимущество предлагаемого решения №n.]

❷ Выделить причины проблемы: проблемы, стоящие за проблемой

Для визуализации удобно использовать диаграмму типа «рыбий скелет» .

Выполнить «анализ корневых причин». ***Метод анализа корневых причин*** – семантический способ нахождения причин, лежащих в основе рассматриваемой проблемы или её проявления. Способы:

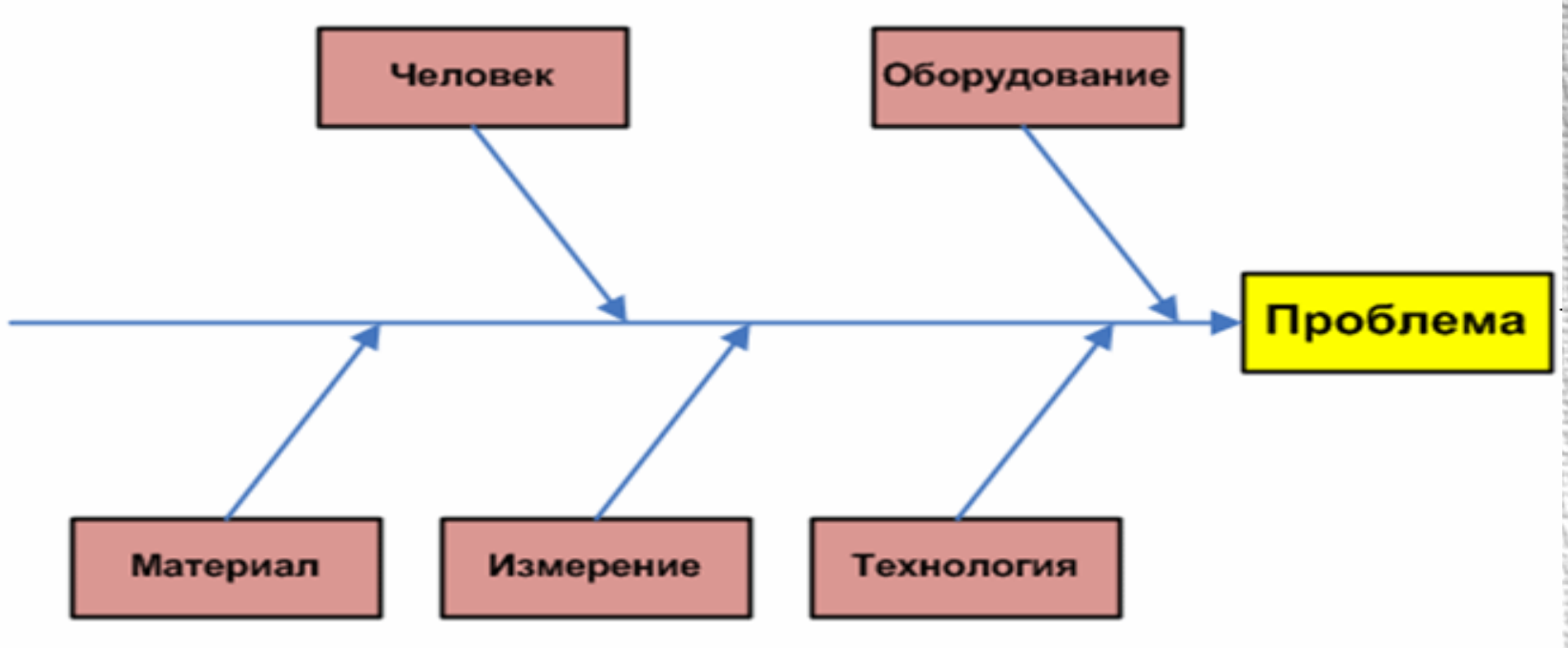
- Спросить людей, занимающих делом, что они считают корневыми причинами. Затем спросить их еще раз.
- Детально исследовать каждую из перечисленных проблем и определить количественный вклад каждой. Методы:
  - Мозговой штурм;
  - Проект по сбору данных.
- Определить влияние каждой причины и сравнить с предполагаемыми затратами на устранение.

Диаграмма Исикавы
(«скелет рыбы»)

Для того чтобы принять решение быстро, проблему лучше визуализировать – т.е. представить в графическом виде связь несоответствия с причинами, влияющими на его возникновение. Это поможет подключить образное мышление, а также позволит коллективно обсуждать проблему.

Для визуализации проблемы удобно применять диаграмму Исикавы («скелет рыбы», Fishbone Diagram), которая представляет собой графическое упорядочение факторов, влияющих на объект анализа. Этот инструмент, вошедший в состав семи инструментов управления качеством, ввел в мировую практику выдающийся японский специалист в области качества Каору Исикава.

Изучаемая проблема на диаграмме – это «голова рыбьей кости». «Хребет» на диаграмме условно изображается в виде прямой горизонтальной стрелки. Причины и факторы, прямо и косвенно влияющие на проблему, изображаются наклонными стрелками – это «кости».



На диаграмму Исикавы по направлению влево наносят основные «кости» (категории), которые имеют отношение к рассматриваемой проблеме. Для начала команда экспертов может рассмотреть общие категории.

Диаграмма Исикавы с основными категориями причин

Все возможные причины обычно распределяют по категориям по принципу «5М»:

- *Man* (Человек) – причины, связанные с человеческим фактором;
- *Machines* (Машины, оборудование) – причины, связанные с оборудованием;
- *Materials* (Материалы) – причины, связанные с материалами;
- *Methods* (Методы, технология) – причины, связанные с технологией работы, с организацией процессов;
- *Measurements* (Измерения) – причины, связанные с методами измерения.

Основные категории распределяют в нисходящем порядке, начиная с той категории, которая имеет наибольшую вероятность того, что ее причины вызвали проблему. То есть та категория с причинами, которая, по мнению экспертной команды, является основной – будет ближе к «голове рыбы».

- Для каждой категории строятся дополнительные «кости», представляющие отдельные причины, а у тех, в свою очередь, добавляются свои подпричины. Продолжая таким образом, мы получаем разветвленное дерево, связывающее причины возникновения несоответствия, находящиеся на разном уровне детализации. Таким образом можно добраться до первичных причин, устранение которых наиболее сильно повлияет на устранение всего несоответствия.
- Устранить все выявленные причины невозможно, или нерентабельно. Требуется выявить две или три наиболее важные первичные причины и управлять ими. Для этого эксперты должны выделить те первичные причины из массы возможных и те последствия несоответствия, которым нужно уделить наибольшее внимание. Для ранжирования причин и оценки тяжести последствий удобно использовать анализ FMEA (Failure Mode and Effects Analysis).

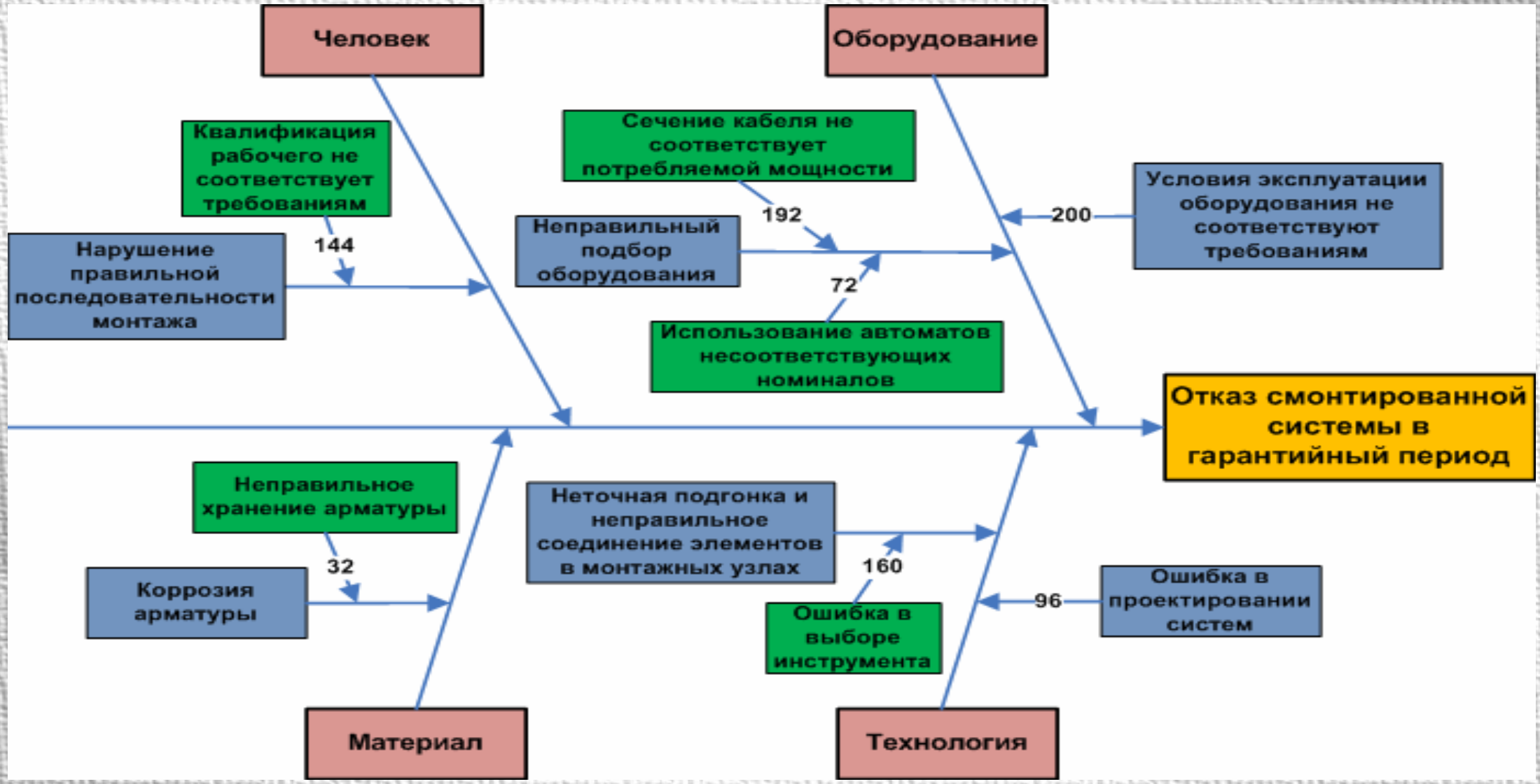
Приоритетное число риска

При проведении анализа FMEA команда рассчитывает обобщенную оценку, указывающую на степень риска возникновения несоответствия. Это **приоритетное число риска** (ПЧР), характеризуемое:

- значимостью последствия несоответствия – *S* [*significance* – значимость];
- возможностью возникновения причины несоответствия – *O* [*origination* – начало, происхождение, originate – возникать];
- возможностью обнаружения причины – *D* [*detection* – обнаружение, выявление].

Каждый из рангов – *S*, *O*, *D* – определяется по 10-балльной шкале, составляемой в форме таблицы или графика. ПЧР причины рассчитывается как произведение этих трех рангов и может принимать значения от 1 до 1000. Те причины, у которых ПЧР больше заданного предельного значения, и подлежат устранению в

Диаграмма Исикавы с указанием ПЧР



Потребности – функции – программные требования (пример)

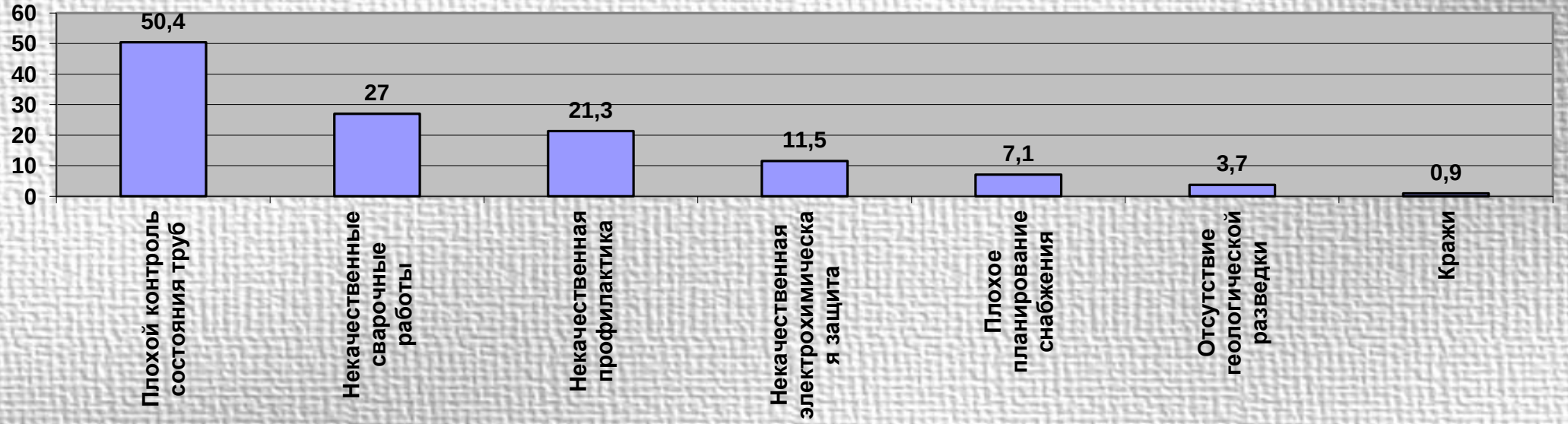
Контроль качества строительства магистральных газопроводов.

Основная проблема заказчика — большой ущерб от аварий. Как его снизить? Что можно сделать для этого?

Рассмотрим сначала причины аварий и факторы, влияющие на причиняемый ими ущерб.

Причины аварий и факторы, влияющие на причиняемый ими ущерб

Анализ влияния этих факторов на ущерб от аварий дает такую картину



Функции

Поскольку первые три фактора покрывают до 80% ущерба от аварий, будем разрабатывать систему, в первую очередь, ориентированную на снижение их влияния.

Постараемся сформулировать набор функций ПО, которые бы позволили уменьшить влияние этих факторов за счет своевременного предоставления нужной информации и помощи в ее обработке.

- Сбор, хранение и возможность просмотра данных о проведении контроля состояния труб.
- Сбор, хранение и возможность просмотра данных о сварочных работах.

- Сбор, хранение и возможность просмотра данных о профилактических работах.
- Сбор, хранение и возможность просмотра данных об авариях.
- Автоматическое сопоставление данных об авариях для выявления общих факторов.
  Эта функция позволит выявить возможные причины аварий и предотвратить аварии на участках, где действуют те же причины.
- Автоматическое построение расписания проведения дефектоскопического контроля труб.
  Эта функция позволит спланировать проведение контроля и избежать появления участков, не проверяемых в течение долгого времени.

Требования

Для преобразования функций в требования к ПО следует провести их дальнейшую конкретизацию.

Функция 5 может быть раскрыта в такое требование:

- Пользователь указывает в списке аварий две аварии и нажимает кнопку «Сопоставить».
  Система выдает список общих атрибутов указанных аварий.
  Атрибутами аварии считаются следующие данные:
  - Место (км по линии газопровода, номер участка, название газотранспортного предприятия)
  - Дата (день, месяц, год, час, минута)
  - Данные об эксплуатации (бригада обходчиков, список 5-ти последних обследований, с их датами, видами проведенного контроля и результатами)
  - Данные о строительстве (название организации-застройщика, бригада землепроходцев, бригада сварщиков, данные контроля качества сварки, бригада укладчиков, бригада электроинженеров)
  - Данные о задействованных трубах (для каждой: номер, партия, название производителя, склад, на которых хранилась, данные предукладочного обследования)
  - Данные об окружающей среде (тип почвы, кислотность, влажность, пластичность, глубина залегания почвенных вод)

Выводы

- Аналогично диаграмму Исикавы можно использовать и для анализа последствий. Изменив направление «костей», можно получить вероятные последствия, которые также ранжируются на основании экспертных оценок и помогают оценить срочность и целесообразность организации корректирующих и предупреждающих мероприятий.
- Трудно найти такие области аналитической деятельности, где бы диаграмма Исикавы не могла найти применение. С помощью диаграммы Исикавы можно решать широкий спектр конструкторских, технологических, технических, экономических, организационных, социальных и других задач.
- Современные программные средства, обеспечивающие поддержку системы менеджмента качества, позволяют построить диаграмму Исикавы. Результаты экспертных оценок заносятся в систему, после чего автоматически производится ранжирование причин. Аналитик может наглядно увидеть те причины, которым следует уделить особое внимание и разработать корректирующие и предупреждающие мероприятия для их устранения или снижения влияния на конечный результат.

3 Выявить заинтересованных лиц и пользователей

Вопросы лицам, принимающим решения, и потенциальным пользователям

- Кто является пользователями системы?
- Кто является заказчиком системы?
- На кого окажут влияние результаты работы системы?
- Кто будет оценивать/принимать систему?
- Кто будет заниматься сопровождением новой системы?
- Существуют ли другие внутренние/внешние пользователи системы, чьи потребности нужно учесть?
- Не забыли ли мы кого-нибудь?

Заинтересованное лицо

В качестве заинтересованного лица можно рассматривать:

- **Любого, участвующего в разработке системы** (бизнес-аналитики, дизайнеры, кодировщики, тестеры, менеджеры проектов, менеджеры по внедрению, дизайнеры сценариев использования, графические дизайнеры).
- **Любого, кто привносит свои знания в систему** (эксперты предметной области, авторы документов, которые были использованы для сбора требований, собственники веб-сайтов, ссылки на которые были предоставлены).
- **Руководство** (президент компании, которого представляют заказчики, директор отдела информационных технологий компании, проектирующей и разрабатывающей систему).

- **Лица, вовлеченные в управление, настройку и сопровождение системы** (хостинговая компания, справочная служба).
- **Поставщики стандартов и регламентов** (стандарты устанавливаются поисковыми механизмами согласно содержанию веб-сайта, политическим нормам, а также порядку налогообложения в конкретном штате).

4 Определить границы решения /определить актёров

Необходимо определить границу между системой и внешним миром: определить, что является частью системы и что находится вне границ системы. Взаимодействие с внешним миром осуществляется посредством интерфейсов.

Актёр – некто/нечто, находящее вне системы и взаимодействующее с ней. Актёр выполняет некоторые действия для того, чтобы система выполняла работу.

Важно помнить:

- Актер всегда внешний по отношению к системе;
- Актер представляет роли людей/сущностей, а не их самих;
- У каждого актера может быть множество ролей;
- У каждого актера должно быть короткое смысловое имя.

Если разрабатываемая системы будет использоваться вместе с существующими системами, то Системный аналитик должен уточнить границу между ними.

Примерные **вопросы**:

- Будут ли данные использоваться совместно с другими системами?
- Должна ли наша система распределяться по разным серверам, клиентам?

Вопросы для выявления актёров:

- Кто/что использует систему?
- Какие роли они играют во взаимодействии?
- Кто/что запускает/выключает систему?
- Кто/что будет поставлять, использовать, удалять информацию системы?
- Кто будет управлять системой?
- Кто будет устанавливать/сопровождать систему?
- Где будет использоваться система?
- Откуда система получает информацию?
- Какие внешние системы будут взаимодействовать с нашей системой?
- Происходит ли что-нибудь в точно установленное время (время как актер)?

5 Выявить ограничения, накладываемые на решение

Ограничение – уменьшает степень свободы, которой мы располагаем при предложении решения. Важно внимательно изучить все возможные ограничения. **Вопросы** для выявления ограничений (примеры) приведены в таблице ниже.

Тип источника ограничения	Примеры вопросов
Экономический	Какие финансовые или бюджетные ограничения следует учесть? Существуют ли соображения, касающиеся себестоимости и ценообразования? Существуют ли вопросы лицензирования?
Политический	Существуют ли внешние или внутренние политические вопросы, влияющие на потенциальное решение? Существуют ли проблемы в отношениях между подразделениями?
Технический	Существуют ли ограничения в выборе технологии? Необходимо ли работать в рамках существующих платформ или технологий? Запрещено ли использовать какие-либо новые технологии? Необходимо ли использовать какое-либо закупаемое ПО?

Системный	Будет ли решение использовать существующие системы? Должны ли мы обеспечивать совместимость с существующими решениями? Какие ОС и среды должны поддерживаться?
Эксплуатационный	Существуют ли ограничения информационной среды или правовые ограничения? Юридические ограничения? Требования безопасности? Какими другими стандартами мы ограничены?
График ресурсы	Определен ли график? Какими ресурсами мы ограничены? Возможно ли привлечение работников со стороны? Возможно ли увеличение штата временно или постоянно?

Следует получить объяснение ограничений для того, чтобы:

- убедиться, что правильно его поняли;
- обнаружить, когда оно больше не применимо.

Тип источника	Ограничение	Объяснение
[см. выше]	[формулировка ограничения]	[почему это ограничение существует]

Препятствия при выявлении требований

При сборе требований основными препятствиями являются следующие:

1. Нераскопанные руины
2. Да, но…
3. Пользователь vs разработчик

1. Нераскопанные руины

– никогда не удается почувствовать, что вы «раскопали» всё. Чем больше находишь, тем больше убеждаешься, что найдено ещё не всё.

- Когда можно прекратить выявление требований?
- Когда можно считать, что выявили достаточное количество существенных требований?

Решение: выявить всех заинтересованных лиц (ЗЛ), а не только пользователей, т.к. все они – источники требований.

При использовании разнообразных методов выявления требований на определенном этапе можно будет уверенно сказать: «Мы выявили достаточно».

2. Ситуация «Да, но…»

– две немедленные и противоположные реакции пользователя, когда он впервые видит реализацию. Он никогда до этого не видел новую систему или что-либо схожее, и не по-другому интерпретировал её описание и требования.

Проблема/причина	Решение
Разработчики редко предоставляют пользователю что-либо для обсуждения/взаимодействия до момента окончания разработки кода.	Выявить «но» как можно раньше. Например, показать макет, модель, пошаговый процесс, прототип, обсудить его, поработать с ним задолго до окончания реализации

3. Конфликт «пользователя и разработчика»

– следствие расхождения взглядов пользователя и разработчика, поскольку они говорят на разных языках, обладают разным опытом, преследуют разные цели.

Проблема/причина	Решение
1. Пользователи либо не знают, чего хотят (ситуация «чистый лист»), либо не могут это выразить.	Признавать пользователя экспертом в предметной области, использовать различные методы выявления требований.
2. Пользователи уверены, что знают, чего хотят. Но лишь до момента, когда разработчики дают им то, что они «хотели».	Как можно раньше предлагать раскадровки, прототипы, макеты, обыгрывание по ролям и т.п.
3. Системные аналитики уверены, что понимают потребности пользователя лучше, чем он сам.	Поставить аналитика на место пользователя и провести ролевую игру.
4. Стороны считают, что противоположная сторона действует так по политическим целям.	Тут ничего не изменишь.

Методы получения требований

В своей практике аналитики используют большое количество различных методов по выявлению требований. Для того, чтобы собрать наиболее полные требования рекомендуется по возможности использовать сразу несколько методов.

Не существует идеальной методики для сбора и анализа требований. Наиболее подходящие методики варьируются от проекта к проекту.

Техники выделения требований

Техники выделения требований можно классифицировать следующим образом.

1. **Аналитические техники.** В их основе лежит анализ каких-либо документов, знаний, систем или деятельностей.
2. **Техники, основанные на общении**. В их основе лежит общение между аналитиком и другими людьми, нацеленное на выявление потребностей заинтересованных лиц и ограничений предметной области.
3. **Контекстуальные техники.** В их основе лежат методы этнометодологии и культурологии. В этих техниках используется наблюдение за действиями пользователей и экспертов в том окружении, в котором надо будет работать будущей системе. При этом пытаются учитывать национальный и культурный контексты их поведения.

1. Аналитические техники

- **Размышление (introspection)** — аналитик размышляет над возможными функциями системы и выбирает те, которые выглядят достаточно привлекательными и полезными.
- **Анализ документов и моделей (review, analysis)** — анализируются нормативные документы, документы, описывающие реальность или пожелания и проблемы, модели, описывающие предметную область. Из них выделяются потребности, возможные функции и критерии их оценки.
- **Анализ имеющихся аналогичных систем.**
  - **Анализ выборки данных (sampling)** — анализируются большие массивы имеющихся данных (отчетов, внутренних данных системы, входной информации и пр.), выделяются сущности, их атрибуты и связи.
  - **Анализ функций** — анализируется работа одной или нескольких имеющихся систем, выделяются выполняемые ими операции и возможные правила их выполнения.

2. Техники, основанные на общении

Индивидуальные техники. В них используется только индивидуальное общение аналитика с другими людьми.

- **Интервью** (interview) — заинтересованное лицо отвечает на вопросы аналитика, часть их которых подготовлена заранее, другая часть возникает в процессе диалога.
- **Опрос** (survey) — ряд заинтересованных лиц отвечает на вопросы из заранее подготовленного списка (вопросника, questionnaire).

Групповые техники. В них используется групповое общение, повышающее отдачу каждого сеанса, но и приводящее к возникновению конфликтов, различных позиций и пр., и поэтому нуждающееся в контроле и управлении.

- **Групповое интервью** (group interview) или фокус-группа (focus group) — группа экспертов собирается и обсуждает совместно с аналитиком возможные функции и требования. Обсуждение направляется вопросами аналитика по заранее подготовленному сценарию.
- **Мозговой штурм** (brainstorming session) — несколько человек, включая аналитиков и заинтересованных лиц, некоторое время выдумывают различные возможности, которые могли бы быть полезны, не ограничивая воображение.

- **Метод Дельфи** (Delphi method) [23,24]— опрос группы экспертов на одну тему. Каждый из них сначала готовит документ с изложением своего мнения. Затем каждый из полученных документов передается для анализа и замечаний другому эксперту. Такая передача может повторяться несколько раз.
- **Ролевая игра** (role playing game) — участники группы разыгрывают возможные сценарии работы, каждый играет свою роль. Выявляемые в ходе игрыпроблемы, потребности, ограничения фиксируются.

Демонстрационные техники. Они используют демонстрацию одному или группе экспертов моделей системы или возможных сценариев работы системы для получения от них откликов и замечаний, которые обычно тут же фиксируются.

- **Раскадровка** (storyboard) — сценарий работы в некоторой ситуации расписывается по отдельным действиям и представляется пользователям и экспертам. Представляться он может в виде описания действий, набора рисунков, анимации, и пр. Собираются замечания и поправки, высказанные по поводу такого сценария

- **Демонстрация прототипа (prototype demo)** — демонстрируется работа прототипа или отдельных модулей системы в определенной ситуации. Собираются замечания пользователей и экспертов.
- **Модели** - техники, использующие в качестве основы общения некоторые модели (model based techniques).
  Таких техник много, любой вид моделей, особенно графических или симуляционных, можно использовать для общения с экспертами и пользователями и фиксации их замечаний к такой модели.
  В качестве моделей часто используются описания сценариев работы системы, варианты использования, диаграммы потоков данных, концептуальные модели данных системы.
  Отдельно можно упомянуть модели, фиксирующие выделяемые цели операций системы и связанные с ними техники (goal-driven, а также модели, фиксирующие различные точки зрения (viewpoint-based).

Когнитивные техники. Это специализированные техники, основанные на некоторых положениях когнитивной психологии. Исторически они возникли как приемы выявления структур мышления у людей с психическими отклонениями. После достижения ряда успехов в этой области техники такого рода стали применятся и для выявления мыслей и знаний в более широком контексте, в частности, для выделения требований.

- **Анализ задач** (task analysis) — анализируются задачи, решаемые пользователем, и операции, предпринимаемые им для их решения. Целью анализа является определение информации, необходимой для выполнения каждой операции, и взаимосвязей между различными используемыми в процессе решения сущностями. Обычно проводится аналитиком, присутствующим при работе пользователя и задающим вопросу по ходу его работы.
- **Анализ протоколов** (protocol analysis) — пользователь или эксперт проговаривает и вслух все выполняемые действия при конкретном или абстрактном сценарии. При этом приводятся их обоснования и возможные варианты.

- **Устойчивые решетки или решетки Келли** (repertory grids). Выбирается набор объектов предметной области, для которых каждый эксперт из группы придумывает систему атрибутов и их значений. Потом эксперты пытаются найти наиболее близкие атрибуты в своей системе к атрибутам в системе другого эксперта, а также оценить значения атрибутов в чужих системах. Есть еще несколько разновидностей этой техники.
- **Сортировка карточек** (card sorting). На карточки заносятся названия объектов предметной области. Обычно один объект присутствует на одной карточке, но в некоторых случаях используется модификация — один объект может быть на нескольких карточках. После этого каждому из нескольких экспертов выдают полный набор карточек и просят разделить карточки на группы по каким-то признакам. Затем каждый формулирует обоснование своей группировки.
- **Лестница** (laddering) — используется иерархическая система вопросов-зондов (probes) для выявления знаний опрашиваемого о предметной области.
- **Масштабирование близости** (proximity scaling). Определяется набор объектов предметной области, список которых выдается экспертам. Те должны определить набор характеристик для их описания и степень близости этих объектов друг к другу по выбранному набору характеристик

3. Контекстуальные техники

В качестве вспомогательных элементов в таких техниках используется следующее.

- Анализ разговоров и речи во время интервью или совещаний.
- Анализ невербального поведения.
- Запись поведения человека на видео и последующий его детальный анализ.
- Запись и анализ физиологических параметров (пульс, давление, электропроводность кожи и пр.).

ТРАДИЦИОННЫЕ МЕТОДЫ ВЫЯВЛЕНИЯ ТРЕБОВАНИЙ

1. *Интервьюирование заказчиков и экспертов проблемной области*
2. *Анкетирование*
3. *Наблюдение* (пассивное и активное)
4. *Изучение документов и программных систем*

1. Интервью

Существуют два основных типа интервью:

1. структурированное (формальное)
2. неструктурированное (неформальное).

Структурированное интервью готовится заранее, отличается ясностью постановки вопросов, а многие вопросы могут быть заданы заранее. Заранее сформулированные вопросы можно разделить на две категории:

- вопросы с открытым множеством ответов (ответы на эту категорию вопросов заранее не готовятся)
- вопросы с замкнутым множеством ответов (ответы на эту категорию вопросов можно выбрать из списка подготовленных ответов).

Неструктурированное интервью больше напоминает неформальную встречу, которой не свойственны заготовленные впрок вопросы или заранее поставленные цели. Цель неструктурированного интервью — подвигнуть заказчика к тому, чтобы он поделился своими мыслями и в процессе беседы подошел к требованиям, которых бизнесаналитик мог и не ожидать и, следовательно, не мог задать нужные вопросы.

Этапы интервью

Хронологически интервьюирование состоит из нескольких этапов:

1. подготовка к интервью;
2. проведение интервью;
3. обработка результатов интервью.

Подготовка к интервью

Любое интервью должно начинаться с подготовки. Нельзя приходить на встречу к Заказчику неподготовленным. Как минимум, Вы будете выглядеть в его глазах непрофессионалом, незаинтересованным в получении желаемого результата. Поэтому перед началом встречи бизнес-аналитик, возможно прибегая к помощи руководителя проекта для решения организационных вопросов, должен проработать следующие обязательные моменты:

- — место, дату и время проведения встречи;
- — состав участников встречи;
- — цель/тему встречи;
- — повестку встречи.

Проведение интервью

- Проведение интервью в большей степени находится в области коммуникации и межличностного общения.
- Бизнес-аналитик должен не только хорошо разбираться в предметной области Заказчика, но и уметь грамотно и профессионально наладить процесс общения с его представителями.
- Для того чтобы докопаться до сути, необходимо как можно чаще задавать вопрос: «Почему?». Также не стоит бояться задавать вопросы, которые могут показаться представителям Заказчика «глупыми». Ответы на эти вопросы позволят выявить требования, которые участники встречи считают само собой разумеющимися.
- В ходе встречи необходимо придерживаться определенного уровня детализации. Например, на начальных этапах проекта не нужно обсуждать мельчайшие подробности интерфейса.

Проведение интервью

- Хорошей практикой во время проведения интервью является не только фиксация той информации, которую говорит Заказчик, но и предложение собственных вариантов, основанных на личном опыте.
- В процессе интервью бизнес-аналитик должен каким-либо образом фиксировать получаемую от Заказчика информацию. Ведь пределы человеческой памяти ограничены. Самый простой способ — записывать основные моменты обсуждения на бумаге.
- Более современный способ фиксации получаемой информации — использование диктофона. В этом случае после проведения встречи можно прослушать сделанную запись сколько угодно раз. Но не каждый человек согласится, чтобы его записывали на диктофон. Поэтому, если Вы решили использовать диктофон, получите у участников встречи согласие на запись.

Обработка результатов интервью

После проведения интервью необходимо составить и согласовать с участниками интервью протокол встречи.

По завершении интервью необходимо оформить протокол встречи, содержащий полученную от Заказчика информацию.

В начале проекта разрабатывается шаблон протокола, который позволяет сократить время на его оформление, а также унифицировать формат протоколов встреч в рамках проекта.

После составления протокола его необходимо согласовать со всеми участниками встречи. Только после того, как каждый участник подтвердил свое согласие с указанной в протоколе информацией, эту информацию можно использовать в проекте.

Интервью является основным способом получения требований от заказчика и пользователя информационной системы. Тема интервьюирования затронута во многих книгах по управлению требованиями. Наиболее полно данный метод выявления требований описан в [1] и [2]. Стоит отметить лишь основные правила, которые необходимо помнить во время и после интервью:

- Аналитик должен быть компетентен в предметной области заказчика;
- Аналитик должен задавать открытые вопросы, которые предполагают развернутый ответ;
- Аналитик может предлагать свои идеи на счет разрабатываемой системы;
- Аналитик должен выяснить ожидания интервьюируемых (что, если…);
- Аналитик должен протоколировать результаты интервью.

Литература

[1] Д. Леффингуэлл, Д. Уидриг, Принципы работы с требованиями к программному обеспечению. Унифицированный подход, Вильямс, 2002
[2] K. Baxter, Understanding Your Users., Morgan Kaufmann Publishers,2005

Подводя итог

Подводя итог всему вышесказанному, надо отметить следующее:

- готовьтесь к интервью;
- учитывайте коммуникативные особенности собеседников;
- выявляйте истинные потребности Заказчика;
- составляйте протокол встреч.

2. Анкетирование

Обычно анкеты используются в дополнение к интервью, а не вместо них. Анкета (или вопросник) должна быть разработана таким образом, чтобы, по возможности, облегчить ответы на вопросы. В частности, следует избегать вопросов с неопределенным множеством ответов — большинство вопросов должны относиться к вопросам с замкнутым списком ответов. Подобные вопросы могут принимать три формы:

- **Многоальтернативные вопросы**. Респондент должен указать один или более ответов.
- **Рейтинговые вопросы**. Респондент должен выразить свое мнение в отношении высказанного утверждения: “абсолютно согласен ”, ”согласен”, ”отношусь нейтрально”, ”не согласен”, ”абсолютно не согласен”, ”не знаю”.
- **Вопросы с ранжированием**. Предусматривает ранжирование ответов с помощью присваивания им последовательных номеров, процентных значений и использования других средств упорядочения.

3. Наблюдение

В некоторых ситуациях бизнесаналитик находит трудным получить полную информацию с использованием интервью и анкет.

Наблюдение может выступать в двух формах.

- **Пассивное наблюдение** (passive observation), в ходе которого бизнесаналитик наблюдает за различными видами деловой деятельности без вмешательства или прямого участия в них. В некоторых случаях для того, чтобы наблюдение было как можно менее навязчивым, можно даже использовать видеокамеры.
- **Активное наблюдение** (active observation), в ходе которого бизнесаналитик участвует в деятельности и становится фактически частью команды.

4. Изучение документов и программных систем

Часто на практике заказчики и пользователи информационной системы отдалены от команды разработчиков, что вызывает трудности при выявлении требований. Однако, кроме заказчиков и пользователей, существуют и другие источники требований:

- документы, описывающие существующие или конкурирующие продукты;
- существующие спецификации требований к системе;
- отчеты об ошибках и запросы от пользователей по совершенствованию системы;
- результаты маркетинговых исследований;
- существующая документация с описанием процессов деятельности заказчика и др.

5. Самостоятельное описание требований

Если опытный аналитик уже исследовал большое число систем такого же типа, что и на предприятии внедрения, он обладает фундаментальными знаниями в соответствующей предметной области, относительно определенного класса систем. Авторы методологии *SADT* рекомендуют проводить самоопрос с тем, чтобы получить максимальную пользу от своих знаний.

По результатам анализа документов и собственных знаний аналитик может составить описание требований и предложить его представителям Заказчика в качестве информации к размышлению, либо - основы для формирования технического задания.

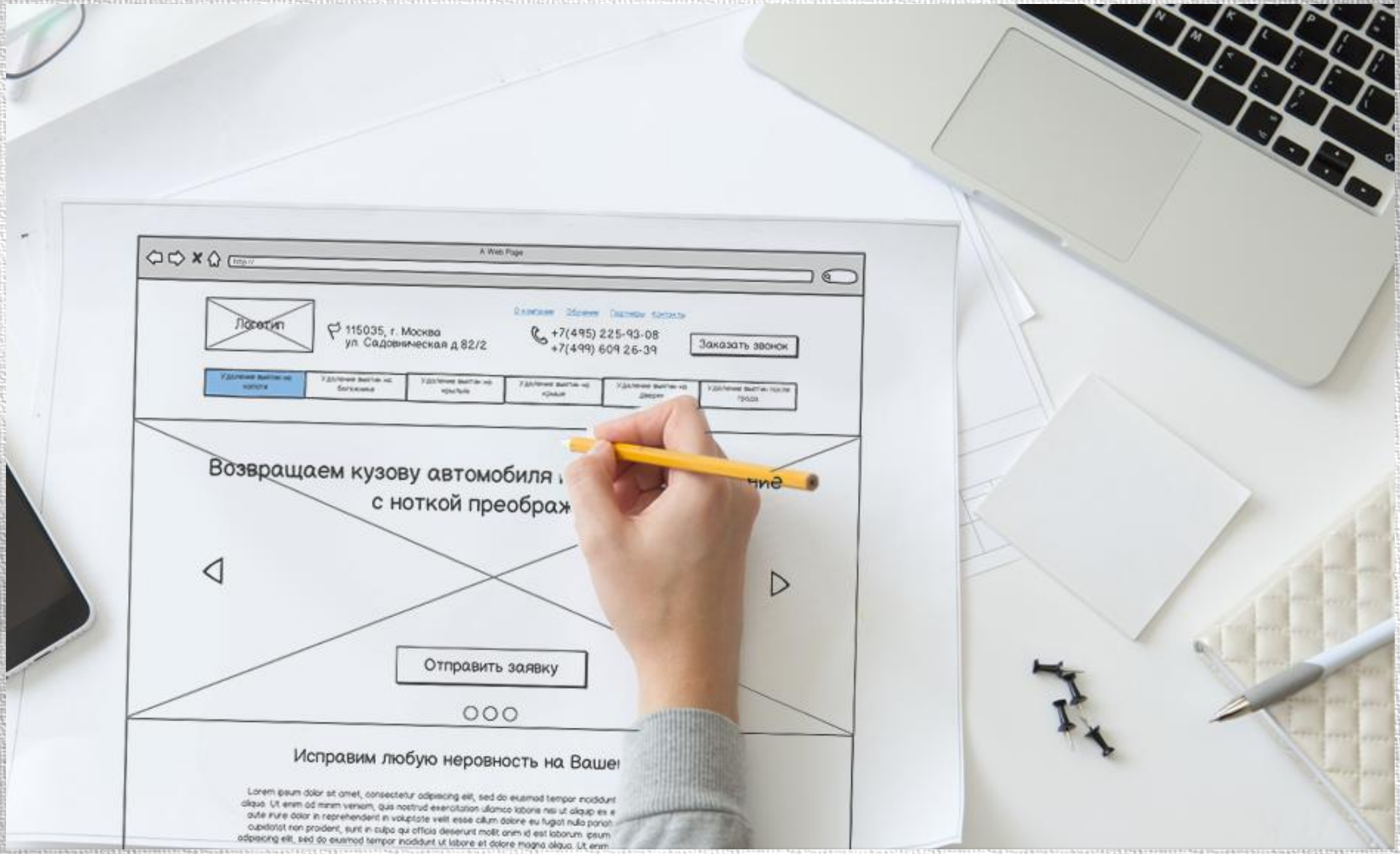
СОВРЕМЕННЫЕ МЕТОДЫ ВЫЯВЛЕНИЯ ТРЕБОВАНИЙ

СОВРЕМЕННЫЕ МЕТОДЫ ВЫЯВЛЕНИЯ ТРЕБОВАНИЙ

- **Прототипирование.** Прототип представляет собой демонстрационную систему – «наскоро и грубо» сделанную рабочую модель решения, которая представляет пользовательский GUI-интерфейс и моделирует поведение системы при инициировании пользователем различных событий
- **Семинары**
- **Пользовательские истории**
- **Сценарии, основанные на персонажах**

Прототипирование

Прототипирование - это техника для построения быстрой и приблизительной версию желаемой системы или части этой системы. Прототип демонстрирует возможности системы пользователям и дизайнерам. Программный прототип - "зеркало", в котором видно отражение того, как понял Исполнитель требования Заказчика. В некоторых случаях, создание прототипов может создать впечатление, что разработчики зашли дальше в развитии проекта, чем есть на самом деле, что может предоставить пользователям нереалистичные ожидания окончания проекта.


A Web Page
Логотип
115035, г. Москва
ул. Садовническая д 82/2
+7(495) 225-93-08
+7(499) 609 26-39
Заказать звонок
Возвращаем кузову автомобиля
с ноткой преображ
Отправить заявку
Исправим любую неровность на Ваше

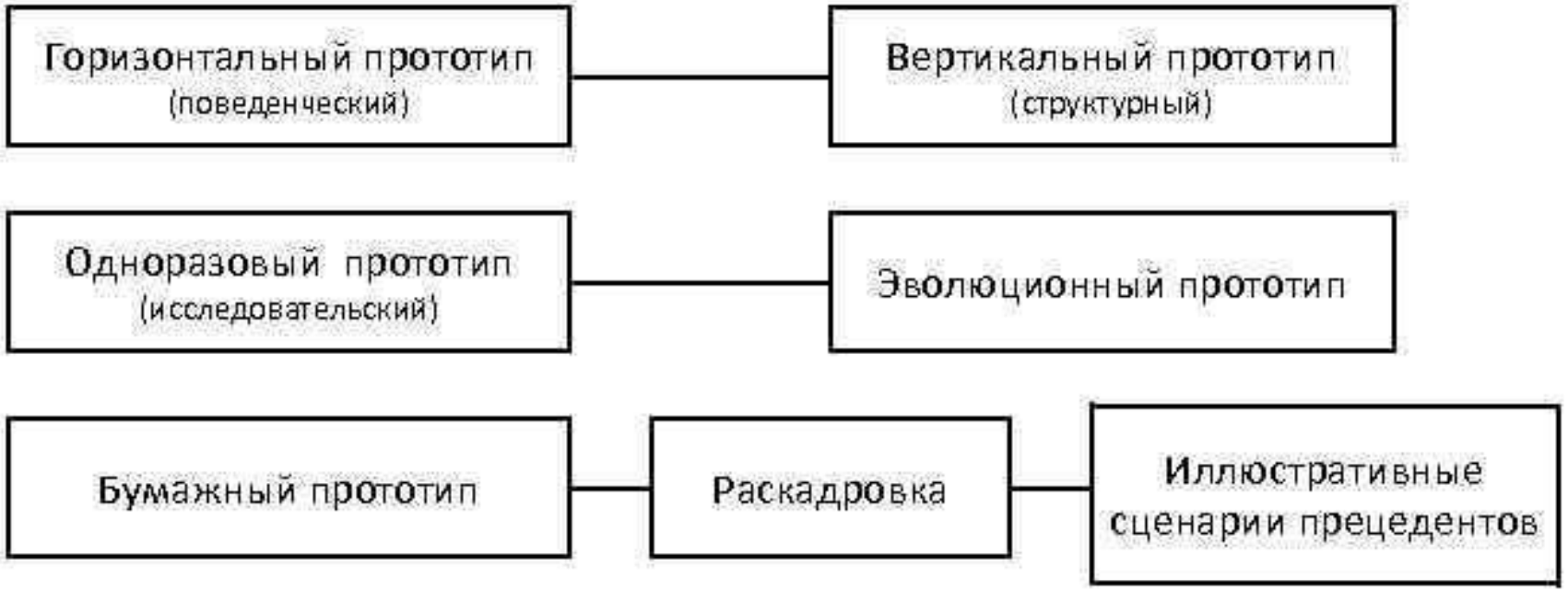
В середине 1980-х прототипирование (англ. *prototyping*) рассматривалось как решение проблемы анализа требований. Прототипы — макеты системы. Макеты дают возможность пользователям представить систему, которая ещё не построена. Опытные образцы помогают пользователям представить на что будет похожа система, и облегчают пользователям принятие проектных решений, не дожидаясь окончания постройки системы. Наибольшие улучшения во взаимопонимании между пользователями и разработчиками часто замечались с введением опытных образцов. Ранние обзоры системы приводят к меньшему количеству изменений на поздних стадиях разработки и следовательно значительно уменьшают затраты..

Работая с прототипом системы пользователь, может высказывать предположения, недочеты, которые будут оформляться в виде требований к программному продукту. Каждый прототип должен иметь определенную цель создания, например, реали-зация функции, представление внешнего вида программы, тестирование варианта архитектуры системы. Существует большое количество разновидностей прототипов.

- ***горизонтальные и вертикальные;***
- ***одноразовые и эволюционные;***
- ***бумажные и электронные, раскадровки***.

Наиболее используемыми являются «горизонтальный» и «вертикальный» прототипы.

Классификация прототипов



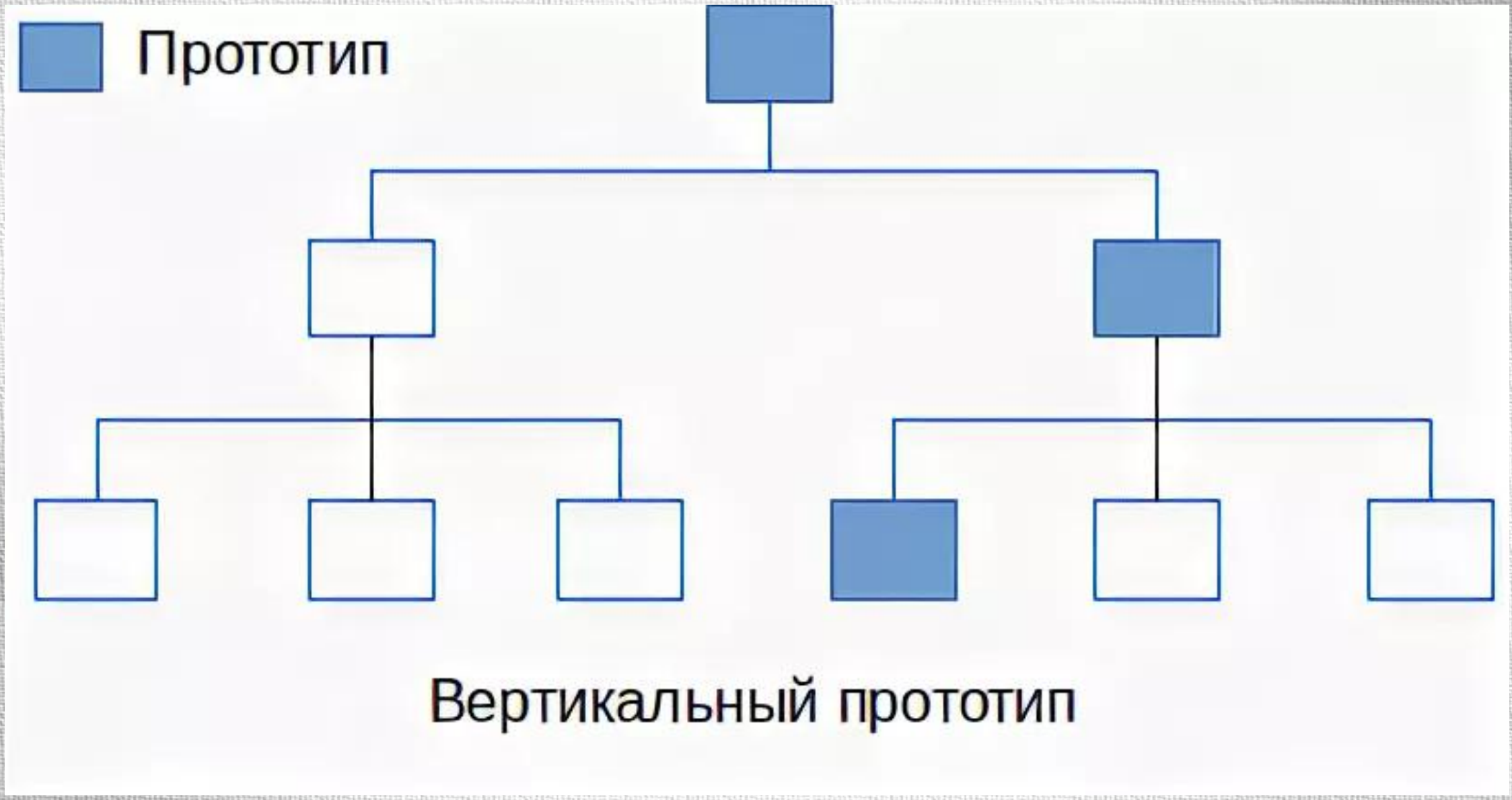
Горизонтальный прототип

Горизонтальный прототип представляет собой изображения пользовательских интерфейсов, которые демонстрируют пользователю внешний вид будущей системы и характеризуют методы работы с системой. Выполнение задач может быть представлено последовательностью сменяющих друг друга пользовательских графических интерфейсов. Горизонтальные прототипы позволяют исследовать поведение системы в различных ситуациях: обработке ошибок, разветвление потоков задач и др.

Это создание интерактивного прототипа будущего приложения или сервиса. В его основу ложится информация о предметной области проекта и специфике бизнеса, собранная нашими специалистами. **Прототип сайта или приложения представляет собой набор экранов, который выглядит почти как готовое приложение или сайт**. На нём видно расположение элементов, по ссылкам можно кликать, на кнопки можно нажимать. Это даст вам ясное понимание того, как будет устроен будущий продукт, какие возможности стоит добавить, а от каких отказаться.

Вертикальный прототип

Вертикальный прототип предназначен для проверки определенной функциональности системы и исследования критически важных требований к интерфейсу, времени выполнения. Вертикальный прототип является законченной реализацией части будущей системы. С помощью вертикальных прототипов можно проверить удовлетворяет ли разработанная системная архитектура требованиям характеристик качества, произвести нагрузочное тестирование приложения.



Вертикальный прототип

Разрабатывайте вертикальный прототип всякий раз, когда сомневаетесь в осуществимости и стабильности предполагаемого подхода к архитектуре системы или когда хотите оптимизировать алгоритмы, оценить предлагаемую схему базы данных или проверить критически важные временные требования. Чтобы получить значимые результаты, вертикальные прототипы создают, используя средства разработки в среде, аналогичной среде разработки. Вертикальные прототипы используются для исследования критически важных требований к интерфейсу и времени исполнения, а также для сокращения рисков на стадии проектирования системы.

Одноразовые прототип

Прежде чем создавать прототип, примите четкое и ясное решение, прекратите ли вы работать с ним после оценки или сделаете его частью выпускаемого продукта. Создайте одноразовый прототип (throwaway prototype) или исследовательский прототип (exploratory prototype), чтобы ответить на вопросы, разрешить неясности и улучшить требования к ПО). Если вы решили, что после выполнения задачи прекратите работу с прототипом, стройте его как можно более быстро и дешево. Чем больше усилий вы вложите в прототип, тем труднее будет участникам проекта отказаться от него.

Создавая одноразовый прототип, разработчики пренебрегают большинством известных им методов конструирования качественного ПО. В одноразовом прототипе предпочтение отдается быстроте реализации и модификации, а не устойчивости, надежности, производительности и долговременному удобству сопровождения. Поэтому внимательно следите, чтобы низкокачественный код одноразового прототипа не попал в окончательный продукт. В противном случае пользователи и персонал технического обслуживания будут страдать от последствий этого в течение всего срока эксплуатации продукта.

Одноразовый прототип наиболее уместен, когда команда разработчиков сталкивается с неизвестностью, двусмысленностью, неполнотой или неопределенностью в требованиях к ПО. Разрешение этих вопросов уменьшает риск продолжения разработки.

Эволюционный прототип

Эволюционный прототип (*evolutionary prototype*) создается, как первое приближение системы, призванное стать впоследствии самой системой.

Код эволюционного прототипа должен последовательно, в течение одной или более итераций, перерасти в код целевого приложения. Поэтому данный вид прототипов требует всего того, от чего следует отказаться при создании одноразовых прототипов: скрупулезной разработки, применения технологических методов и приемов, тестирования результатов и т.п.

Бумажный прототип

Бумажный прототип (*paper prototype*) - отличная альтернатива рассмотренным выше разновидностям электронных прототипов в случае, когда Разработчик ограничен в ресурсах. Наброски интерфейсов на бумаге, конечно, не заменят интерфейс, созданный в среде разработки. Однако, при всех недостатках, у таких прототипов есть два существенных достоинства.

1. Заказчик не станет акцентировать внимание на цветовом решении, форме кнопок и т.п., отвлекаясь от анализа функциональности.
2. Заказчик никогда не скажет, глядя на бумажный интерфейс: "Да вы, я вижу, уже создали систему на 85%! Давайте закончим ее в течении недели".

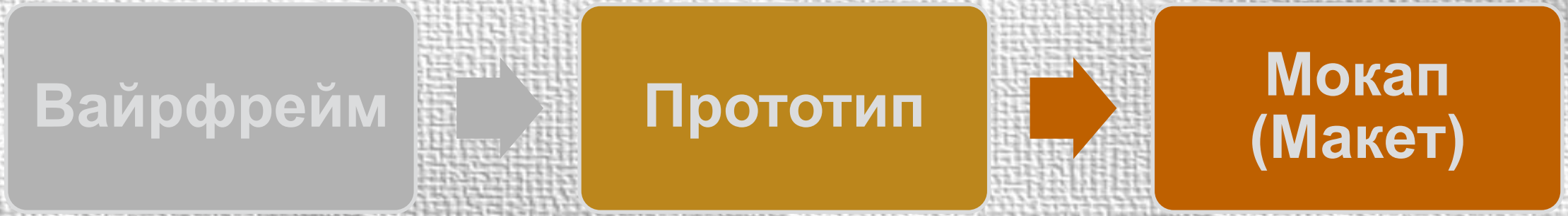
Раскадровка

Решением промежуточного между электронным и бумажным вариантами прототипов UI класса являются презентации, изготовленные при помощи средств электронного офиса (например, комбинации Microsoft Visio и Microsoft PowerPoint). В этом случае пользователь лишен свободы выбора, предоставляемой ему поведенческим прототипом. Но идею пошаговой смены экранов в процессе реализации сценария варианта использования вполне можно реализовать. Данный вид решения определяется, как пассивная раскадровка. Активная раскадровка является дальнейшим развитием понятия пассивной раскадровки, с применением средств анимации и т.п. Третий вид раскадровки - интерактивная представляет собой электронный одноразовый горизонтальный прототип.

Соотношение между рассмотренными выше 4 видами прототипов

	Одноразовые	Эволюционные
Горизонтальные	Прояснение и уточнение примеров использования и функциональных требований Выявление пропущенных требований Исследование возможных вариантов интерфейса пользователя	**Реализация базовых вариантов использования** Реализация дополнительных вариантов использования по приоритетам Реализация и доработка web-сайтов Адаптация системы к быстро меняющимся требованиям бизнеса
Вертикальные	Демонстрация технической осуществимости	Реализация и наращивание ключевой клиент-серверной функциональности и уровней коммуникации Реализация и оптимизация основных алгоритмов Тестирование и настройка производительности

ОТ ВАЙРФРЕЙМА К МАКЕТУ



Блузки

Каталог

Женщинам

Платья
Джемперы и водолазки
Кардиганы и пончо
Юбки
Блузки
Пуловеры и свитеры
Топы
Пальто
Пиджаки
Брюки
Куртки и пуховики
Футболки
Шапки и шарфы

Мужчинам

Bracciallini

Цена + Размер + Цвет + Материал + Сбросить

87 товаров **Сортировать по:** Новизне Цене

Код товара: 34788

Блузка повседневная OLYMPIA

Код товара: 34788

Блузка

Код товара: 34788

Блузка повседневная OLYMPIA с круглым вырезом

Online-инструменты

Cacoo	**Тип**: Online **Цена**: есть бесплатный аккаунт **Link**: http://www.cacoo.com/
iPlotz	**Тип**: Online **Цена**: есть бесплатная версия **Link**: http://iplotz.com
MockFlow	**Тип**: Online **Цена**: есть бесплатные аккаунты **Link**: http://www.mockflow.com/
Mockup Builder (beta)	**Тип**: Online / Silverlight **Цена**: на текущий момент — бесплатная **Link**: http://mockupbuilder.com

Creately	**Тип**: Desktop/Online **Цена**: 5$ **Link**: http://www.creately.com/
Lucidchart	**Тип**: Online **Цена**: 5$ **Link**: http://www.lucidchart.com/
Mockingbird	**Тип**: Online **Цена**: 9$/месяц **Link**: https://gomockingbird.com
Framebox	Platform: Online Price: Free
Gliffy	Platform: Web-based Price: Free, $3.99, $4.99 or $7.99 per month https://www.gliffy.com/

Desktop-инструменты

Pencil	Тип: Firefox Addon Цена: Free Link: http://pencil.evolus.vn
Serena Prototype Composer	**Тип:** Desktop **Цена:** Free **Link:** http://www.serena.com

SketchFlow – это забавный, неформальный, гибкий, быстрый и мощный способ создания динамических набросков и прототипов с богатыми возможностями в Expression Blend.

Прототипирование с помощью Zero-code и Low-code

Прототипирование в IT — это процесс создания ранних версий продукта, который позволяет демонстрировать и тестировать его основные функции и дизайн перед разработкой полноценного решения. Этот процесс играет ключевую роль в разработке программного обеспечения, веб-сайтов и мобильных приложений, потому что он помогает уточнить требования к продукту, обнаружить потенциальные проблемы и собрать обратную связь от пользователей на ранних этапах.

Zero-code и Low-code технологии как нельзя кстати подходят для этой задачи. Быстрое создание прототипа с помощью таких типов разработки становится все более популярным среди предпринимателей и разработчиков, стремящихся проверить свои идеи и получить обратную связь от пользователей без значительных затрат на традиционную разработку.

Семинары

В результате интервью могут появиться нерешенные вопросы. Для решения подобных вопросов и обсуждение требований проводятся специальные семинары.

Семинары по сбору требований предоставляют возможность для совместного выявления требований. Участники семинаром могут уйти с более глубоким пониманием вопросов, и в результате чего могут почувствовать сильное чувство приверженности и заинтересованности в проекте.

На семинаре могут проводится мозговые штурмы для генерации новых идей и ключевых возможностей системы

Семинары рабочих групп

Возможно самый мощный способ сбора требований;

Собирает всех представителей заинтересованных сторон для определения требованиями в небольшом но насыщенном интервале времени;

Используется приглашенный ведущий для руководства мероприятием.

Совместные семинары

Мозговой штурм

Правила мозгового штурма предполагают полную раскрепощённость и свободу мнений, даже самых вычурных и на первый взгляд «бредовых». Первое правило мозгового штурма – «полный запрет на любую критику». Всякое высказанное мнение представляет ценность, а полное отсутствие запретов позволяет полноценным образом подключить творческую фантазию.

Затем, на втором этапе, все высказанные мнения тщательным образом обсуждаются, заведомо неприемлемые варианты отсеиваются, формируются коллективные предложения.

Участники JAD-совещания

- **Ведущий** – специалист в области межличностных коммуникаций. Должен ориентироваться в предметной области, но не обязательно хорошо ориентироваться в проблемах IT.
- **Секретарь** – стенографист встречи. Фиксирует её результаты на компьютере. Возможно применение CASE-средств.
- **Заказчики** – пользователи или руководители, основные участники, формирующие, обсуждающие требования и принимающие решения.
- **Разработчики** – аналитики и другие участники проектной команды. Работают в большей части в пассивном режиме с целью наилучшего понимания проблемной области.

Разъясняющие встречи

“Разъясняющие встречи” или “запланированный мозговой штурм” – термин, пришедший из общей практики менеджмента и базирующийся на идеях сотрудничества заинтересованных лиц для совместного анализа путей решения проблем, определения и предупреждения рисков и т.п.

Воркшоп

Воркшоп — это сбор и обсуждение требований в формате рабочего совещания с заранее продуманной структурой. На воркшоп зовут представителей заказчика и членов команды разработки, участвующих в дизайне решения, чтобы выявить скрытые требования, уточнить общий список потребностей, разрешить противоречия. В итоге группа находит решение, которое устроит все стороны.

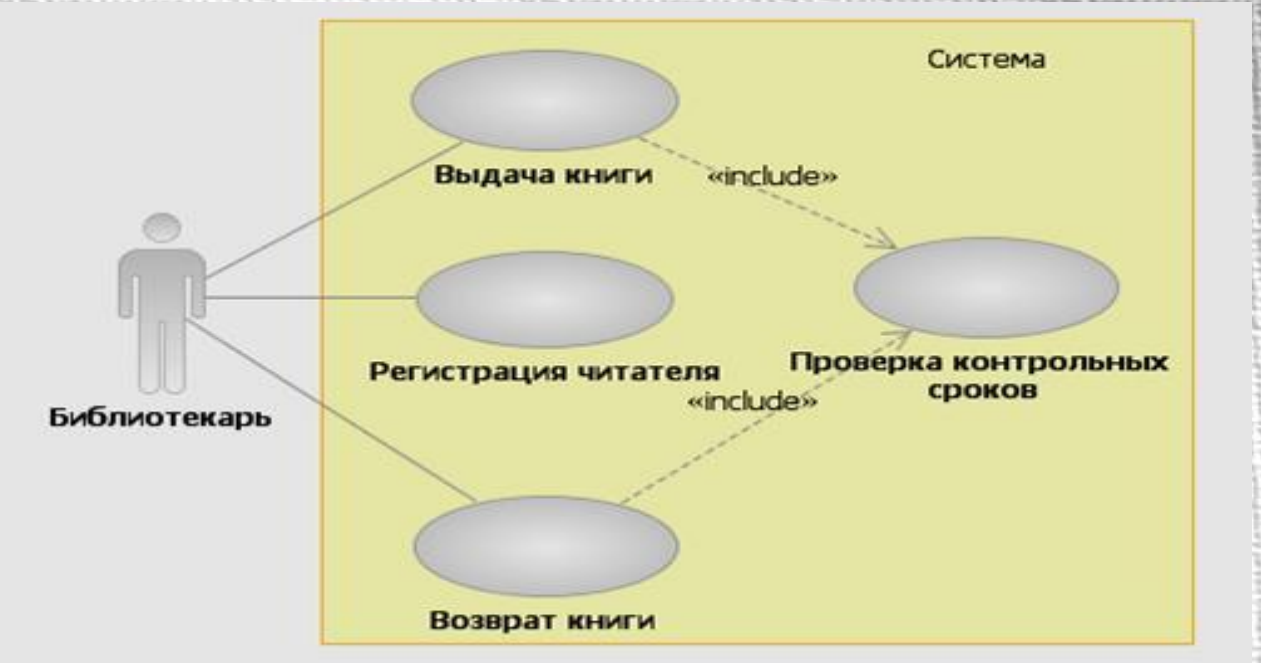
Варианты использования

Вариант использования (use case) является мощным инструментом описания требований к системе с точки зрения пользователей.

Пользовательские требования записываются в виде модели вариантов использования, которая содержит описание внешних по отношению к системе действующих лиц и их взаимодействие с разрабатываемой системой. Варианты использования может иметь от одного до нескольких сценариев, описывающих шаги взаимодействия системы с пользователем или другой системой.

Сценарий варианта использования может быть описан в текстовой форме или с помощью диаграмм деятельности.

На рисунке приведена диаграмма вариантов использования информационной системы для библиотеки в нотации UML



Пользовательские истории

User Story (юзер стори) — описание пожеланий к продукту, его функциям. Написана User Story от лица пользователя, так как этот метод используют именно для того, чтобы описать потребности клиентов.

Пользовательские истории - это простой подход к сбору требований, который отличается от вариантов использования тем, что они написаны клиентами. Клиенты описывают функции, которые система должна выполнять. Пользовательские истории состоят всего из несколько предложений.

На начальном этапе сбора требований можно считать, что именно ПИ ложатся в основу реестра требований. Требованиями ПИ становятся после проведения их анализа.

Допустим, разработчикам нужно выпустить приложение для заказа такси. User Story может выглядеть в таком виде:

Как пассажир, я хочу бронировать поездку заранее, чтобы быть уверенным в подаче машины к определенному времени.

Здесь мы от лица пользователя приложения описали новую функцию, которая важна для клиентов, и указали, почему она необходима. То есть данный инструмент нужен, чтобы изучить контекст: что именно важно аудитории в продукте и по какой причине.

При разработке User Stories важно описать три составляющие:

1. **роль** — в нашем примере это пассажир, а еще может быть пользователь приложения, посетитель ресторана и т. д.;
2. **действие** — что хочет получить пассажир, то есть иметь возможность бронировать поездку;
3. **ценность юзера** — почему пользователю нужно описанное действие.

Проблемы, возникающие при выявлении требований

1. Проблемы классификации требований
2. Проблемы формулировки требований
3. Терминология
4. Предвзятые решения

1 Проблемы классификации требований

Сложной задачей при выявлении требований является идентификация их типов. На практике, особенно при проведении интервью, требования формулируемые пользователем могут относиться к различным типам требований. Существуют следующие рекомендации о том, как «понимать» пользователей и классифицировать требования:

- **Бизнес требования** - любая информация, описывающая рынок или какие-либо интересы бизнеса.
- **Ключевые возможности** - предположения пользователя о выполняемых системой задачах.
- **Вариант использования** - пользователь говорит о том, что ему необходимо что-то сделать

- **Характеристика качества** - необходимые характеристики системы, насколько хорошо работает система.
- **Ограничения** - пользователь говорит о том, что система или функция должна соответствии стандарту или правилу, реализована на предопределенной платформе
- **Функциональные требования** - передаваемая информация, допустимые значения данных, поведенческие характеристики системы.

❷ Проблемы формулировки требований

Во время интервью многие пользователи и заказчики системы сталкиваются с проблемой объяснения выполняемых задач аналитику. Они помнят исключения и забывают порядок действий, забывают рассказать о простых на их взгляд задачах. Данная ситуация может привести к тому, что один и тот же процесс будет описан по-разному несколькими пользователями, что введет в заблуждение остальных и создаст ложное единое мнение.

Для решения подобной проблемы, аналитик может наблюдать за работой пользователя и документировать все действия, выполняемые пользователем в его повседневной работе, или попросить пользователя самостоятельно документировать все выполняемые действия.

❸ Терминология

В процессе разработки системы заказчики и пользователи выражаются с использованием терминологии, являющейся специфичной для их предметной области. Разработчики программного продукта должны понимать терминологию заказчика, для этого аналитик должен собирать термины предметной области заказчика и документировать их в глоссарии. Глоссарий терминов позволит не только понять требования пользователей, но и разобраться в предметной области разрабатываемой системы.

4 Предвзятые решения

Иногда заказчики и пользователи считают, что сами знают решение проблем их деятельности, вместо описания проблемы они высказывают идею решения этой проблемы, которая могут быть неоптимальными или просто ошибочными.

Для решения данной проблемы необходимо анализировать причины, т.е. определять, почему пользователи хотят иметь некоторою функциональность системы, действительно она нужна или является субъективным желанием пользователя.

Составление концепции

Концепцию следует обсудить с командой и опубликовать ее на веб-портале проекта.

Концепция — это краткое изложение тех выгод, которые принесет использование продукта. Что пользователи смогут делать из того, чего раньше сделать не могли? Концепция помогает прояснить сферу применения продукта.

Если продукт уже существует, составляется концепция создаваемой версии. Что пользователи продукта смогут делать из того, чего раньше сделать не могли?

В итоге…

Результатом выявления требований является ***реестр требований***. Требования обычно оформляются в простой письменной форме, без какой-либо особой регламентации. Типовой пример оформления требования к программе электронной почты - "*Система должна позволять набирать текст сообщения с возможностью форматирования текста и вставки смайликов*". Данные требования далеко не во всем могут удовлетворять критериям, сформулированным выше: *они могут противоречить друг другу, быть неясными, неточными* и т.д. Тем не менее, документ "Требования пользователей", несмотря на невысокий уровень формализации, играет очень важную роль: в нем собраны мнения всех заинтересованных сторон и главная цель сбора начальных требований заключалась в том, чтобы получить по возможности как можно более полный набор требований, не пропустив чего-то важного.

Основные ошибки

Основными ошибками, допускаемыми этапе первичного сбора требований, являются:

- Нечеткое определение границ проекта, что приводит к затягиванию сроков проекта, перерасходу бюджета за счет включения в продукт второстепенных возможностей.
- Недостаточно четко определенные группы пользователей продукта
- В каждой группе не выделены представители, наиболее заинтересованные в продукте, готовые продуктивно сотрудничать с командой разработчиков
- Попытка с первого раза максимально детализировать и проанализировать все требования

Рекомендации

Чтобы не допустить этих ошибок можно дать несколько рекомендаций:

- Разделите пользователей продукта на категории в соответствии с их ролями.
- В каждой группе выберите одного - двух человек в качестве представителей интересов группы. Выбранные люди должны быть лидерами (пусть даже и неформальными) в своей группе, четко представлять бизнес процессы, в которых участвует группа и лояльно настроенных по отношению к проекту создания и внедрения разрабатываемой системы.
- Не пытайтесь сразу и досконально описать все требования. Это приведет лишь к затягиванию работ. Требования должны прорабатываться и детализироваться в ходе проекта.
- Каждому требованию присваивайте приоритетность. Это позволит реализовывать в первую очередь самые необходимые требования, откладывая реализацию второстепенных возможностей на более поздний срок. Как правило, достаточно ввести три – четыре градации приоритетности требований.

Интернет-проект

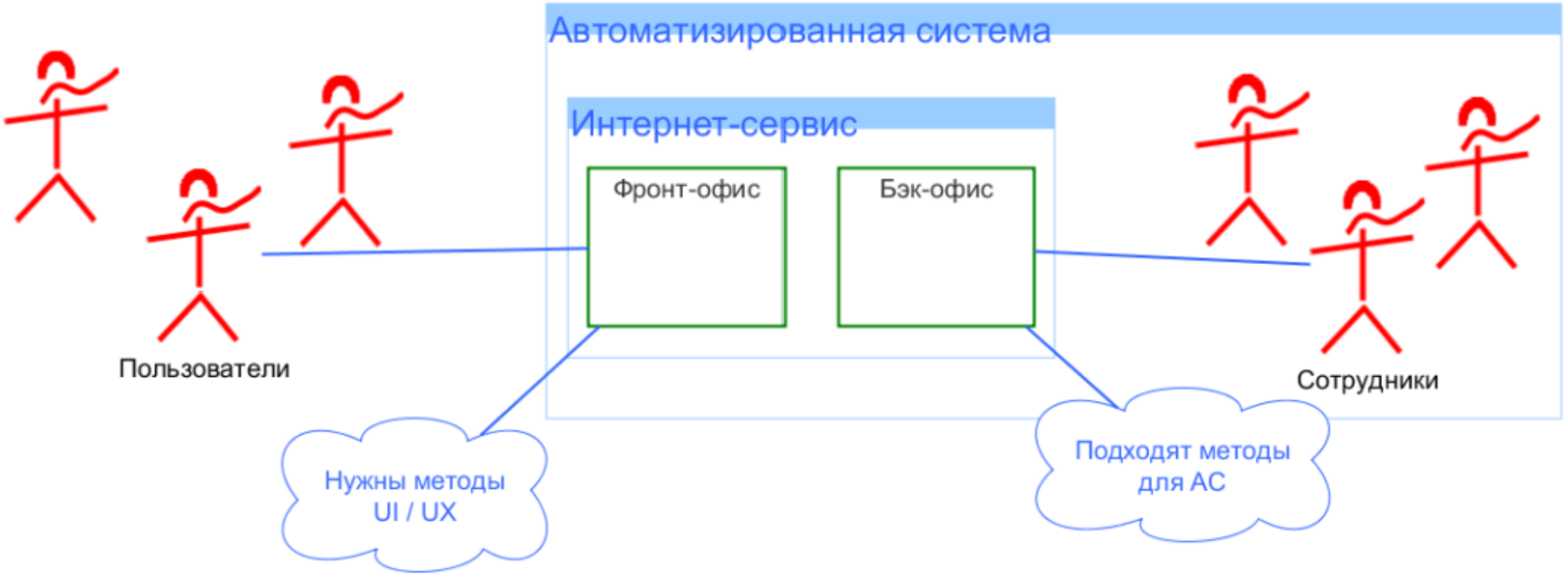
Какие из этих методов применяются, если говорить об интернет-проектах?

У большинства интернет-сервисов есть две стороны. Их часто называют фронт-офис и бэк-офис.

Фронт — это то, чем избушка повёрнута к лесу, где она взаимодействует с пользователем, для которого продукт создан. Это массовый пользователь интернета.

А с другой стороны, для того, что бы этот сервис обслуживать, есть сотрудники компании, которые этот сервис эксплуатируют, и это бэк-офис.

Две стороны сайта



Например, если интернет-сервис принимает деньги от пользователя, то с другой его стороны есть бухгалтерия, и для неё нужно реализовать какую-то функциональность, которая позволяет ей своих целей достигать. Если интернет-сервис продаёт билеты или позволяет выбирать какие-то билеты для авиакомпаний, то, соответственно, есть сотрудники, которые работу этого сервиса обеспечивают. Техническая поддержка есть, наверное, есть служба эксплуатации, есть, опять же, бухгалтерия. И со стороны бэк-офиса вполне применимы методы, которые разработаны для автоматизированных систем. А это в первую очередь Use Cases и бизнес-процессы. Т. е. при разработке интернет-сервиса для бэк-офиса эти методы бывают очень хорошо применимы.

Если говорить о массовом пользователе, то его в эту схему уже не втиснешь. Таких пользователей не заставишь достигать своих целей в рамках вашей системы. И поэтому применяются другие методы. Для них больше подходят User Stories и метод персон. Он, собственно, для решения этой проблемы и предназначен.

Если вы как аналитик участвуете в разработке требований к системе, то вы должны принимать участие в разработке двух сторон системы и применять соответствующие методы.

2

Формирование и анализ требований

- Уточнение требований (полнота требований)
- Структуризация и идентификация требований
- Проверка на непротиворечивость требований
- Установление приоритетов
- Модели требований

Зачем проводить анализ?

- После выявления требований, аналитик имеет большое количество различной информации, полученной из интервью, при обследовании, из анкет и других источников.
- Полученную информацию необходимо уточнить, структурировать, исключить дублирование, сформулировать в виде требований и определить их приоритеты. Эти задачи решает стадия анализа требований.
- Целью анализа требований является получение понятных и непротиворечивых требований, на основе которых можно проектировать и реализовывать программное приложение.
- На данной стадии создаются модели требований, определяются недостающая информация, требования уточняются и для каждого требования задаются значения его атрибутов.

Основная цель анализа требований

Основной целью анализа требований является их систематизация и избавление от дублируемых данных. Это достигается за счет разделения пользовательских историй на отдельные пакеты по функциональному признаку и их иерархической структуризации.

По окончании этапа анализа требований, многостраничный документ, содержащий сотни пользовательских историй, будет разбит на части. Каждая часть будет освещать только необходимую функциональность, а в её основе будет стоять диаграмма вариантов использования, на основе которой можно будет легко увидеть все требуемые функции системы. Описание вариантов использования не будут дублироваться, а лишь дополнять друг друга

Основные задачи этапа

1. **УТОЧНЕНИЕ ТРЕБОВАНИЙ (ПОЛНОТА ТРЕБОВАНИЙ)**
2. **СТРУКТУРИЗАЦИЯ И ИДЕНТИФИКАЦИЯ ТРЕБОВАНИЙ**
3. **ПРОВЕРКА НА НЕПРОТИВОРЕЧИВОСТЬ ТРЕБОВАНИЙ**
4. **УСТАНОВЛЕНИЕ ПРИОРИТЕТОВ**
5. **МОДЕЛИРОВАНИЕ ТРЕБОВАНИЙ**

1 Уточнение требований

Во время анализа требований может возникнуть ситуация, когда собранные требования являются неполными, неточными или одно требование противоречит другому. В данной ситуации аналитик должен снова взаимодействовать с заказчиком, уточнять непонятные моменты, фиксировать все, что необходимо разработчикам и проектировщикам для реализации данных требований.

Пример неадекватной формализации первичных требований

Ян Грэхем приводит следующий пример неадекватной формализации, выполненной в рамках реального проекта.

Одно из требований к системе управления самолетом состояло в том, что при посадке тормозной двигатель малой тяги не должен включаться, пока самолет не коснется взлетно-посадочной полосы. При формализации было принято решение, что условие касания полосы эквивалентно вращению колес шасси самолета — ведь при этом колеса должны войти с ней в контакт.

Все работало хорошо, пока однажды самолету не пришлось садиться на полосу после сильного ливня, покрытую водой, по которой колеса начали скользить. В результате тормозной двигатель не включился, и самолет выехал за пределы полосы.

2

Структуризация и идентификация требований

Целью структуризации требований является

1. Определение границ системы.
2. Разнесение требований по функциональным областям предметной области.
3. Декомпозиция.

1. Структуризация требований
(границы системы)

Для определения границ системы аналитик должен идентифицировать роли пользователей, которые будут работать с системой, и внешние системы, которые будут взаимодействовать с разрабатываемой системой, определить потоки входной и выходной информации.

2. Структуризация требований
(по функциональным областям)

Для структурирования требований по функциональным областям применяются пакеты (каталоги) требований. Пакеты могут объединять требования по разным признакам:

- требования, принадлежащие одному типу, например, все бизнес требования.
- требования, связанные с конкретным пользователем или действующим лицом
- требования, принадлежащие к одной функциональной подобласти предметной области. Например, пакет *Поиск* может содержать все требования, относящиеся к поиску информации.

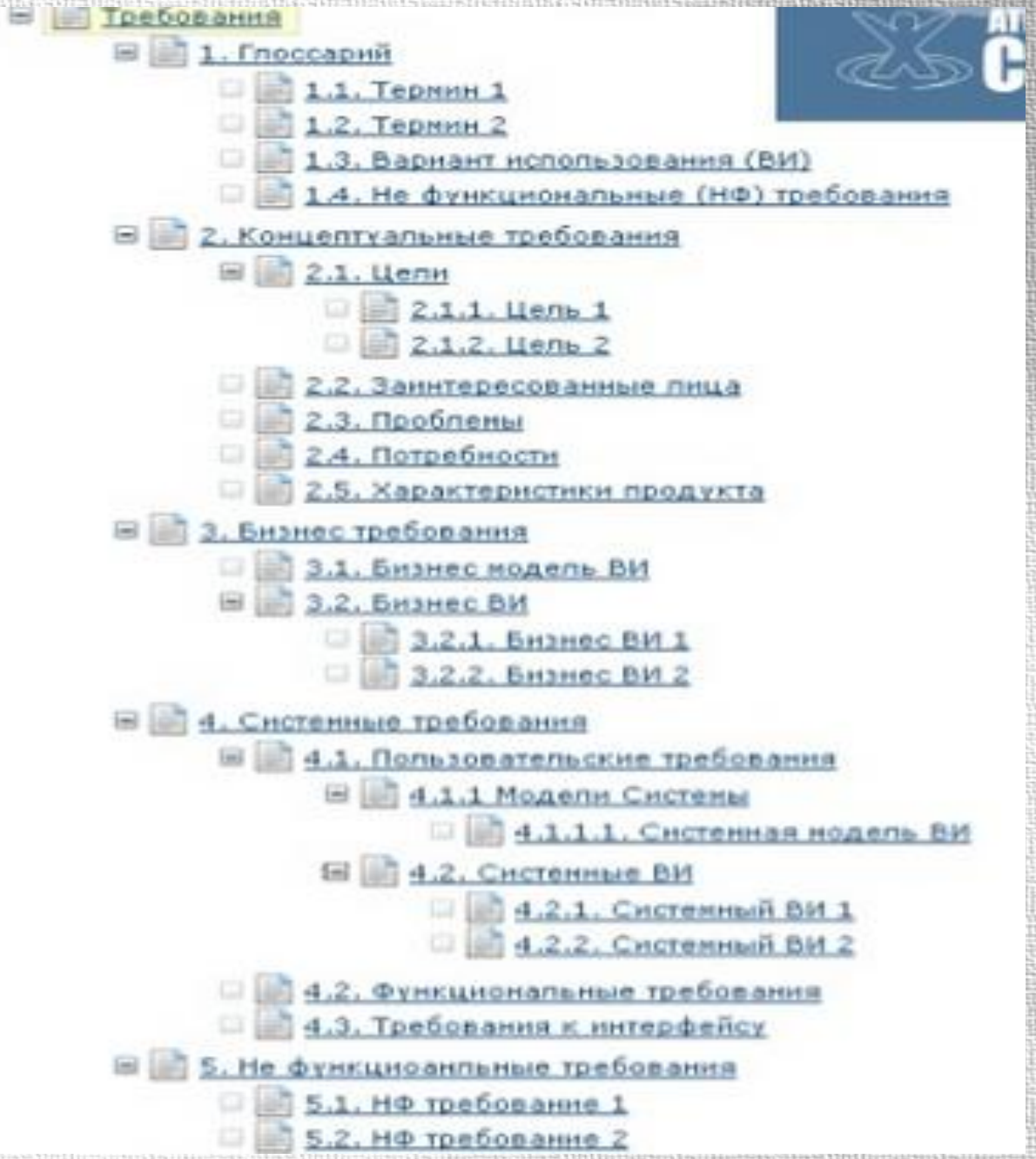
Пакеты могут быть вложенными друг в друга, что позволяет **создавать древовидную структуру требований.**

3. Структуризация требований
(декомпозиция)

Помимо структуризации требований по пакетам, применяется ***декомпозиция требований***.

Декомпозиция требований позволяет разбивать требования на более детальные. Каждое родительское требование может иметь несколько дочерних требований, а одно дочернее требование принадлежать только одному родителю.

Формирование структуры требований с произвольным уровнем вложенности



Идентификация требований

Типичная система может состоять из сотен или тысяч формулировок требований, наподобие тех, что приведены выше. Для надлежащего управления таким огромным количеством требований их необходимо пронумеровать с помощью некоторой *схемы идентификации (identification scheme)*. Схема может включать классификацию требований в виде групп, которые легче поддаются управлению.

Существует несколько методов идентификации и классификации требований.

– *Уникальный идентификатор* – обычно последовательный номер, присвоенный вручную или сгенерированный с использованием базы данных CASE-средства (т.е. базы данных (или репозитория), в которой CASE-средства хранят артефакты, выработанные в результате анализа или проектирования).

– *Последовательный номер внутри иерархии документа* – присваивается с учетом положения требований в пределах документа описания требований.

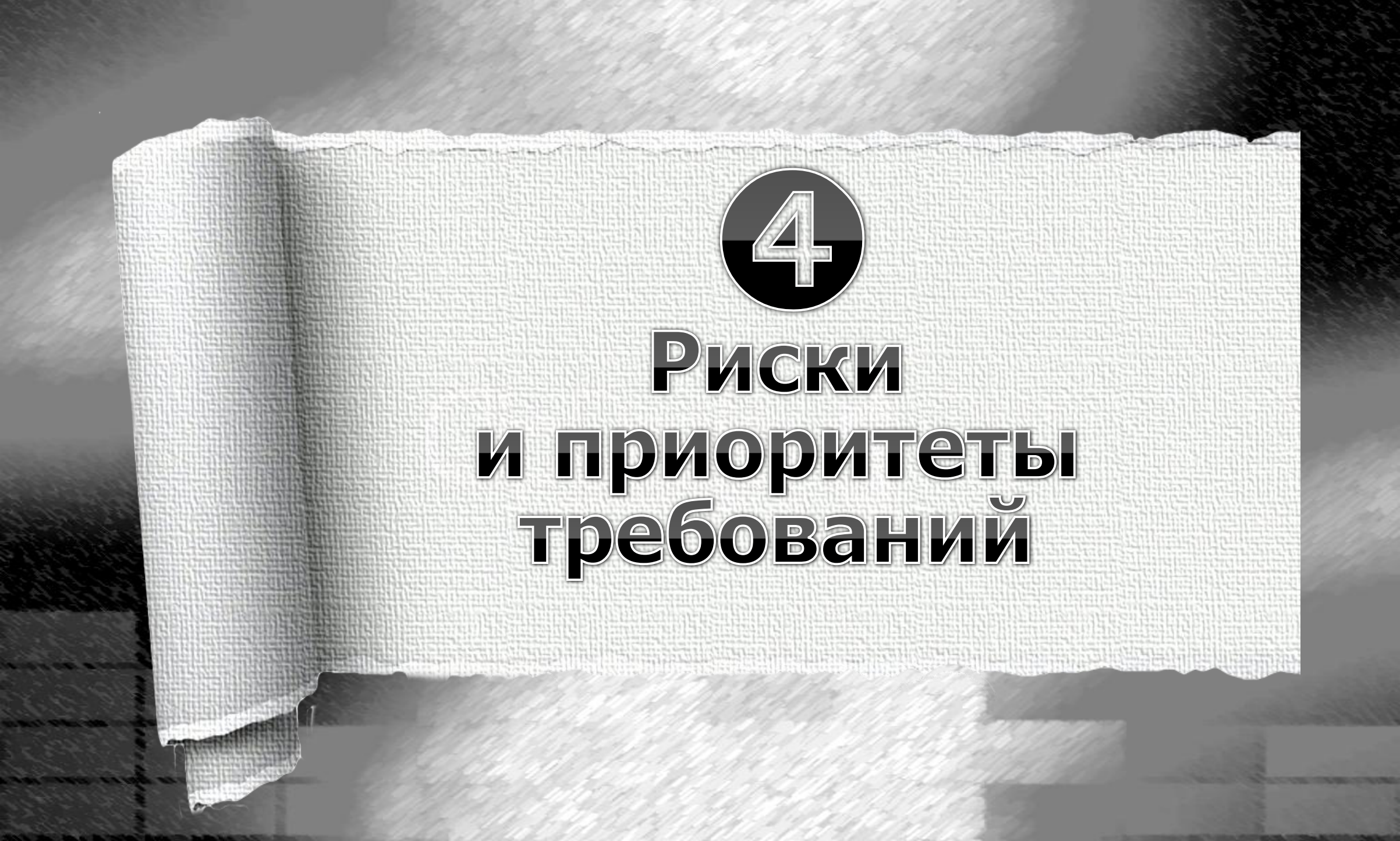
– *Последовательный номер в пределах категории требований* – присваивается в дополнение к мнемоническому имени, которое обозначает категорию требований (где под категорией требований подразумевают функциональное требование, требование к данным, требования к производительности, требования к безопасности и т.д.)

3 Проверка на непротиворечивость требований

Полагая, что раз все требования четко идентифицированы и пронумерованы, можно сконструировать матрицу зависимостей требований (requirements dependency matrix) (или матрицу взаимодействия (interaction matrix требований)) . В столбце и строке заголовка перечислены упорядоченные идентификаторы требований, как показано на рисунке.

Требование	TI	T2	T3	T4
T1	X	X	X	X
T2	Конфликт	X	X	X
T3			X	X
T4		Перекрытие	Перекрытие	X

- Верхняя правая часть матрицы (над диагональю включительно) не используется.
- Оставшиеся ячейки указывают на то, перекрываются ли два любых требования, противоречат друг другу или независимы друг от друга (пустые ячейки). Противоречивые требования необходимо обсудить с заказчиками и при возможности переформулировать для смягчения противоречий (фиксацию противоречия, видимую для последующей разработки, необходимо сохранить). Перекрывающиеся требования также должны быть сформулированы заново, чтобы исключить совпадения.
- Матрица зависимости требований представляет собой простой, но эффективный метод обнаружения противоречий перекрытий, когда количество требований относительно невелико. В противном случае описанный метод все же можно применить, если удается сгруппировать требования по категориям, а затем сравнить их отдельно в пределах каждой категории.



4 Риски и приоритеты требований

Риски и приоритеты

После того, как в результате снятия противоречий и устранения повторов в требованиях выработан пересмотренный набор требований, их необходимо подвергнуть анализу рисков и назначить им приоритеты.

- ***Анализ рисков*** *(risk analysis)* направлен на идентификацию требований, которые являются потенциальными источниками трудностей в разработке.
- ***Назначение приоритетов*** *(prioritization}* необходимо для того, чтобы обеспечить возможность без труда изменить рамки проекта в случае возникновения непредвиденных задержек.

Приоритеты требований

В идеале приоритеты требованиям назначают отдельные заказчики в процессе выявления требований. Затем они согласовываются на совещаниях и снова изменяются после приложения к ним факторов риска.

Для исключения неопределенности и облегчения назначения приоритетов количество групп приоритетов не должно быть слишком велико. Обычно используется от трех до пяти приоритетов: "высокий", "средний", "низкий" и "неопределенный".

Альтернативный перечень приоритетов может выглядеть, например, так: "существенное", "полезное", "трудно определимое" и "отложенное".

Пример учета важности и срочности требований при установке приоритетов

	Важно	Не очень важно
Срочно	Высокий приоритет	Это делать не надо!
Не очень срочно	Средний приоритет	Низкий приоритет

Типичные виды рисков

Требованиям свойственны следующие типичные виды *рисков:*

- *Технический риск*, когда требование технически трудно реализовать.
- *Риск, связанный со снижением производительности*, когда требование — будучи реализовано — может неблагоприятно сказаться на времени реакции системы.
- *Риск, связанный с нарушением безопасности*, когда требование — будучи реализовано - может создать бреши в защите системы.
- *Риск, связанный с процессом разработки,* когда для реализации требования необходимо использование необычных методов разработки, незнакомых разработчикам (например, методов формальной спецификации).

- *Риск, связанный с нарушением целостности баз данных*, когда требование не может быть легко проверено и может привести к противоречивости данных.
- *Политический риск*, когда требование может оказаться трудным выполнить по внутриполитическим причинам.
- *Риск, связанный с нарушением законности,* когда требование может привести к нарушению действующих законов или ожидаемого изменения закона.
- *Риск, связанный с изменчивостью*, когда требование может потенциально изменяться или эволюционировать в течение процесса разработки.

Атрибуты требования

Любое требование в базе проекта имеет следующие атрибуты:

- **Приоритет**
  - (высокий, средний, низкий)
- **Статус**
  - (предложено, одобрено, реализовано, верифицировано)
- **Стоимость реализации**
  - (высокая, средняя, низкая – или числовое значение)
- **Сложность реализации**
  - (высокая, средняя, низкая)
- **Стабильность**
  - (высокая, средняя, низкая)
- **Ответственный исполнитель**
- **№ итерации (этапа) в плане**

Атрибуты требований описываются **матрицей атрибутов требований**, где для каждого типа требований перечисляются требования по одной оси и атрибуты требований этого типа по другой.

Для каждого требования указываются значения его соответствующих атрибутов.

5 Модели требований

Существует два основных метода проектирования —

1. **проектирование на основе вариантов использования и**
2. **проектирование на основе требований**.

Проектирование на базе вариантов использования считается более эффективным, так как этот метод позволяет не терять связь с пользовательскими историями и прекрасно иллюстрирует требуемое поведение системы в целом, а, следовательно, гарантирует востребованность всего функционала, который будет создан (очень расточительно и болезненно для разработчиков писать код, которым никогда не удастся воспользоваться).

Проектирование на основе требований

Но все же есть условия, при которых аналитик может пренебречь этим методом в пользу проектирования на базе требований:

- Проектирование на основе требований следует предпочесть, если необходимо сократить затраты на стадию анализа до минимума, а количество пользовательских историй в пакете не велико (их содержимое можно просмотреть не дольше чем за 10 минут)
- Команда разработки не умеет или не хочет работать с вариантами использования и требует предоставление требований к системе в классическом виде.

Модели визуального представления требований

- диаграммы потоков данных (data flow diagrams, DFD);
- диаграммы рабочих потоков, такие как диаграммы swimlane;
- диаграммы переходов состояний (state-transition diagrams, STD) и таблицы состояний;
- **карты диалоговых окон**;
- таблицы и деревья решений;
- таблицы событий и реакций;
- деревья функций
- диаграммы вариантов использования;
- диаграммы процессов;
- диаграммы «сущность-связь» (entity-relationship diagrams, ERD)

Карта диалоговых окон

Карта диалоговых окон представляет дизайн пользовательского интерфейса на высоком уровне абстракции. На ней показаны элементы диалоговых окон в системе и ссылки навигации между ними, но не показан подробный дизайн экрана.

Пользовательский интерфейс также можно рассматривать как набор изменений состояний. Только один элемент диалогового окна (такой, как меню, рабочая область, диалоговое окно, командная строка или дисплей сенсорного экрана) доступен в определенный момент времени для ввода информации пользователем. Пользователь может перейти к другим определенным элементам диалогового окна, связанным с действием, которые он выполняет в активной области ввода.

Карта диалоговых окон на самом деле представляет собой всего лишь пользовательский интерфейс, смоделированный в диаграммы переходов между состояниями.

Так же, как и обычные диаграммы перехода состояния, карта диалоговых окон показывает каждый элемент диалогового окна как состояние (прямоугольник) и каждую допустимую возможность перемещения как переход (стрелка). Условие, инициирующее перемещение по пользовательскому интерфейсу, показано в виде текстового ярлыка на стрелке перехода.

Существует несколько типов инициализирующих условий:

- **действие пользователя**, например нажатие функциональной клавиши, щелчок гиперссылки или кнопки диалогового окна или жест на сенсорном экране;
- **значение данных**, такое как недостоверная информация, в результате чего появляется сообщение об ошибке;
- **системное условие**, например отсутствие бумаги в принтере;
- **некоторые комбинации вышеперечисленных элементов**, например ввод номера элемента меню и нажатие клавиши Enter.

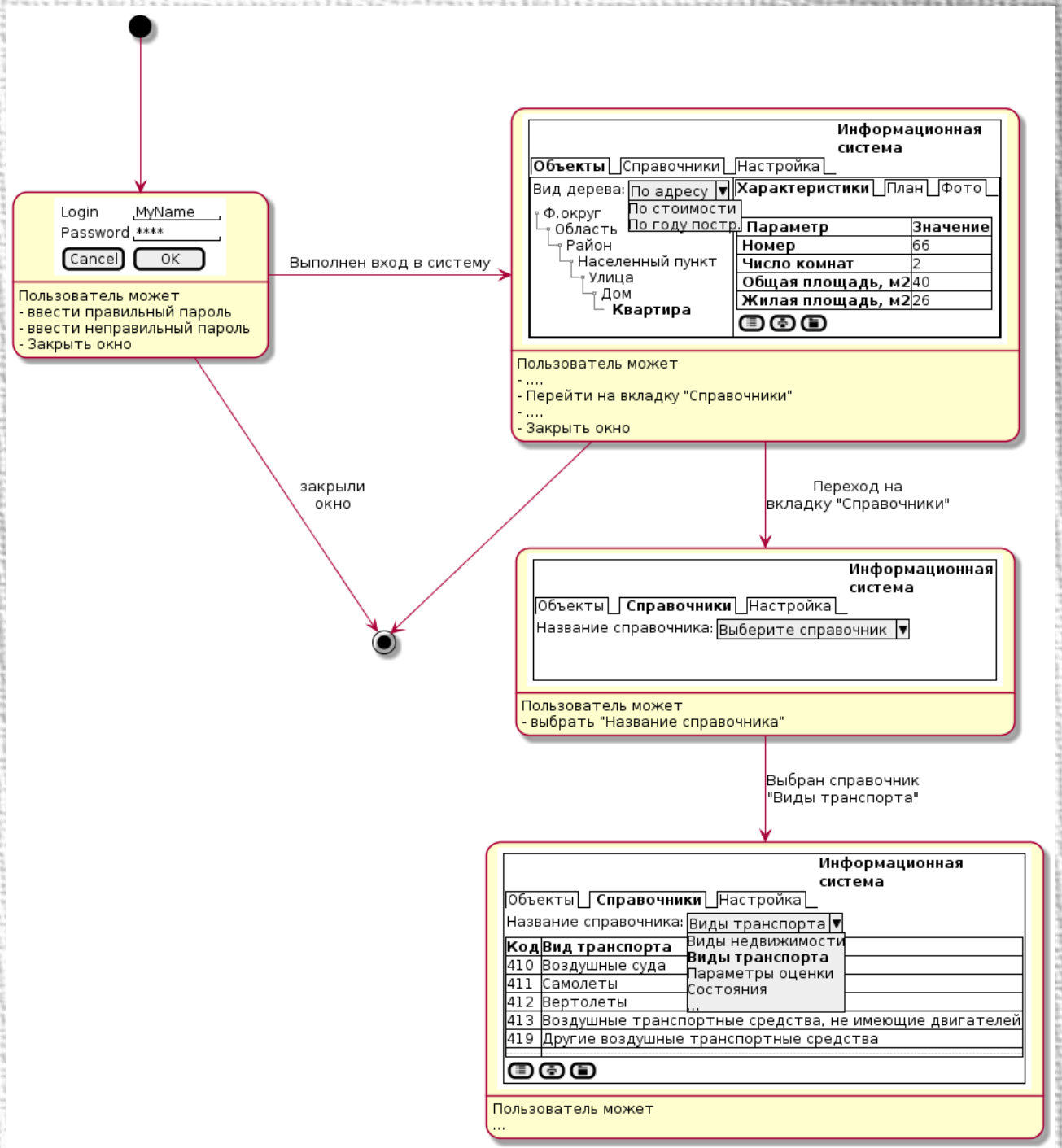
Карта диалоговых окон — это прекрасный способ представить взаимодействия действующего лица и системы, описываемые вариантом использования. Карта диалоговых окон позволяет отобразить альтернативные направления в виде ответвлений от нормального направления.

Некоторые переходы на карте диалоговых окон позволяют пользователю откатить операцию. Пользователей раздражает, если они передумали, а им все-таки приходится завершать задачу. Карты диалоговых окон позволяют облегчить и упростить работу, предлагая функции отката и отмены в стратегических точках.

Карта создается для каждого прецедента!

Карта диалоговых окон

Эта карта создана с помощью PlantUML



Основные приемы работы

При анализе и детализации требований основными приемами работы с участниками проекта являются проведение круглых столов, обсуждений, «мозговых штурмов», демонстраций предварительных версий разрабатываемой системы.

Важным моментом проведения обсуждений является их **документирование и протоколирование**. Это позволяет не держать постоянно в голове все важные детали проекта, а так же отследить источники возникновения тех или иных требований. Довольно часто представители пользователей боятся подписывать протоколы обсуждений. Страх связан с ожиданием каких-либо карательных мер. Следует объяснить пользователям, что цель получения их подписи под протоколом не в том, чтобы потом наказать их, а всего лишь в том, чтобы убедиться, что между участниками обсуждения достигнуто взаимопонимание и выработано совместное решение.

Вывод

На этапе анализа требований проходит структуризация уже собранных ранее требований. Цель этапа — предоставить четкий список не дублируемых требований к системе, которые должны быть выделены из избыточных и частично дублирующихся сценариев и пользовательских историй, которые были получены на предыдущем этапе.

Писать просто и ясно так же трудно, как быть искренним и добрым.

Сомерсет Моэм, (William Somerset Maugham).
писатель, 1874-1965

3

Документирование требований

К этапу документирования проект приходит, уже располагая достаточным объемом требований, чтобы изучив их, можно было получить представление о продукте.

Как их обрабатывают дальше?

Часто на практике, разрабатываемое программное приложение предназначается для автоматизации деятельности некоторых бизнес процессов заказчика. В большинстве случаев, данные процессы уже задокументированы в различных регламентах, инструкциях, правилах и другой документации. В таком случае аналитику не требуется проводить интервью с заказчиком и требуется лишь изучить существующую документации и получить из нее необходимую информацию для дальнейшего формирования из нее требований к программному обеспечении.

Спецификации на естественном языке

Независимо от вида требований, документировать их нужно так, чтобы это обеспечивало удобный доступ и просмотр. Простейший способ – использовать естественный язык. Для проектов, в которых заняты интернациональные команды, таковым является английский. В остальных случаях можно использовать русский.

Специфицирование на естественном языке является основным методом при составлении **концепт-документов и бизнес-правил**. **Пользовательские требования** изначально записываются на естественном языке, но затем в процессе анализа модифицируются и документируются с помощью более формальных технологий. **Технические требования**, как правило, содержат лишь отдельные части, специфицированные на естественном языке.

Правила описания

При документировании на естественном языке необходимо руководствоваться несколькими важными правилами. Их соблюдение позволяет избежать наиболее распространенных ошибок.

- Используйте шаблоны спецификации требований к ПО
- Сопровождайте требования указанием их источников
- Присваивайте требованиям уникальные идентификаторы
- Указывайте количественные характеристики

  Нефункциональные требования желательно сопровождать количественными характеристиками.
- Не включайте в требования элементы дизайна

  В процессе анализа требований появляется значительное число материалов, содержащих дизайн продукта. Необходимо избежать включения этой информации в спецификацию требований к ПО. Детали архитектуры и внутреннего устройства разрабатываемой системы не являются требованиями. Они разрабатываются и перерабатываются инженерами в зависимости от требований, но представляют информацию иного уровня. Требования показывают ЧТО необходимо сделать, дизайн – КАК достигнуть результата.

- Определяйте термины в начале спецификации
  - Для понимания текста важно, чтобы читатель знал, что обозначает каждое слово. Поэтому в начале документа необходимо отвести место для словаря терминов. В нем должны объясняться все использованные сокращения, аббревиатуры и специальные имена. Читатель должен иметь представление о терминах до того, как углубится в изучение спецификации.
- Используйте одинаковую форму требований
  - Для функциональных требований рекомендуется использовать запись в следующей форме: **Триггер Объект Действие Условие**.
  - Объект и Действие должны присутствовать обязательно. Триггер и Условие опциональны. Объект – это действующее лицо, которым может быть как элемент системы, так и внешний пользователь. Действие описывает функционал. Триггер специфицирует, что инициирует действие. Условие – это описание особенностей указанного действия. Например, «При нажатии кнопки Старт в главном окне должно появиться основное меню, если загрузка прошла без ошибок.» Здесь Триггер – «При нажатии кнопки», Объект – «в главном окне», Действие – «должно появиться основное меню», Условие – «если загрузка прошла без ошибок».
- Используйте слово «должен»
  - Для того чтобы не возникало сомнений в поведении и характеристиках системы, необходимо употреблять глагол «должен» и его формы. Важно отказаться от использования других слов, являющихся более мягкими.

- Избегайте в требованиях негативных утверждений
- Количество деталей требований может варьироваться

  Один из самых сложных вопросов при описании требований на естественном языке: каков должен быть уровень детализации? Необходимо ли тщательно расписывать все нюансы продукта? Однозначного ответа на эти вопросы нет. Все зависит от проекта. Ориентируйтесь на следующие моменты:

  - Если готовится новая версия уже известного продукта, или продукт имеет широкораспространенный аналог, то вместо описания можно использовать ссылки на сходные части аналога.
  - Для опытной команды многими деталями можно пренебречь.
  - Недопонимание требований при реализации одних частей системы является критическим, других – нет. Поэтому детализация зависит от тех рисков, которые связаны с частями системы.
  - Если команда расположена в одном помещении и может свободно общаться, то важность деталей спецификации ниже, чем в случае, когда команда интернациональна и географически распределена на тысячи километров.

Какие документы использовать?

Не так важно, как будут называться документы с требованиями и сколько их будет. Главное, чтобы в них нашли отражение 5 ключевых вопросов.

Что хотят?

Как будут использовать?

Как достичь?

Как разделить по задачам?

Как проверить результат?

Основные документы

К основным документам, описывающим требования, относятся:

1. **Глоссарий**
2. **Документ-концепция**
3. **Спецификация требований**

1. Глоссарий

Глоссарий (Glossary) представляет собой документ, описывающий все основные определения, встречающиеся в проекте. Сосредоточение определений в одном документе позволяет отслеживать изменения определений, добавление новых и удаление старых определений. Глоссарием могут пользоваться все участники проекта по разработке программного обеспечения: аналитики, разработчики, системные архитекторы, инженеры по тестированию и заказчики. В основном глоссарий содержит определения предметной области и дает участникам проекта возможность понимания предметной области. Также глоссарий позволяет участникам проекта разговаривать на «одном» языке. За ведение глоссария обычно отвечает системный аналитик.

БЕЗ ГЛОССАРИЯ

Я знаю, что Вы полагаете, что Вы поняли то, что Вы думаете, что я сказал, но я не уверен, что Вы сделали то, что Вы слышали, и не то, что я подразумевал.

2. Документ – концепция (Vision)

Документ-концепция, или документ об образе и границах проекта (Vision), описывает разрабатываемую систему в терминах заказчика и с точки зрения основных потребностей заказчика (бизнес - требований) и ключевых возможностей. В этом документе объясняется, почему заказчику нужна система, то есть, описаны цели, которые заказчик намерен достичь с ее помощью, основные заинтересованные в системе лица, преимущества системы перед аналогичными системами, концептуальная модель системы и среда, в которой будет работать система. Данный документ определяет границы проекта и в дальнейшем используется для определения бюджета и сроков реализации проекта.

Содержание концепции должно ответить на следующие вопросы, которые также могут быть разбиты на отдельные пункты:

- Каковы ключевые термины? (Глоссарий)
- Какую проблему мы пытаемся решить? (Формулировка проблемы)
- Кто является заказчиками? Кто пользователи? Каковы их запросы?
- Каковы свойства продукта?
- Каковы функциональные требования? (Прецеденты)
- Каковы нефункциональные требования?
- Каковы конструктивные ограничения?

Концепция в ГОСТ РФ

В соответствии с ГОСТ 34.601-90 "Автоматизированные системы. Стадии создания" [7.2], после этапа формирования (выявления) требований к системе выполняется этап разработки концепции системы.

Основные работы этапа:

- *Изучение объекта;*
- *Проведение научно-исследовательских работ (НИР);*
- *Разработка вариантов концепции АС;*
- *Оформление отчета о выполненной работе.*

Так как данный этап хронологически стоит на втором месте, к его началу у Разработчика на руках уже имеется документ, в котором собраны основные требования пользователей.

3. Спецификация требований (ТЗ)

Чтобы требования, выявленные и описанные, приняли силу соглашения между Заказчиком и Разработчиком, их необходимо оформить в виде документа.

Собранные требования оформляются в виде спецификации требований (Software Requirement Specification - SRS).

- **Спецификация** – достаточно точное и полное описание задачи, доступное для понимания всех заинтересованных лиц.
- **Спецификация требований** – процесс документирования системы в структурированном, доступном всем и управляемом формате.

Спецификация требований (Software Requirements Specification) используется для документирования требований к разрабатываемой системе или части системы. В спецификацию требований должна попадать информация о границах системы, выполняемых функциях, характеристиках качества, ограничениях и распределении приоритетов. В зависимости от заказчика, специфики проекта, применяемых методологий и процессов разработки спецификация требований может различаться по своему составу и структуре. В отечественных стандартах по разработке программного обеспечения, аналогом спецификации требований является техническое задание на разработку системы.

Спецификация требований является фундаментальным документом, потому что на ее основе осуществляется планирование, проектирование и реализация проекта, разработка сценариев тестирование и пользовательской документации.

Какими характеристиками должны обладать спецификации требований?

Набор требований, составляющий спецификацию, должен отвечать характеристикам:

Полнота. Никакие требования или необходимые данные не должны быть пропущены.

Согласованность. Согласованные требования не конфликтуют с другими требованиями такого же типа или с высокоуровневыми пользовательскими, системными или бизнес-требованиями. Несогласованность документов следует устранить до начала процесса разработки. Вы не всегда знаете, какое именно положение некорректно (если какое-то некорректно), пока не выполните исследование. Рекомендуется записывать автора каждого требования, чтобы узнать, кто его высказал, если конфликт все-таки будет обнаружен.

Способность к модификации. Необходимо обеспечить возможность переработки требований, если понадобится, и поддерживать историю изменений для каждого положения. Для этого все они должны быть уникально помечены и обозначены, чтобы вы могли ссылаться на них однозначно. Каждое требование должно быть записано в спецификации только единожды. Иначе вы легко получите несогласованность, изменив только одно положение из двух одинаковых. Лучше используйте ссылки на первоначальные утверждения, а не дублируйте положения. Модификация спецификации станет гораздо легче, если вы составите содержание документа и указатель. Сохранение спецификации в базе данных коммерческого инструмента управления требованиями сделает их пригодными для повторного использования.

Трассируемость. Трассируемость, или возможность для анализа, можно реализовать как в направлении назад, к первоисточникам, так и вперед, к элементам дизайна и исходному коду, который его реализует, а также к вариантам использования, которые позволяют проверить корректность, реализации. Трассируемые требования помечены соответствующими идентификаторами. Они записаны в структурированной, детализированной форме, в противоположность параграфам в длинной повествовательной форме. Избегайте слипания множества требований в один ком, отдельные требования можно трассировать к различным элементам дизайна и кода.

Каких требований не должно быть?

Спецификация требований не содержит деталей дизайна или реализации (кроме известных ограничений), данных о планировании проекта или сведений о тестировании. Для проекта, как правило, создаются требования других типов: документ, где описана среда разработки, ограничения бюджета, руководство пользователя или требования для выпуска продукта и продвижения его в поддерживаемую среду. Все это относится к требованиям к проекту, но не к продукту.

Спецификация нефункциональных требований

Описание нефункциональных требований обычно осуществляется в форме, близкой к свободному формату описания варианта использования.

RUP рекомендует концентрировать нефункциональные требования в документе, описывающем вариант использования во всех случаях, когда это возможно.

В случае, если нефункциональные требования носят общий характер, они выносятся в документ «Дополнительная спецификация».

Утверждение – завершающая стадия документирования. В англоязычных странах ее называют **Baseline**. На этой фазе принимается решение о том, что требования к продукту могут являться полноценной основой для разработки.

Все последующие исправления в спецификации вносятся в согласии с практиками этапа управления требованиями.

Шаблоны описания требований

Описание шаблона

Предлагается описывать функциональное требование к системе и функции, его реализующие, с использованием следующего шаблона.

1. **Общие сведения о требовании**
2. **Функции, реализующие требования**
3. **Связи между требованием и функциями**
4. **Описание процесса выполнения функции**
5. **Состав, экранных форм, связанных с функцией**
6. **Описание печатных форм**
7. **Описание импортируемых/экспортируемых данных (импортируемых данных нет)**
8. **Нефункциональные требования, связанные с функциональным требованием**

Описание шаблона ниже дано на примере описания конкретного требования.

Шаблон описания требования

1. Общие сведения о требовании

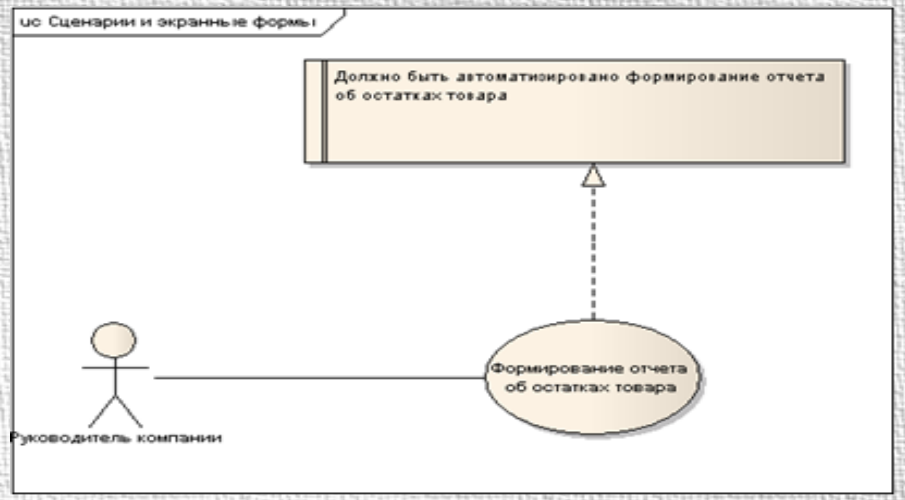
1.	**Требование**	Должно быть автоматизировано формирование отчета об остатках товара
2.	**Цель, которая будет достигнута при реализации требования**	Оперативное получение информации о текущих остатках на складе компании
3.	**Причина возникновения требования**	Требование руководителя компании
4.	**Пользователи, которым доступна работа с функциями системы, реализующими требование**	Руководитель компании
5.	**Источник данных (ручной ввод, использование записей БД, данных из смежной системы)**	Отчет должен формироваться на основе записей в базе данных, содержащих информацию о количестве остатков товара на складе
6.	**Правила, связанные с требованием**	Отчет формируется в двух экземплярах

2. Функции, реализующие требования

№	Название функции
1.	Формирование отчета "Остатки товара"

3. Связи между требованием и функциями

В разделе должно быть представлена модель, отображающая связи между требованием и функциями, реализующими требование, и если требуется, описание связей. Модель "Требование и функции":

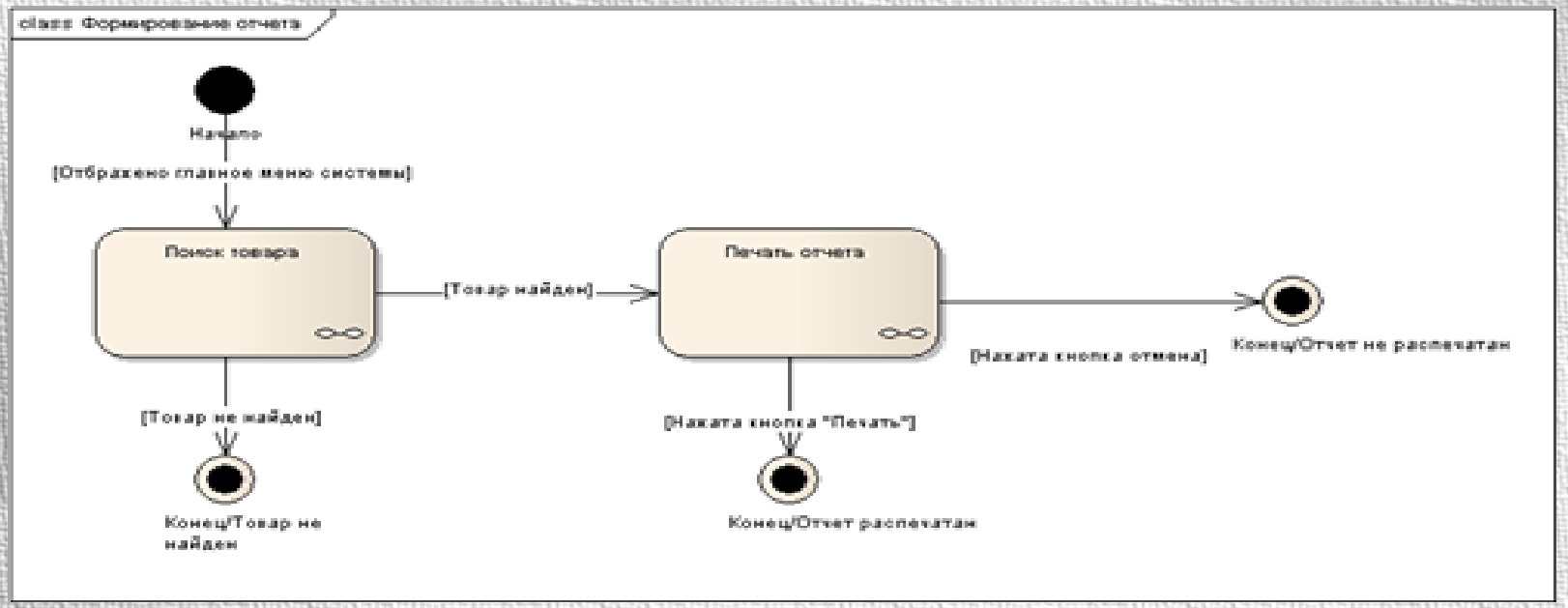


4. Описание процесса выполнения функции

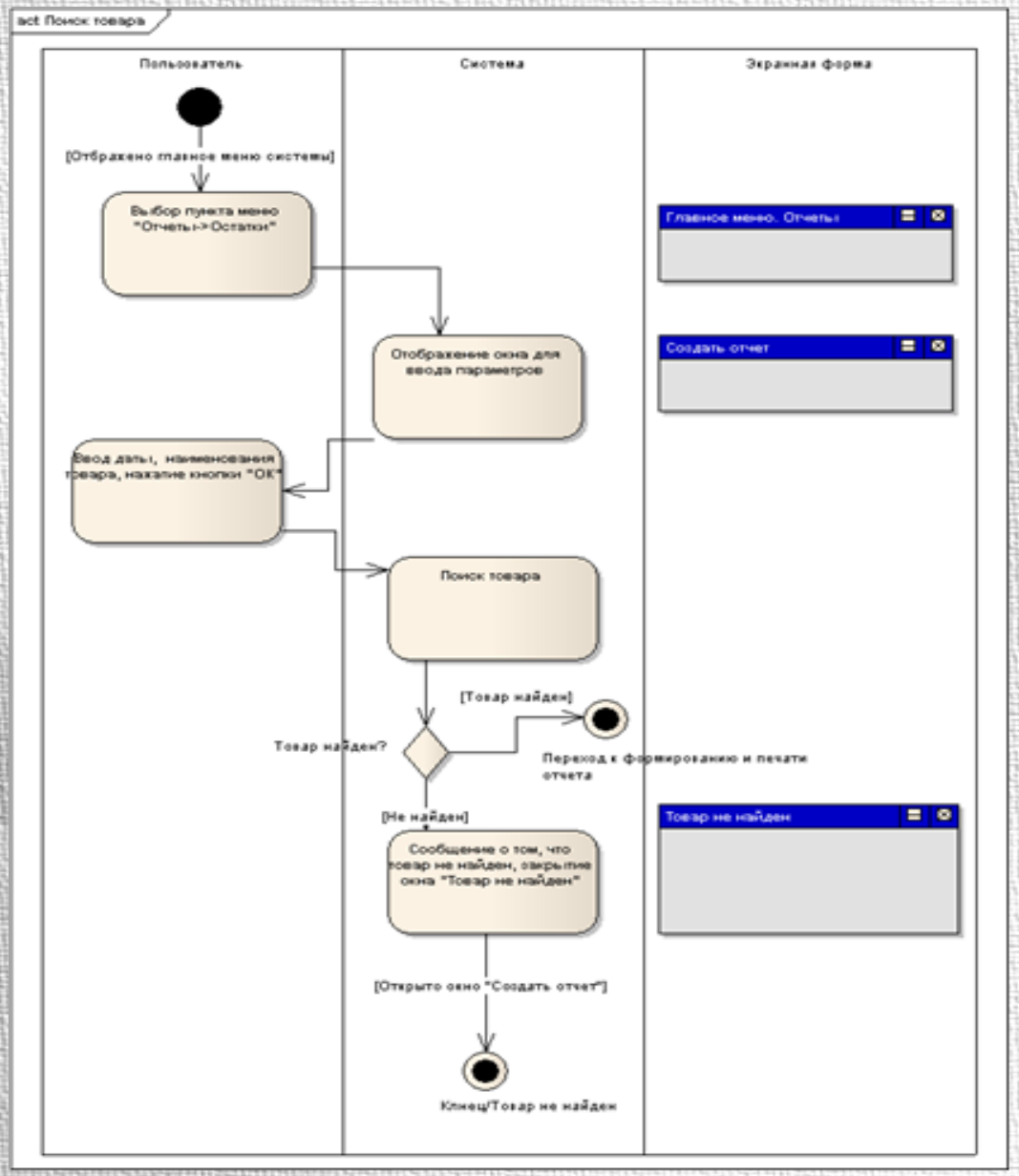
В разделе должны быть представлены

- модели процесса выполнения функции
- и их описание.

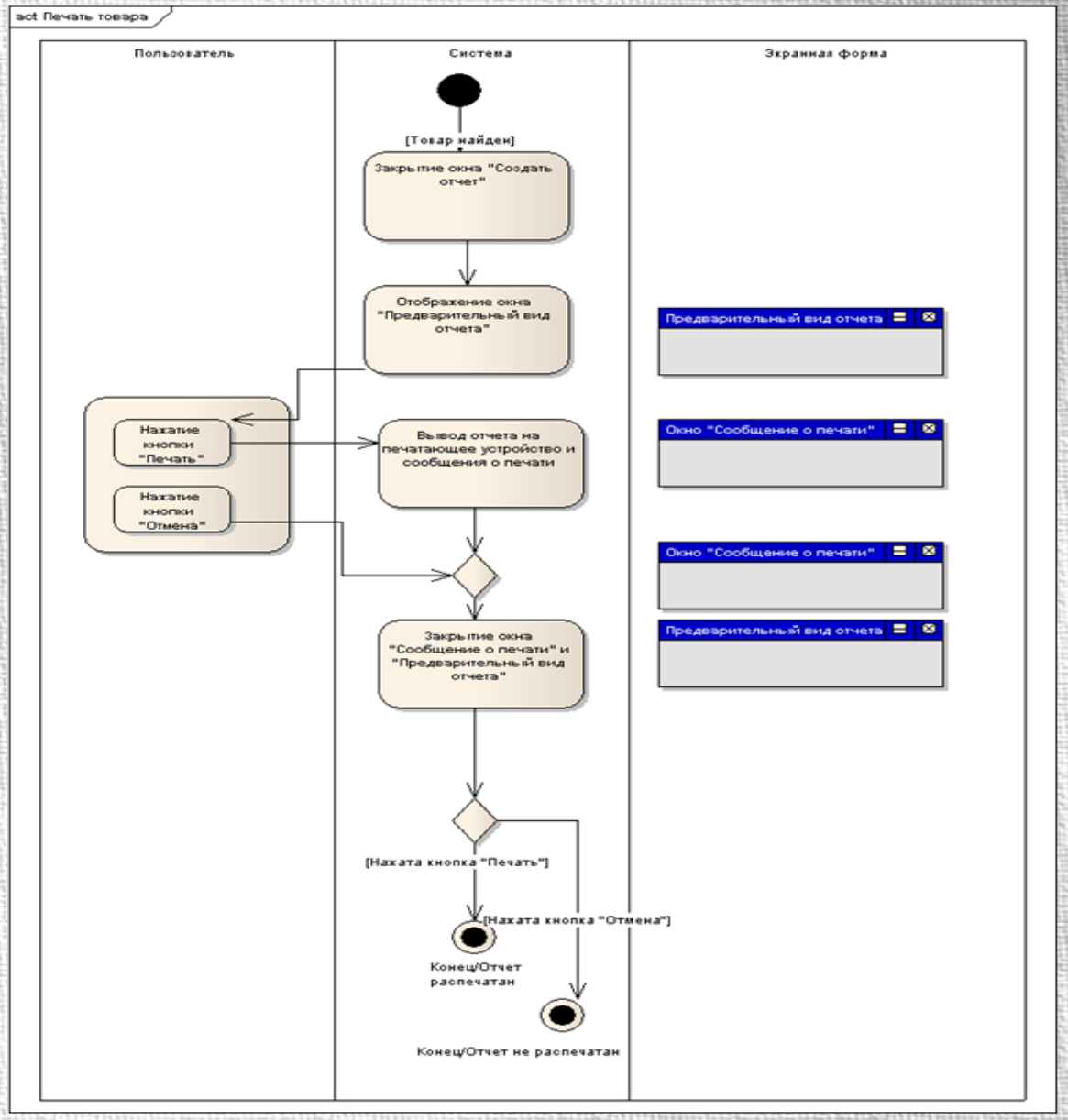
Основные этапы формирования отчета:



Поиск товара:



Печать отчета:



Шаблон описания требования

Описание процесса выполнения функции "Формирование отчета об остатках"

№	Пользователь	Система	Экранная форма	Условие: последующий шаг
Предусловие: Отображено окно с главным меню системы				
1.	Выбор пункта меню "Отчеты->Остатки"		Главное меню	
2.		Отображение окна для ввода параметров	Создать отчет	
3.	Ввод даты, наименования товара, нажатие кнопки "ОК"		Создать отчет	
4.		Поиск товара	Создать отчет	Товар найден: 5 Товар не найден: 10
5.		Закрытие окна "Создать отчет"	Создать отчет	
6.		Отображение окна "Предварительный вид отчета"	Предварительный вид отчета	
7.	Нажатие кнопки "Печать"		Предварительный вид отчета	"Печать": 8 "Отменить": 9

8.	Вывод отчета на печатающее устройство	Окно "Сообщение о печати"	
9.	Закрытие окна "Сообщение о печати" и "Предварительный вид отчета"	Окно "Сообщение о печати", Предварительный вид отчета	"Печать": П1 "Отменить": П3
Постусловие 1: Отображено окно "Создать отчет". Содержимое полей окна "Создать отчет" сохранило введенные пользователем значения. Отчет распечатан			
10.	Сообщение о том, что товар не найден, закрытие окна "Товар не найден"	Товар не найден	
Постусловие 2: Отображено окно "Создать отчет". Содержимое полей окна "Создать отчет" сохранило введенные пользователем значения. Отчет не сформирован			
Постусловие 3: Отображено окно "Создать отчет". Содержимое полей окна "Создать отчет" сохранило веденные пользователем значения. Отчет не распечатан			

8. Нефункциональные требования, связанные с функциональным требованием

- **Временной регламент**
  Если требуется в разделе указывается время выполнения функции. Время формирования отчета не должно превышать 5 сек.
- **Надежность**
  Если требуется в разделе указывается требования к надежности выполнения функции.
- **Точность**
  Если требуется, в разделе указывается требования к характеристики необходимой точности выполнения функции.
- **Достоверность**
  Если требуется, в разделе указывается требования к достоверности результатов выполнения функции.
- **Связи с другими функциональными требованиями**
  Если требуется, в разделе указываются связи между требованиями.

Рекомендации по формулировке требований

В формулировках функциональных требований необходимо избегать любой размытости и неопределенности. Если это невозможно - исключайте требование вообще. Вот ряд рекомендаций:

- Избегайте банальных слов: простой, эффективный, надежный, быстрый, удобный, качественный, многоцелевой, гибкий, настраиваемый, улучшенный, дружественный и пр.
- Избегайте «заумных» формулировок. Чем проще, тем лучше. Легче запоминается и понятно большинству членов команды
- Если что-то легче понять по графику, рисунку, схеме и т.п., чем по текстовому описанию – лучше нарисуйте;
- Не используйте в одном документе различные синонимы для обозначения одного и того же. Выберите один и применяйте его.
- Если при прочтении документа у других членов команды возникают вопросы в одних и тех же местах, скорее всего такое требование следует обдумать еще раз;

Критерии законченного набора требований

- **полнота:** все необходимые требования записаны;
- **непротиворечивость:** не существует требований противоречащих друг другу;
- **отсутствие избыточности:** каждое требование сформулировано только один раз (нет повторов);
- **модульность:** требования близкие друг другу содержатся в одном разделе;
- **структурированность:** наличие ясной четкой структуры документа с требованиями;
- **удовлетворенность:** достигнут необходимый уровень покрытия требований связями типа «удовлетворяет»
- **тестируемость:** достигнут необходимый уровень покрытия требований тестами.

Функциональные требования. Ключевыми словами для записи функциональных требований являются:

- **Tag (таг)** – уникальный, неизменный идентификатор
- **Gist (суть)** – название требования, раскрывающее его сущность
- **Requirement (требование)** – описание требования со всеми необходимыми деталями
- **Rationale (обоснование)** – обоснование необходимости требования
- **Priority (приоритет)** – приоритет требования (обычно используется от трех до пяти приоритетов: "высокий", "средний", "низкий" и "неопределенный").

- **Stakeholders (участники)** – заинтересованные лица, для которых важно требование
- **Owner (владелец)** – лицо, ответственное за имплементацию требования
- **Author (автор)** – лицо, записавшее требование
- **Revision (версия)** – версия требования
- **Date (дата)** – дата записи последней версии
- **Assumptions (предположения)** – все предположения о проблемах, которые связаны с требованием
- **Risks (риски)** – все, что может негативно сказаться на результате имплементации требования
- **Defined (определения)** – определение термина

Ниже приведен пример требования записанного на Planguage.

Ключевое слово	Описание
Tag	REQ1000
Gist	Создать новый документ
Requirement	При выборе пункта меню File/New в папке новых документов создается новый документ с именем New Document N, где N уникальный номер документа, не совпадающий с уже присутствующими в папке.
Rationale	Необходимо иметь возможность создавать новые документы
Priority	Наивысший
Revision	1.0
Author	Иванов И.И.
Date	15 марта 2007
Assumptions	Без этого требования нормальное функционирование продукта невозможно

Нефункциональные требования

Ключевых слов для записи нефункциональных требований больше, чем для записи функциональных. К уже данному списку добавляются:

- **Ambition (назначение)** – описание цели требования
- **Scale (шкала)** – шкала, используемая для количественной характеристики требования
- **Meter (измеритель)** – процесс или устройство, используемые для установления количественной характеристики
- **Minimum (минимум)** – минимальное допустимое значение количественной характеристики
- **Target (цель)** – значение количественной характеристики, при котором требование считается реализованным хорошо

- **Outstanding (достижение) –** наилучшее значение количественной характеристики, которое может быть достигнуто, если все будет реализовано наилучшим образом
- **Wish (идеал) –** значение количественной характеристики, к которому надо стремиться, но достигнуть которого практически невозможно
- **Past (прошлое) –** предыдущий результат, который можно использовать для сравнения с текущим.
- **Trend (тенденция) –** массив исторических данных по развитию требования
- **Record (рекорд) –** лучший достигнутый результат в индустрии

Ключевое слово	Описание
Tag	REQ EDITOR LAUNCH TIME
Gist	Время подготовки редактора для полноценной работы
Requirement	Редактор должен быть готовым к полноценной работе за время, сравнимое с временем аналогичных продуктов на той же конфигурации.
Ambition	Конкурирующие продукты не должны превосходить разрабатываемый редактор.
Priority	Высокий
Revision	1.0
Author	Иванов И.И.
Date	16 марта 2007
Assumptions	Без выполнения этого требования возможны проблемы со сбытом продукта.
Scale	Секунды
Meter	Тест на время запуска редактора
Minimum	60 сек в мин. конфигурации
Target	45 сек в мин. конфигурации
Record	Продукт конкурентов показывает на тесте 57 сек.

4

Проверка требований

(верификация и валидация)

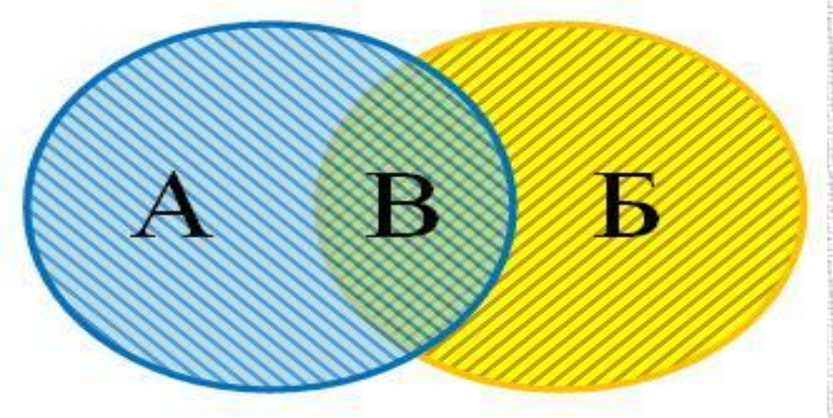
Почему это надо делать?

Требования, полученные от пользователей, могут дублироваться или противоречить друг другу. Некоторые требования могут быть неясными или нереальными Другие требования могут остаться невыясненными. По этой причине прежде, чем требования попадут в документ описания требований, их необходимо согласовать.

В действительности ***согласование и проверка обоснованности требований*** осуществляется параллельно с *выявлением требований.* После того как требования выявлены, они подвергаются определенному уровню проверки.

Правильные требования

Корректность требований отражает степень того, насколько задокументированные требования к разрабатываемой системе соответствуют потребностям пользователей и предметной области. На рисунке показано множество потребностей пользователей (А) и множество сформулированных и специфицированных требований (Б). Область пересечения (В) областей А и Б является областью правильных требований, т.е. тех требований, которые точно выражают потребности заказчика.



Очевидно, что чем больше потребностей выявлено и задокументировано, тем большая вероятность получения правильных требований. Оптимальным является вариант совпадения всех трех областей, но, к сожалению, на практике такого никогда не бывает.

Для увеличения области В необходим четко поставленный процесс выявления, документирования и проверки требований.

Наверное хочется задать вопрос откуда взялись требования в области Б не совпадающие с областью В. Информация из области Б может представлять собой детали проектирования и/или реализации системы, а также требования, которые пользователь или заказчик никогда не высказывали. Иногда подобную информацию высказывают маркетологи и технические специалисты, которые уверены, что пользователи и заказчик придут в восторг от добавленной функциональности, и аналитики, которые считаю, что знают предметную область заказчика или проблемы пользователя лучше самих заказчиков.

К таким требованиям необходимо относится очень серьезно и убеждаться в их действительной необходимости.

Единственным способом оценки правильности требований является их проверка заказчиком системы, пользователем или экспертом предметной области.

Общие рекомендации

- В проверке качества требований участвует вся команда разработки программного обеспечения, а также эксперты предметной или другие представители заказчика.
- Наиболее распространенным методом проверки спецификаций требований является проверка экспертом предметной области.
- После проверки и исправления замечаний спецификация требований утверждается заказчиком и команда приступает к проектированию и разработке программного обеспечения.

- Проверка одним лишь экспертом часто приводит к тому, что спецификация не содержит ошибок в предметной области и весь функционал описан правильно, но, как показывает практика, требования могут быть сложно реализуемыми, не тестируемыми, дорогостоящими. Для выхода из подобной ситуации в проверке требований *также должны участвовать инженеры по тестированию, системные архитекторы и руководитель проекта*.
- Для проверки требований рекомендуется также использовать руководства и контрольные списки, которые содержат рекомендации и критерии «хороших» требований.

Верификация и валидация

Верификация и валидация

Уже перевод с английского этих терминов дает определенную пищу для понимания разницы: *verification* - проверка, *validation* - придание законной силы, проверка правильности, *аттестация*, утверждение.

- ***Верификация*** — проверка того, что продукт делался правильно, т.е. проверка того, что он разрабатывался в соответствии со всеми требованиями по отношению к процессу и этапам разработки.
- ***Валидация (аттестация)*** — проверка того, что сам продукт правилен, т.е. установление того, что он удовлетворяет требованиям, ожиданиям **пользователя**, **заказчика** и других заинтересованных сторон.

Кратко различие между верификацией и аттестацией можно определить следующим образом:

- Верификация отвечает на вопрос, правильно ли **создана** система
- Валидация (аттестация) отвечает на вопрос, правильно ли **работает** система

Примеры

1. В результате проведения испытаний было выяснено, что некоторое лекарство изготовлено в соответствии со всеми необходимыми требованиями. Данное лекарство поступило в продажу, но будет ли оно использовано неким конкретным человеком? Не обязательно, поскольку у одного больного может быть индивидуальная непереносимость каких-то компонентов данного препарата, другой не может принимать его из-за несовместимости с другими лекарственными средствами и т.д. Все это учитывает врач, когда назначает препарат данному конкретному пациенту, т.е. он подтверждает, что данное лекарство может быть использовано этим больным. В указанном случае врач осуществляет валидацию.
2. Предприятие производит лебедки для осуществления наземного строительства. Данная продукция соответствует всем необходимым Техническим условиям. Однако предприятие получает заказ от некой организации, осуществляющей укладку трубопроводов в подводные траншеи. Можно ли в данном случае использовать изготовленные по ТУ лебедки в подводных условиях? Ответ на этот вопрос можно дать, только проведя валидацию.

Суммируя вышеизложенное, дадим определение валидации и верификации:

- Верификация представляет собой обязательный процесс, который осуществляется методом **сравнения** характеристик продукции и нормативных требований. По проведении верификации делается вывод о соответствии/несоответствии продукции заданным требованиям
- Валидация осуществляется по необходимости путем анализа возможности применения продукции в конкретных условиях и оценки соответствия характеристик продукции заданным условиям. По проведении верификации делается вывод о возможности или невозможности использования продукции в конкретных условиях.

Верификация и валидация

Применительно к требованиям , это можно трактовать следующим образом:

- если ***верификация*** связана с выяснением того, удовлетворяет ли разрабатываемый *объект*, либо процесс его создания сформулированным требованиям, то ***валидация*** отвечает на вопрос - правильно ли разработан целевой *объект* (продукт), удовлетворяет ли он потребностям заказчика. Другой аспект валидации заключается в том, что она обычно увязывается с формальной приемкой (аттестацией) системы.

Итак, если критерием проверки АИС служат требования, то что может послужить критерием проверки самих требований? Ответ заключается в том, что требования должны удовлетворять свойствам, сформулированным выше.

Еще одно замечание

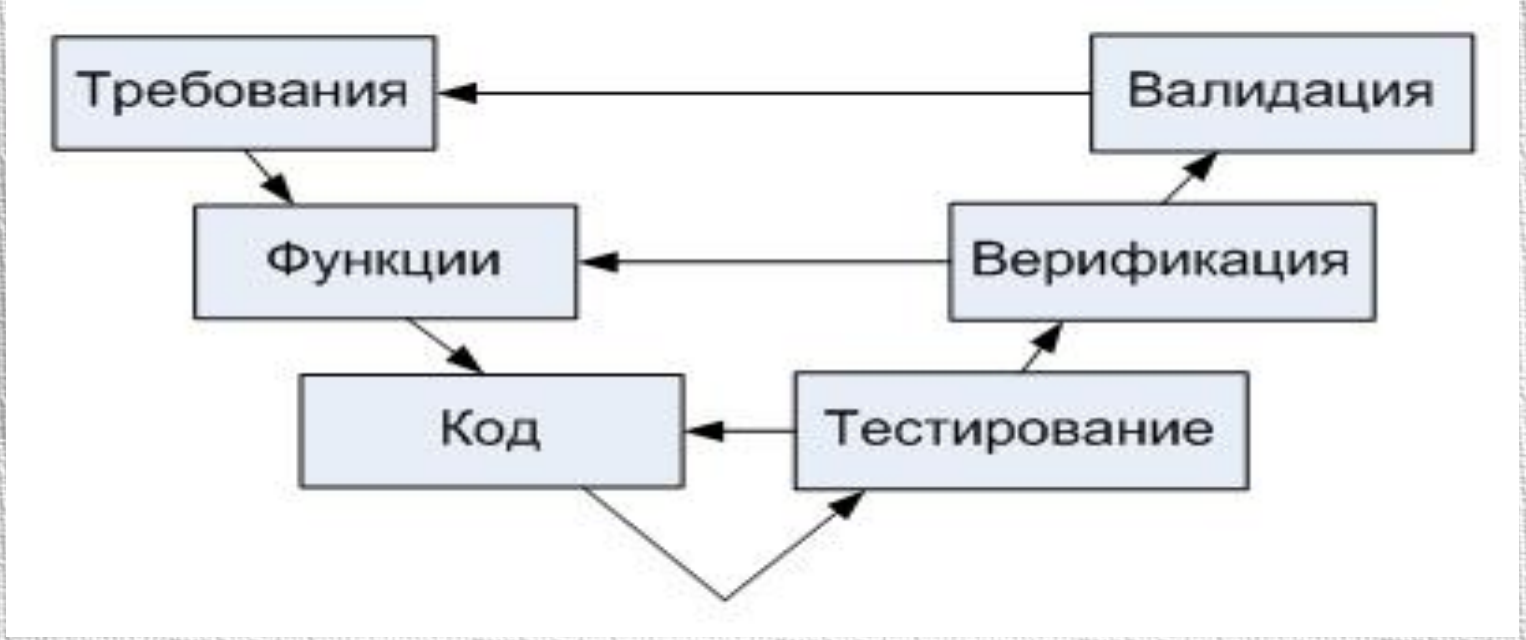
- Верификация проверяет артефакт на соответствие требованиям, описанным-оговоренным РАНЕЕ, т.е. смотрит в прошлое. Например, заказчик, читающий и утверждающий спецификацию, по которой уже были обсуждения делает верификацию.
- Валидация же проверяет артефакт на возможность его ДАЛЬНЕЙШЕГО использования, т.е. смотрит в будущее. Например, Тестировщик, «идущий» по тестовым сценариям, которые составлены на базе спецификации, делает верификацию. Но если ту же спецификацию инспектирует Тестировщик на предмет тестируемости, а разработчик на предмет реализуемости, то это уже валидация.

Вообще говоря, принято считать, что требования описаны не полностью, если для них не заданы **правила V&V** (*verification&validation – проверка и аттестация*), то есть не определены способы проверки и утверждения.

Процедуры проверки являются отправной точкой для инженеров-тестировщиков и специалистов по качеству, непосредственно отвечающих за соответствие получаемого программного продукта предъявляемым к нему требованиям.

Верификация, валидация и тестирование в связи с объектами анализа

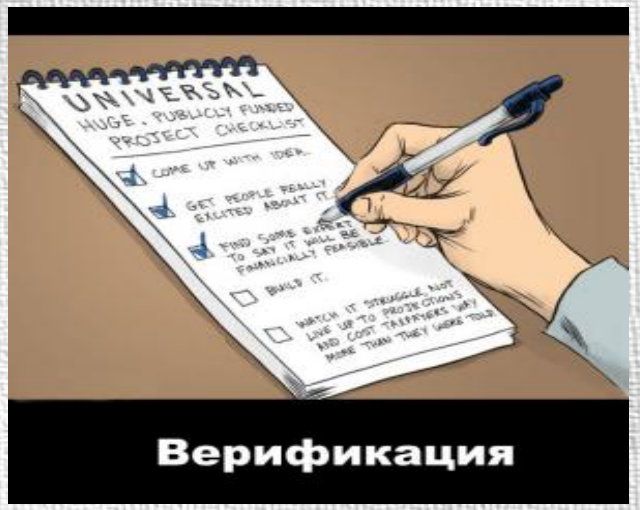
По этой схеме очень легко запомнить, что тестируется код, верифицируется функциональность системы, и валидируются высокоуровневые требования (бизнес-требования). Т.е. эти три вида деятельности отличаются тем, что имеют различные объекты анализа.



Верификация на всех уровнях

Целью верификации является удостоверение в том, что программное обеспечение соответствует предъявляемым требованиям. Параллельно с этим фиксируются новые дефекты, добавленные в процессе разработки. Верификацию можно применять на все стадиях разработки. Методы не ориентированы на какую-то одну фазу. Можно и нужно верифицировать элементы требований, проектирования, реализации, тестирования … Верификация позволяет убедиться в корректности переходов между фазами:

1. Потребности пользователей (stakeholder requests)
2. Функции продукта (функции, features)
3. Требования (requirements)
4. Архитектура
5. Модель проектирования (design model)
6. Реализация
7. Планирование тестов

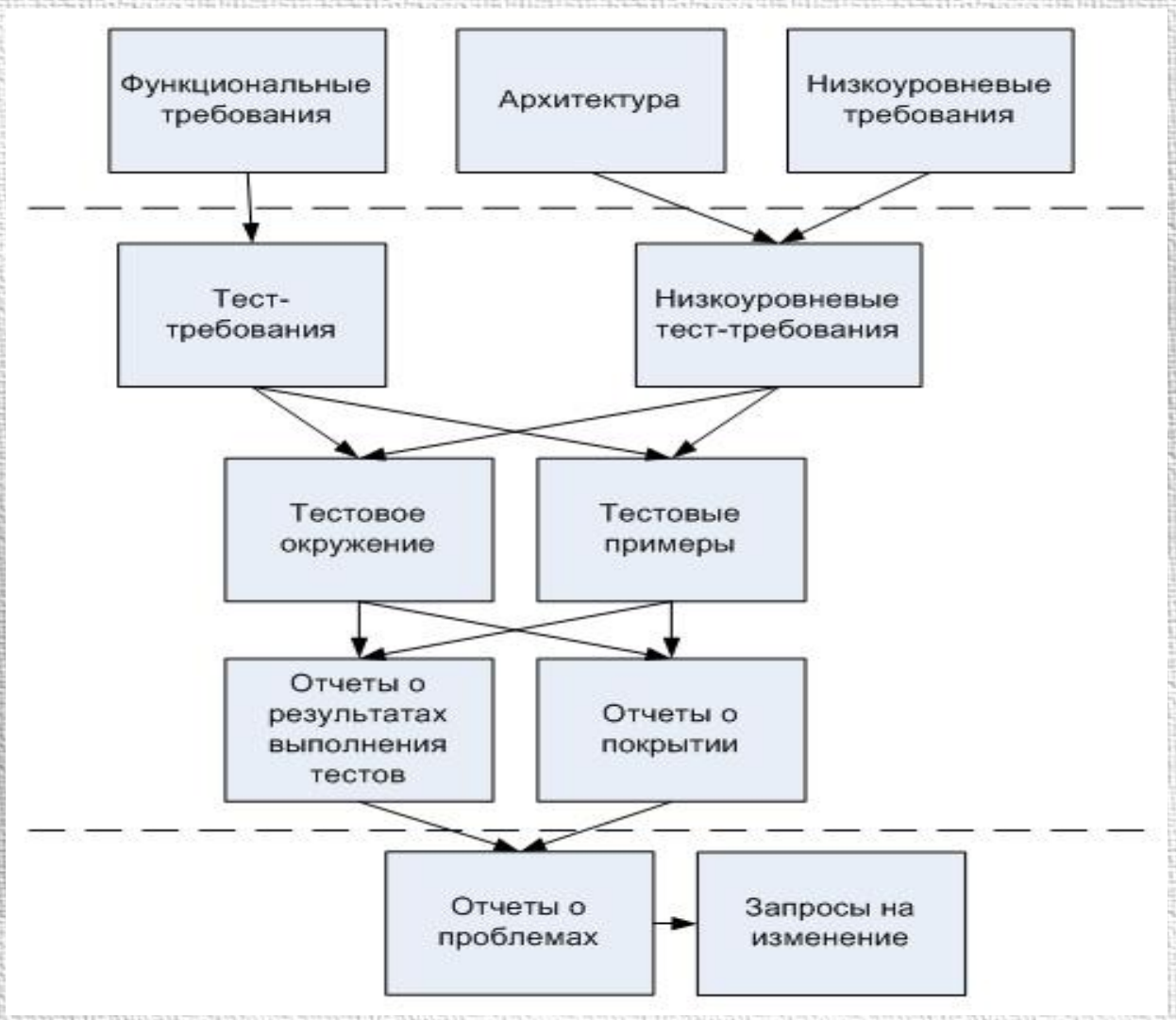


Верифицировать следует:

- *Описанные функции системы действительно соответствуют потребностям клиента/пользователей.*
- *Варианты использования и требования, созданные на основе функций, поддерживают эти функции.*
- *Варианты использования реализуются при проектировании.*
- *Проектирование поддерживает функциональное и нефункциональное поведение системы.*
- *Программный код соответствует целям и результатам проектирования.*
- *Тесты полностью покрывают варианты использования и требования*.

Нужно исследовать требования и ВИ и убедиться, что они полно и неизбыточно отвечают потребностям пользователей (функциям) верхнего уровня (п.2 выше). Далее убедиться, что при проектировании использовались эти требования и ВИ, а технический проект получился полным и неизбыточным.

Документация, сопровождающая процесс верификации



Валидация требований

Валидация гарантирует, что требования отвечают желаемым характеристикам правильно поставленных требований (полнота, правильность, возможность, необходимость, приоритеты, однозначность и проверяемость), а также правильной спецификации требований (полнота, последовательность, модифицируемость и трассируемость)

Валидация представляет собой процесс оценки, будет ли конечный продукт удовлетворять требованиям заказчика. Валидация помогает удостовериться, что требования были правильно поняты.

Такой подход к поставке в последнее время называют «Test-First Development» или Requirements-Based Testing.

С валидацией участник проекта устанавливает критерии приемки для достижения соглашения с заинтересованными лицами о том, что будет поставлено. Это включает в себя проверку бизнес-требований, функциональных требований, нефункциональных требований, сценариев использования, моделей, прототипов и т.д.

Валидация, как и любая другая часть процесса развития требований, не является независимой от других видов деятельностей и должна осуществляться итеративно. Материалами для валидации могут быть и одно небольшое требование, и большой полный набор требований, описанных в спецификациях.

Для проверки правильности проводятся приёмо-сдаточные испытания. Они основаны на сценариях тестирования, которые пользователь согласовывает, а затем выполняет в среде использования системы. Стандарт IEEE 829-2008 (Standard for Software and System Test Documentation) содержит образцы 8 документов, которыми следуют руководствоваться при планировании, организации и проведении тестирования:

1. План тестирования. Руководящий документ, отражающий:
   - Как будет проводиться тестирование, включая конфигурации тестируемой системы (ТС).
   - Кто будет тестировать.
   - Что будет тестироваться.
   - Сколько времени займёт тестирование.
   - Какой уровень качества тестирования необходим.
2. Критерии успешности тестирования (Test Design Specification).
3. Данные для тестирования (Test Case Specification).

4. Сценарии тестирования (Test Procedure Specification), включая предусловия и шаги тестов.
5. Отчет о переходах между этапами тестирования (Test Item Transmittal Report).
6. Протокол тестирования (Test Log).
7. Отчет об инцидентах (Test Incident Report), включая ожидаемый результат, фактический результат, время, предполагаемые причины инцидента и всё, что может помочь с разрешением ситуации. Инцидент — не обязательно подразумевает ошибку в системе: ожидаемый результат мог быть неверным, проверка могла проводиться неверно, требование могло толковаться по-другому.
8. Отчет от тестировании (Test Summary Report).

Утверждение & Документирование

Согласование и проверка требований не могут быть отделены от процесса подготовки документа описания требований.

- Обычно **согласование требований** основано на черновом варианте документа. Требования, перечисленные в черновом варианте документа, обсуждаются и при необходимости модифицируются. Побочные требования удаляются. Вновь выявленные требования добавляются.
- Для *проверки* ***обоснованности требований*** необходима более полная версия документа, в котором все требования четко идентифицированы и классифицированы. Участники проекта изучают документ и проводят совещания по их формальному пересмотру. *Пересмотры* часто структурированы в виде так называемых процедур *сквозного контроля* или *инспекций*. *Пересмотры являются* разновидностью *тестирования.*

Некоторые типичные проблемные ситуации процесса формирования и оценки требований

1. **Двусмысленность требований**
   - На практике требование, сформулированное двусмысленным образом, может привести к различным его интерпретациям представителями Разработчика и Заказчика.
2. **"Золочение" продукта**
   - Под «золочением» понимают такие ситуации, когда разработчики добавляют функции, которых нет в спецификации, но им кажется, что это понравится пользователям. Зачастую же клиентам не нужны такие избыточные возможности, получается, что время, отведенное на реализацию, тратится впустую.
3. **Минимальная спецификация**
   - Создавать полную документацию требований в соответствии с вышеизложенными принципами, или ограничиться наброском требований на 2-3 страницы– как говориться, дело вкуса.
4. **Пропуск типов пользователей**

Методы и средства проверки требований

Методы и средства проверки требований

Наработано значительное количество методов и средств проверки требований. Они разнятся по ряду параметров. Так, различают:

- **по широте анализа** - просмотр (выборочная проверка) и сквозной контроль (тотальная проверка);
- **по степени формализации** - неофициальные процедуры, процедуры, проводимые по формальным правилам (инспекции, экспертизы);
- **по составу группы проверки** - с (без) участием автора, с (без) участием менеджера проекта, с (без) участием представителей внешних организаций;
- **по используемым средствам** - тексты требований, тестовые сценарии, критерии приемлемости, прототипы.

1. Неофициальные просмотры требований
2. Инспекции
3. Разработка тестов
4. Трассирование
5. Формирование критериев приемлемости

1.Неофициальные просмотры требований

Различают несколько способов неофициальных просмотров требований:

- *просмотр «за столом»,*
- *коллективная проверка,*
- *критический анализ.*

В первых двух случаях автор требований обращается за помощью к коллегам (соответственно, к одному, либо к нескольким) с целью выдачи практических рекомендаций по улучшению продукта. В третьем случае автор осуществляет презентацию разработанных им требований на совещании с последующим обсуждением.

Неофициальные просмотры используют для знакомства с разработкой, сбора отзывов, формирования обратной связи. По статистике неофициальные просмотры позволяют выявить до 60% ошибок в требованиях.

2. Инспекции

Основным методом верификации спецификаций требований к ПО является инспектирование. Инспектирование – это формализованный процесс, выполняемый небольшой группой специалистов. Члены группы системно анализируют объект инспекции и определяют возможные дефекты. Члены группы должны исполнять различные роли.

Роль	Описание
Автор	Специалист, ответственный за создание спецификации, вынесенной на инспекцию. Отвечает за исправление дефектов, обнаруженных во время инспекции
Инспектор	Эксперт, анализирующий спецификацию на предмет обнаружения ошибок
Рецензент	Человек, ответственный за чтение спецификации во время инспекции
Секретарь	Человек, ответственный за запись результатов инспекции
Модератор	Руководитель, управляющий и организующий процесс инспектирования

Одно лицо может исполнять несколько ролей, поэтому количество членов в группе инспектирования может варьироваться.

Для начала процесса инспектирования необходимы следующие условия:

1. Наличие всех документов, на основании которых готовилась инспектируемая спецификация.
2. Члены группы должны хорошо знать стандарты документирования требований.
3. В распоряжении группы должна быть синтаксически корректная последняя версия спецификации.

Инспекция проходит в несколько этапов:

- **Планирование** – модератор подбирает инспекционную группу, организует собрание и убеждается, что все начальные условия для инспекции есть.
- **Предварительный просмотр** – спецификация передается инспекционной группе, автор сообщает об особенностях инспектируемого объекта, члены группы оценивают временные затраты на инспекцию.
- **Индивидуальная подготовка** – члены инспекционной группы изучают спецификацию.
- **Собрание инспекционной группы** – докладываются все обнаруженные дефекты, они фиксируются в записях инспекции.
- **Исправление ошибок** – автор исправляет обнаруженные ошибки.
- **Оценка** – модератор проверяет исправления и принимает решение, нужна ли повторная инспекция.

Принято считать, что максимальная длительность инспектирования не должна превышать 2 часов, поскольку потом эффективность обнаружения дефектов снижается.

3. Разработка тестов

Механизм вариантов использования (uses cases) позволяет ответить на вопрос: как будет использоваться система. Чтобы проверить систему, используется аналогичный механизм: **тестовых сценариев** (test cases).

Тестовые сценарии (ТС) рекомендуется создавать уже на ранних стадиях работы с требованиями, в идеале - после получения запросов ЗЛ, параллельно с разработкой вариантов использования.

Тестовые сценарии, как и варианты использования, могут поддерживать разные уровни абстракции. Различаются *концептуальные* и *детальные* ТС. Концептуальный уровень предполагает проработку процедуры тестирования, инвариантную к конкретной реализации UI.

Как использовать тестовые сценарии для тестирования требований?

Предлагается следующая процедура.

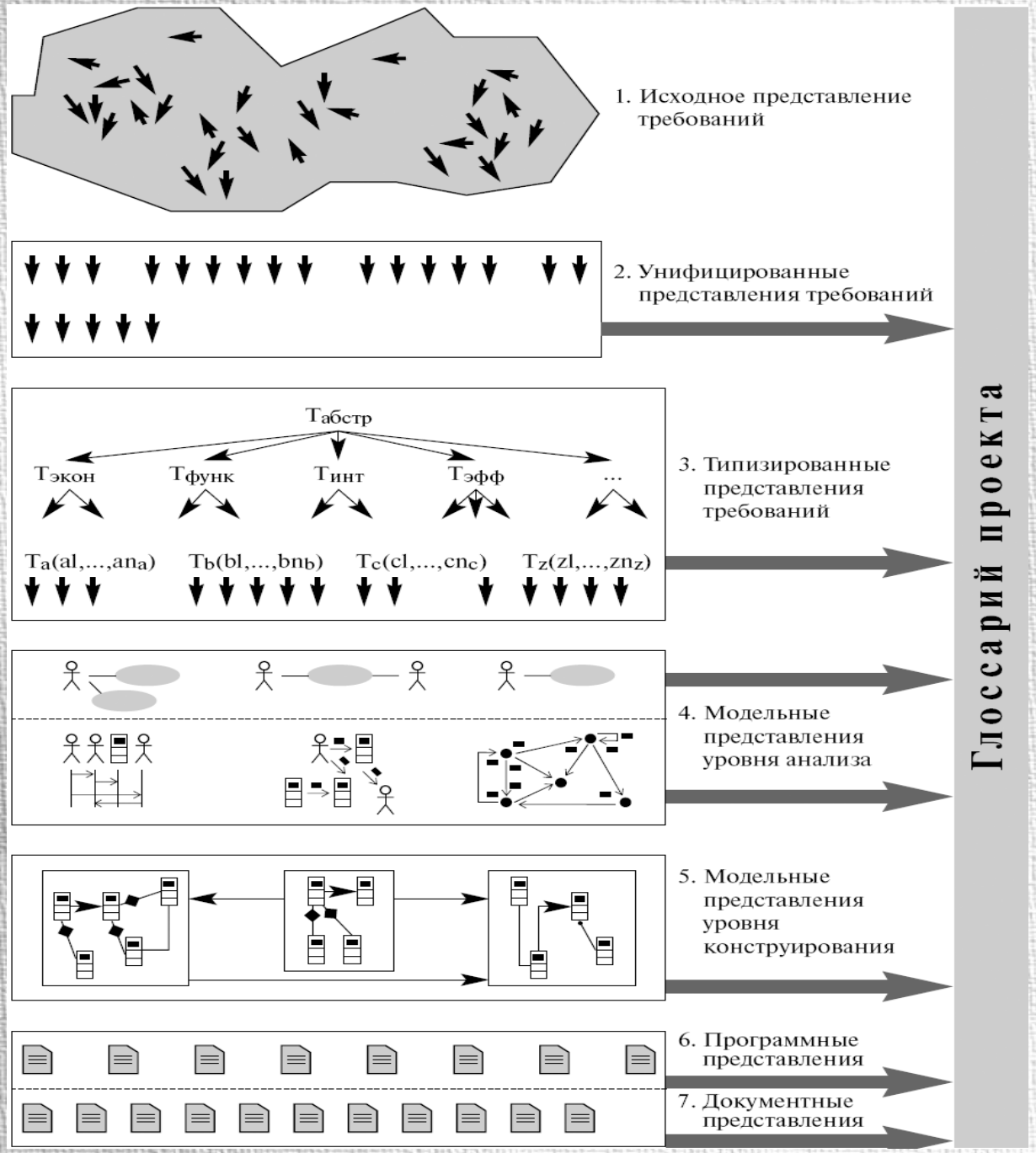
- Построить матрицу, где по вертикали отмечены функциональные требования, а по горизонтали - тестовые сценарии.
- Убедиться, что каждый из ТС осуществим на существующем наборе требований.
- Убедиться, что для каждого требования представлен как минимум один ТС.
- Прочертить "путь" каждого из ТС на карте диалогов. Это позволит:
  - обнаружить некорректные или пропущенные требования,
  - исправить ошибки на карте диалогов и отшлифовать варианты тестирования.

4. Трассирование

Все более популярным методом верификации спецификаций требований к ПО становится трассирование спецификаций. Данная технология базируется на отслеживании взаимосвязей между требованиями различных видов и характеристиками созданного продукта. То есть между тем, что задумывалось, и тем что получилось.

При трассировании мы ориентируемся на схему трансформации требований.

Схема трансформации требований



1. **Исходное представление** — текстовое описание пожеланий к системе, заданное в свободной форме. Это описание, в частности, может фактически содержать несколько *требований*, отражающих разные аспекты проекта, — элементарные составляющие *требования*.
2. **Унифицированные представления** — исходное *представление требования* разбивается на элементарные составляющие, которые описываются в базовом виде, приспособленном для дальнейшего использования на всех проектных уровнях. В частности, здесь могут применяться формализованные описания элементарных составляющих *требования*. Во всяком случае, на уровне унифицированного *представления* достигается однозначность понимания *требований*.

3. **Типизированное представление** — каждое из элементарных составляющих *требования* приписывается к некоторому типу. В результате формируется набор атрибутов элементарных *требований* и их значений. Эта информация допускает почти формальное сопоставление элементарных *требований* с различными *требованиями*, уже *представленными* в проекте. Сопоставление проводится на разных уровнях иерархии типов *требований* к системе.
4. **Модельные представления уровня анализа** — образы элементарных требований как элементы аналитических моделей системы. Если следовать RUP, то нужно говорить о моделях ситуаций использования и динамики взаимодействий, которые применяются для оценки требований.

Подчеркнем, что в п. 4 говорится именно об элементах моделей, поскольку сами модели, как правило, отражают, пусть даже фрагментарно, сразу несколько *требований*. Это же относится и к последующим *представлениям требований*. Для *трассировки* важно знать, какие модели и в каких частях затрагивает то или иное *требование*, что позволяет отслеживать связи, возникающие при трансформации *требований*. Не менее существенно иметь возможность оперативно получать соответствующую информацию о текстах программ и о документации. Если *требование* принимается на уровне анализа, то *трассировка* продолжается на следующих уровнях и можно говорить о продолжении последовательности трансформаций в реализации *требования* в компонентах программного изделия.

5. **Модельные представления уровня конструирования** — образы элементарных *требований* в диаграммах классов, состояний и других компонентах архитектуры системы. На этом уровне *требования* принимаются или отклоняются в зависимости от их соответствия уже разработанной части проекта.
6. **Программные представления** — программные *рабочие продукты* и их фрагменты, которые рассматриваются в качестве образов *требований*, представленных очередной версией системы.
7. **Документные представления** — фрагменты документов, сопровождающих программный код и предназначенных для поддержки деятельности пользователей.

Типы трассируемости требований

Трассирование помогает следить за качеством требований в двух направлениях – от первоисточника к реализации и наоборот. Если связи в обоих направлениях четкие, то требования имеют высокое качество. Если в связях есть разрывы или неясности, то требования нуждаются в корректировке.

На диаграмме показаны четыре типа связей трассируемости. Потребности клиента отслеживаются в направлении от потребностей к требованиям. Благодаря этой связи можно определить, какие требования будут затронуты, если в течение разработки продукта потребности изменятся.

Это также дает уверенность, что в спецификации требований указаны все потребности пользователя. В направлении от требований к потребностям устанавливается связь, чтобы определить происхождение каждого требования к ПО. Если представить потребности в форме пользовательских требований, то левая половина рисунка будет показывать трассирование между пользовательскими требованиями и требованиями к продукту. В направлении от требований к характеристикам продукта определяются связи между отдельными требованиями и особенностями продукта. Этот тип связи гарантирует, что каждое требование удовлетворено, поскольку известно, какая особенность соответствует каждому требованию. Четвертый тип связи ведет от характеристик продукта к требованиям, давая знать причину создания каждой особенности продукта. Характеристики продукта соответствуют результатам тестирования. Поэтому третий и четвертый типы связей вводят взаимозависимость требований и тестов.

Назначение матрицы прослеживаемости требований заключается в отображении каждого требования на проектные компоненты, программные коды и тестовые случаи. Пример матрицы прослеживаемости требований показан на рис. 2.5. В этом примере каждое требование, фигурирующее в документе определения требований, отображается

- на одно или большее количество требований из спецификации,
- на проектные компоненты
- на программные коды,
- на тестовые случаи модульного тестирования,
- на проверки взаимодействия и функционирования компонентов программного продукта, системных и приемочных испытаний.

5. Формирование критериев приемлемости

Крайне важно, наряду с формированием требований, вовлечь Заказчика на ранних стадиях создания продукта в процесс формирования ***критериев приемлемости***. Критерии приемлемости (acceptance criteria) должны отразить точку зрения Заказчика на то, что он считает ***правильной системой***.

Делегирование разработки ***тестов на приемлемость*** пользователям — эффективная стратегия разработки требований. Это позволяет уже на этапе сбора информации перейти от вопроса «*Что вам нужно делать с помощью системы?*» к «*Как вы делаете вывод о том, что система удовлетворяет вашим потребностям?*». Если клиент не может описать, как он оценит, что конкретное требование удовлетворено системой, значит, требование сформулировано недостаточно ясно.

Раннее формирование тестов для проверки приемлемости позволяет обнаружить дефекты в требованиях.

Проверка приемлемости базируется на ключевых (существенных) вариантах использования. При этом следует абстрагироваться от альтернативных сценариев и исключений и сосредоточить внимание на основном потоке событий.

Необходимо учесть также и нефункциональные требования, такие, как производительностью, легкость и простота использования.

Требования, выходящие за рамки проекта

Чтобы иметь возможность решить, лежит ли конкретное требование в рамках системы или выходит за рамки, необходима эталонная модель, по отношению к которой и должно приниматься решение.

- Исторически роль подобной эталонной модели сыграла *контекстная диаграмма (context diagram)* — высокоуровневая диаграмма метода структурного моделирования под названием диаграмма потоков данных - DFD. Хотя в языке UML место этой диаграммы заняла диаграмма прецедентов, контекстная диаграмма по-прежнему остается превосходным методом установления границ системы .

Существуют другие причины, по которым требование может быть расценено, как выходящее за рамки проекта .

- Например, требование может представлять большую трудность для реализации в компьютеризованной системе и остается прерогативой человека, участвующего в процессе.
- Либо требование может иметь низкий приоритет и исключается из первой версии системы.
- Требование может быть также реализовано с помощью аппаратного обеспечения или внешних устройств и, таким образом, может оказаться вне контроля со стороны программного обеспечения.

Управление — искусство карабкаться по лестнице вверх. Лидерство — умение определить, к той ли стене приставлена лестница.
П. Друкер, У. Беннис

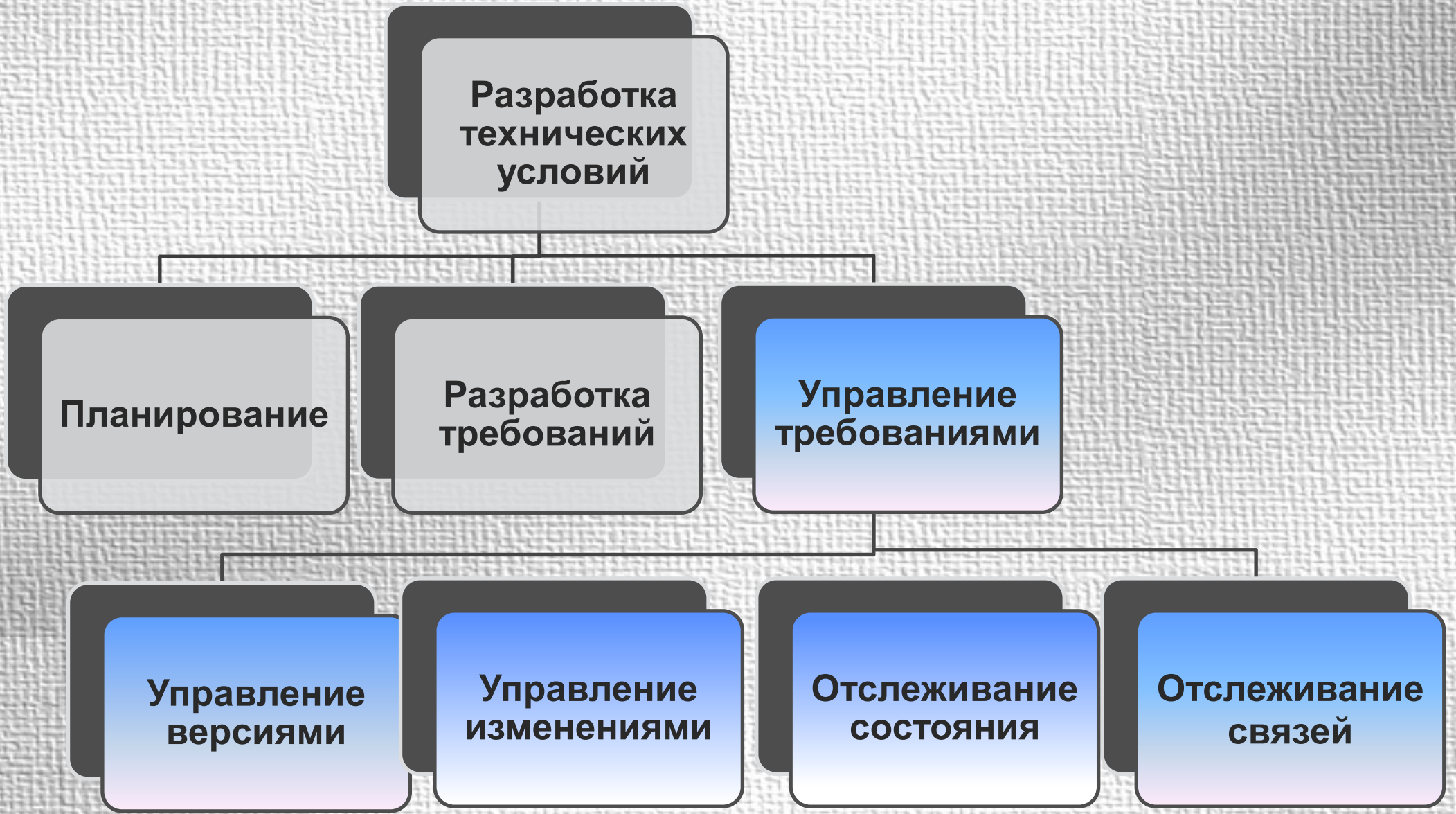
- Управление версиями
- Изменение требований.
- Прослеживаемость требований.
- Отслеживание состояния требований

УПРАВЛЕНИЕ ТРЕБОВАНИЯМИ

Определение

Управление требованиями (requirements management):

- набор организованных, стандартизованных, систематических процессов и методов работы с требованиями в значительном/сложном проекте;
- системный подход к выявлению, организации, документированию требований к системе, в ходе которого вырабатывается и обеспечивается соглашение между заказчиком и исполнителем по поводу меняющихся требований к системе.

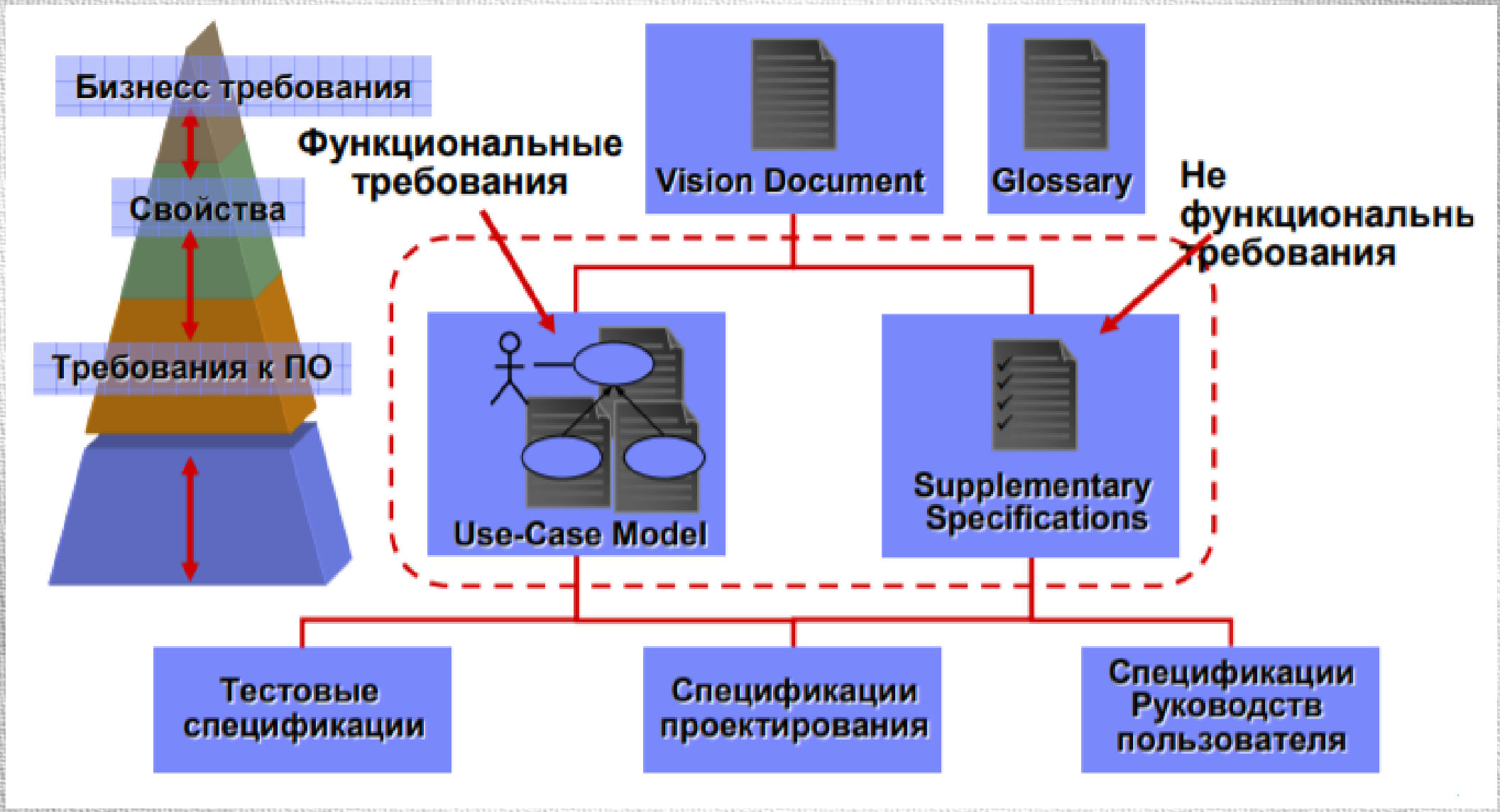

Разработка технических условий
Планирование
Разработка требований
Управление требованиями
Управление версиями
Управление изменениями
Отслеживание состояния
Отслеживание связей

Управление требованиями
Управление версиями
• Определение схемы идентификации версий
• Отслеживание версий отдельных требований
• Отслеживание версий наборов требований
Управление изменениями
• Предложение изменений
• Анализ влияния изменений
• Принятие решений
• Обновление отдельных требований
• Обновление наборов требований
• Обновление планов
• Оценка изменчивости требований
Отслеживание состояния требований
• Определение возможных состояний требований
• Фиксирование состояния каждого требования
• Отслеживание состояния распределения всех требований
Отслеживание связей требований
• Определение связей с другими требованиями
• Определение связей с другими элементами системы

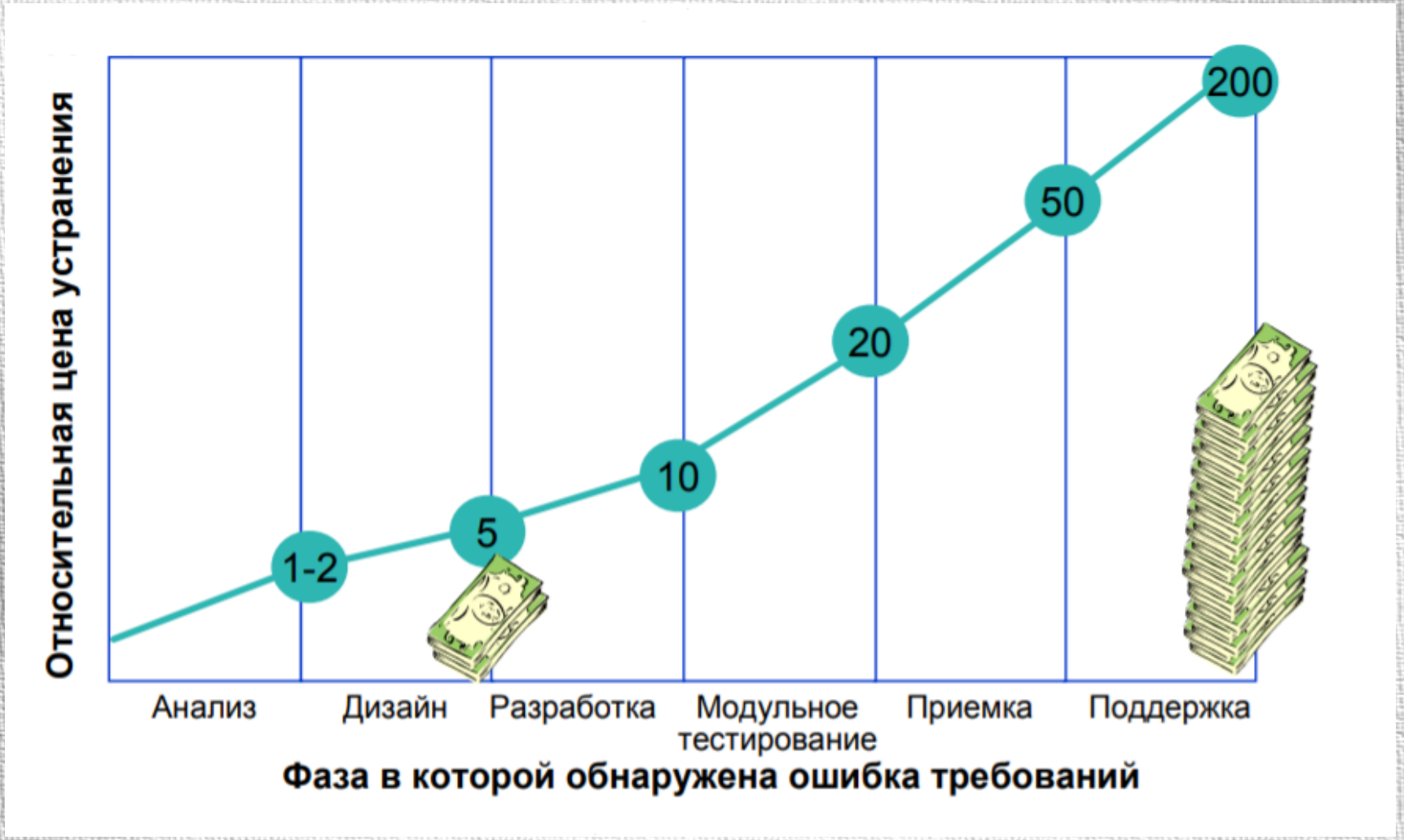
Задачи работы с требованиями

Непосредственное управление требованиями включает следующие задачи работы с требованиями:

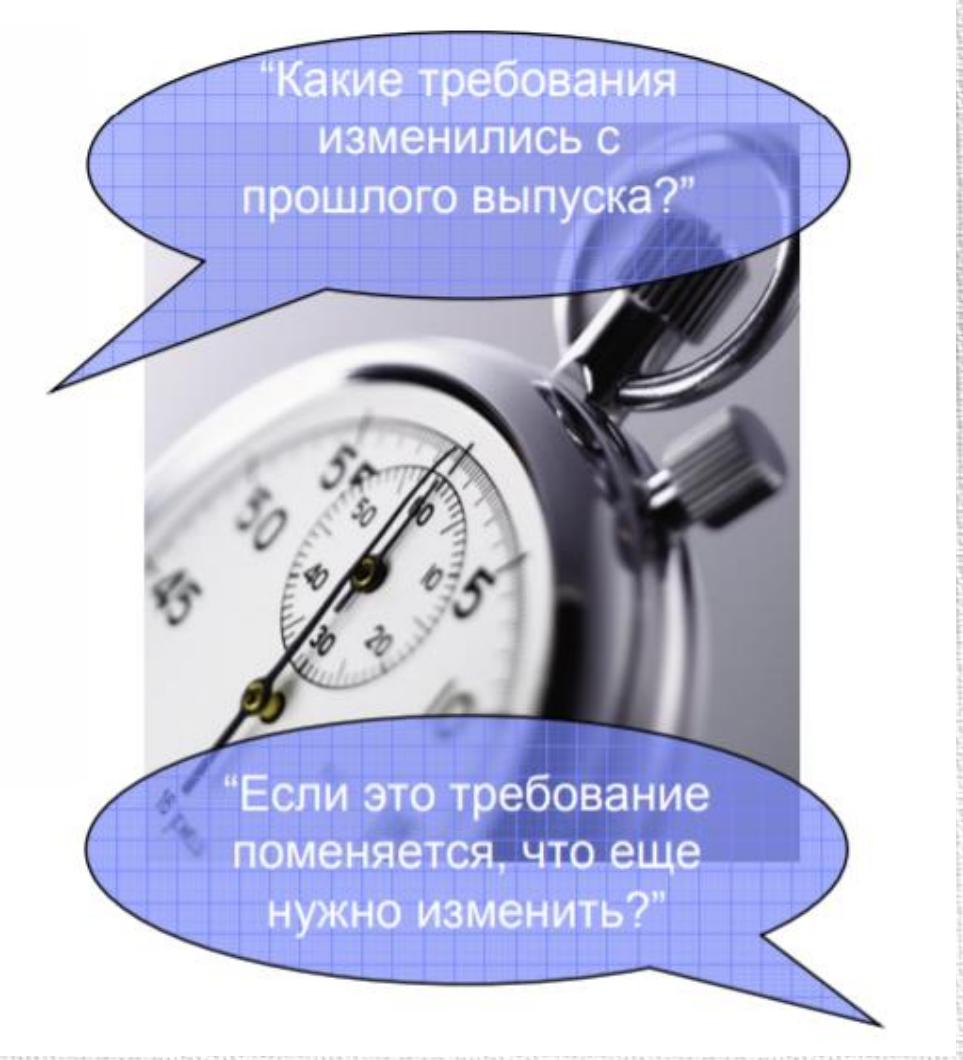
- Трассировка (Trace Requirements)
- Поддержание (Maintain Requirements)
- Приоритизация (Prioritize Requirements)
- Оценка изменений (Assess Requirements Changes)
- Утверждение (Approve Requirements)


Бизнесс требования
Свойства
Требования к ПО
Функциональные требования
Vision Document
Glossary
Не функциональнь требования
Use-Case Model
Supplementary Specifications
Тестовые спецификации
Спецификации проектирования
Спецификации Руководств пользователя

Неуправляемые требования – причина неконтролируемых бюджетов



Проблемы управления требованиями



Заказчик

Руководитель проекта

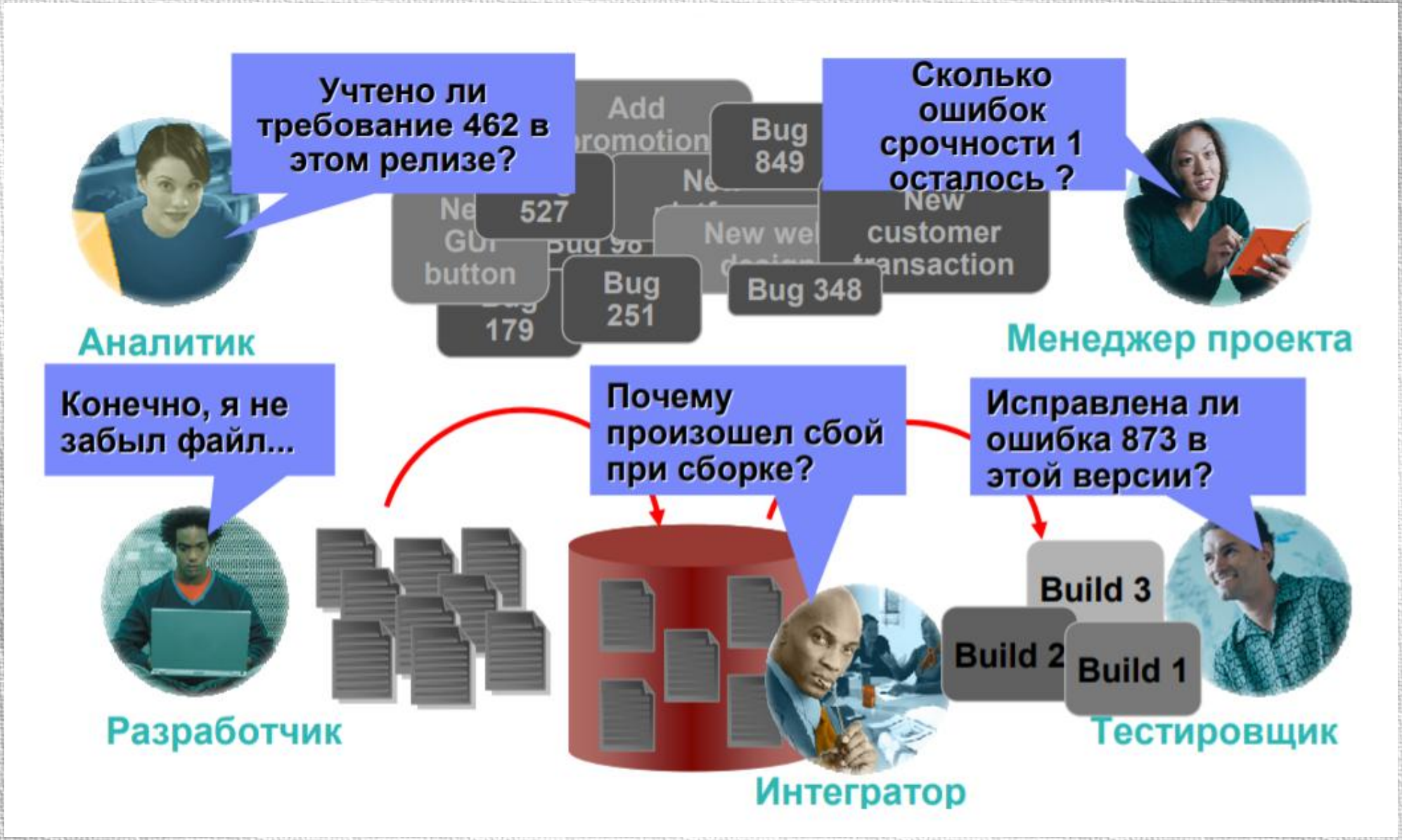
Аналитик

Разработчик

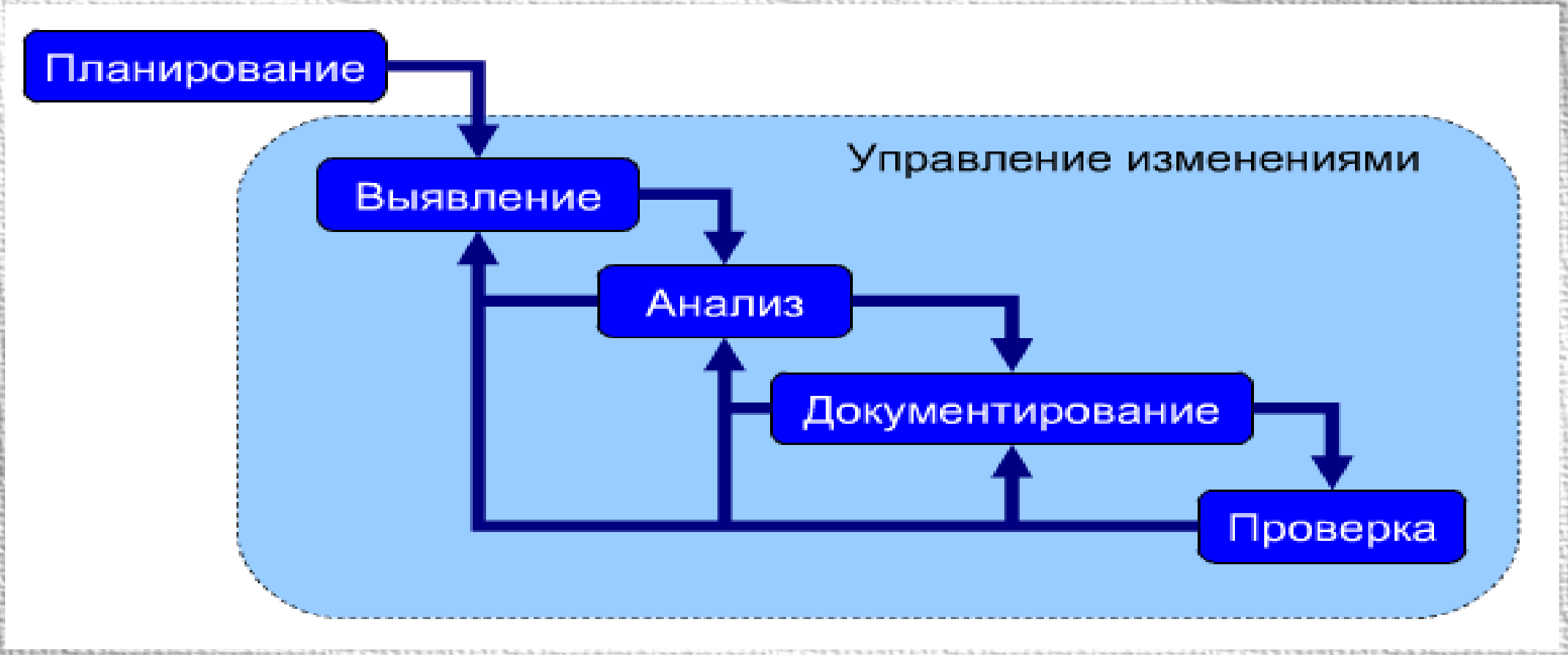
Тестировщик

Администратор

Управление изменениями



Какими изменениями мы управляем?



Фундаментальная особенность разработки ПО:

Требования изменяются всегда!

За исключением самых простых случаев, требования изменяются всегда и везде, вне зависимости от того, насколько хорошо был сделан первичный анализ! Вместо того чтобы жаловаться на изменение исходно предоставленных требований, необходимо так модифицировать процесс разработки, чтобы обеспечить эффективную обработку любых возникающих изменений.

Мы живем в мире изменяющихся требований, и наша задача состоит в том, чтобы написанные нами программы смогли успешно пережить эти изменения.

Процесс управления требованиями может быть полезен только в том случае, если он позволяет выявлять и решать проблему изменений.

Невозможно предотвратить изменения, но можно научиться ими управлять.

Каскадный подход

Каскадный подход представлял собой одну из первых систематизаций потоков работ программной инженерии и на момент своего появления представлял безусловную ценность. Однако практика выполнения проектов автоматизации в рамках данного подхода показала низкий (порядка 20%) процент успешных проектов.

1. Первая причина – лавинообразное разрастание цены исправления ошибок, возникших на ранних этапах создания системы от этапа к этапу (схема не имела обратных связей и, соответственно, ошибки копились вплоть до этапа внедрения).
2. Вторая – статичность схемы. Крупный проект автоматизации может длиться 2, 3 года, а требования, замороженные в SRS, перестают соответствовать бизнес-реалиям предприятия внедрения, которое за столь долгий период может существенно измениться.

Современные методологии управления проектами разработки ПО

Подавляющее большинство современных методологий управления проектами разработки программного обеспечения при всём своём разнообразии сходятся в одном: **требования могут меняться!** Причём практически на любой фазе производства АИС. Эта новая парадигма работы с требованиями, безусловно, импонирует Заказчикам. Теперь они имеют право ошибиться и исправить свою ошибку. Исключение составляют, например, некоторые классы военных разработок, где до сих пор применяются модификации каскадного подхода, базирующиеся на очень тщательной проработке и верификации спецификаций и гарантирующие точное соответствие продукта спецификациям.

Легко ли управлять требованиями?

Классика:

- На 1й встрече заказчик рассказал, чего он хочет
- Перед 2й встречей заказчик поменял своё представление
- Разработчик показал работы, основанные на 1й встрече
- На 2й встрече заказчик **«уточнил», чего «на самом деле» хочет**

Разработчик недоволен

Я уже столько сделал по его предыдущей просьбе. Он сам **не понимает, чего хочет!**

Заказчик недоволен

Я изъясняюсь предельно ясно. **Что тут сложного**? Нужно просто понять и сделать!

Управлять требованиями сложно!

- Желания заказчика **не ясны** ему самому + **изменчивы**
- В одном проекте «Заказчиком» может быть множество людей разных специальностей и положений
- Встречаются унаследованные системы и **нелогичные** «У нас так сложилось»
- Программы **неосязаемы** – нельзя потрогать, сложно чётко вообразить и представить
- Теоретически программы могут дорабатываться бесконечно, но это стоит очень **недёшево**

Итак…

- Если «***двусмысленность – страшилка любой спецификации требований***», то неконтролируемое изменение и разрастание требований – ходячий кошмар Разработчика.

- Вопрос контроля процесса изменений требований и его влияния на другие рабочие потоки программной индустрии настолько серьёзен, что породил отдельную инженерную дисциплину **– управление требованиями.**

Почему изменяются требования

Внешние факторы

- Произошли ***изменения проблемы***, которую мы пытались решить с помощью новой системы (возникли изменения в экономике, в правительственных инструкциях, на рынке или же изменились предпочтения потребителей).
- ***Пользователи изменили свое мнение*** о том, чего они хотят от системы, или свои предпочтения.
- ***Изменилась внешняя среда***, что привело к появлению новых ограничений и/или новых возможностей. Одним из наиболее очевидных примеров изменения среды является постоянно происходящее совершенствование систем аппаратного и программного обеспечения.
- Вошла в эксплуатацию ***новая система.*** Одним из самых неожиданных внешних факторов возникновения изменений является то, что само появление новой системы приводит к тому, что меняются требования к ней.

Внутренние факторы

Помимо внешних факторов, существует ряд внутренних, которые также приводят к возникновению изменений.

- При первоначальном выявлении требований нам не удалось задать правильные вопросы нужным людям и в нужное время. Если процесс коснулся не всех заинтересованных лиц или им не были заданы правильные вопросы, это усугубляет проблему изменений, так как нет понимания истинных требований к системе, и в ходе разработки приходится производить значительные изменения, которых можно было бы избежать, если бы на более раннем этапе удалось добиться более полного понимания.

- Нам не удалось создать практический процесс, позволяющий справиться с изменениями требований, которые являются нормой при пошаговой разработке. Возможно, мы пытались "заморозить" требования, и "латентные" необходимые изменения накапливались до тех пор, пока не "взорвались" перед лицом разработчиков и пользователей, вызвав переделки и стресс. Или процесс внесения изменений отсутствовал вовсе; все могли изменять все, что угодно и когда угодно. В таком случае на некотором этапе практически все оказывается измененным и невозможно понять, что к чему.

Наш враг — мы сами

Вайнберг (Weinberg, 1995) заметил, что изменения могут быть скрытыми. . Данное явление Вайнберг назвал "просачиванием" требований. Среди них можно указать следующие.

- Упомянутые дистрибьюторами улучшения, о которых программисты случайно услышали на переговорах.
- Прямые запросы клиентов, обращенные к программистам.
- Аппаратные функции, которые в итоге не вошли в продукт или не работают.
- Изменение масштаба в ответ на действия конкурентов.
- Функции, включенные программистом из "лучших побуждений" (в расчете, что это понравится клиенту).
- "Сюрпризы" программистов.

Как менеджер проекта может учесть эти изменения и при этом не выйти из графика и удовлетворить критерии качества? **Это сделать невозможно!** Чтобы иметь шансы на успех, "просачивание" требований должно быть остановлено или, по крайней мере, сведено к минимуму.

1

Управление версиями

Контроль версий

Каждая версия документа требований должна содержать историю переработки, где указываются внесенные изменения, дата каждого из них, лицо, внесшее изменение, а также причина. Целесообразно добавлять номер версии к названию каждого отдельного требования, который можно последовательно увеличивать при модификации требований.

Для документирования версий используются текстовые процессоры, электронные таблицы. Существуют специализированные средства для контроля версий и конфигураций.

Управление версиями — идентификация уникальным образом разных версий каждого элемента — применяется к отдельным требованиям и их наборам, которые обычно представлены в форме документа. Каждая версия требований должна уникальным образом идентифицироваться. У каждого члена команды должен быть доступ к текущей версии требований, а изменения необходимо ясно документировать и доводить до всех заинтересованных лиц. Чтобы минимизировать путаницу и непонимание, назначьте право обновления требований строго определенным лицам и убедитесь, что идентификатор версии изменяется при каждом изменении требования. Каждая версия документа требований или каждое требование должно содержать историю изменений, где указываются внесенные изменения, дата каждого из них, лицо, внесшее изменение, а также причина изменения.

Пример

Конфуз может произойти, когда над проектом работает несколько бизнес-аналитиков. Один бизнес-аналитик может начать редактировать версию 1.2 спецификации требований к ПО, а несколькими днями позже другой бизнес-аналитик может поработать над теми же требованиями, назначив им ту же версию 1.2, даже не подозревая о конфликте версий.

Очень скоро изменения теряются, требования перестают быть актуальными, работу приходится переделывать и углубляется недопонимание.

Наиболее надежный метод управления версиями заключается в хранении требований в базе данных серийного средства управления требованиями. Эти средства отслеживают полную историю изменений требований, что особенно важно, если нужно вернуться к более ранней версии требования.

Если вы сохраняете требования в документах, храните документы требований в таком средстве управления версиями, в которой применяется контроль исходного кода с помощью системы управления версиями (check-in и check-out).

Бизнес-аналитику этого проекта и его заместителю, отвечающему за архивирование, предоставлен доступ на чтение и запись; остальным членам команды — доступ только на чтение.

Возможные атрибуты требований

- дата создания требования;
- номер текущей версии требования;
- автор, создавший требование;
- приоритет;
- состояние требования;
- происхождение или источник требования;
- логическое обоснование требования;
- номер выпуска или итерации, на которую назначено требование;
- контактное лицо или ответственный за принятие решений по внесению изменений в требование;
- метод проверки или критерий приемки.

2

Управление изменениями

Как-то, оценивая процесс разработки ПО, я спросил специалистов, работающих над проектом, как они включают изменения в требования к продукту. После неловкой паузы, один из них сказал: «Каждый раз, когда специалист отдела маркетинга хочет внести изменение, он обращается к Брюсу или Сэнди, потому что они всегда соглашаются, а мы нет». Эта процедура внесения изменений не показалось мне хорошей.

Карл И. Вигерс

Процесс управления изменениями

Поскольку изменения являются неотъемлемой частью процесса, а источники их поступления могут быть как внешними, так и внутренними, необходим процесс управления изменениями требований. Согласно рекомендациям Вайнберга (Weinberg), процесс обработки изменений должен включать в себя следующие шаги.

1. *Осознать, что изменения неизбежны, и разработать план управления изменениями.*
2. *Сформировать базовый уровень требований (requirements baseline).*
3. *Установить единый канал контроля изменений.*
4. *Использовать систему контроля изменений для их фиксации.*
5. *Обрабатывать изменения по иерархическому принципу.*

Рассмотрим более подробно каждый из этих шагов.

Шаг 1. Осознать, что изменения неизбежны, и разработать план управления изменениями

Команда должна признать, что изменения требований к системе неизбежны и даже необходимы. Изменения будут возникать, и команда, зная об этом, должна разработать соответствующий план управления изменениями, который разрешит внесение определенных изменений в исходный базовый уровень.

Что касается легитимности изменений, все они (за исключением сюрпризов) могут считаться таковыми, так как исходят от заинтересованных лиц, которые имеют как реальную потребность, так и возможность внести реальный вклад в приложение.

Шаг 2. Формирование базового уровня требований

Ближе к концу фазы исследования жизненного цикла разработки команда должна скомпоновать все известные требования к системе. Процесс формирования базового уровня может заключаться просто в наложении контроля исправлений на соответствующие артефакты (документ-концепцию, программные требования, модели вариантов использования) и публикации базового уровня для команды разработчиков. Собранные в этих документах отдельные требования создают базовый уровень информации о требованиях и предполагаемых вариантах использования системы.

Этот простой шаг позволяет команде различать известные ("старые") требования и новые (те, которые были добавлены, удалены или модифицированы).

Чтобы договориться об изменении требований, сначала нужно их зафиксировать в "первозданном виде".

Базовая версия (baseline) - *это набор функциональных и нефункциональных требований, которые разработчики обязались реализовать в определенной версии (итерации).*

Управление требованиями - это рабочий процесс, следовательно, он должен подчиняться определенным правилам и процедурам.

После задания базового уровня гораздо легче выявлять и обрабатывать новые требования. Запрос на новое требование можно сравнить с существующей базой и определить, где оно будет размещаться и не будет ли оно конфликтовать с другими требованиями

Процесс формирования базового уровня может заключаться просто в наложении контроля исправлений на соответствующие артефакты (документ-концепцию, программные требования, модели вариантов использования) и публикации базового уровня для команды разработчиков. Собранные в этих документах отдельные требования создают базовый уровень информации о требованиях и предполагаемых вариантах использования системы.

Этот простой шаг позволяет команде различать известные ("старые") требования и новые (те, которые были добавлены, удалены или модифицированы). После задания базового уровня гораздо легче выявлять и обрабатывать новые требования. Запрос на новое требование можно сравнить с существующей базой и определить, где оно будет размещаться и не будет ли оно конфликтовать с другими требованиями.

Однако упорядочение процесса внесения изменений не означает, что можно вносить их сколько угодно. С точки зрения, как пользователей, так и разработчиков жизнь будет гораздо проще, если удастся создать набор стабильных корректных требований. Даже при достаточно хорошо организованном процессе управления изменениями существуют ограничения на количество изменений, которые разработчик сможет учесть, особенно на стадиях проектирования и реализации.

Как правило, коэффициент изменения требований во время разработки **составляет 1-4% в месяц**. Но если его величина **превышает 2% в месяц**, проект подвергается высокому риску "перемешивания" требований.

Управление незапланированным ростом объема

Незапланированным изменением требований считается предложение новой функциональности и существенной модификации после утверждения базовой версии требований к проекту. Чем дольше продолжается работа над проектами, тем больше их объем.

Проблема заключается не в изменении требований, а в том, что запоздалые изменения оказывают существенное влияние на уже проделанную работу.

Ключевая стратегия ограничения роста незапланированных требований - разработка хороших требований, руководствуясь приемами и методами, рассмотренными ваше, в максимальном контакте с Заказчиком.

Другая стратегия - это умение сказать: "Нет" . Психология большинства людей устроена так, что им тяжело отказывать, что в данном случае может привести к состоянию постоянного прессинга. К.Вигерс предлагает "смягчить" этот подход, заменив "Нет", на "Не сейчас" (требование обязательно будет выполнено, но не в текущей версии).

Можно добавить в эту копилку фразу "Зачем?". Опыт показывает, что данная фраза значительно упрощает общение и позволяет сподвигнуть представителя Заказчика на размышления: а действительно ли его идея хороша, а какую конкретную пользу она принесет бизнесу его предприятия?

Совет по управлению изменениями

После регистрации запроса на изменение необходимо принять решение о его дальнейшей судьбе. Совет по контролю за изменениями (change control board, **CCB**) (иногда его называют советом по управлению конфигурацией) был признан лучшим практическим решением при разработке ПО . На практике функции Совета могут быть делегированы и одному человеку (в зависимости от размеров проекта).

Запрос на изменение может быть принят, либо отклонен. В первом случае его необходимо включить в план работ над проектом, во втором - сформулировать мотивированный отказ.

При принятии решения по запросу необходимо исходить:

a) из степени важности запроса для Заказчика и
b) из его стоимости для Разработчика. Стоимость определяется на основании анализа влияния изменения.

После утверждения базовой версии набора требований придерживайтесь этого порядка для внесения всех предлагаемых изменений в нее.

Изменение системы нельзя инициировать до тех пор, пока механизм контроля за изменениями не признает его "официальным«»!

Обработка запроса

Шаг 4. Использование системы контроля изменений для их фиксации

Проще всего дело обстоит с внешними изменениями, которые производятся по запросу клиента. Их легко выявлять, и они будут естественным образом включены в проект руководством или органом, осуществляющим контроль за изменениями. Но во время разработки возникает огромное множество иных изменений системы.

Многие предлагаемые изменения, возникающие во время проектирования, кодирования и тестирования системы, могут казаться не связанными с требованиями (например, исправление ошибок кода или проектирования). Тем не менее необходимо оценить их воздействие.

А если подходит срок сдачи, следует даже принять сознательное решение о том, какие ошибки оставить в системе (из-за того, что их исправление может дестабилизировать систему в целом и тем самым поставить под угрозу дату сдачи), а какие — устранить. Помимо этого, многие ошибки могут влиять на требования, вызывать необходимость их согласования или устранения неоднозначности отдельного известного требования.

Команде следует разработать некую систему для фиксации всех запросов на изменения.

Это может осуществляться с помощью системы отслеживания изменений и неполадок (*bug tracking*), которая обеспечивает создание централизованного архива запросов.

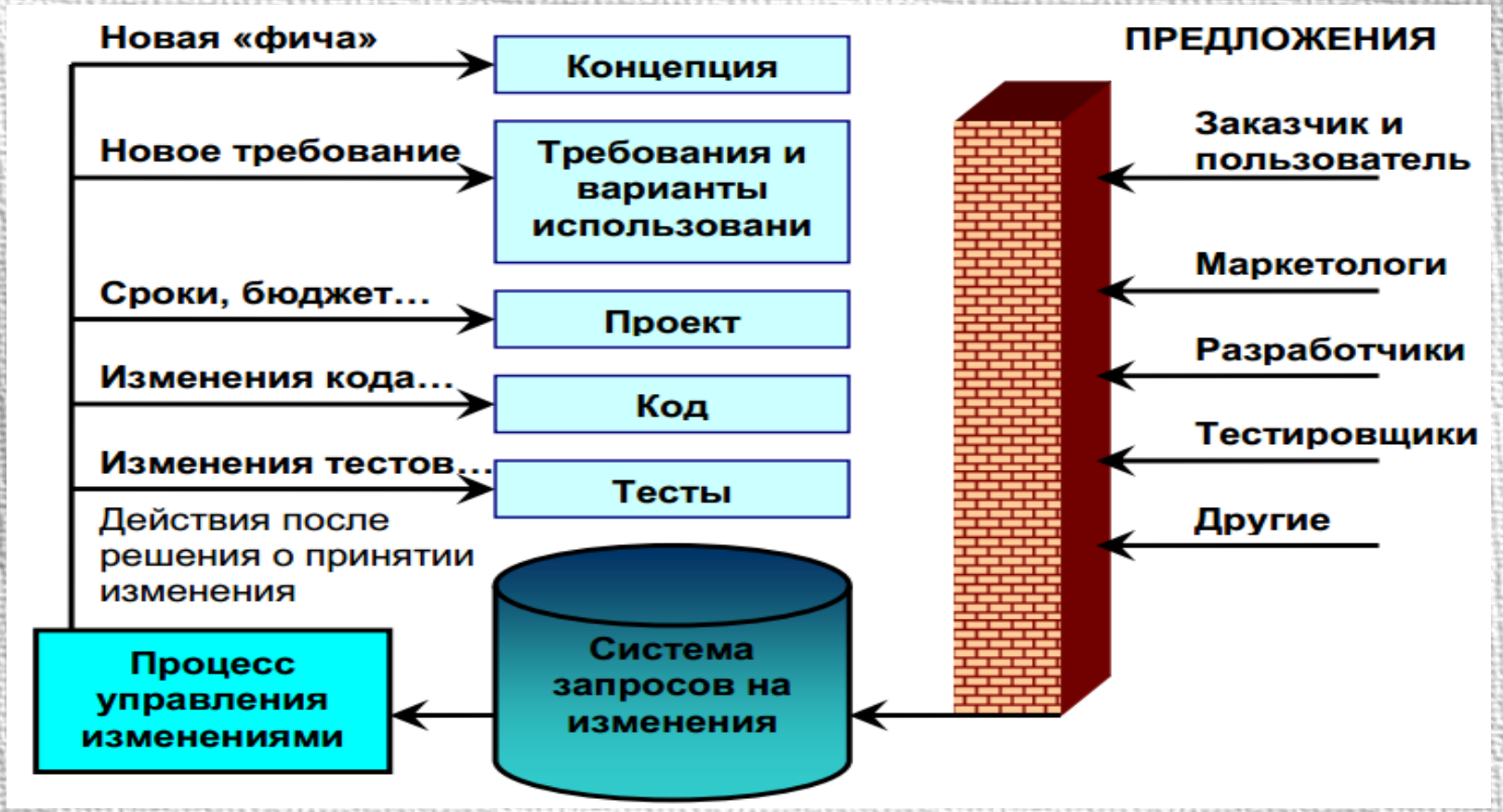
Эту систему следует использовать, чтобы фиксировать все предложения и передавать их руководству **Совета по контролю за изменениями** (ССВ) для принятия решения. Совет играет ключевую роль в успехе проекта. Он должен состоять из трех-пяти человек, представляющих интересы основных участников проекта: заказчиков, представителей маркетинга и руководства проекта. Когда решается вопрос, принять ли запрос об изменении, ССВ должен учитывать следующие факторы.

- Влияние изменения на стоимость и функциональные возможности системы.
- Воздействие изменения на заказчиков и внешних участников.
- Возможность того, что изменение дестабилизирует систему.

При принятии решения ССВ также несет ответственность за то, чтобы отметить все, на что повлияет изменение, даже если оно не принимается.

Источники изменений и процесс внесения изменений в проект

После принятия изменения необходимо решить, куда его вставить. Например, нужно определить, предлагается ли изменить требование или тест. Последующие изменения будут распространяться по иерархии так, как показано на рисунке ниже.



Шаг 5. Иерархическое управление изменениями

То, что все вокруг заинтересованы во внесении изменений, не так уж плохо. Можно даже предположить, что все эти изменения, за исключением сюрпризов, полезны. Проблемой является то, что изменения могут не документироваться и не анализироваться. Если их тщательным образом не обрабатывать, это может привести к неприятным последствиям. Изменение одного требования может оказать возмущающее воздействие на связанные с ним требования, проектирование или другие подсистемы. К сожалению, это не учитывают представители маркетинга, которые часто просят программиста "быстро и просто" изменить систему.

Проблема осложняется тем, что при отсутствии явно заданного процесса изменения обычно производятся "восходящим" образом, **что конечно неправильно!**. Это означает, что если изменение появляется во время написания кода новой системы, оно, как правило, вносится непосредственно в программный код. Если разработчики предельно дисциплинированы, они могут затем задуматься: "Не вызовут ли изменения, которые мы вносим в код, изменений в проекте? А повлияют ли изменения на уровне проектирования на требования? Окажут ли изменения требований к программе воздействие на документ-концепцию?" Тем не менее никто не вспомнит о том, что надо хоть что-нибудь сказать об этом изменении группе тестирования, члены которой полагают, что им нужно создавать планы тестирования, ориентируясь на исходные программные требования!

Чтобы ослабить возмущающий эффект изменений требований, необходимо выполнять их в иерархии **нисходящим образом** от документа концепции к требованиям, коду и далее к тестам. Так как изменение документа-концепции может заключаться в удалении функций, нам может понадобиться создать совершенно новый исходный набор требований к программному обеспечению, а это может привести к соответствующим изменениям проектирования, кода и планов тестирования.

Контрольный журнал изменений

Полезно вести контрольный журнал изменений, где в частности, фиксируются изменения отдельных требований. При поддержке вспомогательной программы это позволит работать с каждым требованием отдельно, независимо от того, частью какого документа или модели оно является. Все изменения требований автоматически фиксируются (образуя историю изменения одного требования), и их можно впоследствии проверять и пересматривать. В истории изменений указывается текущая формулировка требования, в том числе текущие значения всех его атрибутов. Кроме того, в истории изменений в хронологическом порядке представлена последовательность всех предшествующих изменений данного требования и его атрибутов. Также записываются автор, дата и время изменения.

Программа также должна предоставлять возможность вводить описание изменения. Как правило, вводится одно - два предложения, чтобы объяснить, почему было сделано изменение, сослаться на другие документы и т.д.

Сохранение истории изменения требований

.....		
0627 (12.12.2003)	ТР0 поступило	Иванов
.....		
0632 (13.12.2003)	ТР1 получено из ТР0	Петров
0633	ТР2 получено из ТР0	Петров
0634	ТР3 получено из ТР0	Петров
.....		
0651 (14.12.2003)	ТР1 принято	Петров
0652	ТР2 отклонено	Петров
0653	ТР3 принято	Петров
0654	ТР1У получено из ТР1	Петров
0655	ТР3У получено из ТР3	Петров
.....		
0681 (15.12.2003)	М01 расширено ТР1У, ТР3У	Петров
0682	ТР3У отклонено	Петров
0683	ТР3 отклонено	Автоматически 0682
0684	М01 восстановлено по 0681	Автоматически 0682
0685	М01 расширено ТР1У	Петров

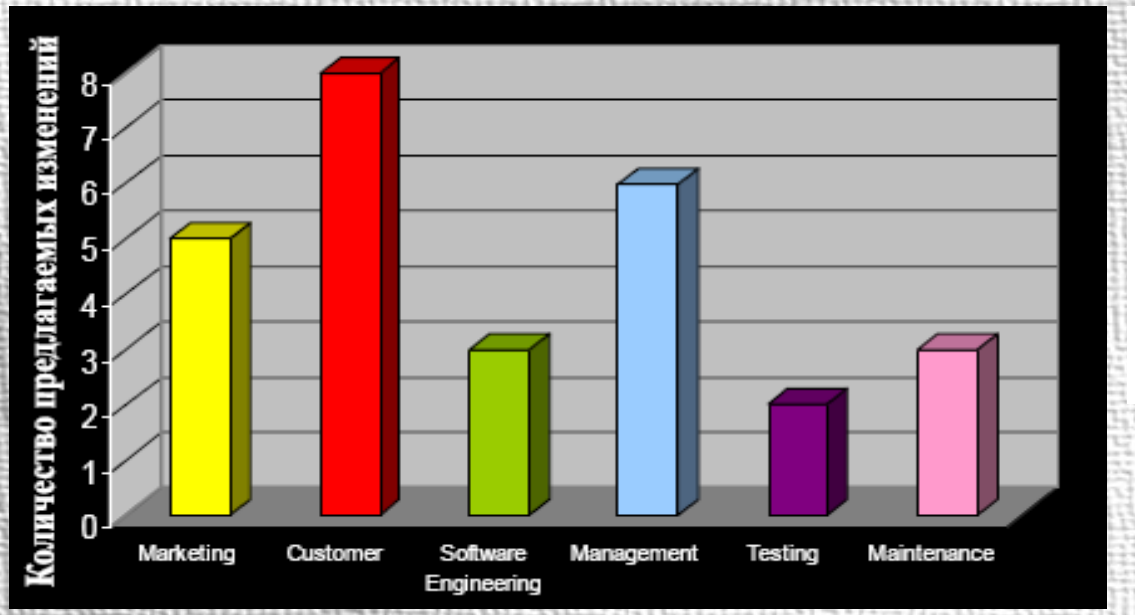
На рисунке приведен условный пример фрагмента протокола оперирования с требованием ТР0, поступившим от Иванова (авторство указано в правой графе протокола) 12.12.2003. Все записи пронумерованы и справа от номера указана дата – 13.12.2003 к работе с этим требованием приступил Петров. Прежде всего, он трансформирует исходное представление в три типизированных представленияТР1, ТР2 и ТР3, из которых второе требование отклоняется (запись 0652 от 14.12.2003). Для унификации ТР1 и ТР3трансформируются в ТР1У и ТР3У (записи 0654 и 0655 от 14.12.2003). 15.12.2003 Петров строит расширение модели М01 посредством ТР1У и ТР3У (запись 0681).

Анализ показал неприемлемость такого расширения из-за ТР3У. Как следствие, решено отклонить ТР3У (запись 0682). Это влечет за собой автоматическое отклонение ТР3 (в графе авторства записи 0683есть соответствующая пометка и указана причина отклонения — ссылка на предыдущую запись).

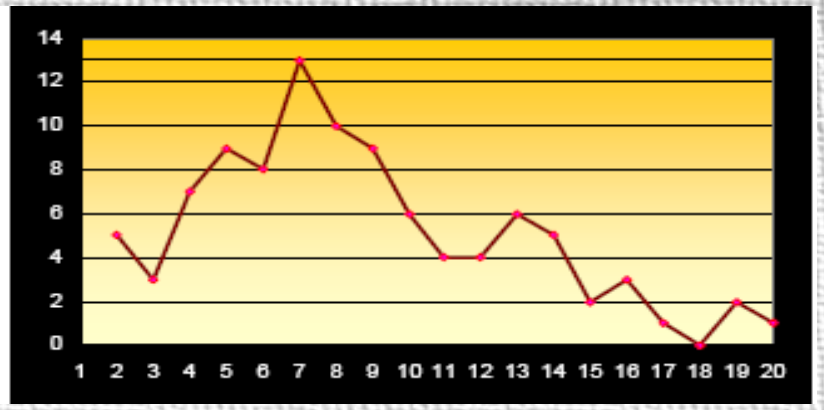
По этой же причине автоматически восстанавливается модель М01. Завершает фрагмент запись, свидетельствующая о том, что Петров строит расширение модели с учетом лишь ТР1У.

Измерение активности изменений

Измерение активности изменений — это способ оценки стабильности требований. Он позволяет также обнаружить возможности для улучшения процесса, чтобы снизить количество изменений в будущем.



Источники изменений



Количество изменений в требованиях по неделям

Шаблон описания процесса контроля изменений

1. Введение
   1. Назначение
   2. Границы
   3. Определения
2. Роли и ответственности
3. Состояние запроса на изменение
4. Входные критерии
5. Задачи
   1. Оценка запроса
   2. Принятие решения
   3. Внесение изменений
   4. Уведомление всех затронутых сторон
6. Проверка
   1. Проверка изменения
   2. Установка продукта
7. Критерии выхода
8. Отчет контроля изменений о статусе запроса

Приложение. Элементы данных, сохраненные для каждого запроса

❸ Отслеживание состояния требований

Отслеживание состояния каждого функционального требования на протяжении всей разработки позволяет более точно оценивать готовность проекта.

Иногда разработчики ПО слишком оптимистичны, когда сообщают об объеме выполненных работ по задаче. Обычный синдром «90 процентов» не говорит руководителю проекта, насколько Иван на самом деле приблизился к завершению своей подсистемы. Но Иван может сообщить о состоянии дел и так:

«*Все очень даже неплохо, шеф. В моей подсистеме 84 требования: 61 из них реализованы и проверены, 14 реализованы, но не проверены и 9 требований ожидают реализации*».

Возможные атрибуты требований

- дата создания требования;
- номер текущей версии требования;
- автор, создавший требование;
- приоритет;
- состояние требования;
- происхождение или источник требования;
- логическое обоснование требования;
- номер выпуска или итерации, на которую назначено требование;
- контактное лицо или ответственный за принятие решений по внесению изменений в требование;
- метод проверки или критерий приемки.

На определение и обновление этих атрибутов требуется часть затрат, отведенных на управление требованиями, однако эти инвестиции в значительной степени окупаемы. В одной компании периодически генерировался отчет о требованиях, в котором показывалось, какое из 750 требований в трех связанных спецификациях назначено каждому дизайнеру. Один из дизайнеров обнаружил несколько требований, которые он не считал своими.

Состояние требования

Состояние	Определение
Proposed (Предложено)	Требование запрошено авторизированным источником
Approved (Одобрено)	Требование проанализировано, его влияние на проект просчитано, и оно было размещено в базовой версии определенной версии. Ключевые заинтересованные в проекте лица согласились с этим требованием, а разработчики ПО обязались реализовать его
Implemented (Реализовано)	Код, реализующий требование, разработан, написан и протестирован. Требование отслежено до соответствующих элементов дизайна и кода

Verified (Проверено)	Корректное функционирование реализованного требования подтверждено в соответствующем продукте. Требование отслежено до соответствующих вариантов тестирования. Теперь требование считается завершенным
Deleted (Удалено)	Утвержденное требование удалено из базовой версии. Опишите причины удаления и назовите того, кто принял это решение
Rejected (Отклонено)	Требование предложено, но не запланировано для реализации ни в одной будущих версий. Опишите причины отклонения требования и назовите того, кто принял это решение

Состояние — один из атрибутов требований, предложенный в предыдущем разделе. Отслеживание состояния означает определение готовности по отношению к тому, что это означает в текущем цикле разработки. Например, вы планируете реализовать в следующей версии только часть варианта использования, оставив полную реализацию до будущих версий. В этом случае вам нужно отлеживать состояние тех функциональных требований, которые запланированы для ближайшего выпуска, потому что этот набор требований должен быть выполнен на 100% до выпуска продукта.

4

Отслеживание связей требований

Отслеживаемость – это наличие ссылок, облегчающих поиск требований. Чтобы реализовать отслеживаемость, каждое требование должно быть уникально идентифицировано, чтобы можно было однозначно ссылаться на него в ходе работы над проектом.

Кажущиеся простыми изменения ПО часто влияют на многие элементы продукта, поэтому возникает необходимость в их изменении. Трудно обнаружить все компоненты системы, которые могут быть затронуты модификацией требования

Преимущества

1. Обнаружение пропущенных требований.
2. Поиск ненужных требований.
3. Анализ влияния изменения.
4. Поддержка и обслуживание.
5. Отслеживание проекта.
6. Реинжиниринг.
7. Тестирование
8. и др.

Общее определение

Очевидно, что наиболее результативным и менее затратным механизмом для этого является РЕГИСТРАЦИЯ выполненных технологических и контрольных работ.

Для этого существует механизм под названием **ПРОСЛЕЖИВАЕМОСТЬ.**

Прослеживаемость является одним из требований ИСО 9001:2000, которое звучит так:

"Если прослеживаемость является требованием, то организация должна управлять обеспечивающей идентификацией продукции и регистрировать ее".

Прослеживаемость требований

Прослеживаемость — возможность установления связей между требованиями и их источникам, с одной стороны, а также с разделами и элементами возникающих при разработке текстов программ, документов и моделей, с другой стороны.

Основано на выделении элементарных требований и присвоении им идентификаторов.

Каждое требование должно иметь уникальный идентификатор, который позволяет прослеживать его разработку на протяжении всего жизненного цикла.

В рабочих продуктах, которые появляются на более поздних этапах жизненного цикла, таких как план тестирования, каждая ссылка на свойство системы должна прослеживаться до определения и спецификации требований. Матрица прослеживаемости требований, является отличным инструментальным средством для решения упомянутой задачи.

Обеспечение прослеживаемости — определяет процедуры прослеживания требований в документах, моделях и коде, вырабатываемых при создании системы.

Основной техникой прослеживания является процедура трассирования

Анализ влияния изменения

Анализ влияния обеспечивает точное понимание подтекста предложенного изменения, что помогает команде принимать информированные бизнес-решения о том, какое изменение одобрить. Анализ позволяет выявить компоненты, которые может понадобиться создать, изменить или отклонить, и оценить затраты, связанные с реализацией изменения. До того, как разработчик ответит: "Конечно, без проблем", он должен потратить время на анализ результата изменения.

Председатель *совета по управлению изменениями* обычно просит опытного разработчика выполнить анализ определенного результата.

Анализ результатов изменений затрагивает три аспекта

1. ***Определите возможные последствия изменения***. Часто они вызывают значительный волновой эффект. Включение множества функций в продукт может снизить его производительность до неприемлемого уровня, например, когда системе, запускаемой ежедневно, потребуется 24 часа для завершения одного запуска.
2. ***Определите все файлы, модели и документы, которые, возможно, придется изменить***, если команда включит все запрошенные изменения.
3. ***Определите задачи, необходимые для реализации изменения***, ***и оцените усилия***, необходимые для выполнения этих задач.

Анализ результатов изменений затрагивает три аспекта

1. ***Определите возможные последствия изменения***. Часто они вызывают значительный волновой эффект. Включение множества функций в продукт может снизить его производительность до неприемлемого уровня, например, когда системе, запускаемой ежедневно, потребуется 24 часа для завершения одного запуска.
2. ***Определите все файлы, модели и документы, которые, возможно, придется изменить***, если команда включит все запрошенные изменения.
3. ***Определите задачи, необходимые для реализации изменения***, ***и оцените усилия***, необходимые для выполнения этих задач.

Техникой, позволяющей проводить подобный анализ является **трассирование**.

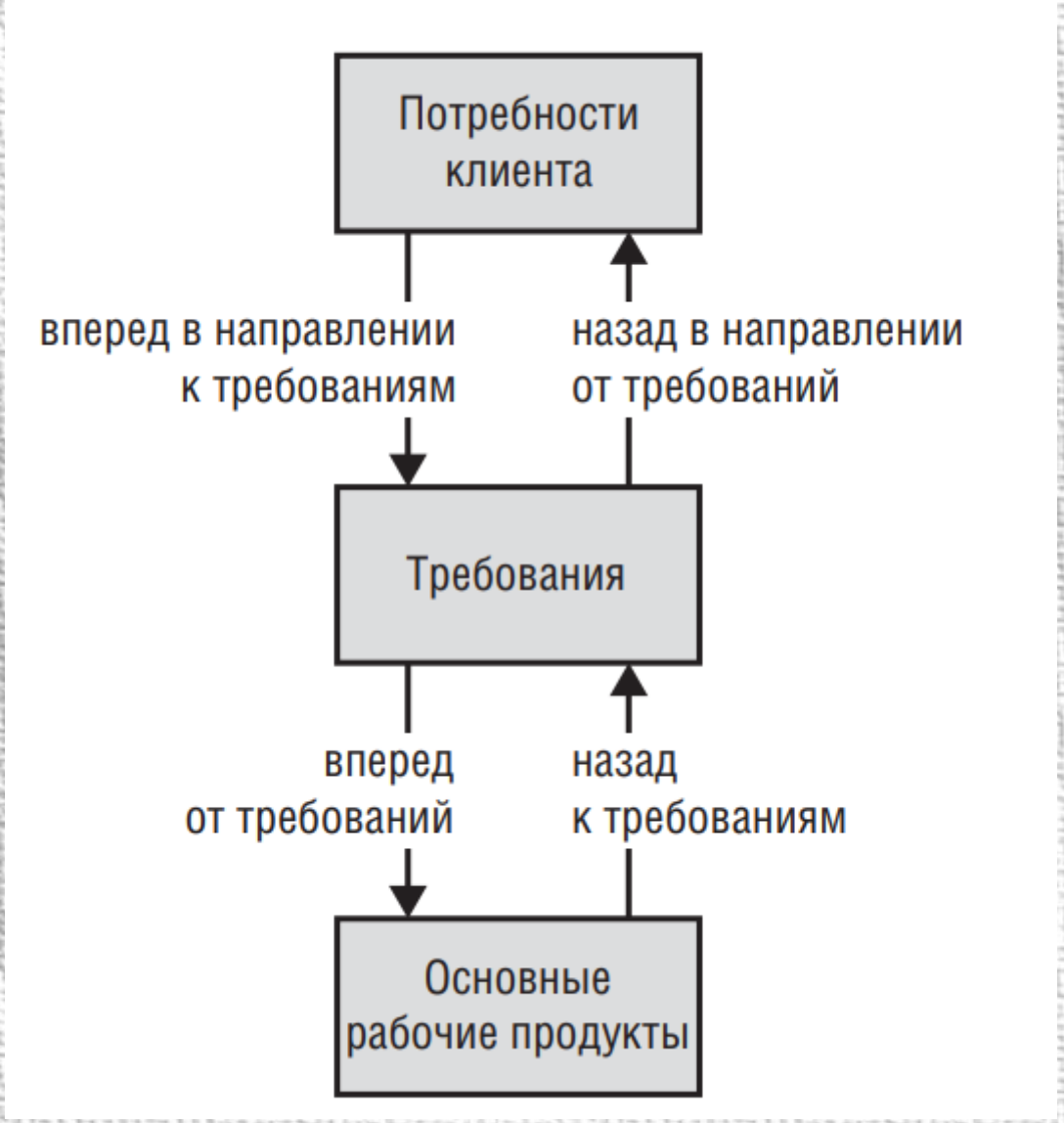
Выполнять анализ влияния проще, если вы построите «дорожную карту», на которой показано, где именно в ПО реализовано каждое требование и бизнес-правило.

Что такое трассируемость?

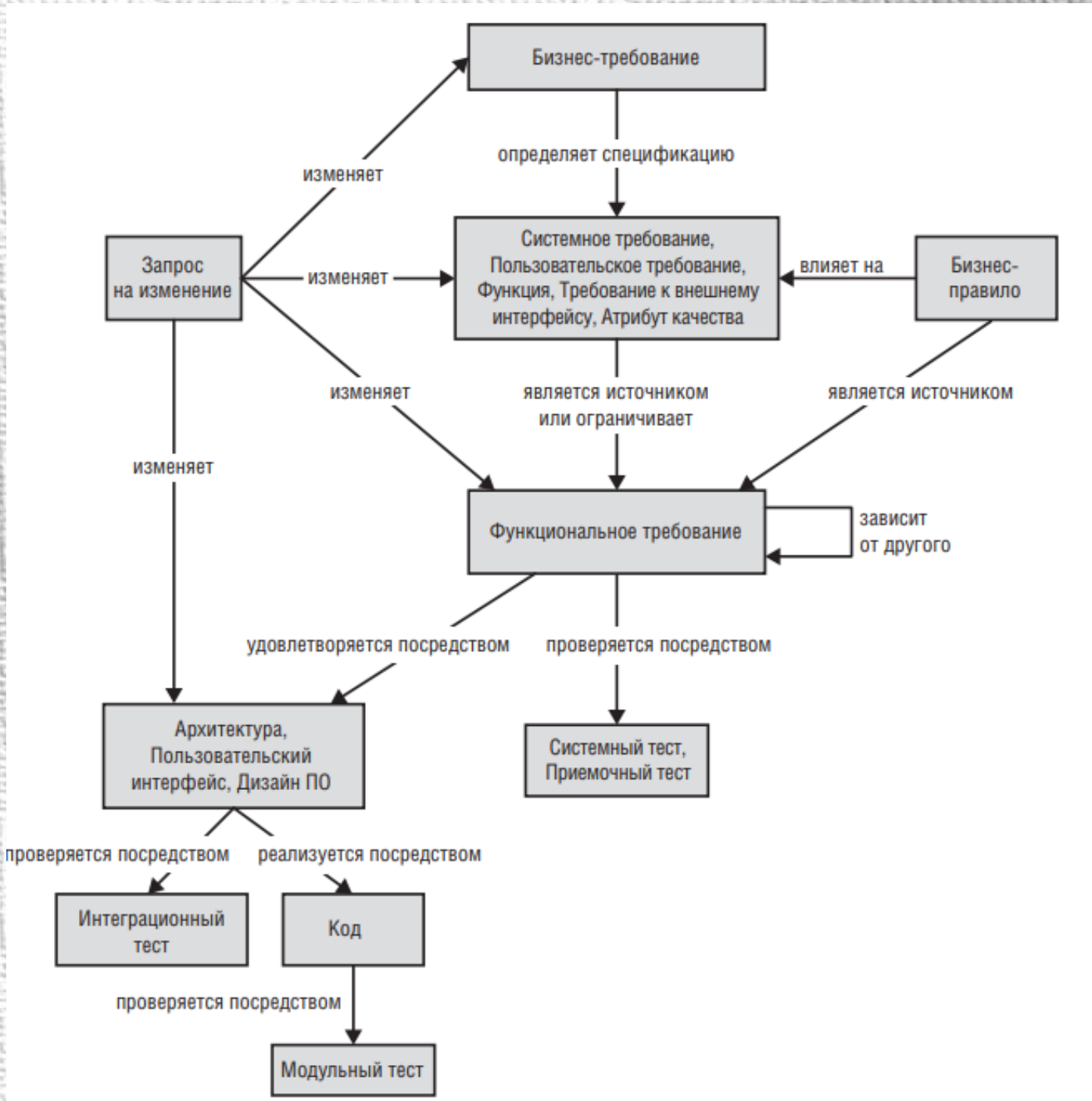
Rational Unified Process (RUP) определяет понятие трассируемости, как

"***способность соотнести какой-либо элемент проекта с другим, связанным с проектом элементом, особенно с теми, которые имеют отношение к техническим требованиям***."

Четыре типа отслеживаемости требований



Возможные связи отслеживания требований



Зачем нужна трассируемость?

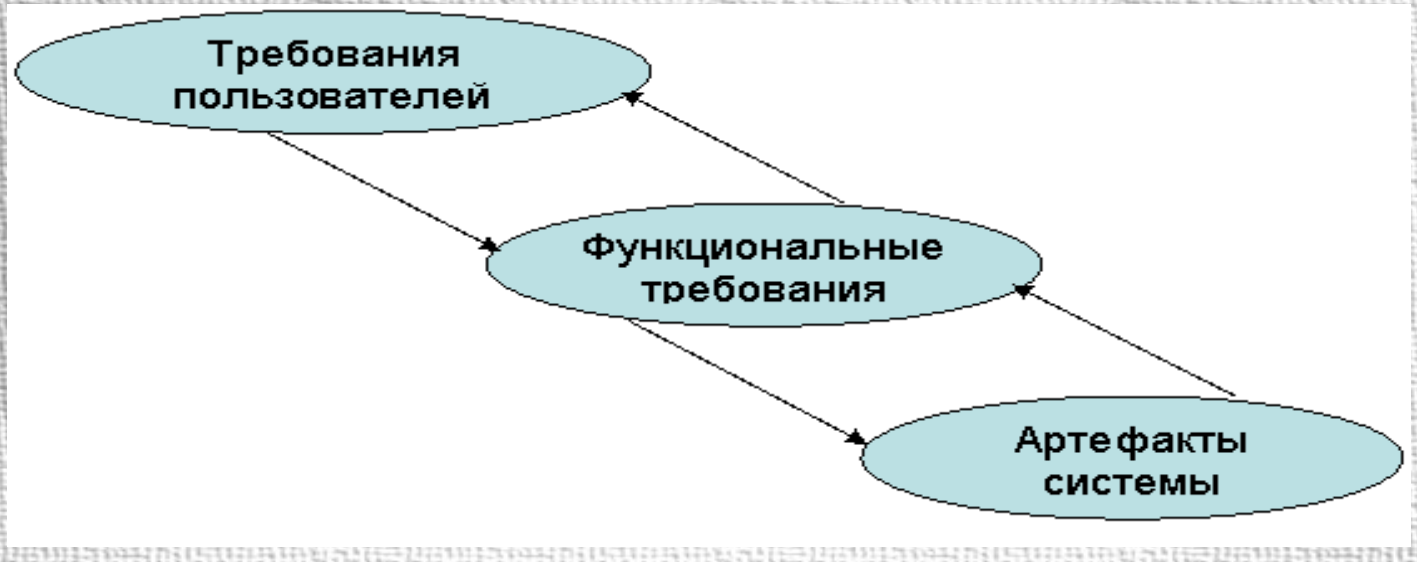
Трассируемость служит двум основным целям:

1. Обеспечению *качества* продукта через
   - Валидация - придание ему всех тех возможностей, которые затребовал заказчик, а также обязательное отсутствие тех качеств, которые не были им запрошены ("создание правильного продукта").
   - Верификация - другой аспект качества заключается в обеспечении правильного функционирования всех затребованных возможностей («построение правильного продукта").
2. *Анализу влияния* изменений через определение затронутых требований, проектных решений и артефактов реализации, а также связанных с ними тестов.

Трассируемость требований

Связи трассируемости помогают следить за развитием требования в обоих направлениях- от первоисточника к реализации и наоборот. Трассируемость представляет собой одно из качеств хороших требований.

Для осуществления анализа трассируемости каждое требование должно быть уникально идентифицировано.



Матрица отслеживаемости требований
(матрица трассируемости)

Наиболее типичный способ представления связей между требованиями и другими элементами системы — матрица отслеживаемости требований, которую также называют таблицей трассируемости (requirements traceability matrix).

Таблица трассируемости показывает связи между требованиями и различными другими объектами.

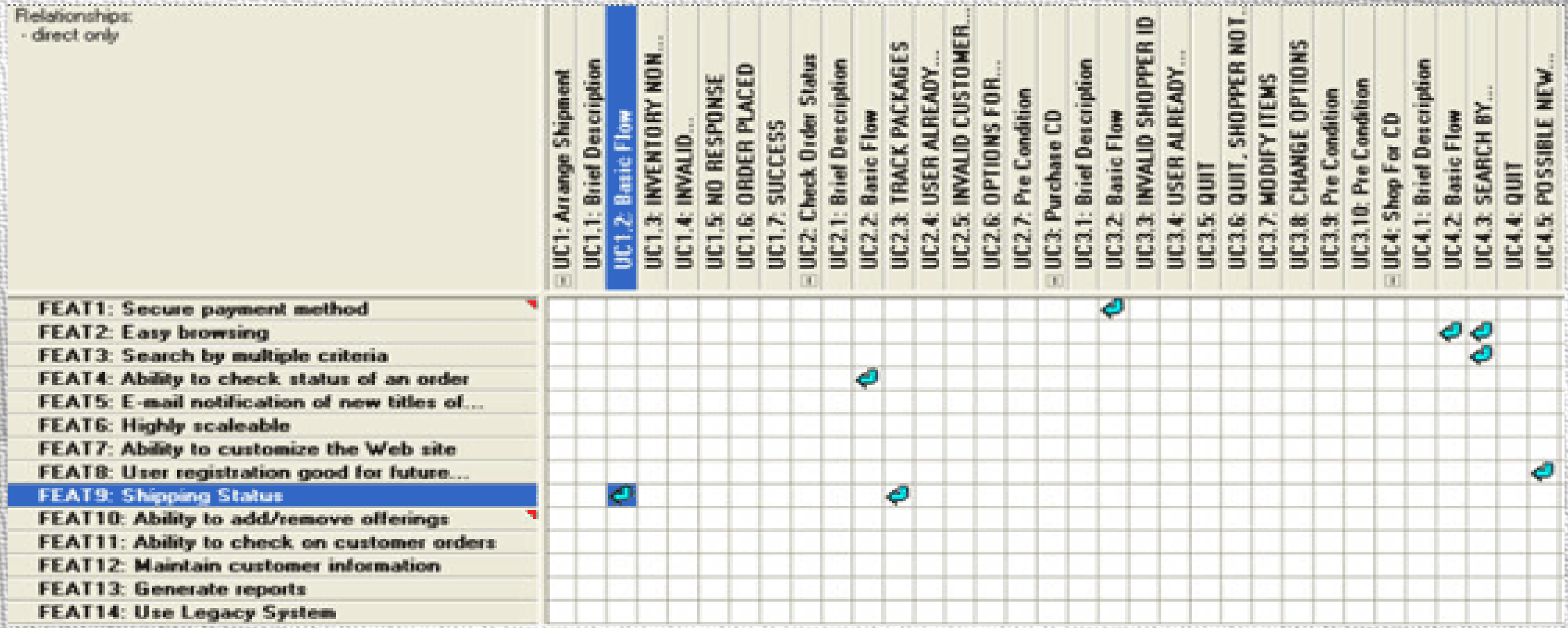
В матрице трассируемости в строках указываются **пользовательские требования**, а столбцах - функциональные требования, элемент проектирования, вариант версии и др. В этих столбцах заполняются данные о степени выполнимости системных требований на каждом элементе создаваемого продукта. Механизм ссылок в таблице позволяет проверять связанные с каждым элементом продукта *диаграммы вариантов использования*, потоки данных, классы и др.

Степень заполненности этой таблицы характеризует степень покрытия одних требований другими. Например, покрыты ли исходные бизнес-требования функциональными спецификациями. Это достаточно простой и эффективный инструмент для контроля за полнотой описания требований к разрабатываемому продукту.

Матрица трассируемости
(requirements traceability matrix)

Пользовательское требование	Функциональное требование	Элемент дизайна	Модуль кода	Вариант тестирования
UC-28	catalog.query.sort	Каталог класса	catalog.sort()	search.7 search.8
UC-29	catalog.query.import	Каталог класса	catalog.import() catalog.validate()	search.12 search.13 search.14

Матрица трассируемости, доступная в IBM Rational RequisitePro, позволяет выполнять визуализацию и составлять отчёты о трассируемости.



Product backlog

pages: [1]

Tracing | Releases: all | Not resolved | Transition: all | Type: all | Priority: all | Actions

Workload: | Submitted: | Author: | Tags: all | Feature | Submitted in

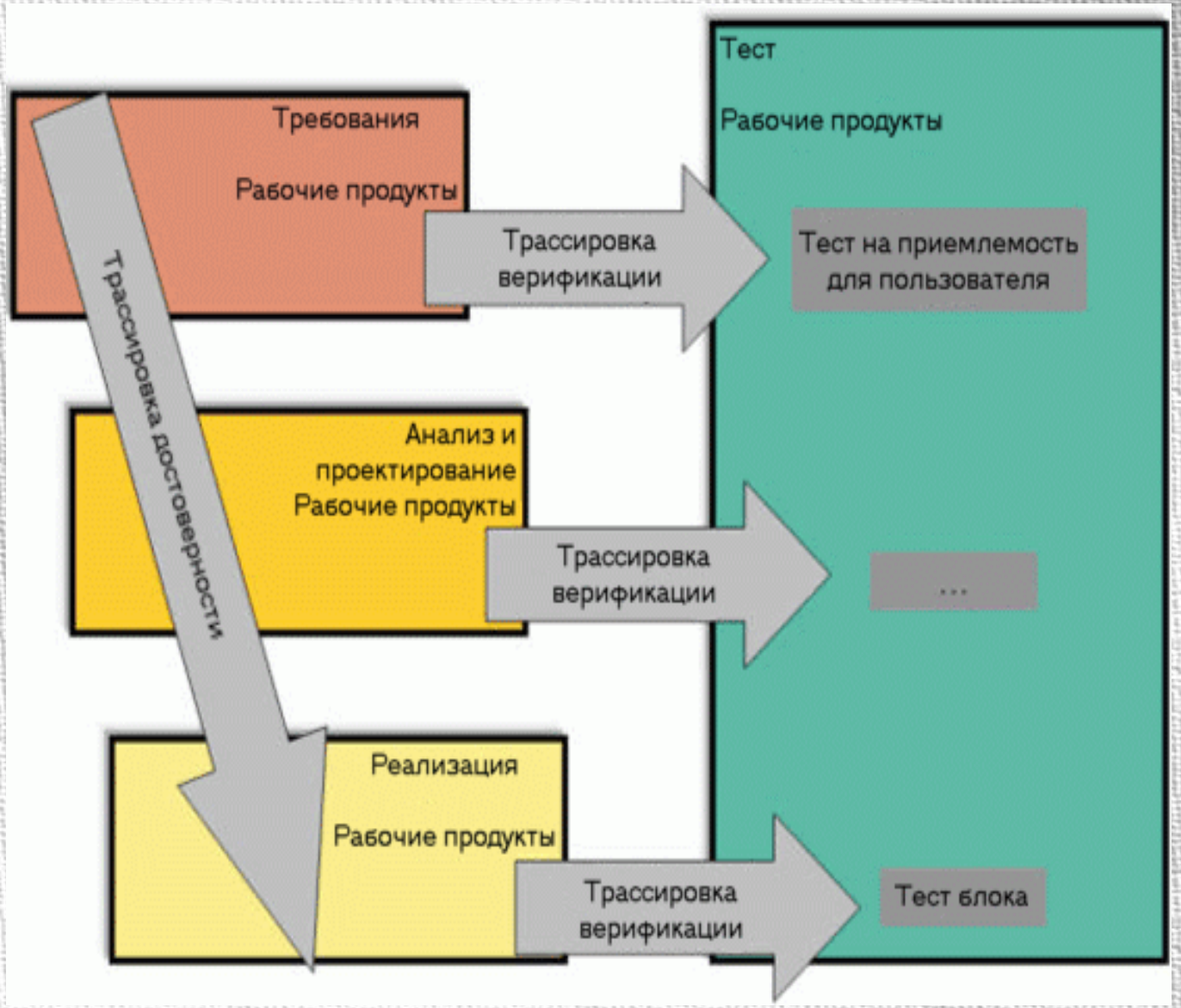
No.	Description	Tasks	Requirements	Test documents	Testing results	Documentation	Source code
[F-15] Распределенное хранение файлов системы (Must have)							
1	[I-65] Enhancement: Синхронизация файлов в хранилище		[R-331]	[S-378]			[C-1]
2	[I-63] Issue: Разработка механизма резервного копирования файлов с распределенного хранилища	T A D	[R-332] [R-335]				
[F-16] Метрики (Should have)							
3	[I-32] Enhancement: Подсистема сбора ключевых индикаторов производительности	T T A D	[R-330] [R-331] [R-334] [R-338]	[S-373] [S-374] [S-376]	1.0.1 [Inception] 1.0.4 [Transition]		

Total issues: 3

Аспекты трассируемости

Параметр "**достоверность**" трассирует различные уровни абстракции или детализации во время разработки относительно друг друга.

Трассировка "**верификации** (формальной правильности)" отслеживает требования, анализ и реализацию рабочих продуктов по сравнению с соответствующими тестами.



Процедура трассирования

Процедура трассирования состоит в следующем:

1) выбирается элемент (функция, фрагмент или некоторая часть) из матрицы трассирования требований, за которым проводится прослеживание на этапах ЖЦ;
2) составляется список вопросов, по которым на каждом этапе проверяются связи при *реализации требований* в продукте, и если изменяется какое-то звено в цепочке требований, то может модифицироваться процедура разработки этого элемента на последующем этапе ЖЦ;
3) проводится мониторинг статуса каждого требования на соответствие выполнения согласно принятому плану;
4) уточнение ресурсов выполнения проекта при необходимости проведения изменений в требования и в элементы проекта.

Управление — искусство карабкаться по лестнице вверх. Лидерство — умение определить, к той ли стене приставлена лестница.
П. Друкер, У. Беннис

СИСТЕМЫ УПРАВЛЕНИЯ ТРЕБОВАНИЯМИ

Системное моделирование

Перечень программных продуктов, позволяющих моделировать на SysML:

- 3SL Cradle
- Modelio
- SysML Designer
- Visual Paradigm for UML

SysML (язык системного моделирования) — это язык моделирования общего назначения для системных инженерных приложений. Он поддерживает спецификацию, анализ, проектирование, проверку и валидацию широкого спектра систем и систем-из-систем

Системы управления требованиями (СУТ)

1. Jama Connect
   https://www.jamasoftware.com/platform/jama-connect/trial/
2. Visure
3. Reqview

В основном на проектах пользуются Excel Word, Polarion, Enterprise Architect, TFS, Wiki, Caliber, Jira.

Devprom и «Техэксперт» — достойные российские аналоги

В отличие от зарубежных сервисов лицензию на использование этих инструментов можно приобрести.

Devprom. Программа позволяет совместно создавать полноценные требования из браузера, документировать UML-модели, формулы и алгоритмы, просматривать изменения в моделях и формулах.

«*Техэксперт*». Позволяет создавать требования из нормативных документов, технических заданий и других технических документов. В системе автоматизирована работа специалистов по согласованию и утверждению требований. Есть много функций по работе с требованиями

Программная инженерия - Постановка и автоматизация процессов разработки ПО

Devprom ALM охватывает не только разработку и трассировку всех видов требований между собой, а также их связь с задачами, тест-кейсами и релизами. Эта система позволяет автоматизировать планирование всех этапов ИТ-проекта, от сбора требований до сопровождения реализованного продукта, автоматизируя ежедневные задачи каждого участника процесса разработки ПО.

Каждому требованию автоматически присваивается идентификационный номер и автоматически заполняются некоторые метаданные (время создания, стадия ЖЦ, автор). Остальные свойства (приоритет, важность, трудоемкость, бюджет и пр.) можно заполнить в настройках и связать требование с исходными, производными и связанными артефактами, чтобы обеспечить трассируемость, которая отлично визуализируется в виде матрицы.

Трассируемость требований